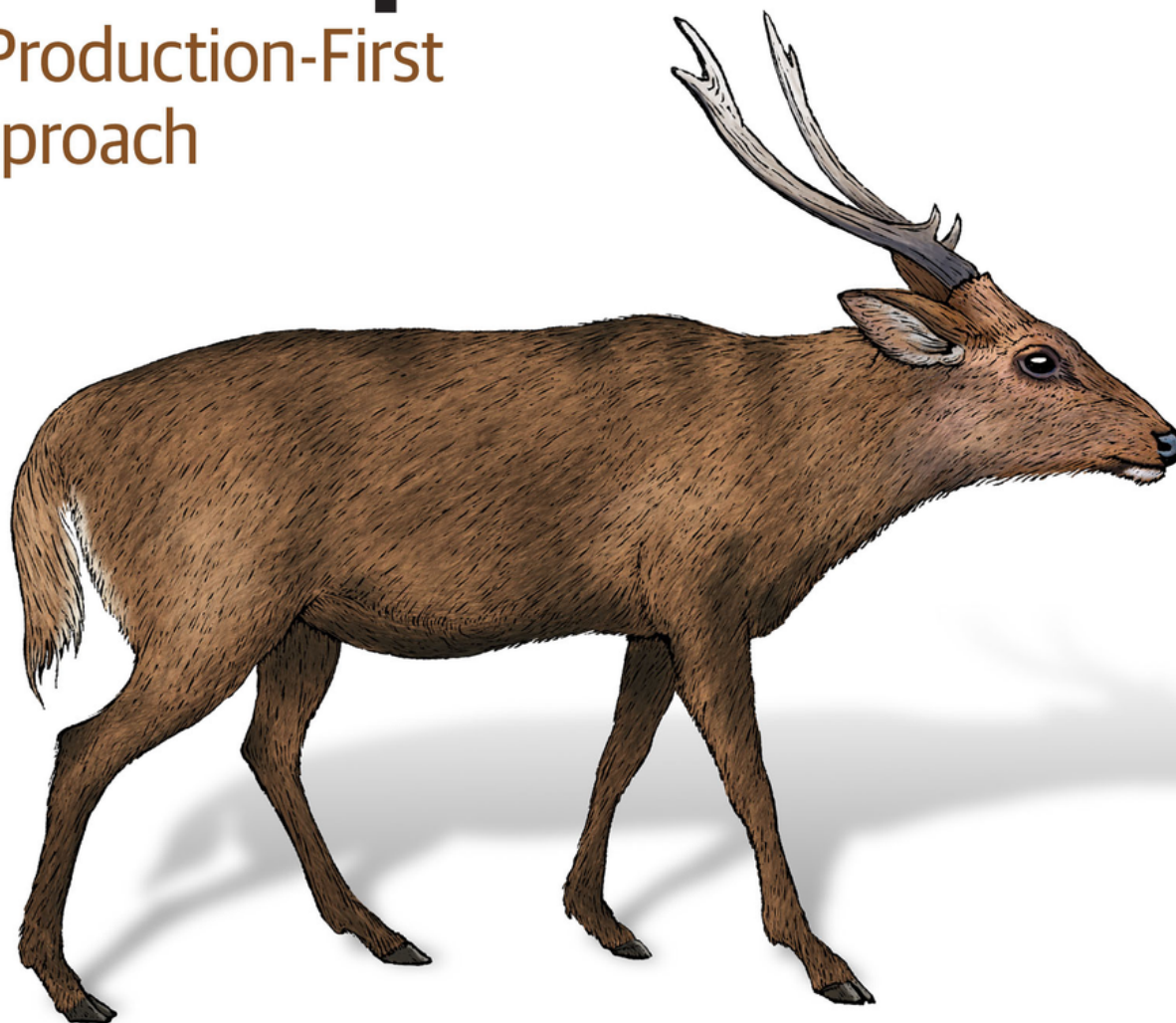


O'REILLY®

Implementing MLOps in the Enterprise

A Production-First
Approach



Yaron Haviv
& Noah Gift

Implementing MLOps in the Enterprise

A Production-First Approach

Yaron Haviv and Noah Gift



Beijing • Boston • Farnham • Sebastopol • Tokyo

OceanofPDF.com

Implementing MLOps in the Enterprise

by Yaron Haviv and Noah Gift

Copyright © 2024 Yaron Haviv and Noah Gift. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisition Editor: Nicole Butterfield
- Development Editor: Corbin Collins
- Production Editor: Beth Kelly
- Copyeditor: Piper Editorial Consulting, LLC
- Proofreader: Heather Walley
- Indexer: WordCo Indexing Services, Inc.
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- December 2023: First Edition

Revision History for the First Edition

- 2023-11-30: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098136581> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Implementing MLOps in the Enterprise*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-13658-1

[LSI]

OceanofPDF.com

Preface

As MLOps veterans, we have often seen the following scenario play out across enterprises building their data science practices.

Traditionally, when enterprises built their data science practice, they would start by building a model in the lab, with a small team, often working on their laptops and with a small, manually extracted dataset. They developed the model in operational isolation, and the results were incorporated manually into applications. Then, once the model was complete and predicting with accuracy, the true struggle of trying to bring it to production, to generate real business value, began.

At this point, the enterprise faced challenges such as ingestion of production data, large scale training, serving in real-time, and monitoring/management of the models in production. These hurdles would often take months to overcome, presenting a huge cost in resources and lost time.

The AI pipeline is siloed, with teams working in isolation and with many different tools and frameworks that don't necessarily play well with each other. This results in a huge waste of resources and businesses not being able to capitalize on their investment in data science. According to [Gartner](#), as many as 85% of data science projects fall short of expectations.

In this book, we propose a mindset shift, one that addresses these existing challenges that prevent bringing models to production. We recommend a production-first approach: starting out not with the model but rather by designing a continuous operational pipeline, and then making sure the various components and practices map into it. By automating as many components as possible and making the process fast and repeatable, the pipeline can scale along with the organization's needs and provide rapid business value while answering dynamic and enterprise MLOps needs.

Today, more businesses understand the vast potential of AI models to positively impact the business across many new use cases. And with generative AI opening up new opportunities for business innovation across industries, it seems that AI adoption and usage are set to skyrocket in the coming years. This book explores how to bring data science to life for these real-world MLOps scenarios.

Who This Book Is For

This book is for practitioners in charge of building, managing, maintaining, and operationalizing the data science process end to end: the heads of data science, heads of ML engineering, senior data scientists, MLOps engineers, and machine learning engineers.

These practitioners are familiar with the nooks and crannies (as well as the challenges and obstacles) of the data science pipeline, and they have the initial technological know-how, for example, in Python, pandas, sklearn, and others.

This book can also be valuable for other technology leaders like CIOs, CTOs, and CDOs who want to efficiently scale the use of AI across their organization, create AI applications for multiple business use cases, and bridge organizational and technological silos that prevent them from doing so today.

The book is meant to be read in three ways. First, in one go, as a strategic guide that opens horizons to new MLOps ideas. Second, when making any strategic changes to the pipeline that require consultation and assistance. For example, when introducing real-time data into the pipeline, scaling the existing pipeline to a new data source/business use case, automating the MLOps pipeline, implementing a Feature Store, or introducing a new tool into the pipeline. Finally, the book can be referred to daily when running and implementing MLOps. For example, for identifying and fixing a bottleneck in the pipeline, pipeline monitoring, and managing inference.

Navigating This Book

This book is built according to the phases of the MLOps pipeline, guiding you through your first steps with MLOps up to the most advanced use cases:

- Chapters 1–3 show how organizations should approach MLOps, how data science teams can get started, and what to prepare for your first

MLOps project.

- Chapters 4–7 explain the components of a resilient and scalable MLOps pipeline and how to build a machine learning pipeline that scales across the organization.
- Chapter 8 covers deep learning pipelines and also dives into GenAI and LLMs.
- Chapters 9 and 10 show how to adapt pipelines for specific verticals and use cases, like hybrid deployments, real-time predictions, composite AI, and so on.

Throughout the book, you will find real code examples to interactively try out for yourself.

After reading this book, you will be a few steps closer to being able to:

- Build an MLOps pipeline.
- Build a deep learning pipeline.
- Build application-specific solutions (for example, for NLP).
- Build use-case specific solutions, (for example, for fraud prediction).

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function

names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, and so on) is available for download at <https://github.com/mlrun/demo-fraud> and <https://github.com/mlrun/demo-llm-tuning>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Implementing MLOps in the Enterprise* by Yaron Haviv and Noah Gift (O'Reilly). Copyright 2024 Yaron Haviv and Noah Gift, 978-1-098-13658-1.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-829-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *<https://oreil.ly/mlops-in-the-enterprise>*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *<https://oreilly.com>*.

Find us on LinkedIn: *<https://linkedin.com/company/oreilly-media>*.

Follow us on Twitter: *<https://twitter.com/oreillymedia>*.

Watch us on YouTube: *<https://youtube.com/oreillymedia>*.

Acknowledgments

We'd like to thank the people behind the scenes who assisted, guided, and supported us throughout this book's journey. Without them, this book wouldn't have been brought to life.

Thank you to the dedicated team at O'Reilly, who provided feedback and guidance, drove the writing process of this book, and helped polish the content. We'd especially like to thank Corbin Collins for being our partner throughout the process, paying close attention to all the details and helping us meet deadlines, and to Nicole Butterfield, for her unwavering support and valuable input.

We're deeply appreciative of our tech reviewers, Dhanasekar Sundararaman, Tigran Harutyunyan, Nivas Durairaj, and Noga Cohen for their expertise and wisdom.

Yaron

I am thrilled to present my first book, a culmination of years of experience and knowledge, as I eagerly share it with readers worldwide.

I am deeply grateful to my family, Dvori, Avia, Ofri, and Amit, for their love and support throughout my career and the long process of writing this book. Their patience and encouragement have meant a lot to me.

Special thanks go to Sahar, who encouraged me to write this book, and to Guy and the Iguazio team, who shared their knowledge, experiences, and code examples.

Noah

It is always an honor to have the opportunity to work on an O'Reilly book. This book marks my fifth O'Reilly and likely my last technical book as I shift to other writing and content creation forms. Thank you to everyone I worked with at O'Reilly, including current and former editors and collaborators and authors of the recent book.

Also, thanks to many of my current and former students, faculty, and staff at **Duke MIDS, Duke Artificial Intelligence Masters in Engineering**, as many ideas in this book came from courses I taught and questions brought up by students.

Finally, thank you to my family, Leah, Liam, and Theodore, who put up with me working on weekends and late at night to hit deadlines.

OceanofPDF.com

Chapter 1. MLOps: What Is It and Why Do We Need It?

At the root of inefficient systems is an interconnected web of incorrect decisions that compound over time. It is tempting to look for a silver bullet fix to a system that doesn't perform well, but that strategy rarely, if ever, pays off. Consider the human body; there is no shortage of quick fixes sold to make you healthy, but the solution to health longevity requires a systematic approach.¹

Similarly, there is no shortage of advice on "getting rich quick." Here again, the data conflicts with what we want to hear. In *Don't Trust Your Gut* (HarperCollins, 2022), Seth Stephens-Davidowitz shows that 84% of the top 0.1% of earners receive at least some money from owning a business. Further, the average age of a business founder is about 42, and some of the most successful companies are real estate or automobile dealerships. These are hardly get-rich-quick schemes but businesses that require significant skill, expertise, and wisdom through life experience.

Cities are another example of complex systems that don't have silver bullet fixes. WalletHub created a list of **best-run cities in America** with San Francisco ranked 149 out of 150 despite having many theoretical advantages over other cities, like beautiful weather, being home to the top tech companies in the world, and a 2022-2023 budget of **\$14 billion** for a population of 842,000 people. The budget is similar to the entire country of **Panama**, with a population of 4.4 million people. As the case of San Francisco shows, revenue or natural beauty alone isn't enough to have a well-run city; there needs to be a comprehensive plan: execution and strategy matter. No single solution is going to make or break a city. The WalletHub survey points to extensive criteria for a well-run city, including infrastructure, economy, safety, health, education, and financial stability.

Similarly, with MLOps, searching for a single answer to getting models into production, perhaps by getting better data or using a specific deep learning framework, is tempting. Instead, just like these other domains, it is essential to have an evidence-based, comprehensive strategy.

What Is MLOps?

At the heart of MLOps is the continuous improvement of all business activity. The Japanese automobile industry refers to this concept as *kaizen*, meaning literally “improvement.” For building production machine learning systems, this manifests in both the noticeable aspects of improving the model’s accuracy as well the entire ecosystem supporting the model.

A great example of one of the nonobvious components of the machine learning system is the business requirements. If the company needs an accurate model to predict how much inventory to store in the warehouse, but the data science team creates a computer vision system to keep track of the inventory already in the warehouse, the wrong problem is solved. No matter how accurate the inventory tracking computer vision system is, the business asked for a different requirement, and the system cannot meet the goals of the organization as a result.

So what is *MLOps*? A compound of *Machine Learning* (ML) and *Operations* (Ops), MLOps is the processes and practices for designing, building, enabling, and supporting the efficient deployment of ML models in production, to continuously improve business activity. Similar to *DevOps*, MLOps is based on automation, agility, and collaboration to improve quality. If you’re thinking continuous integration/continuous delivery (CI/CD), you’re not wrong. MLOps supports CI/CD. According to Gartner, “MLOps aims to standardize the deployment and management of ML models alongside the operationalization of the ML pipeline. It supports the release, activation, monitoring, performance tracking, management, reuse, maintenance, and **governance of ML artifacts**”.

MLOps in the Enterprise

There are substantial differences between an enterprise company and a startup company. Entrepreneurship expert Scott Shane wrote in *The Illusions of Entrepreneurship* (Yale University Press, 2010) “only one percent of people work in companies less than two years old, while 60 percent work in companies more than ten years old.” Longevity is a characteristic of the enterprise company.

He also says, “it takes 43 startups to end up with just one company that employs anyone other than the founder after ten years.” In essence, the enterprise builds for scale and longevity. As a result, it is essential to consider technologies and services that support these attributes.

NOTE

Startups have technological advantages for users, but they also have different risk profiles for the investors versus the employees. Venture capitalists have a portfolio of many companies, diversifying their risk. According to [FundersClub](#), a typical fund “contains 135 million” and is “spread between 30-85 startups.” Meanwhile, startup employees have their salary and equity invested in one company.

Using the [expected value](#) to generate the actual equity value at a probability of 1/43, an enterprise offering a yearly 50k bonus returns 200k at year four. A startup produces \$4,651.16 in year four. For most people, on average, startups are a risky decision if judged on finance alone. However, they might offer an excellent reward via an accelerated chance to learn new technology or skills with the slight chance of a huge payout.

On the flip side, if a startup’s life is dynamic, it must pick very different technology solutions than the enterprise. If there is a 2.3% chance a startup will be around in 10 years, why care about vendor lock-in or multicloud deployment? Only the mathematically challenged startups build what they don’t yet need.

Likewise, if you are a profitable enterprise looking to build upon your existing success, consider looking beyond solutions that startups use. Other metrics like the ability to hire, enterprise support, business continuity, and price become critical key performance indicators (KPIs).

Understanding ROI in Enterprise Solutions

The appeal of a “free” solution is that you get something for nothing. In practice, this is rarely the case. **Figure 1-1** presents three scenarios. In the first scenario, the solution costs nothing but delivers nothing, so the ROI is zero. In the second scenario, high value is at stake, but the cost exceeds the value, resulting in a negative ROI. In the third scenario, a value of one million with a cost of half a million delivers half a million in value.

The best choice isn’t free but is the solution that delivers the highest ROI since this ROI increases the velocity of the profitable enterprise. Let’s expand on the concept of ROI even more by digging into bespoke solutions, which in some sense are also “free” since an employee built the solution.

#1 no value/no cost/ no ROI	#2 high value/high cost/ negative ROI	#3 high value/low cost/ positive ROI
Value = 0	Value = 1 Million	Value = 1 Million
Cost = 0	Cost = 2 Million	Cost = 1/2 Million
ROI = 0	ROI = -1 Million	ROI = 1/2 Million

Figure 1-1. Evaluating ROI for technology platform solutions

In **Figure 1-2**, a genuinely brilliant engineer convinces management to allow them to build a bespoke system that solves a particular problem for the Fortune 100 company. The engineer not only delivers quickly, but the system exceeds expectations. It would be tempting to think this is a success story, but it is actually a story of failure. One year later, the brilliant engineer gets a job offer from a trillion-dollar company and leaves. About three months later, the system breaks, and no one is smart enough to fix it. The company reluctantly replaces the entire system and retrain the company on the new proprietary system.

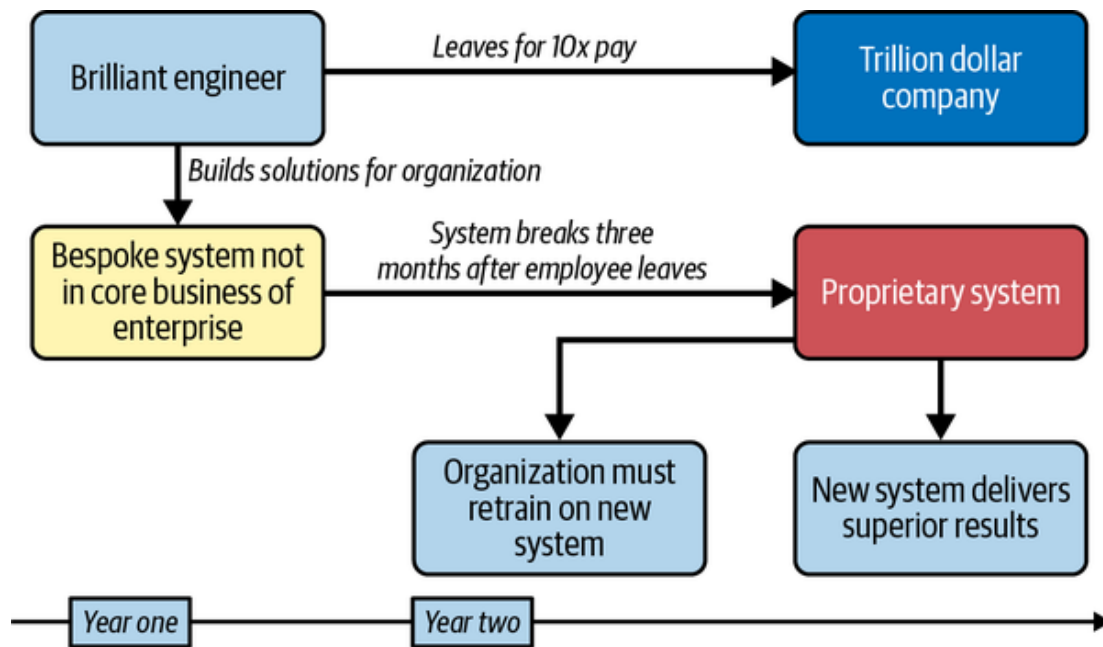


Figure 1-2. Bespoke system dilemma

The ultimate cost to the organization is the lack of momentum from using a superior system for a year, alongside the training time necessary to switch from the old system to the new system. Thus, a “free” solution with positive ROI can have long-term negative ROI for an organization. This scenario isn’t just hypothetical; you may have seen it yourself.²

In *Fooled by Randomness: The Hidden Role of Chance in Life and the Markets* (Random House, 2008), Nassim Taleb argues, “it does not matter how frequently something succeeds if failure is too costly to bear.” This statement directly applies to a successful enterprise that wants to implement MLOps. Taking the right kind of strategic risk is of critical importance. In the following section, we discuss the concept of risk in more detail.

Understanding Risk and Uncertainty in the Enterprise

Not all risk is the same, just as not all uncertainty is the same. Unlike a startup, an enterprise has made it to the survival phase. There are some risks that enterprises do not need to take. In his book about the enterprise, *Good to Great* (Harper Business, 2011), Jim Collins asks, “How do good-to-great organizations think differently about technology?” He found that in every

case a “good-to-great” company found technological sophistication and became a pioneer in applying technology. Further, Collins states that technology is an accelerator, not a creator, of momentum.

NOTE

Mark Spitznagel makes a case for considering the geometric mean in financial investment in *Safe Haven* (Wiley, 2021). He states, “Profit is finite. Risk is infinite.” The percentage of your wealth you can lose is more important than the absolute value of the wealth you could lose when investing. This fact is well suited to the enterprise. Why take a risk with unbounded loss?

Collins’ key point about technology directly applies to MLOps in the enterprise. The purpose of machine learning is to accelerate the business value that is already there. The reason to use machine learning isn’t to pivot the organization to becoming machine learning researchers competing with companies that specialize in research; it is to accelerate the strategic advantages of the organization through technology.

The calculated risk of adopting machine learning as a business accelerator is acceptable if done in a manner that allows an organization to limit the downsides of technology change management. There is essentially unbounded risk in a company creating bespoke machine learning solutions and platforms when its core strength is in some other industry, such as manufacturing, hospitality, or financial services.

Many options exist to accelerate technological advancement in the enterprise, including using pretrained models like **Hugging Face** or **TensorFlow Hub**, computer vision APIs like **AWS Rekognition**, or open source AutoML solutions like **Ludwig** or MLOps orchestration frameworks like **MLRun**. Enterprises that adopt MLOps with an approach of using the right level of abstraction give themselves a “good-to-great” advantage over organizations that “hired 15 data scientists” who do “research.” In the latter example, it is often the case that after years of research, in the best case

nothing is done, but in the worst case, a lousy solution creates a worse outcome than doing nothing.

Economist **Frank Knight** clearly articulates the difference between risk and uncertainty: the reward for taking a known risk is very different than a risk that is immeasurable and impossible to calculate. This form of risk, called *Knightian uncertainty*, was named after Knight. An enterprise doing machine learning should deeply consider which risk they are taking: a regular risk that is knowable, or are they embarking on a path with Knightian uncertainty? In almost all cases, it is better to take knowable risks in machine learning and AI since technology is not the creator of growth; instead, it is the accelerator.

Knowing that acceleration is the crucial insight into great companies that use technology, let's look at some of the differences in technology acceleration between MLOps and DevOps.

MLOps Versus DevOps

Without DevOps, you cannot do MLOps. DevOps is a foundational building block for doing MLOps, and there is no substitute. DevOps is a methodology for releasing software in an agile manner while constantly improving the quality of both business outcomes and the software itself. A high-level DevOps practitioner has much in common with a gourmet chef. The chef has deep knowledge of ingredients and years of practical experience creating beautiful and delicious meals, and they can make these meals in an industrialized and repeatable manner. The repetition allows a restaurant to stay open and earn a profit.

Similarly, with DevOps, an expert in the domain has detailed knowledge of how to build software and deploy it in a high-quality and repeatable manner. One of the biggest challenges for experts in data science to transition to MLOps is a lack of experience doing DevOps. There is no substitute for experience; many data science practitioners and machine learning researchers should get experience building and deploying software

with the DevOps methodology to get the foundational knowledge and experience necessary to be an expert at MLOps.

NOTE

You can learn more about DevOps from *Python for DevOps* (O'Reilly) by Noah Gift, Kennedy Behrman, Alfredo Deza, and Grig Gheorghiu.

There are apparent differences, though, between traditional DevOps and MLOps. One clear difference is the concept of *data drift*; when a model trains on data, it can gradually lose usefulness as the underlying data changes. A tremendous theoretical example of this concept comes from Nassim Taleb in *Fooled by Randomness* (Random House, 2021), where he describes how a “naughty child,” as shown in **Figure 1-3**, could disrupt the understanding of the underlying distribution of red versus black balls in a container.

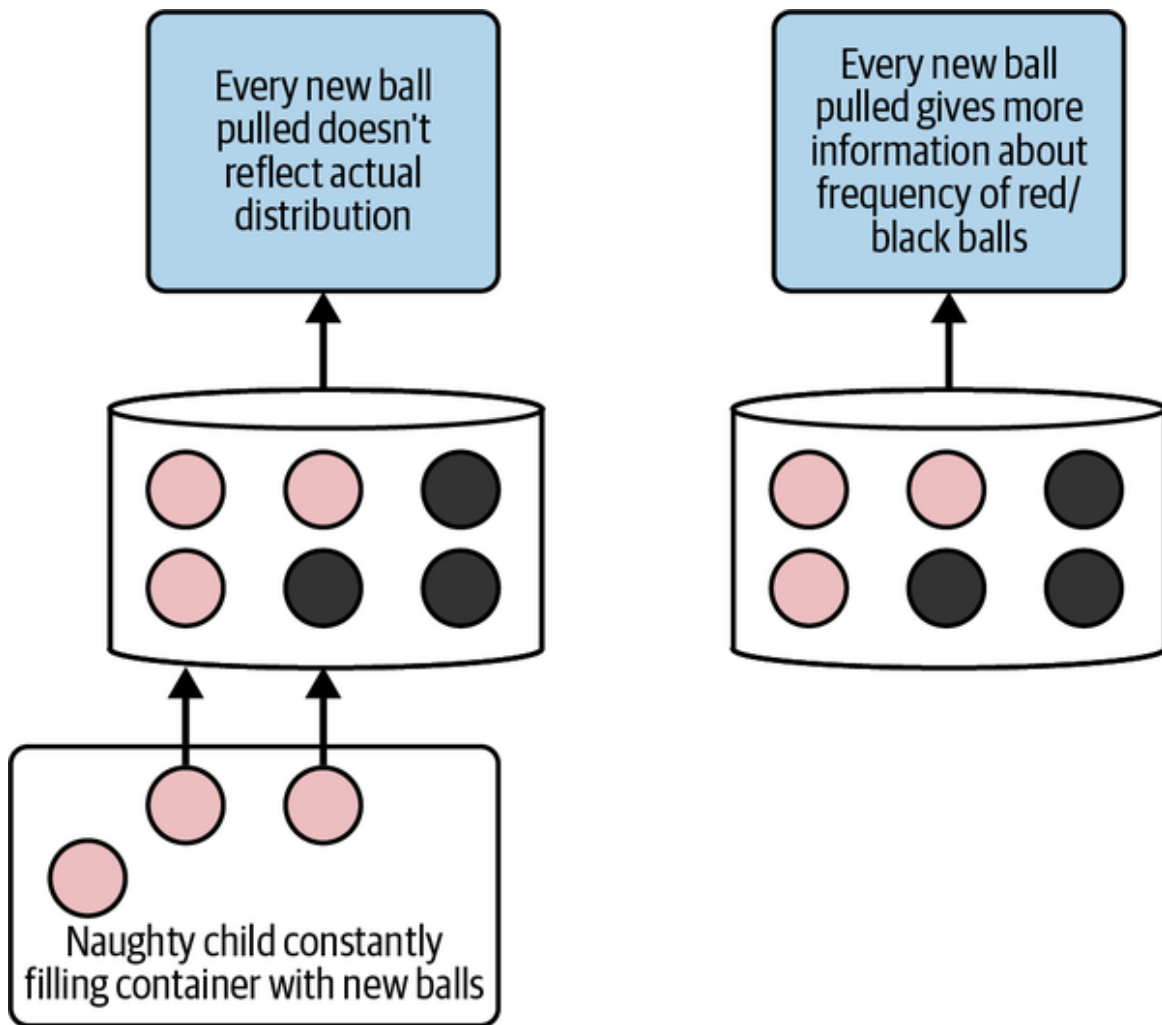


Figure 1-3. "Naughty child" data drift problem

In a static condition, the more balls pulled from a container, the more confident a person can be of the underlying distribution of red versus black balls. In a dynamic condition, if the balls are constantly changing, then a model trained on an older data version won't be accurate. This example captures one of many unique elements specific to MLOps not found in DevOps.

The takeaway is that DevOps is a necessary foundation for MLOps, but MLOps' additional requirements, like data drift, don't appear in traditional DevOps.

NOTE

Microsoft notes, “Data drift is one of the top reasons model accuracy degrades over time.”

What Isn't MLOps?

One way to understand more about MLOps is to define what it is not. Here are some common MLOps anti-patterns:

Hiring a team of data scientists and hoping for the best

Perhaps the most common of the MLOps anti-patterns is hiring a team of data scientists and expecting an excellent solution to appear. Without organizational support that understands MLOps and technology infrastructure to support them, there will not be an ideal outcome.

Building only bespoke machine learning solutions

A fundamental problem with building only customized solutions is that they may not be necessary for an organization's business goals. Training a bespoke machine learning model on propriety data for a self-driving company is essential to a competitive advantage. Training a similar model for a Fortune 500 delivery company could be a costly experiment adding no real value to the business.

Dismissing DevOps importance

Teams that work in silos are not following the best practices of DevOps. For example, it is impractical to have a data science team in Texas that builds models in R and then throws them over to the DevOps team in San Francisco's financial district to put into the software stack in Python.

Ultimately, MLOps requires a business and production-first mindset. The purpose of machine learning is to accelerate business value. This means the teams building solutions must be agile in their approach to solving machine learning problems.

Mainstream Definitions of MLOps

A challenge in technology is separating marketing strategy from technology strategy. In the case of MLOps, it is not a marketing strategy; it is a specific solution to a severe problem in the enterprise. The bottom line is that models are not making it into production; if they do, they are brittle and fall apart when faced with the complexities of the actual world. Various surveys show that **50-70% of organizations have failed to deliver AI pilots or models to production.**

With the condition identified, let's find the cure. The cure needs to address the following key issues (among others):

- Model deployment and development time
- Collaboration between different teams
- Operational excellence of ML systems
- Data governance
- Enhancing the ROI of the enterprise deploying the model

One minimalist way to define MLOps is that it supports ML development like DevOps supports software development.

What Is ML Engineering?

One way to define ML engineering is to look at popular certifications. **Google's Professional Machine Learning Engineer** explains the following criteria for a professional ML engineer:

Frame ML problems

Which model to choose depends on business constraints and the context. For example, a business may decide to classify damaged shipped boxes versus successfully delivered packages. In that context, a classification model would be more appropriate than a regression model.

Architect ML solutions

An ML engineer develops a solution to solve the correctly framed problem using machine learning alongside other team members.

Design data preparation and processing systems

Two **critical steps** in data preparation and processing are constructing the dataset and then transforming the data.

Develop ML models

The **detailed modeling process** involves a team or individual that creates a model correctly suited to initial model framing.

Automate and orchestrate ML pipelines

A **pipeline** serves to create a process for reproducible and maintainable ML.

Monitor, optimize, and maintain

It is better to be proactive than reactive in building complex systems. Building monitoring allows for a **proactive approach** to maintaining ML systems.

ML engineering aims to build high-quality ML models that solve specific business problems while creating ROI.

NOTE

Several O'Reilly books discuss machine learning engineering, including *Data Science on the Google Cloud Platform*, *Machine Learning Design Patterns*, and *Practical MLOps*.

MLOps and Business Incentives

A classic problem in business school is incentives, often described as “who moved the cheese?” This scenario refers to a rat in a maze that moves depending on where the cheese is. Similarly, there are two common incentives worth discussing in MLOps: negative externalities and hiring data scientists without regard for ROI:

Negative externalities

Negative externalities, like a company creating a profit dumping toxic waste into a river instead of the more expensive appropriate disposal, are classic examples of the fundamental problems in capitalism. In machine learning, the negative externalities could be biased algorithms that send an innocent person to jail or deny a person credit based on race, religion, national origin, and other categories. Even an unintentionally created bias in a model is still illegal (e.g., denying credit based on age). Enterprises that fail to look into the future could expose themselves to existential risk if system bias against elderly applications, for example, were accidentally baked into a machine learning model.

Hiring data scientists without regard for ROI

It has recently been in vogue to hire data scientists without regard for the problem they are solving. As we discussed, this strategy ultimately doesn't work because models are not in production at most organizations doing AI and ML.

MLOps in the Cloud

MLOps methodology leverages several critical advantages of cloud computing. First, the cloud is an elastic resource that enables both the efficient use of computing and storage and the ability to scale to meet almost any demand. This capability means that cloud computing has on-demand access to essentially infinite resources.

Second, the cloud has a network effect in that cloud technologies benefit from integrating other cloud technologies. A great example is AWS Lambda, a serverless technology. AWS Lambda is a valuable service to build applications with, not because of what it does alone, but because of the deep integration with other AWS services like AWS Step Functions, Amazon SageMaker, or AWS S3. For any active cloud platform, you can assume that the integrated network of services further strengthens its capabilities as the platform develops more features.

Third, all cloud vendors have MLOps platforms. AWS has [SageMaker](#), Azure has [Azure Machine Learning](#), and Google has [Vertex AI](#). Even smaller niche clouds like Alibaba Cloud has their [Machine Learning Platform for AI](#). By using a cloud platform, an organization will likely use some of the offerings of the native ML platform and potentially augment it with custom solutions and third-party solutions.

Fourth, all cloud vendors have Cloud Development Environments. A significant trend is the use of a combination of lightweight CloudShell environments like [AWS CloudShell](#), heavier full interactive development environment (IDE) options like [AWS Cloud9](#), and notebook environments, both free like [SageMaker Studio Lab](#) or [Google Colab](#) and those with rich IDE integration like [SageMaker Studio](#).

Finally, depending on what a company is doing, it may have no option but to use cloud computing. Some cloud computing components are a hard requirement for organizations specializing in building bespoke deep learning solutions because deep learning requires extensive storage and compute capabilities.

In addition to the public cloud vendors, several additional players offer MLOps solutions in the cloud (see later in this section). These vendors can operate on the public cloud or on private clouds. The advantage of using a smaller vendor is the customization level that such a company provides its customers. In addition, an MLOps vendor will have more in-depth expertise in MLOps since that is its only focus. Integrated vendors often ensure more relevant features and many more integrations. Finally, by choosing a vendor that is agnostic to a specific cloud provider, you, as a customer, aren't connected to it either. Instead, you can use the vendor across multiple clouds or on additional infrastructure that you may have (see later in this section).

NOTE

One helpful resource for machine learning vendor analysis is the AI Infrastructure Alliance (AIIA). This organization provides data scientists and engineers with clarity and information about AI/ML tools to build robust, scalable, end-to-end enterprise platforms. One resource is [a comprehensive MLOps landscape](#) that maps out all the players in the industry. This document includes an updated MLOps landscape that will map out open source and enterprise solutions for MLOps. The new landscape will encompass multiple categories and hundreds of companies while detailing the capabilities of each vendor solution.

In [Figure 1-4](#), notice a typical pattern among all clouds in which there is a set of cloud development environments, flexible storage systems, elastic compute systems, serverless and containerized managed services, and third-party vendor integration.

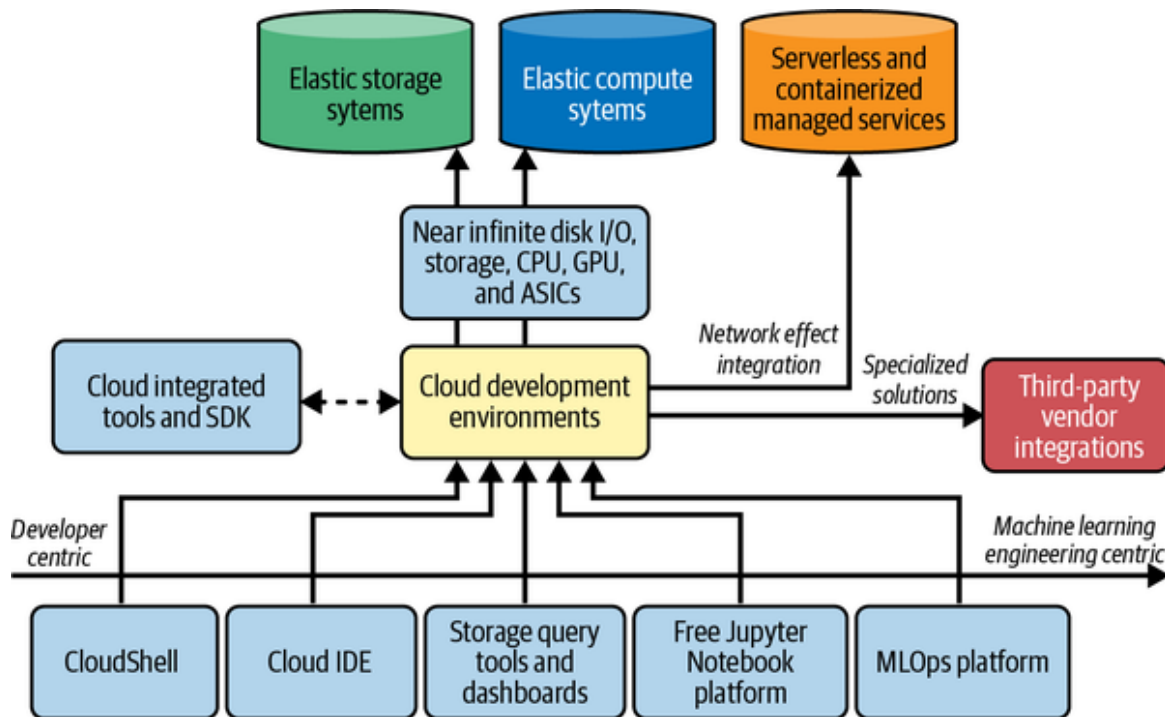


Figure 1-4. Cloud MLOps landscape

Here is more detail about these categories:

Cloud development environments

Generally, developer-centric tools like cloud shells and IDEs are on one extreme and machine learning-centric tools on the other. Storage query tools like **Google BigQuery**, **Amazon Athena**, or **Azure Databricks Integration** are in the middle.

MLOps platforms that operate in the cloud

MLOps platforms are built specifically for running MLOps for enterprises on the cloud or across any environment. Solutions like **Iguazio**, **Valohai**, **DataRobot**, **Azure Databricks** and **Outerbounds**, and many others offer a wide variety of MLOps solutions for the enterprise.

Elastic storage systems and elastic computing systems

Deep learning systems thrive on big data, and flexible compute capabilities from GPUs, CPUs, and AI Accelerator

application-specific integrated circuits (ASICs) like Tensor Processing Units (TPU). As a result, MLOps platforms, both native and third party, heavily use this elastic capability to provide managed solutions.

Serverless and containerized managed services

Cloud platforms evolve toward more serverless solutions like [AWS Lambda](#) or [Google Cloud functions](#) and solutions with fully managed containerized solutions such as [Google Cloud Run](#) or [AWS Fargate](#). These managed services, in turn, have deep platform integration, which enhances the value proposition of the cloud platform through a network effect.

Third-party vendor integrations

A cloud platform can't have the exact right mix of everything and at the right quality. A trip to a large warehouse store yields a wide variety of offerings at a reasonable price. However, they may not have the authentic gourmet food you like or the exact appliance features you need. Just like that large warehouse store, a cloud provider cannot go deep on everything. As a result, third-party integrations handle these specialized or advanced use cases.

With the common aspects of cloud computing for MLOps covered, let's move on to discuss the cloud environments in more detail.

Key Cloud Development Environments

One of the best new products from Microsoft is [GitHub Codespaces](#), a cloud-based development environment with many customizable features and a great place to practice MLOps. In particular, what is helpful about this environment is the deep integration with GitHub and the ability to customize it with a specialized runtime. Finally, the synergy with [GitHub Actions](#) allows for a great CI/CD story.

NOTE

Learn more about GitHub Codespaces with the following videos:

- “Building with the GitHub EcoSystem: Copilot, Codespaces, and GitHub Actions”
- “GitHub Codespaces and Custom Dotfiles”
- “Compiling Python from Scratch with GitHub Codespaces”
- “GitHub Copilot Driven: Python DevOps from Functions to Continuous Delivery of Microservices on AWS”
- “GitHub Codespaces Course”

Three different flavors of cloud-based developments are available from Google: Colab notebooks, Google Cloud Shell, and Google Cloud Shell Editor.

Figure 1-5 shows a full editor available for Google Cloud Platform (GCP).

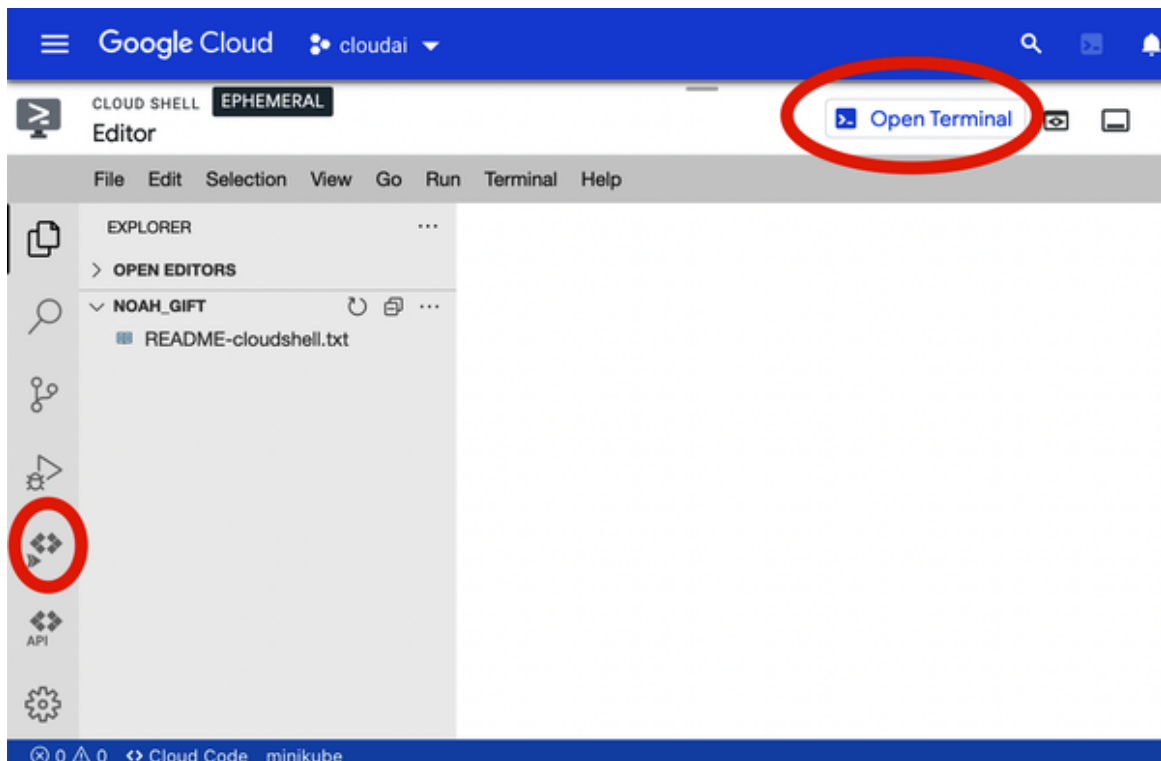


Figure 1-5. Google Cloud Shell Editor

In Figure 1-6, API docs integrate with the development environment.

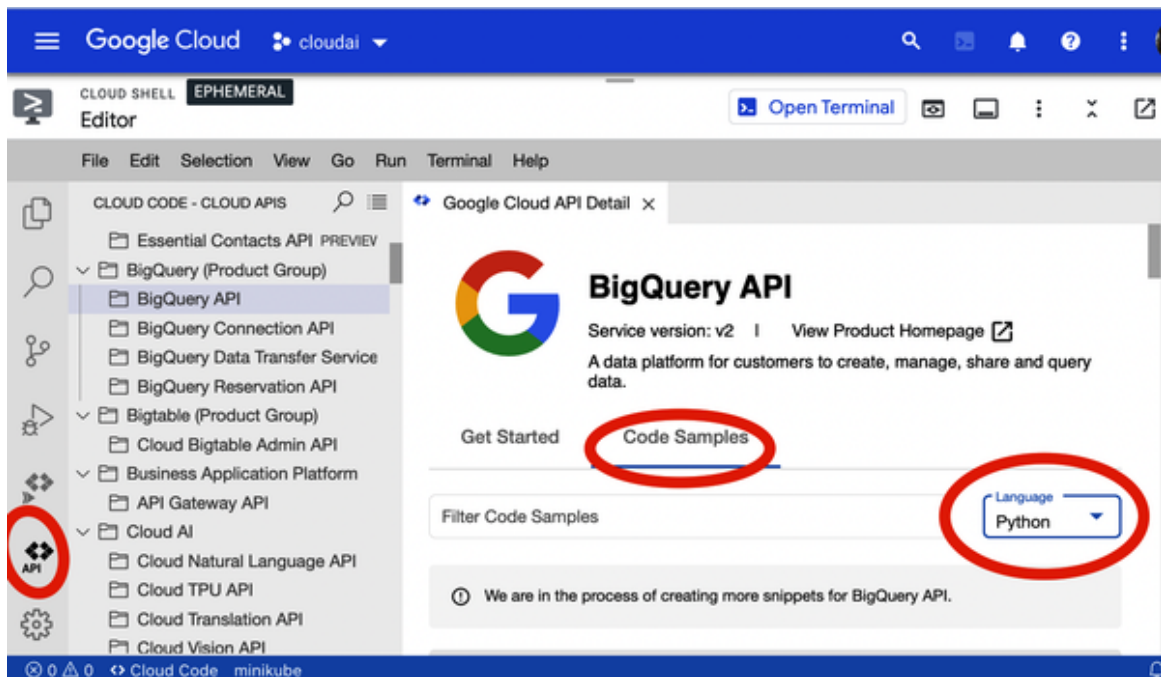


Figure 1-6. Google Cloud Shell Editor API

In Figure 1-7, the terminal shows a standard view of the experience using the cloud shell.

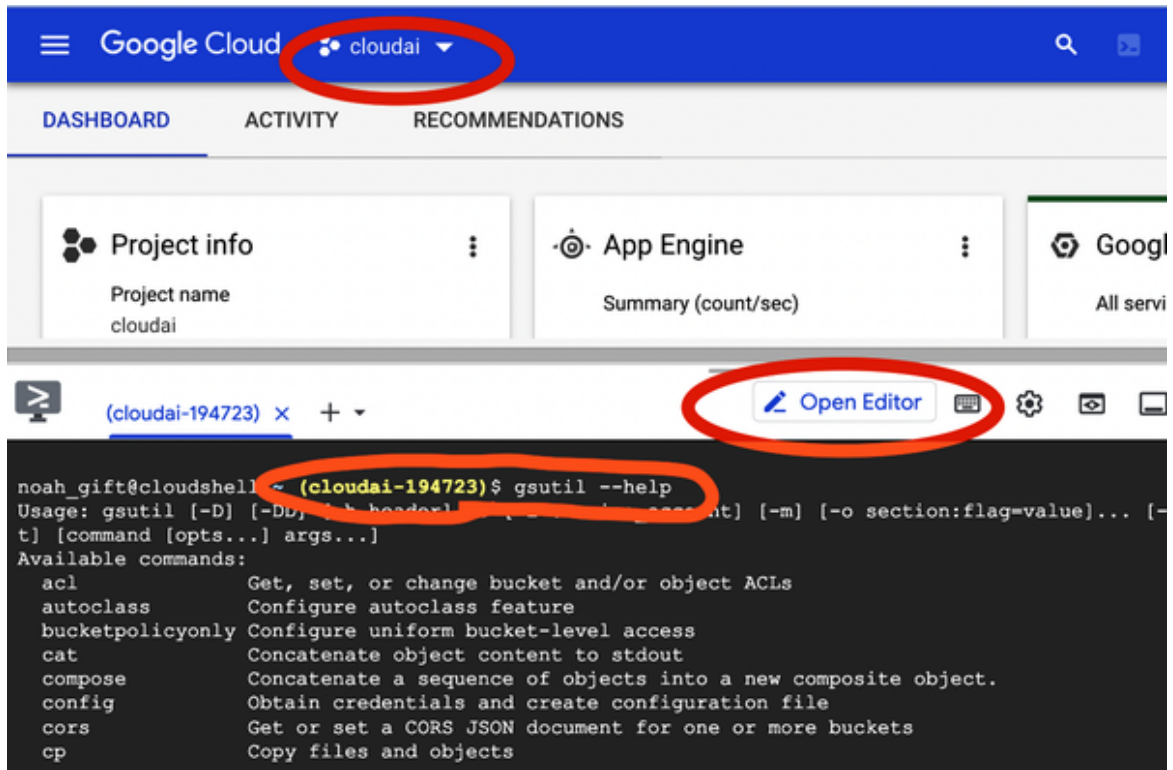


Figure 1-7. Google Cloud Shell terminal

NOTE

Learn more about Colab notebooks from the following videos:

- “Data Science on Your First Day with Python”
- “Python for Data Science with Colab and pandas in One Hour Video Course”
- “What are Google Colab Notebooks and How Do You Share Them for Data Science Projects?”

Finally, the AWS platform has cloud shell environments, as shown in Figure 1-8.

NOTE

One quick way to learn about multiple clouds simultaneously is by setting up a multicloud continuous integration. You can learn how to set this up with the video [“GitHub Actions Hello World All Cloud and Codespaces”](#).

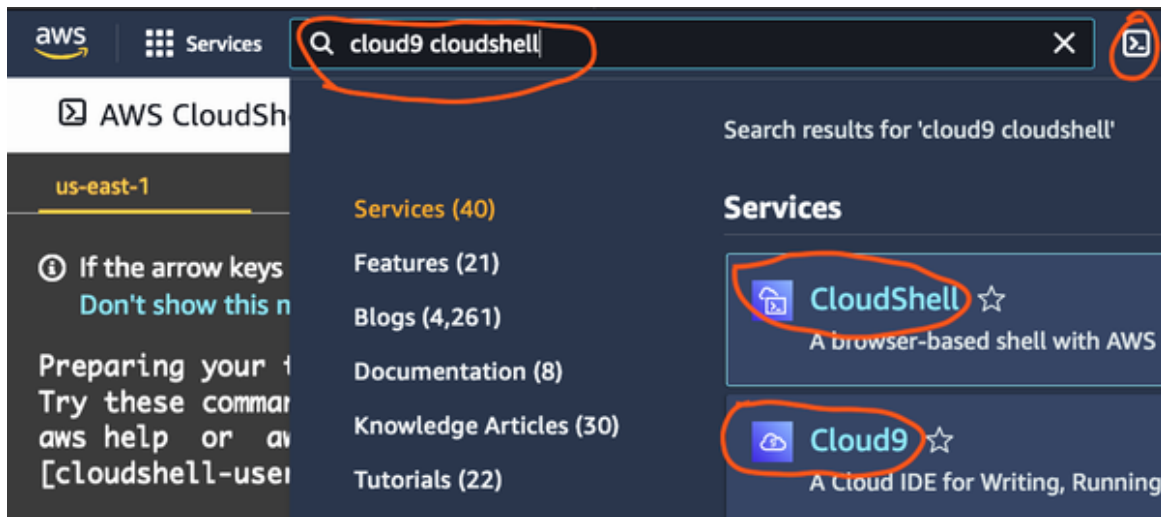


Figure 1-8. AWS Cloud Shell terminal

All of this leads to the concept of the cloud developer workspace advantage, as shown in [Figure 1-9](#). A laptop or workstation is expensive and nondeterministic due to preinstalled software and, by definition, not the deploy target. When you look at a cloud-based workspace, it has many incredible advantages, including power, disposability, preloading, and deep integration with advanced tools.

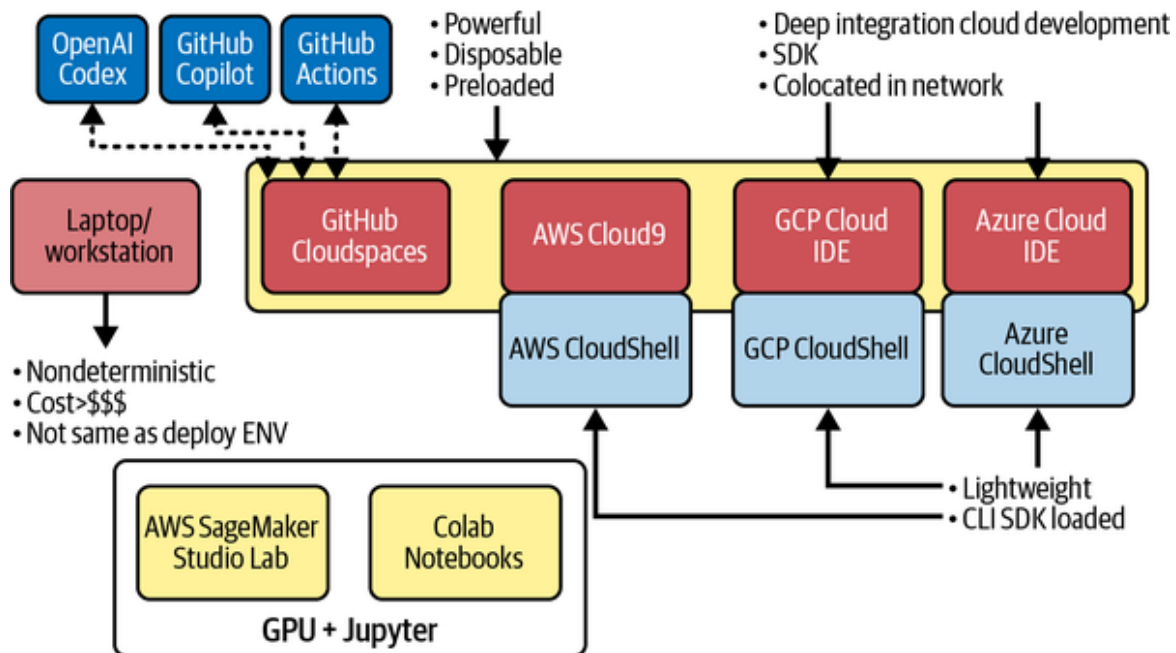


Figure 1-9. Cloud developer workspace advantages

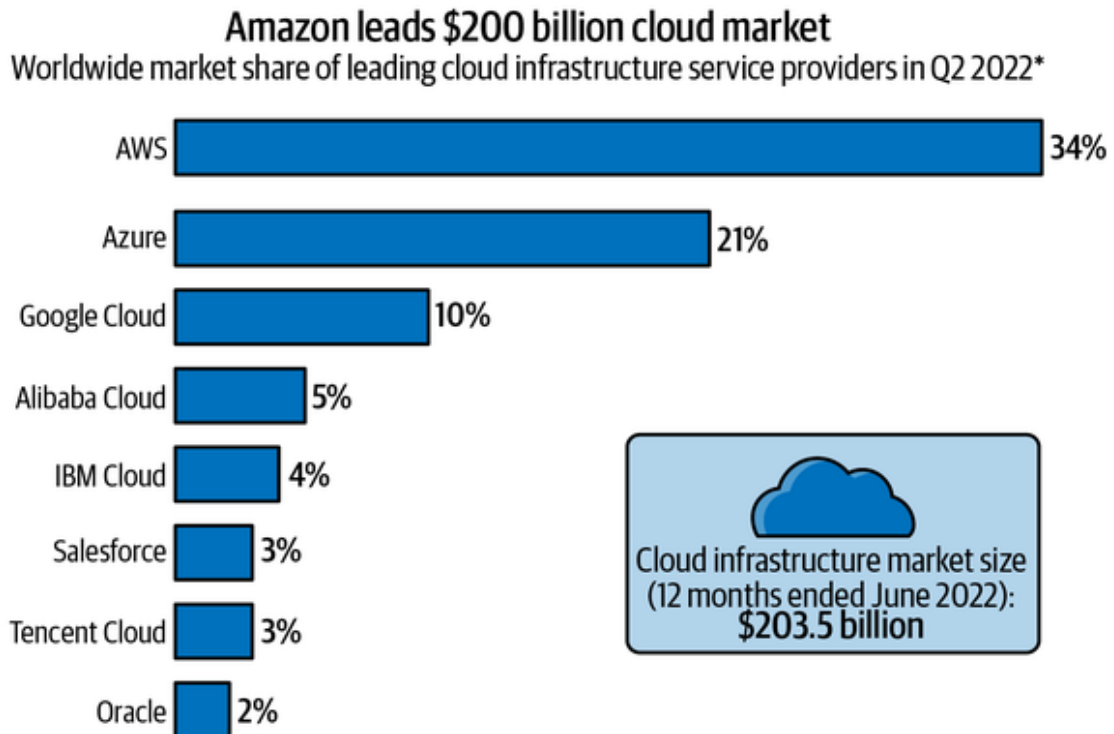
NOTE

You can learn more about the cloud developer workspace advantage in the video “52 Weeks of AWS-The Complete Series” or on [YouTube](#).

The Key Players in Cloud Computing

Know someone who wants to earn \$200k or more a year? According to the [2022 Cloud Salary Survey](#) by Mike Loukides (O’Reilly), the average salary for certified professionals on AWS, Azure, and GCP is over 200k.

Further backing this up is the data from Statista, as shown in [Figure 1-10](#). As of Q2 2022, there were three key players in the worldwide market. AWS had about 33% of the market share, Azure had about 21%, and Google Cloud had about 10%. Combined, these three vendors controlled two-thirds of a market that generates almost \$200 billion in revenue. Service revenue increased by 37% from the last year.



*Includes platform as a service (PaaS) and infrastructure as a service (IaaS) as well as hosted private cloud services.

Source: Synergy Research Group

Figure 1-10. Cloud computing market

A reasonable strategy for an organization wishing to use cloud computing is to use the platform of the largest providers. The Matthew effect³ saying, “the rich get richer, and the poor get poorer,” applies to cloud computing for several reasons:

Available employees and vendors to hire

Leveraging the most prominent cloud platforms makes hiring employees and finding vendors that work with the platform more accessible.

Training material available

The availability of training material for the most prominent platforms makes it easier to train employees.

Services available

Larger platforms can hire more software engineers and product managers, meaning you can count on a continuation of new features and maintenance in their platform.

Cost of service

Economies of scale mean that the most significant providers benefit the most from economies of scale. They can leverage pricing advantages by buying in bulk and then passing them on to the customer.

NOTE

You can study for the AWS Cloud Certifications by viewing “[AWS Solutions Architect Professional Course](#)” and “[AWS Certified Cloud Practitioner Video Course](#)” by Noah Gift.

Now that you know the top providers in cloud computing, let’s discuss how each vendor views the world of cloud computing as it relates to MLOps.

AWS view of cloud computing as it relates to MLOps

The best place to get a high-level summary of AWS cloud computing is the [Overview of Amazon Web Services AWS Whitepaper](#). In particular, they mention [six advantages of cloud computing](#):

Trade fixed expense for variable expense

Avoiding large capital expenditures encourages agility and efficiency.

Benefit from massive economies of scale

As prices decrease for the supplier, they fall for the customer, allowing for lower pricing than if the customer bought the

same product. Similarly, managed services on the platform will have a steady schedule of new features.

Stop guessing capacity

There isn't a need to preprovision resources since systems get built with an elastic ability to scale as needed.

Increase speed and agility

Focusing on an organization's comparative advantage and not building nonessential-to-business IT allows an organization to move faster.

Stop spending money running and maintaining data centers

Cost savings accumulate from outsourcing this component of IT.

Go global in minutes

Going global is a highly challenging problem that goes away with AWS due to its comprehensive offerings.

NOTE

You can learn more about AWS in *Developing on AWS with C#* (O'Reilly) by Noah Gift and James Charlesworth.

These features ultimately drive into the core MLOps offering of Amazon SageMaker in **Figure 1-11** as the project's lifecycle goes from preparation to building to training, to finally deploying and managing the solution. At the center of the workflow is tight integration with developer tools from Studio and RStudio.

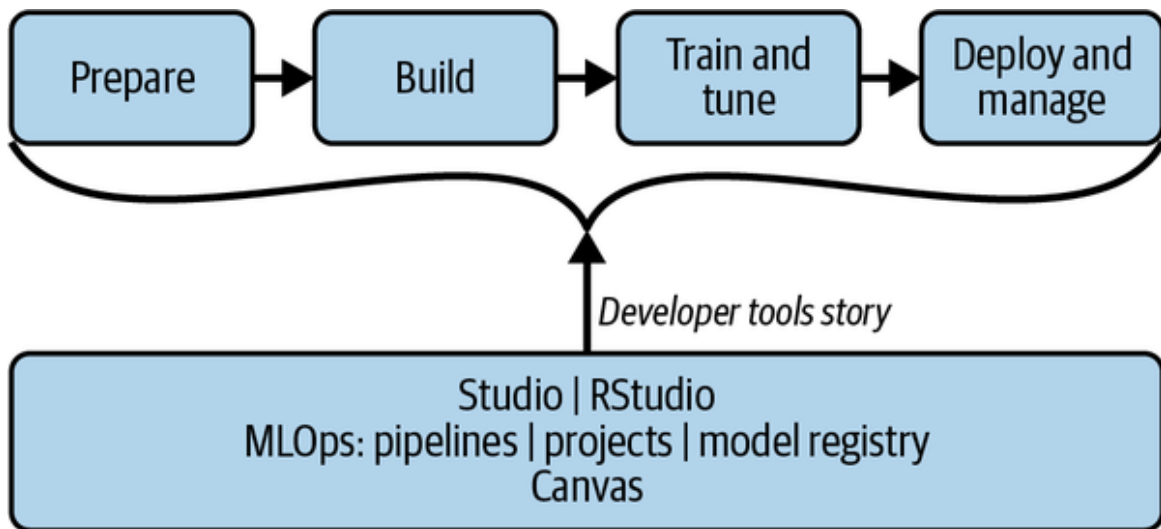


Figure 1-11. Amazon SageMaker MLOps workflow

NOTE

In the video “[Amazon SageMaker Studio Labs: First Thoughts](#)”, you can see a complete walkthrough of SageMaker Studio Lab.

With the AWS view of the MLOps complete, let’s look at Azure next.

Azure view of cloud computing as it relates to MLOps

Microsoft Azure **sees the world of MLOps** as a way to “efficiently scale from a proof of concept or pilot project to a machine learning workload in production.” As shown in **Figure 1-12**, the model’s lifecycle includes training, packaging, validating, deploying, monitoring, and retraining.

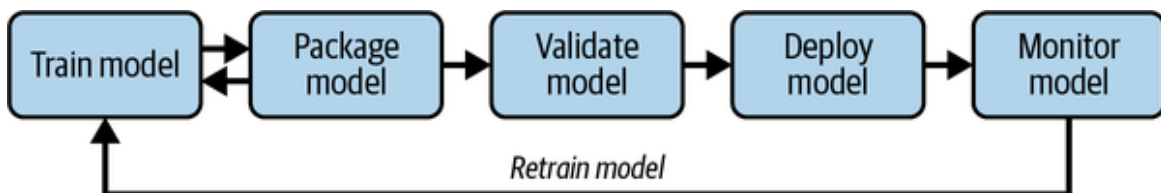


Figure 1-12. Azure MLOps

Next, let’s next look at how Google views MLOps.

GCP view of cloud computing as it relates to MLOps

An ideal place to look at how Google sees the world is by looking through the [Production ML Systems crash course](#). One of the items the company points out is how tiny the modeling part of the problem is, as shown in [Figure 1-13](#). Instead, the combination of other tasks, including data collection, serving infrastructure, and monitoring, take up much more of the problem space.

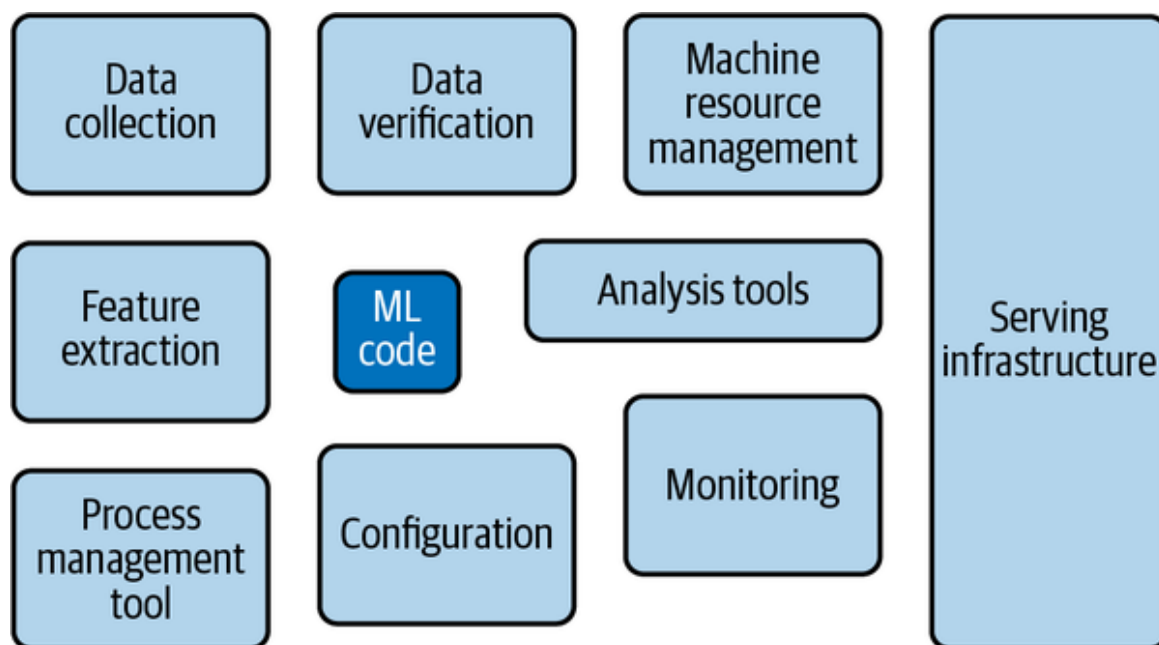


Figure 1-13. Google's view of MLOps

Ultimately this leads to how Google's [Vertex AI platform](#) handles the MLOps workflow, shown in [Figure 1-14](#). The ML development process occurs, including model framing for the business problem. The data processing phase leads to an operationalized training process that can scale up as needed. Then the model deployment occurs along with a workflow orchestration alongside artifact organization. The model has monitoring baked into the deployment process.

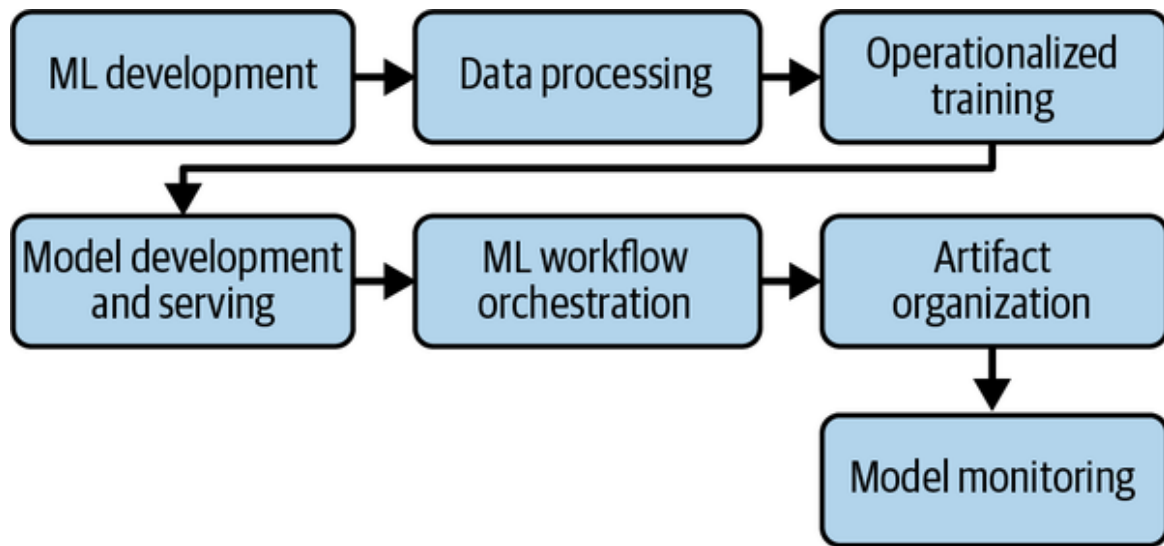


Figure 1-14. Google's view of MLOps

While public cloud providers offer their own solutions, sometimes enterprises might need a solution that is more tailored to their specific needs. Let's look at two more deployment options: on-premises deployment and hybrid cloud deployment.

MLOps On-Premises

In some use cases, enterprises cannot use the public cloud. Business restrictions like the need to secure sensitive data or having to adhere to strict regulations (e.g., data localization privacy regulations) require an MLOps solution that can operate on-premises. Many MLOps solutions offer the ability to deploy them either in the cloud or on-premises. The only downside to this approach is that on-premises solutions require the enterprise to provide the servers and equipment that will support the intense computing power needed to run ML algorithms at scale. They will also need to update and maintain the infrastructure.

On the other hand, an on-premises deployment will almost certainly require some sort of customization. This installation gives enterprises more control over the product, and they can make specific requests to tailor it to their needs. More specifically, if the deployed solution is a startup solution, they will be attentive and work hard to ensure satisfaction and adoption. If it's an

open source product, then enterprises not only can leverage the community's development power but also go inside with their own developers and tinker with the product to ensure it suits their needs.

MLOps in Hybrid Environments

Similar to on-premises deployment, some enterprises might prefer a hybrid cloud deployment. This involves deploying on the public cloud(s), on-premises, and perhaps even on a private cloud or on edge devices. Naturally, this makes things a lot more complex, since the MLOps solution must enable total separation of the data path from the control path and must be delivered by a highly available, scalable entity that orchestrates, tracks, and manages ML pipelines across types of infrastructure deployments. Lest we forget, this has to occur at high speed and with optimal performance. Finally, the solution ideally provides a single development and deployment stack for engineers across all infrastructure types.

Finding a vendor or open source solution that meets all these requirements might not be simple, but as mentioned before, your best bet is with startups or mature OSS solutions that can be customized to the specific needs of your infrastructure.

Enterprise MLOps Strategy

With a high-level overview of the critical issues involved in MLOps completed, it is time to turn to strategy, as shown in **Figure 1-15**. There are four key categories to consider when implementing an MLOps strategy: cloud, training and talent, vendor, and executive focus on ROI.

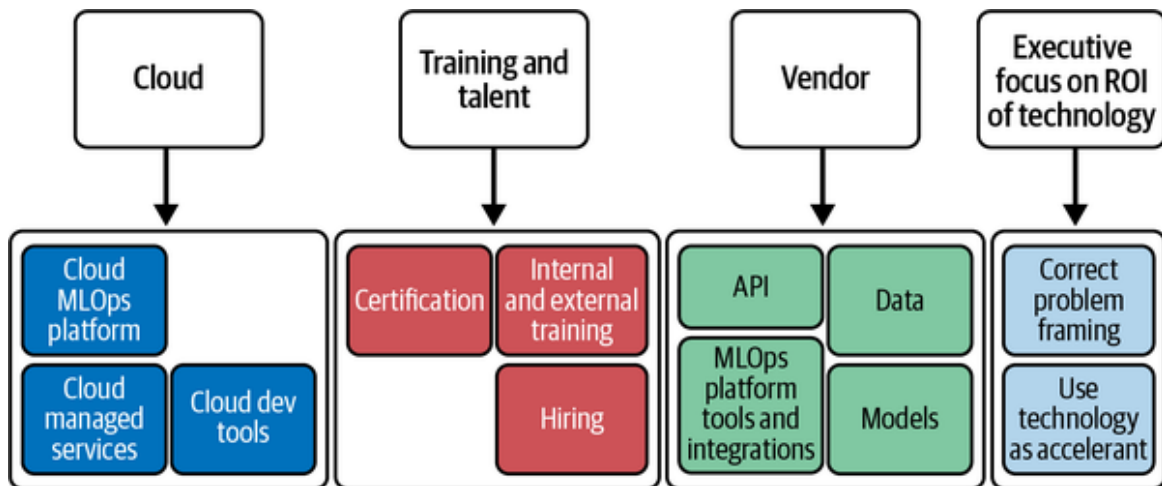


Figure 1-15. Enterprise MLOps strategy

Let's discuss each of these four categories:

Cloud

There is no perfect answer for which cloud platform to use. Any central platform will offer the advantages of economies of scale. What is essential in an MLOps strategy is to be aware of how a cloud platform fits into the unique goals of each organization and how it aligns with other strategic components like hiring or third-party vendor integration.

Training and talent

Often, organizations look only at the power of new technology and don't consider the training and talent component of using the technology. In almost all cases, an organization should use a less powerful technology if hiring and training are better with a less powerful solution. This fact means widespread technology is crucial when implementing new technology. Ultimately, the latest technology is dead on arrival if you cannot hire or train your staff.

Vendor

An often overlooked issue with using cloud computing is that it usually needs to be augmented by specialized vendors to help an organization reach its goals with the technology. These strategic choices can lead to better ROI for both the cloud and the business strategies. Examples include using vendor technology specializing in Hadoop, Kubernetes, or pretrained models. The vendors will be unique to each organization and its business goals.

NOTE

In “**Enterprise MLOps Interviews**”, CEO of Outerbounds and author of Metaflow, Ville Tuulos, mentions that while all companies use the base layer of the cloud, say storage and databases, they often need to augment with vendors at higher layers.

Executive focus on ROI

Ultimately, the preceding three categories don’t mean anything if the executive focus isn’t on ROI. The purpose of technology is to drive long-term business value, meaning problems need accurate scoping.

Conclusion

This chapter sets the stage for understanding the crisis in enterprises getting machine learning and AI into production. From a common sense approach, the idea of “just hiring more data scientists” to increase ROI is as sensible as “just hiring more software engineers” to make a traditional software project go faster. In the case of the conventional software company, if there is no product, no goal, and no oversight, then hiring more developers increases the capital expenditure of the organization without any added value.

Instead of this scenario, MLOps aims to add a methodology that builds on the successful lessons of DevOps while handling the unique characteristics of machine learning. Finally, at the enterprise level, ultimately data science comes down to ROI. Technology is an accelerant of value for most organizations, not the value. Organizations that create a hunger for ROI can quickly adopt the MLOps mindset.

Critical Thinking Discussion Questions

- There are many methods for deploying machine learning models to production, including pretrained models, APIs, AutoML, and bespoke training. What are the pros and cons of each of these approaches?
- What strategies could an enterprise implement to attract new machine learning engineering talent and train and retrain current talent?
- If your organization currently doesn't do any DevOps, a foundational component necessary for MLOps, how could they start a first DevOps project to test concepts like CI/CD and infrastructure as code (IaC)?
- If your organization doesn't have large quantities of proprietary data, how can it use machine learning to gain a competitive advantage anyway?
- What is your organization's cloud strategy: single cloud, multicloud, hybrid cloud, private cloud, or something else? How does this help your organization reach your MLOps goals?

Exercises

- Go to a popular model hosting site like [TensorFlow Hub](#) or [Hugging Face](#) and deploy one of their models to your favorite cloud platform.
- Pick a cloud-based development environment like [GitHub Codespaces](#), [Amazon SageMaker Studio Lab](#), or [Google Colab](#) and explore the

interface with an eye for building a machine learning engineering project.

- Use a machine learning app framework like **Gradio** or **Streamlit** to build a simple machine learning application.
- Brainstorm several organizational problems that may benefit from using machine learning and build a simple prototype using an MLOps technology.
- Convert a Kaggle project to an MLOps project by downloading the dataset and coding an MLOps technology to serve predictions.

¹ Dr. Luks summarizes the systematic evidence-based strategy: “Create a caloric deficit, then stay lean. Get sleep. Eat real food. Move often, throughout the day. Push and pull heavy things. Socialize. Have a sense of purpose.”

² In *Principles of Macroeconomics* (McGraw Hill, 2009), Ben S. Bernanke shares the story of how a talented chef could extract all of the profit from restaurants in a scenario of perfect competition since they would continuously leave for a higher salary at a competing restaurant, ultimately removing all profit for the owner.

³ Sociologists Robert K. Merton and Harriet Zuckerman **first coined this term**.

Chapter 2. The Stages of MLOps

MLOps is not about tracking local experiments and is not about placing an ML model behind an API endpoint. Instead, MLOps is about building an automated environment and processes for continuously delivering ML projects to production.

MLOps consists of four major components (and is not confined to model training):

- Data collection and preparation
- Model development and training
- ML service deployment
- Continuous feedback and monitoring

This chapter explores these components in detail.

Getting Started

Begin with the end in mind. The first step in any ML project is to articulate:

- The problem that needs to be solved using ML.
- What you want to predict.
- How to extract business value from the answer. Examples of business value we might require include decreasing fraud, increasing revenue by attracting new customers, cutting operational costs by automating various manual processes, and so on.

Once you define the goal, don't rush straight into implementation. First, consider the following:

- Which historical and operational data can be gathered and used in both the training and **serving pipelines**
- How to incorporate the ML model results in a new or existing application in a way that can make an impact
- How to verify and reliably measure that the ML model meets the target and generates valuable business outcomes

Figure 2-1 illustrates the different stages in an ML project. Note the feedback loop where the observations are used to recalibrate the business goals, data collection, and preparation logic.

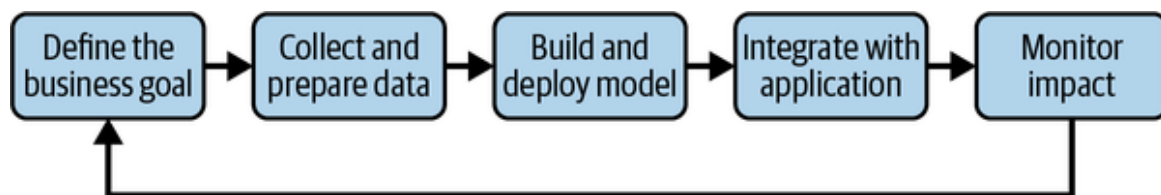


Figure 2-1. ML project life cycle

If you focus only on the ML model, you may encounter pitfalls such as these:

- Using the wrong datasets, which can easily lead to inaccurate or biased results
- Lacking enough labeled data to build a model
- Finding out historical features used to train the model are unavailable in the production or real-time environment
- Discovering there is no practical way to integrate the model predictions into the current application
- Realizing the ML project costs are higher than the generated value or, in a worst-case scenario, cause losses in revenue or customer satisfaction

Choose Your Algorithm

The next phase is to determine the type of ML problem and algorithm.

In supervised learning, labels are required and known:

Classification

The algorithm will answer binary yes-or-no questions (fraud or not, is it an apple, will the customer churn) or make a multiclass classification (type of tree, and so on). You also need enough labeled data for the algorithm to learn from.

Regression

The algorithm predicts continuous numeric values based on various independent variables. For example, regression algorithms can aid in estimating the right price for a stock, the expected lifetime of a component, temperature, and so on.

Figure 2-2 compares the two algorithms.

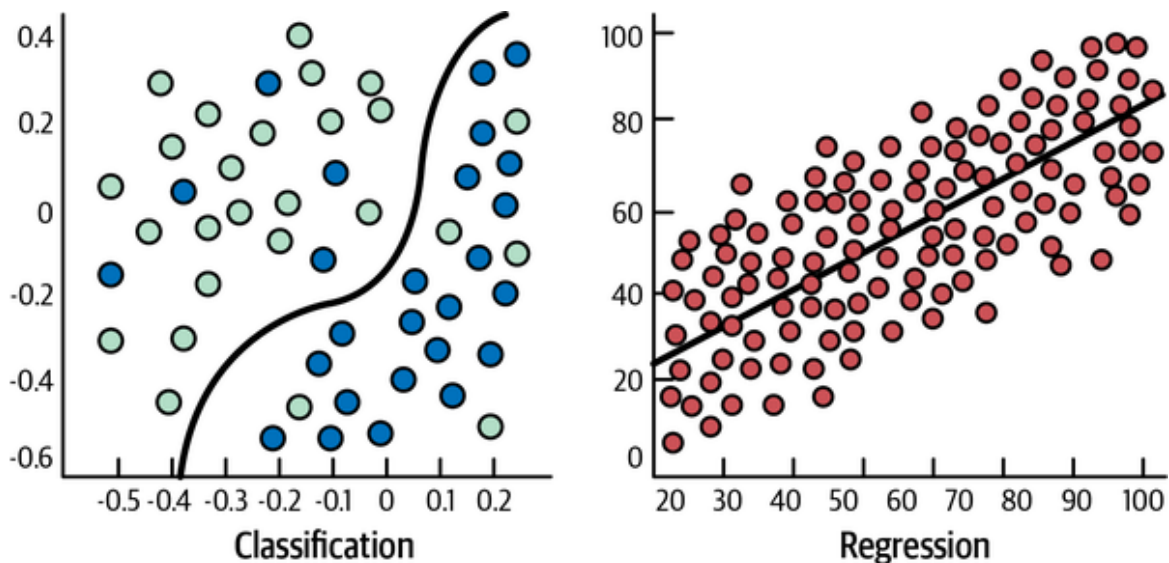


Figure 2-2. Regression versus classification

In unsupervised learning, labels are not required and known:

Clustering

The algorithm will look for meaningful groups or collections in the data (customer segmentation, medical imaging, music genre, anomaly detection, and so on) based on their similarity without the help of pre-labeled data.

Dimensionality reduction

The algorithm will reduce the *dimensionality* (the number of input variables in a dataset) from a high-dimensional space into a low-dimensional space so that the low-dimensional representation retains some meaningful properties of the original data, ideally close to its intrinsic dimension. Dimensionality reduction allows you to avoid overfitting, reduce the model computation overhead, and handle fewer features than originally required.

Recommendation and ranking

The algorithm recommends or ranks objects by considering their relevance, importance, and content score. Recommendation algorithms can be used to rank web pages, recommend movies or music in streaming services, or show the products that a customer might purchase with a high probability based on their previous search and purchase activities. Recommendation engines can be used either for supervised or unsupervised learning.

Transformers and generative AI

A neural network architecture that can automatically transform a sequence of inputs into another a set of outputs; for example transforming a chat question into an answer (like Chat GTP), or text description into a relevant image.

Note that ranking algorithms relies on search queries provided by users who know what they are looking for. Recommender systems, on the other hand,

operate without any explicit inputs from users and aim to discover things the users might not have found otherwise.

Some applications may incorporate multiple algorithms. For example, using a natural language processing (NLP) algorithm to determine the sentiment in the text and using the sentiment as an input for making a purchase decision.

Design Your Pipelines

ML models have a limited lifetime since data patterns change (**drift**) over time, and models may have limited scope. For example, when creating specific models per user or device (trained on the relevant subset of the data). In many cases, we would like to train multiple models using different parameters or algorithms and compare or combine them.

For those reasons, the goal is *not* to build a model but rather to create an automated **ML pipeline** (factory) that can accept inputs (code, data, and parameters), produce high-quality model artifacts, and deploy them in the application pipeline.

The ML pipelines can be triggered every time the data, code, or parameters change or can be executed in a loop (each time with a different dataset or parameters) to produce multiple models. To understand, compare, or explain the model results, all the inputs (code, data, parameters), operational data (type of hardware, logs, and so on), and results must be recorded and versioned.

A model is usually deployed as part of a more extensive application pipeline, including API integration, real-time data enrichment and preparation, model serving, actions, and monitoring. The automated deployment cannot focus solely on the model but on deploying or updating the entire application pipeline.

The typical ML pipeline consists of data preparation, training, testing, registering, and deployment. In real life, the ML pipelines can incorporate

additional steps for data validation, optimization, and so on. In addition, some ML pipelines build and use multiple models.

Figure 2-3 demonstrates a recommendation engine application that uses two models in cascade. The first model is used to identify similar products. The second model will use the output from the first model and other user data to determine the buying probability (and filter the results).

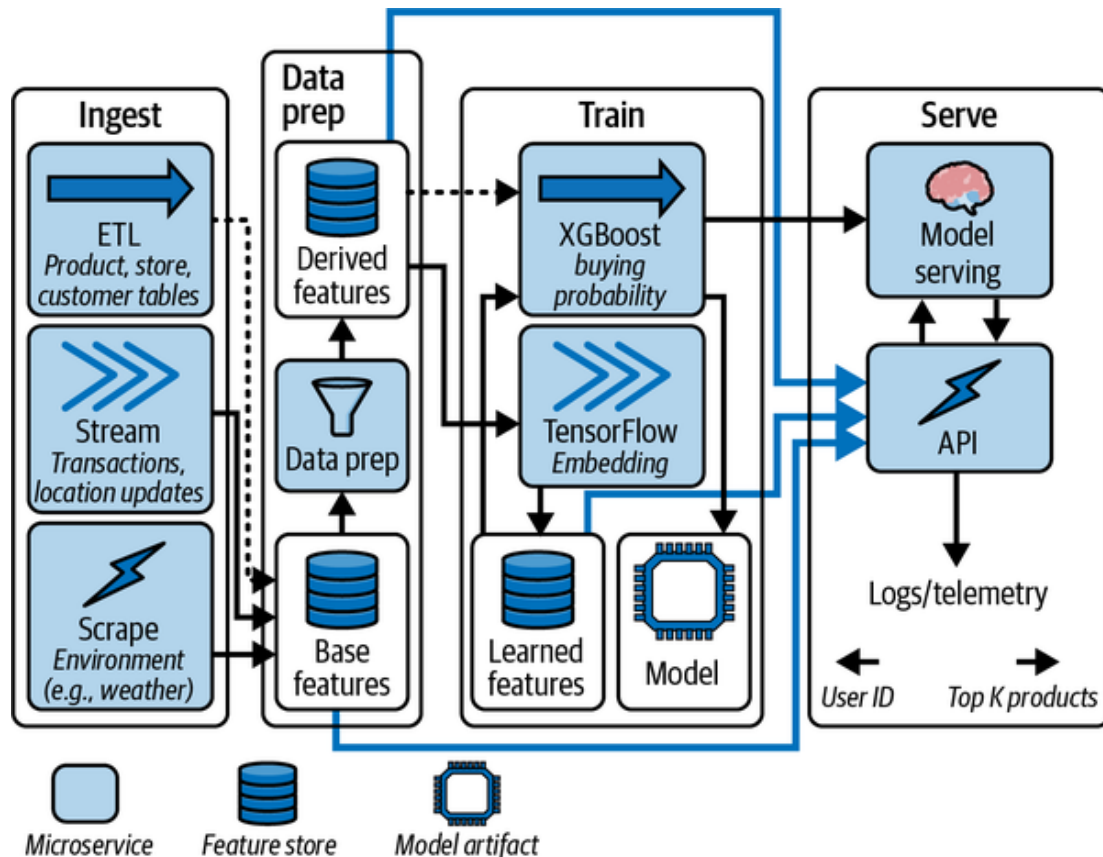


Figure 2-3. ML pipeline example: real-time product recommendations

Data Collection and Preparation

There is no ML without data. Before everything else, ML teams need access to historical or online data from multiple sources. They must ingest, prepare, and explore the data before building any model.

The first step is to define your goal, which problem or challenge you intend to solve, and which data sources or features can help you predict the

outcome. Once you identify the target and raw datasets, you must gather enough data, prepare, label, and explore it for use in your model.

In most cases, the raw data cannot be used as-is for machine learning algorithms, for various reasons, including the following:

- The data is low quality (missing fields, wrong spelling, null values, and so on) and requires cleaning and imputing.
- The data needs to be grouped or aggregated to make it meaningful.
- The data needs to be converted to numerical or categorical values, which algorithms can process.
- Feature values should be normalized and scaled to guarantee they have equal importance.
- The data is encoded or requires joins with reference information.

According to IDC, by 2025, 90% of data will be unstructured, so an essential part of building operational data pipelines is to convert unstructured textual, audio, and visual data into machine learning- or deep learning-friendly data organization or vector formats.

The ML process starts with manual exploratory data analysis and feature engineering on small extractions from historical data. However, to bring accurate models into production, ML and data engineering teams must work on larger, more up-to-date datasets and automate the collection and preparation process.

Furthermore, batch collection and preparation methodologies and batch analytics don't work well for operational or real-time pipelines. As a result, ML teams often build separate *real-time data pipelines* (pipelines that handle a very large number of events at scale in real time) that use *stream processing* (the ingestion and processing of a continuous data stream).

Some vendors provide data labeling as a service using a combination of automated tools and crowd-sourcing (for example Amazon SageMaker Ground Truth). Many algorithms require labeled data for training the

model. Therefore, you must design and implement labeling solutions for the historical data as part of the data preparation process.

In addition, many applications require constant retraining to maintain the model's accuracy and relevancy. Therefore, you should design a pipeline for automatically generating data labels in such cases.

Models are as good as the data they are trained on. To compare or explain model behavior and to address regulatory compliance, you must have access to the data used in training. Therefore, you must save information about the data origin with the model or save a unique copy of the dataset used for every training run. Data lineage and versioning solutions are a must in every MLOps solution.

A key component in any modern MLOps solution is a **feature store**, which automates the collection, transformation, cataloging, versioning, and serving of offline and online data.

Data Storage and Ingestion

Data is the foundation for AI and ML. It can be persistent or in transit and can be broken into two main categories: structured and unstructured.

Unstructured data is usually stored in file systems, object storage (data lakes), logging, or messaging systems (such as email). *Structured* data has some schema and is stored in tables, documents, or graphs.

Since it is scalable and cost-effective, we usually use object storage for deep learning workloads that process images, video, and text (NLP). In some cases, we will use local or distributed file storage.

When the data is structured, we can use files (CSV, Excel, and so on) to do simple exploration and model training, but this cannot scale for production. In production, we store data in one of those two categories:

Archival data systems

These are data warehouses or objects with structured file formats like CSV, Parquet, JSON, and so on. They record all

the historical transactions and allow efficient analytics queries.

Operational or real-time databases

These are frequently updated and enable fast data retrieval by index.

Use archival storage (data warehouses or data lakes with structured objects) for the training process since a model is an equation that learns how to predict results based on historical data patterns. Suppose the data source is a real-time or operational data system. In that case, you first need to copy and transform the data to the archival system, which is better at analytical workloads, for example, using an ETL process (Extract, Transform, Load). Structured object formats are usually the cheapest storage option, especially when using efficient compression techniques (like Parquet files). But data warehouses (like [Google BigQuery](#), [Snowflake](#), [Amazon Redshift](#), and so on) support faster and more flexible data queries and are easier to update.

WARNING

When you collect data for training, it is essential to make sure there is no bias in the data since this can lead to poor model results and even a total failure of your project (see [Amazon scrapped sexist AI tool](#)).

MLOps solutions and the training flow should incorporate data version control. Every training job should point to a unique version of the data, which allows for reconstructing the exact content of the data. While this may be simple for static historical content, it is harder for continuous and dynamic data like user information or transactions, which can change frequently.

The solution is to snapshot and store the dataset in archival storage and add the appropriate link (data lineage) to the job and model objects, allowing viewing of the data associated with each run easily. Some MLOps

frameworks and feature stores (like **MLRun**) provide this as a built-in feature.

In the serving process, a request arrives with partial data, for example, a user ID; you enrich the data with additional features for that user (such as age, gender, income, and so on) from an online database and pass it to the model. You cannot use archival storage for serving since it's too slow and cannot support a high number of concurrent requests. Instead, indexed NoSQL or SQL databases (like **Redis**, **DynamoDB**, or **MySQL**), also referred to as the *online feature store*, are better since they are faster and you have the index key (user ID).

To use the online features, you must first copy them to the online database; this can be simple with static features (like age or gender) but challenging with transactional features (like the total number of purchases in the last hour) that are frequently updated. Stream processing is usually used to calculate and update real-time features efficiently. This means the real-time data pipeline uses a different implementation than the offline feature calculation (implemented for training).

Figure 2-4 demonstrates different components used in the data ingestion flow.

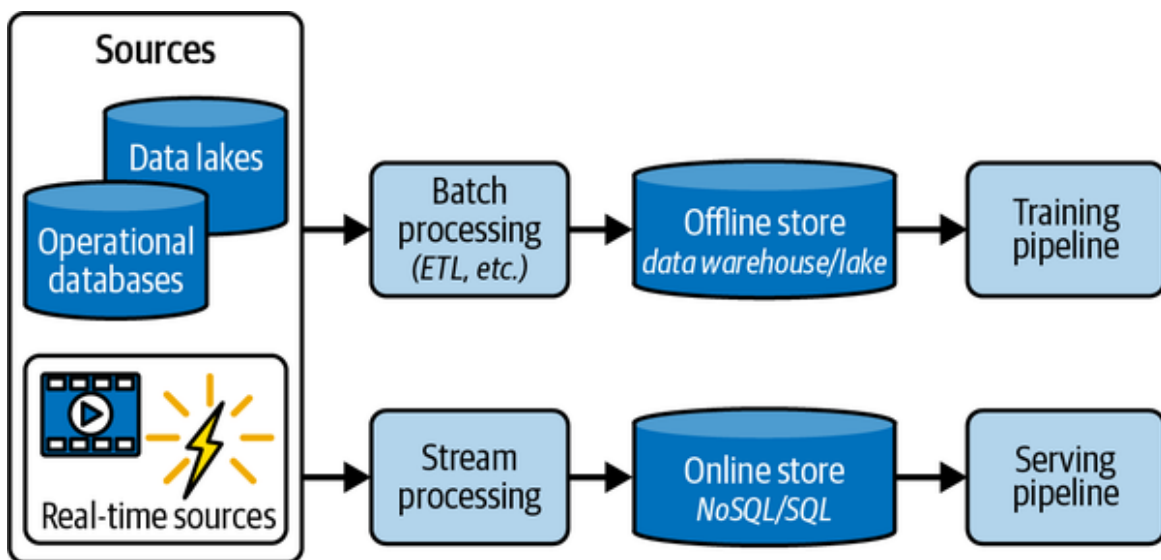


Figure 2-4. Offline and online data ingestion flow

Using different databases and data processing technologies in training and serving leads to higher complexity and data synchronization challenges. Feature stores, which we discuss in the next sections, are used to abstract away much of that complexity.

NOTE

Learn more about feature stores from the blog post [“What Are Feature Stores and Why Are They Critical for Scaling Data Science?”](#) by Adi Hirschtein.

Data Exploration and Preparation

In most cases, you cannot use data in its raw format, so the first step is applying cleaning, transformations, or calculations to the data. Once you have a clean set of meaningful features, you can start evaluating the data and selecting the best features for your model.

Here are some examples of required data conversions:

- Data arrives in a JSON format, and you need to convert it to an array or vector.
- Data contains a string (like a city name), and you need to convert it to a numeric value using some encoding strategy.
- You have a transaction log, but you need the total value of transactions in the last month.
- You have a person’s zip code, but you need to translate it to a numeric value representing a social-economical score.
- Dataset has missing values or misspelled names.

It is easier to start data exploration with a subset of the data and use interactive visual tools or standard Python packages like [pandas](#), [Matplotlib](#), [Bokeh](#) and [Plotly](#).

First you should visually inspect the data's nature and quality (inconsistencies, outliers, anomalies, missing data, and so on) and clean the data. Next, transform and add derived features, examine the correlation between the data or its derivatives and the target feature (goal), to support or disprove your theory, and generate a training set (feature vector). Creating new derived features to improve a model's output is the main craft of data scientists. Choosing relevant features to analyze and eliminate irrelevant or redundant ones is also essential.

Note that in the production implementation, there is a need to process more significant amounts of data in an automated way. Therefore, you must reimplement the data cleansing and transformations steps as part of a scalable and automated data processing pipeline and may need to use scalable or real-time data processing engines (like **Spark**, **Flink**, **Nuclio**, and so on) instead of interactive tools.

Figure 2-5 illustrates the data preparation and feature engineering flow.

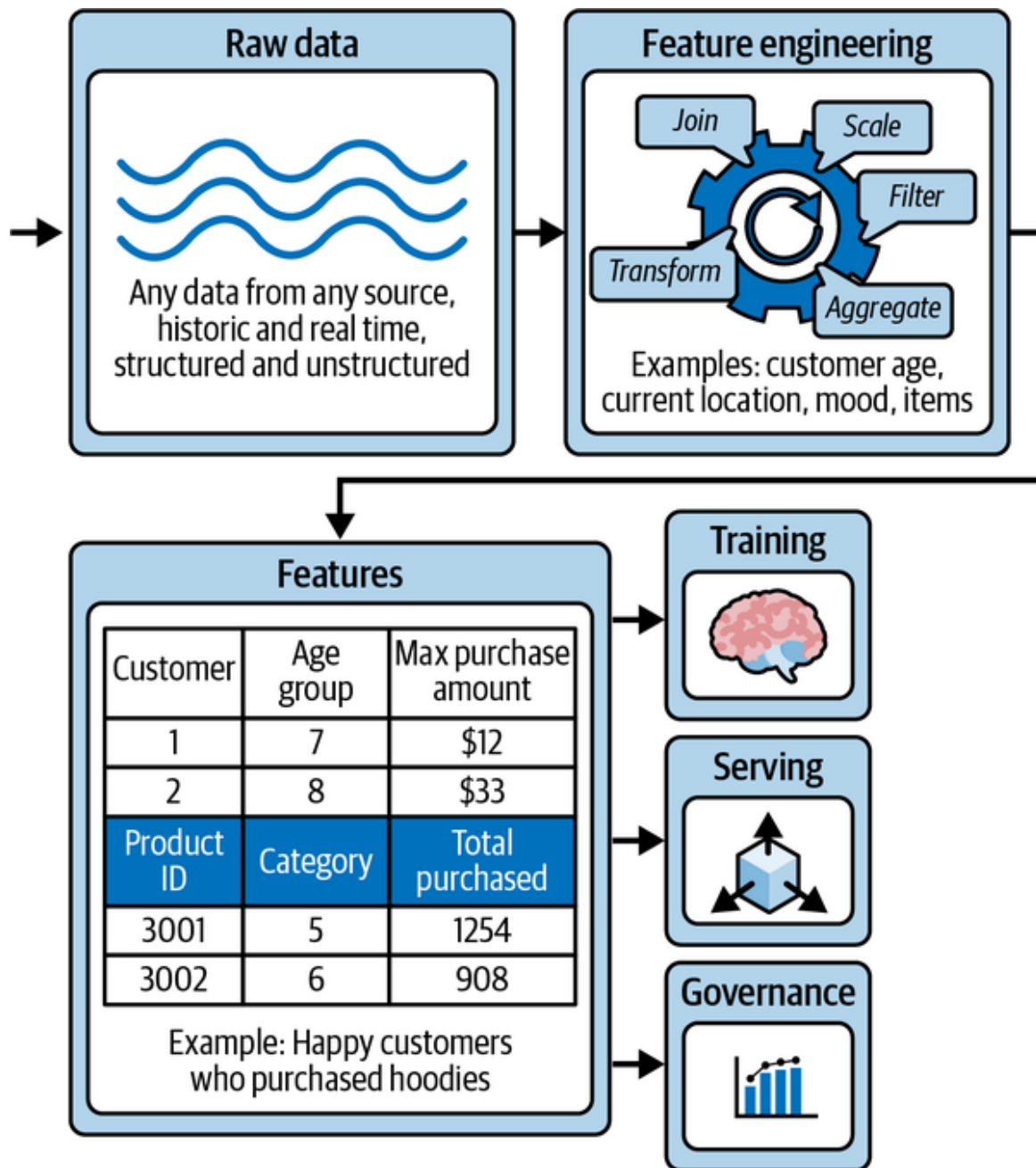


Figure 2-5. Feature engineering flow

The most common data transformations operations include:

Drop rows/columns

Drop rows/columns with too much missing data.

Imputing

Replace missing values with a constant or a statistical value (for example, median of the column).

Outlier detection

Drop rows where the values don't fall under the expected range (for example, compare the row value with $\text{mean} \pm N * \text{stddev}$).

Binning

Group multiple values into a single category (for example, Chile and Brazil map to South America).

Log transform

Convert a linear scale to a log scale.

One-hot encoding

Map different categorical values to a binary (yes/no) feature.

Grouping and aggregations

Aggregate column values by time (hour, day, month, and so on) or by category (for example, number of units sold by product type).

Scaling

Rescale column values (normalization, standardization).

Date extractions

Convert a date time to the hour, day of the week, month, season, is it a holiday, and so on.

Time recency

The time distance between two events (for example, time from the last login).

For unstructured data, there can be many more types of transformations (extract text elements, resize or rotate an image, and so on).

In training and during serving, you must use the same features; this requires you to implement two data pipelines: a batch pipeline for training and a real-time (streaming) pipeline for serving.

Some feature stores provide simple ways to define the data transformation logic and will automatically deploy and manage both offline and online data pipelines for you.

Data Labeling

Data labeling, or *data annotation*, is part of the preprocessing stage required for supervised learning. You add tags to raw data (numeric, text, images, and so on) to show a machine learning model the expected target attribute (prediction). Some prominent examples include [Amazon SageMaker Ground Truth](#), [Label Studio](#), [DataTurks](#), and [CVAT](#).

For numeric values, labeling can be deducted from the raw data. So, for example, in a churn model that tries to predict which customers are about to churn, you can examine historical records and mark the customers who churned by looking to see whether they remained a customer in the consecutive month. A simple analytics query will do the trick and shift the results back by one month.

Labeling is harder for unstructured data (text, images, video, audio, and so on) and usually involves a manual labeling process (by a human). However, many solutions in the market can simplify and automate parts of the process. Nevertheless, some challenges remain, like the need for domain expertise, the risk of inconsistency, and the error proneness of the process.

When the historical datasets are static, the labeling is done once. So, for example, the problem of classifying images as cats or dogs probably won't

change anytime soon. But when the data is dynamic, for instance, in a face or finger recognition application, new people can be added any day. In such cases, the labeling solution must be part of the application. For example, new users can take their pictures and attach their ID (for the application to verify their identity). If an image is not classified, it should alert or fall into a manual identification flow. When new pictures are added, the model training process needs to be triggered, and the online models must be refreshed to take the new images into account.

Data can be associated with labels and tags during ingestion time. For example, images arrive from a car along with metadata (car ID, model, driver) and telemetry (geolocation, timestamp, speed, weather, sensor metrics, and so on). This information should be stored and linked to the image and can be used to generate labels.

When considering MLOps with an automated (re)training flow, you should consider a mechanism for automated labeling. In some applications, the labels arrive in a delay (for example, if the user churned, if the stock price went up, or if the customer purchased the product). Therefore, the training dataset should be shifted to accommodate the delay (if you retrain the churn model based on the last three months, the data range should be between four and one months ago).

Feature Stores

As we've established, most of the complexities in any ML project arise from the data:

- Work typically done in silos (data scientists and engineers)
- Labor-intensive data engineering to produce high-quality features
- Duplicate efforts and resources in generating offline and online features that also lead to inaccurate results
- Hard to incorporate data versioning and governance
- Feature development work duplicated for every new project

- Lack of simple access to production-ready features at scale
- Disjointed or nonexistent model and feature monitoring

ML teams need to continuously deploy AI applications in a way that creates real, ongoing business value for the organization. Features are the fuel driving AI for the organization, and feature stores are the architectural answer that can simplify processes, increase **model accuracy**, and accelerate the path to production.

A feature store provides a single pane of glass for sharing all available features across the organization along with their metadata. When data scientists start a new project, they can access this catalog and easily find features. But a feature store is not just a data layer; it is also a data transformation service enabling users to manipulate raw data and store it as features ready to be used for offline (training) and online (serving), without duplicating the work. In addition, some feature stores support strong security, versioning, and data snapshots, enabling better data lineage, compliance, and manageability.

Some of the largest tech companies that deal extensively with AI have built their own feature stores (Uber, Twitter, Google, Netflix, Facebook, Airbnb, and so on). The open source and commercial landscape for feature stores has exploded in the last few years. This is a good indication to the rest of the industry of how important it is to use a feature store as part of an efficient ML pipeline.

Most feature stores are limited to structured data handling (ML), but some can support both structured and unstructured data (text, documents, images, audio, and so on).

Feature stores are described in detail in **Chapter 4**. As illustrated in **Figure 2-6**, they provide a mechanism to read data from various online or offline sources, conduct a set of data transformations, and persist the data in online and offline storage. Features are stored and cataloged along with all their metadata (schema, labels, statistics, and so on), allowing users to compose *feature vectors* (joint multiple features from different feature sets)

and use them for training or serving. The feature vectors are generated when needed, taking into account data versioning and time correctness (time traveling). Different engines are used for feature retrieval, a real-time engine for serving, and a batch one for training.

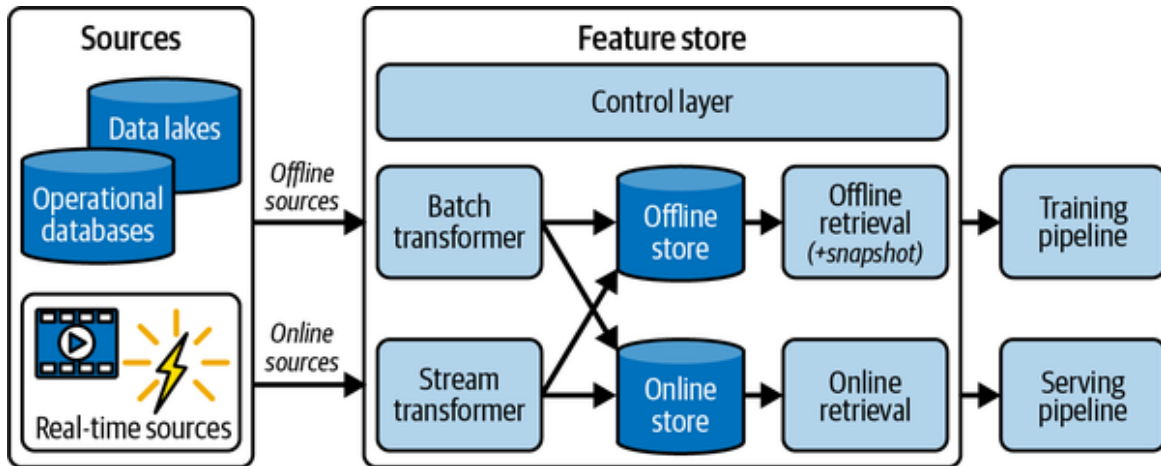


Figure 2-6. Common feature store architecture

Here are some major benefits of a feature store:

- Faster development with far fewer engineering resources
- Smooth migration from development to production
- Increased model accuracy (same pipeline for online and offline)
- Better collaboration and security across teams
- Ability to track lineage and address regulatory compliance

NOTE

Not all feature stores are born equal. Some are focused on cataloging and don't automate the process of ingestion and online or offline transformation, which are the most labor-intensive tasks. Therefore, make sure you properly evaluate before selecting a solution.

Model Development and Training

Data scientists generally go through the following process when developing models:

1. Extracting data manually from external sources
2. Data labeling, exploration, and enrichment to identify potential patterns and features
3. Model training and validation
4. Model evaluation and testing
5. Going back to step one and repeating until the desired outcomes (accuracy, loss, and so on) have been achieved

The traditional way is to use notebooks, small-scale data, and manual processes, but this does not scale and is not reproducible. Furthermore, to achieve maximum accuracy, experiments often need to be run with different parameters or algorithms (AutoML).

With MLOps, ML teams build machine learning pipelines that automatically collect and prepare data, select optimal features, run training using different parameter sets or algorithms, evaluate models, and run various model and system tests. All the executions, along with their data, metadata, code, and results, must be versioned and logged, providing quick results visualization, comparing them with past results, and understanding which data was used to produce each model.

Pipelines can be more complex: for example, when ML teams need to develop a combination of models or use deep learning or NLP. You can see a basic model development flow example in [Figure 2-7](#).

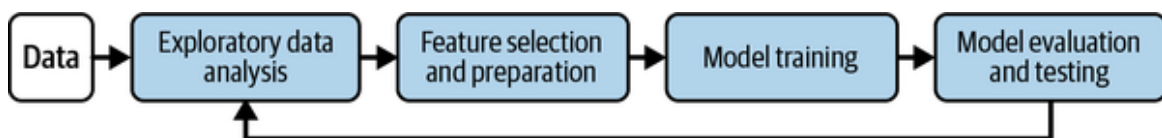


Figure 2-7. Model development flow

ML pipelines can be started manually or (preferably) triggered automatically when:

- The code, packages, or parameters change.
- The input data or feature engineering logic change.
- Concept drift is detected, and the model needs to be retrained with fresh data.

ML pipelines have the following features:

- Built using microservices (containers or serverless functions), usually over Kubernetes.
- Track all their inputs (code, package dependencies, data, parameters) and the outputs (logs, metrics, data/features, artifacts, models) for every step in the pipeline in order to reproduce or explain experiment results.
- Version all the data and artifacts used throughout the pipeline.
- Store code and configuration in versioned Git repositories.
- Use CI techniques to automate the pipeline initiation, test automation, review, and approval process.

Pipelines should be executed over scalable services or functions, which can span elastically over multiple servers or containers. This way, jobs complete faster, and computation resources are freed up once they are complete, saving high costs.

The resulting models are stored in a versioned model repository along with metadata, performance metrics, required parameters, statistical information, and so on. Models can be loaded later into batch or real-time serving microservices or functions.

Writing and Maintaining Production ML Code

Many data scientists like the usability and interactivity of Jupyter Notebook when they develop and evaluate models. It is convenient indeed to manipulate some code and immediately see a visual table or a chart, and

most ML tutorials, examples, and Kaggle projects are consumed as notebooks.

You can find projects where the data preparation, training, evaluation, and even prediction are all made in one huge Notebook, but this approach can lead to challenges when moving to production, for example:

- Very hard to track the code changes across versions (in Git).
- Almost impossible to implement test harnesses and unit testing.
- Functions cannot be reused in various projects.
- Moving to production requires code refactoring and removal of visualization or scratch code.
- Lack of proper documentation.

The best approach is to use functional programming for code segments and notebooks for interactive and visualization parts. **Example 2-1** implements a data preparation function that accepts a dataset (DataFrame) and some properties as inputs and returns the manipulated dataset. The function is documented and allows users to understand the purpose and usage.

Example 2-1. Data prep function (data_prep.py)

```
import pandas as pd

def add_date_features(
    data, time_column: str = "timestamp", drop_timestamp: bool = False
):
    """Add numeric date features (day of week, hour, month) to a dataframe

    :param time_column: The name of the timestamps column in the data
    :param drop_timestamp: set to True to drop the timestamp column from
                           the original dataframe
    :return dataframe
    """
    timestamp = pd.to_datetime(data[time_column])
    data["day_of_week"] = timestamp.dt.day_of_week
    data["hour"] = timestamp.dt.hour
    data["month"] = timestamp.dt.month
    if drop_timestamp:
```

```

    data.drop([time_column], axis=1, inplace=True)
    return data

```

Place the function in a separate Python file *data_prep.py*, and you can call it from the Notebook, inject data, and examine or visualize its output using the following code cell:

```

import pandas as pd
from data_prep import add_date_features

df = pd.read_csv("data.csv")
df = add_date_features(df, "timestamp", drop_timestamp=True)
df.head()

```

Once the code is well defined, use the Python test framework (pytest) and implement unit testing for each of the functions as show in **Example 2-2**:

Example 2-2. Data prep test function (test_data_prep.py)

```

import pytest
import data_prep
import pandas as pd

# tell pytest to test both drop values (True/False)
@pytest.mark.parametrize("drop_timestamp", [True, False])
def test_add_date_features(drop_timestamp):
    df = pd.DataFrame({'times': ['2022-01-01 08:00',
                                '2022-02-02 09:00',
                                '2022-03-03 10:00'],
                       'vals': [1, 2, 3]})
    new_df = data_prep.add_date_features(df, "times",
                                         drop_timestamp=drop_timestamp)

    # verify the results are as expected
    assert new_df["day_of_week"].to_list() == [5, 2, 3]
    assert new_df["month"].to_list() == [1, 2, 3]
    assert new_df["hour"].to_list() == [8, 9, 10]
    assert ("times" in new_df.columns.values) != drop_timestamp

```

The code in **Example 2-2** will execute the `add_date_features()` function with different input options and verify that the outputs are correct.

Using this approach, you gain some immediate benefits:

- Easily see changes to your data prep code in the version control.

- The same code can be tested later with a test harness (for example, using `pytest`).
- The function can be moved to production without the need to refactor the notebook.
- The function is documented, and you can easily understand how to use it and what to expect.
- The function can later be saved to a shared library and used across different projects.
- The code becomes more readable.

Another benefit of the functional approach is demonstrated in the upcoming chapters: an automated way to convert development code into production services and pipelines using tools such as **MLRun** (MLOps orchestration framework).

Tracking and Comparing Experiment Results

When running ML experiments, it is essential to track every run so that you can reproduce experiment results (for example, which parameters and inputs yield the best results), visualize the various metrics, and compare the results of different algorithms or parameter sets.

Each execution involves input and output datasets. It is crucial to track and version the datasets, not just the parameters. Any MLOps solution should provide a mechanism to version data and track the data propagation (lineage) together with the rest of the execution parameters, outputs, and metadata.

Today various open source and commercial frameworks track the results of every experiment run, store it in a database, and visualize it. Some examples shown in **Figure 2-8** include **MLflow**, **Weights & Biases**, **MLRun** and **ClearML**.

In the real world, experiments can run in an automated ML pipeline (see **Figure 2-9**), which comprises different steps (data prep, train, test, and so

on). Each stage of the pipeline accepts parameters, inputs data, and generates results such as output values, metrics, and data to be used in subsequent pipeline steps. In addition, the tracking should be extended to operational data (which code was used, packages, allocated and used resources, systems, and so on).

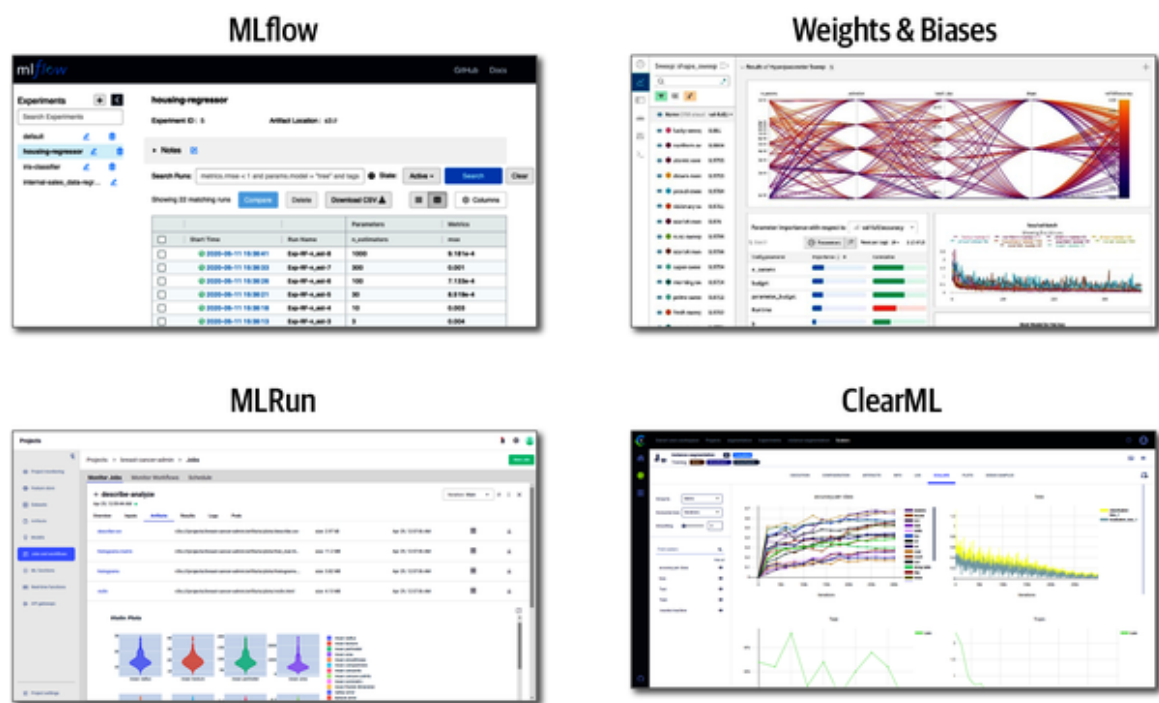


Figure 2-8. Different tools for ML execution tracking

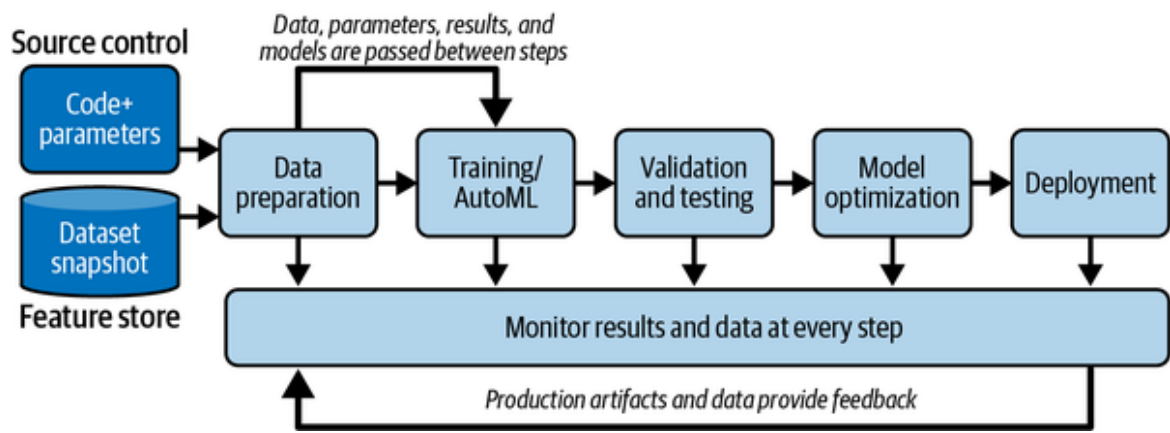


Figure 2-9. Multi-stage (pipeline) execution tracking

Figure 2-10 shows the general architecture of an execution tracking system. Inputs may include parameters, the user, or system-defined tags (to allow filtering and comparisons), secrets (hidden credentials used by the

execution), and data objects (files, tables, and so on). Outputs include the result metrics, logs, usage data, output data objects, and artifacts. A good tracking system also records the code version, used packages, runtime environment and parameters, resources, code profiling, and so on.

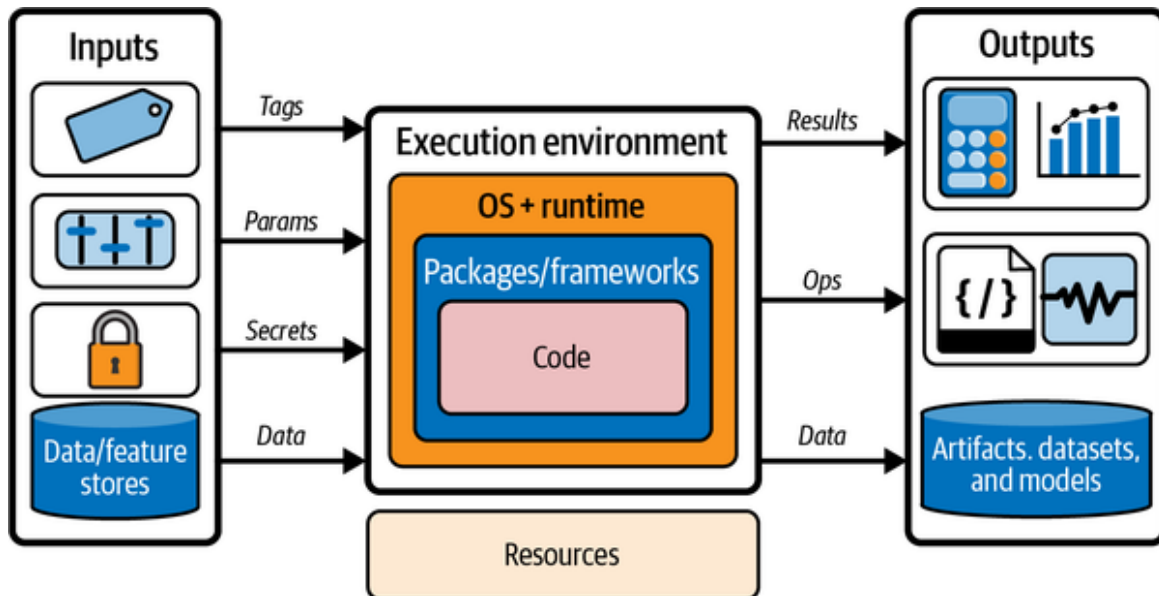


Figure 2-10. Execution tracking: what and how do we track?

The downside of execution tracking is that it requires *code instrumentation* (adding code to explicitly log parameters, tags, results, and data). Some MLOps frameworks provide auto-logging for ML/DL workloads where you can import a library that automatically records all the ML framework-specific metrics.

A new technology, AutoMLOps, is pioneered in the MLRun framework. It records metrics along with the parameters, data lineage, code versioning, and operational data. It also automatically adds production features for auto-scaling, resource management, auto-documentation, parameter detection, code profiling, security, model registry, and so on, eliminating significant engineering efforts.

Distributed Training and Hyperparameter Optimization

To get to the best model results, try out various algorithms or parameter combinations and choose the best one based on a target metric like best

accuracy. This work can be automated using multiple hyperparameter optimization and AutoML frameworks, which try out the different combinations, record all the metrics for each run, and mark the best. To shorten training time, some frameworks support executing each individual run on a different compute resource. **Figure 2-11** shows the tracking of multiple children runs in a hyperparameter job and the best-selected result.

The screenshot shows the MLRun 'Monitor Jobs' interface. The breadcrumb path is 'Projects > breast-cancer-admin > Jobs'. There is a 'New Job' button in the top right. The interface has tabs for 'Monitor Jobs', 'Monitor Workflows', and 'Schedule'. The selected job is 'trainer-train', with a timestamp of 'Apr 29, 01:01:04 AM'. Below the job name are tabs for 'Overview', 'Inputs', 'Artifacts', 'Results', 'Logs', and 'Pods'. The 'Results' tab is active, displaying a table of hyperparameter runs.

Iter	State	N_estimators	Learning_rate	Max_depth	Accuracy	F1_score	Precision_score	Recall_score
1	completed	10	0.10	2	0.96	0.97	0.96	0.97
2	completed	100	0.10	2	0.96	0.97	0.96	0.97
3	completed	1000	0.10	2	0.96	0.97	0.96	0.99
4	completed	10	0.00	2	0.62	0.77	0.62	1
5	completed	100	0.00	2	0.62	0.77	0.62	1
6	running	1000	0.00	2	0.96	0.97	0.96	0.97

Figure 2-11. Execution tracking of a hyperparameter job (in the MLRun framework)

Parallel hyperparameter jobs are not limited to model training. They can be used for parallel loading and preparation of many data objects, parallel testing of different test cases, and so on.

There are several hyperparameter execution strategies:

Grid search

Running all the parameter combinations

Random

Running a sampled set from all the parameter combinations

Bayesian optimization

Building a probability model of the objective function and using it to select the most promising hyperparameters to evaluate in the true objective function

List

Running the first parameter from each list followed by the second from each list and so on

You can specify selection criteria to select the best run among the different child runs (for example, the model's accuracy) and the stop condition to stop the execution of child runs when certain criteria, based on the returned results, are met (for example: `stop condition="accuracy>=0.9"`).

Some data engineering, ML, or DL jobs cannot fit into a single container or virtual machine and must be distributed across multiple containers. A few open source frameworks, including Spark, **Dask**, **Horovod**, and Nuclio, support workload distribution. When distributing the workload in combination with the parallel run of child (hyperparameter) tasks, you need to control and limit the total amount of resources used.

Tracking a distributed workload may be more challenging. Make sure the MLOps framework you use supports that.

Building and Testing Models for Production

When models are used in real-world applications, it is critical to ensure they are robust and well-tested. Therefore, in addition to traditional software testing (unit tests, static tests, and so on), testing should cover the following categories:

Data quality tests

The dataset used for training is of high quality and does not carry bias.

Model performance tests

The model produces accurate results.

Serving application tests

The deployed model along with the data pre- or post-processing steps are robust and provide adequate

performance.

Pipeline tests

Ensuring the automated development pipeline handles various exceptions and the desired scale.

When the training dataset is of low quality, you may presume that the model is accurate, but it can make harmful predictions. Therefore, it is essential to validate that the data is high quality. Here are some examples of data quality tests:

- There are no missing values.
- Values are of the correct type and fall under an expected range (for example, user age is between 0-120, with anticipated average and standard deviation).
- Category values fall within the possible options (for example, city names match the options in a city name list).
- There is no bias in the data (for example, the gender feature has the anticipated percentage of men and women).

The data quality tests can be implemented in the data pipeline (and feature store) or in the ML pipeline before the training. Note that some feature stores automate the data quality validation using built-in functions.

Once you train the model, the next step is to make sure it is accurate and resilient. Beyond the common practice of setting aside a test dataset and measuring the model accuracy using that dataset, several additional tests can improve the model quality:

- Verify the performance is maintained across essential slices of the data (for example, devices by model, users by country or other categories, movies by genre) and that it does not drop significantly for a specific group.

- Compare the model results with previous versions or a baseline version and verify the performance does not degrade.
- Test different parameter combinations (hyperparameter search) to verify you chose the best parameter combination.
- Test for bias and fairness by verifying that the performance is maintained per gender and specific populations.
- Check feature importances and whether there are features with a marginal contribution that can be removed from the model.
- Test for immunity to fake, random, or malicious input vectors to increase robustness and defend against adversarial attacks.

Particular attention should be given to how you generate the test set independently that considers fairness and lack of bias and minimizes the dependencies on the training set.

When the models are deployed into production serving applications, they contain additional data pre- or post-processing logic (extraction, formatting, validation, transformations, API integration, and so on). In addition, the model code may depend on various software packages or infrastructure (memory, CPUs, GPUs, and so on). Therefore, models must be thoroughly tested in their target serving application environment and through the API before they are deployed into the production environment.

Here are some examples for serving application tests:

API coverage

All serving APIs behave as expected.

Performance tests

Verify the serving application can sustain the target number of requests per second and respond within the required latency.

Package consistency

Verify that the model training and serving are using the same framework version (for example, sklearn).

Test data validation logic

Verify the model endpoint fails or logs the request if improper data is sent to the model.

Test resiliency

Test that the serving application can resist malicious attacks and impersonation.

Test correctness

Verify that the model prediction results via the serving API are the same as those in the model validation step.

Test the outcome

Verify that prediction results translate to the proper action (writing to a database, generating an alert, updating the user interface, and so on).

The different tests all should be part of an automated CI/CD pipeline. Every time the dataset or code changes, the pipeline is executed and produces a new set of deployable objects (models, applications, features, and so on) and logs all the results to enable reproducibility and explainability.

Some attention should be given to testing the ML pipeline, ensuring that it will run correctly every time it's triggered, will not fail due to missing parameters or inadequate resources, and can handle data at scale.

Once the model and other production artifacts are ready, they must be stored in a versioned artifacts repository along with all their metadata and the parameters required to generate the production deployments.

In many cases, the trained model can be further optimized for production and higher performance, for example, by performing feature selection and

removing redundant features or by compressing the models and storing them in more machine-efficient formats like **ONNX**. Therefore, ML pipelines may incorporate model optimization steps.

Figure 2-12 illustrates how different test and optimization steps can be used as part of an ML pipeline.

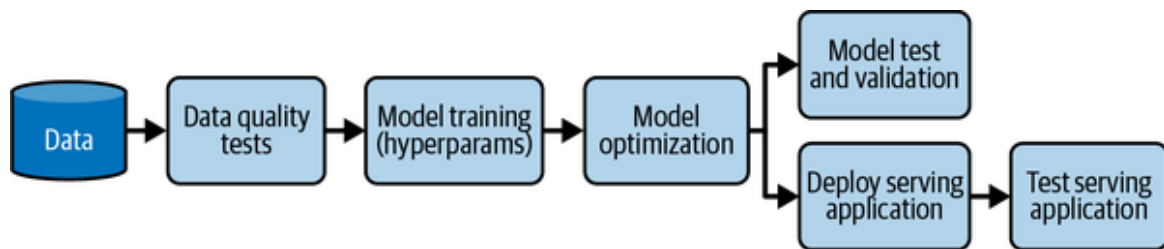


Figure 2-12. Adding tests and optimizations to an ML pipeline

Deployment (and Online ML Services)

Once an ML model has been built, it needs to be integrated with real-world data and the business application or frontend services. The whole application or parts thereof need to be deployed without disrupting the service. Deployment can be extremely challenging if the ML components aren't treated as an integral part of the application or production pipeline.

ML application pipelines usually consist of the following:

- API services or application integration logic
- Real-time data collection, enrichment, validation, and feature engineering logic
- One or more model serving endpoints
- Data and model monitoring services
- Resource monitoring and alerting services
- Event, telemetry, and data/features logging services
- A set of actions following the prediction results

You can see a real-time pipeline example in [Figure 2-13](#).

The different services are interdependent. For example, if the inputs to a model change, the feature engineering logic must be upgraded along with the model serving and model monitoring services. These dependencies require online production pipelines (graphs) to reflect these changes.

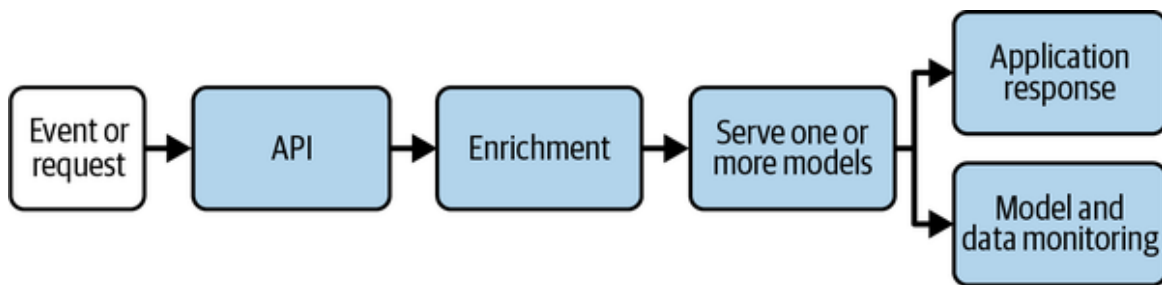


Figure 2-13. Building online ML services

Application pipelines can be more complex when using unstructured data, deep learning, NLP, or model ensembles, so having flexible mechanisms to build and wire up our pipeline graphs is critical.

Application pipelines are usually interconnected with fast streaming or messaging protocols, so they should be elastic to address traffic and demand fluctuations, and they should allow nondisruptive upgrades to one or more elements of the pipeline. These requirements are best addressed with fast serverless technologies.

Application pipeline development and deployment flows do the following:

- Develop production components:
 - API services and application integration logic
 - Feature collection, validation, and transformation
 - Model serving graphs
- Test online pipelines with simulated data.
- Deploy online pipelines to production.
- Monitor models and data and detect drift.

- Retrain models and reengineer data when needed.
- Upgrade pipeline components (nondisruptively) when needed.

From Model Endpoints to Application Pipelines

Today's common practice is to build model serving endpoints that merely accept the numeric feature vector and respond with a prediction. The pre- or post-processing logic, usually tightly coupled with the model, is done in separate microservices. This complicates the delivery, scaling, and maintenance of the ML application.

In some cases, the prediction is made using a combination of models, for example, by implementing an ensemble of models that cover different time scopes (recent time and seasonal models) or other algorithms. Another example is cascading two models. The first extracts sentiments from text, and the second makes a prediction based on the sentiments and other features.

A preferred approach is to design online (or real-time) application pipelines where the model serving is just one step, and be able to deploy, upgrade, or roll back that pipeline as a whole. Unlike the data and model training pipelines that run slow batch tasks, the application pipeline should process thousands of requests per second and use streaming or serverless processing engines.

Figure 2-14 demonstrates a simple application pipeline that accepts a user request (via HTTP or a stream message), processes it, predicts a result using a three-model ensemble, and does post-processing (for example, response to the user, updated the result in a database, generates an alert, and so on).

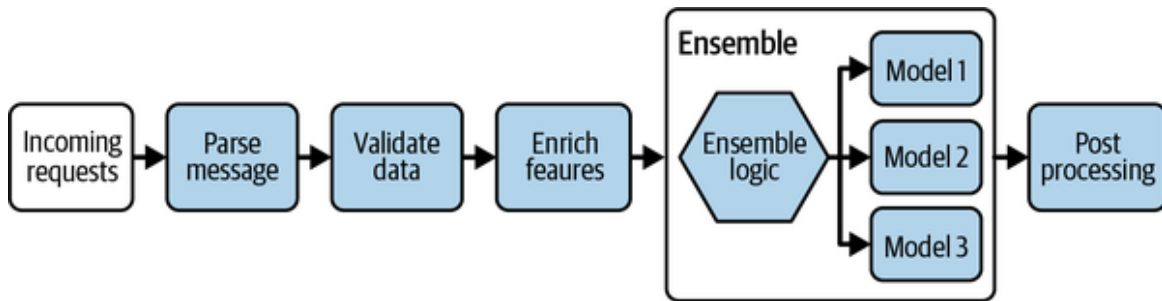


Figure 2-14. Online application pipeline example

ML or DL applications may work with unstructured data and complex processing stages such as image manipulations (detect objects, resize, sample, recolor, crop, and so on) or text manipulations (parse, format, tokenize, and so on). Application pipelines are not limited to structured data. As illustrated in **Figure 2-15**, a pipeline can branch and process different parts of the data using various technologies and models. In the example, a document URL is sent to the pipeline (via a Kafka stream) and the first step fetches the document from an object storage repository. This is followed by text and image processing steps, and finally the results are combined and a search database is updated that hosts the document information.

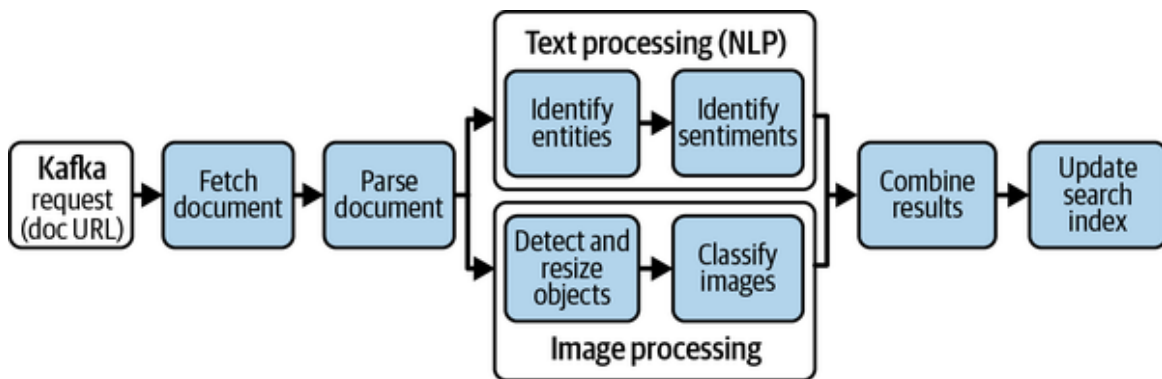


Figure 2-15. Advanced online application pipeline

Some examples of open source and commercial frameworks for building multistage online pipelines:

AWS Step Functions

AWS cloud service composing online pipelines from AWS Lambda serverless functions and other AWS cloud services.

MLRun serving graphs

Open source and commercial MLOps framework, its serving layer enables the composition of online data and ML/DL pipelines (graphs), provisioned automatically into auto-scaling real-time serverless functions.

Apache Beam

Open source stream processing framework, focused on online structured data processing. (**Google Dataflow** is a managed version of Apache Beam.)

Seldon

Open source and commercial model serving framework with basic online pipeline capabilities.

Online Data Preparation

A dominant part of the online application pipeline is data processing, with tasks such as data parsing, formatting, validations, transformations, aggregations, logging, persisting, joining, and so on.

Processing data in a batch is a common practice. For example, you can use data warehouse queries, ETL processes, Spark, and so on. But the same technologies don't work for online pipelines where thousands of events or user requests arrive every second and may need to be answered within milliseconds.

In online data pipelines, the features are accumulated in memory or a fast SQL/NoSQL database, fetched per event to enrich the user request and passed into the model for prediction. When the features are based on historical or static data (such as gender, age, annual income, and so on), you can use a periodic batch process to copy such features to the online database. However, this won't work when the features are frequently

updated (current geolocation, last transaction value, money spent last hour, time from the previous login, and so on).

Online data pipelines are implemented using stream processing (Spark Streaming, Flink, **Amazon Kinesis Data Analytics**, Nuclio, and so on), where events are ingested, transformed, or aggregated on the fly to form real-time feature vectors and a fast key/value database is used to persist and share the distributed state. **Figure 2-16** illustrates how stateful stream processing works. Events arrive and are distributed to stream workers (partitioned by the user key). Each worker processes the data and merges or aggregates it with the accumulated state.

The major challenge is that stream processing code and methodologies differ quite a bit from batch data analytics approaches and require reimplementing the batch pipeline used for the training into a real-time streaming pipeline. However, some feature stores allow you to define the data pipeline using high-level primitives and automatically generate the batch or streaming pipelines, ensuring the same logic is preserved and saving you significant engineering effort.

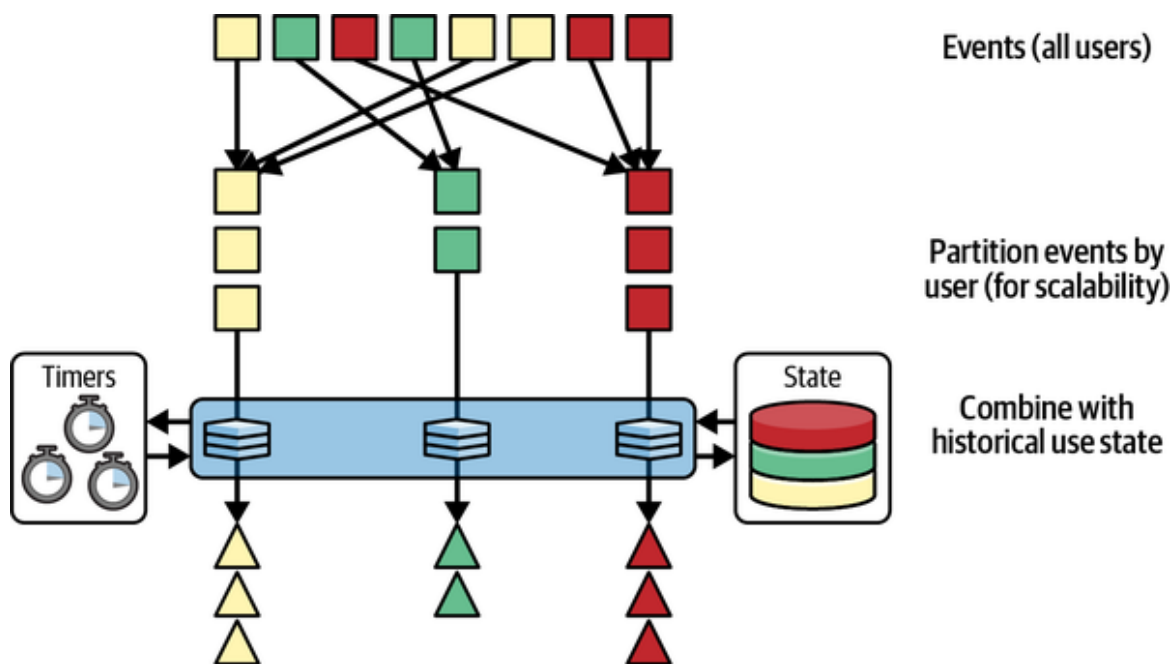


Figure 2-16. How stateful stream processing works (source: <https://beam.apache.org>)

Online data pipelines are not limited to structured data. Modern applications need to process unstructured visual and textual data with operations such as resizing or rotating images, parsing text, tokenizing statements, and so on. Therefore, the technology and framework you select need to support such applications.

NOTE

The line between data processing and ML or DL can be blurry. For example, text can be converted to sentiment or a category feature using an NLP model. Is that model serving or a data transformation?

Continuous Model and Data Monitoring

AI services and applications are becoming an essential part of any business. Poor model performance can lead to liabilities, revenue loss, damage to the brand, and dissatisfied customers. Therefore, it is critical to monitor the data, the models, and the entire online applications pipeline, and guarantee that models continue to perform and that business KPIs are met. Thanks to well-implemented monitoring solutions, you can quickly react to the problems by notifying users, retraining models, or adjusting the application pipeline.

Monitoring systems track various infrastructure, data, model, and application metrics and can report or alert on different situations, including the following:

Data or concept drift

The statistical attributes of the model inputs or outputs change (an indication that the model will underperform).

Model performance problems

The results of the model are inaccurate.

Data quality problems

The data provided to the model is of low quality (missing values, NaNs, values are out of the expected range, anomalies, and so on).

Model bias

Detect changes between the overall scoring and scoring for specific populations (such as male and female and minorities).

Adversarial attacks

Malicious attempts have been made to deceive the model.

Business KPIs

Verify that the model meets the target business goals (revenue increase, customer retention, and so on).

Application performance

The application manages to properly serve requests without delays.

Infrastructure usage

Track the usage of computing resources.

Model staleness

Alert if it is too long since the last time a model version was deployed.

Anomaly detection

Model data or results don't fall under the expected norm or classes (for example, using an encoder-decoder neural network model).

Figure 2-17 shows a typical model monitoring architecture. The data inputs, outputs, and application metrics are sent to a stream. A real-time stream processing application reads the data. It can detect or alert on immediate problems, aggregate the information, and write to various data targets (key/value, time series, and files or data warehouse).

Alerts generated by the monitoring system can notify users (via emails, Slack, and so on) or trigger a corrective action such as retraining a model with newer data, changing model weights, and so on.

Feature stores can play a significant part in monitoring data and models. They store the schema and statistics per feature, which can be used in the different validation and analysis tasks. If the production data is returned to the feature store, it's easier to analyze, join, and compare production datasets with other historical or offline datasets.

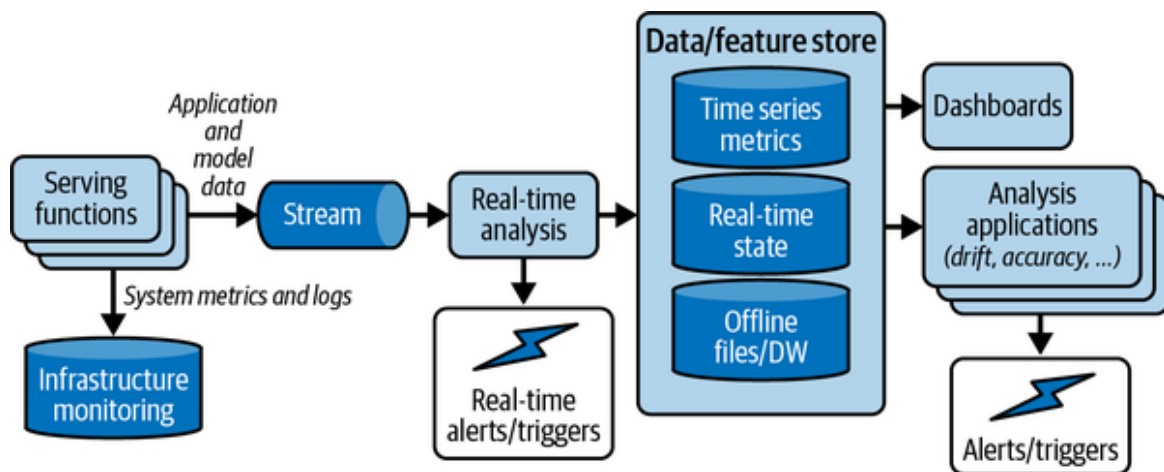


Figure 2-17. Online model and data monitoring architecture

Monitoring Data and Concept Drift

Concept drift is a phenomenon where the statistical properties of the target variable (y, which the model is trying to predict) change over time. Data drift (virtual drift) happens when the statistical properties of the inputs changes. In drift, the model built on past data no longer applies, and assumptions made by the model on past data need to be revised based on current data. **Figure 2-18** illustrates the differences between concept drift and virtual (data) drift.

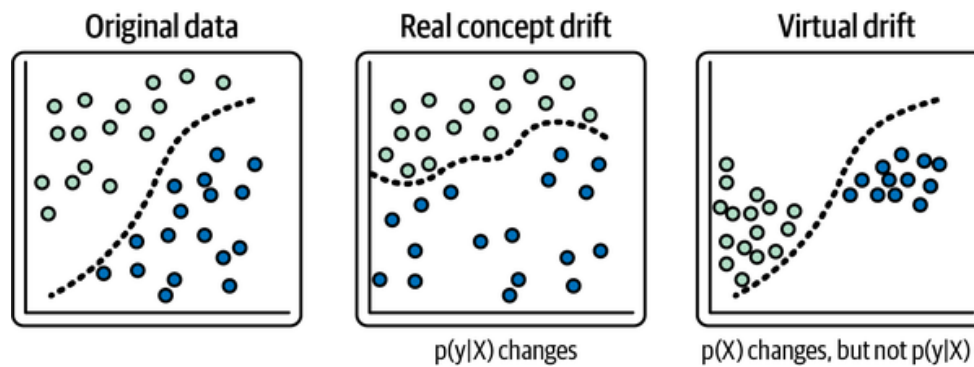


Figure 2-18. Concept drift versus virtual (data) drift

Going back to the business level, you can see examples of drift in the following use cases:

Wind power prediction

When predicting the electric power generated from wind from an offline dataset based model, we have concept drift versus online training models due to the nonstationary properties of winds and weather.

Spam detection

Email content and presentation change constantly (data drift). Since the collection of documents used for training changes frequently, the feature space representing the collection changes. Also, users themselves change their interests, causing them to start or stop considering some emails as spam (concept drift).

Concept drift changes can be:

Sudden

The move between an old concept and a new one happens simultaneously. The behavioral patterns associated with the COVID-19 pandemic have provided us with striking examples, like the lockdowns that abruptly changed population behaviors worldwide.

Incremental/gradual

The change between concepts happens over time as the new concept emerges and starts to adapt. The move from summer to winter could be an example of gradual drift.

Recurring/seasonal

The changes recur after the first observed occurrence. An example is a seasonal shift in weather, which dictates that consumers buy coats in colder months, cease these purchases when spring arrives, and then begin again in the fall.

Figure 2-19 shows how model drift detection works. First, the model inputs and outputs are collected, and the system calculates the statistics over a time window and compares them with the sample set statistics (saved at training time) or with the data from an older time window.

The monitoring system saves the various feature statistics (min, max, average, stddev, histogram, and so on), and the drift level is calculated using one or more of the following metrics:

- Kolmogorov–Smirnov test
- Kullback–Leibler divergence
- Jensen–Shannon divergence
- Hellinger distance
- Standard score (Z-score)
- Chi-squared test
- Total variance distance

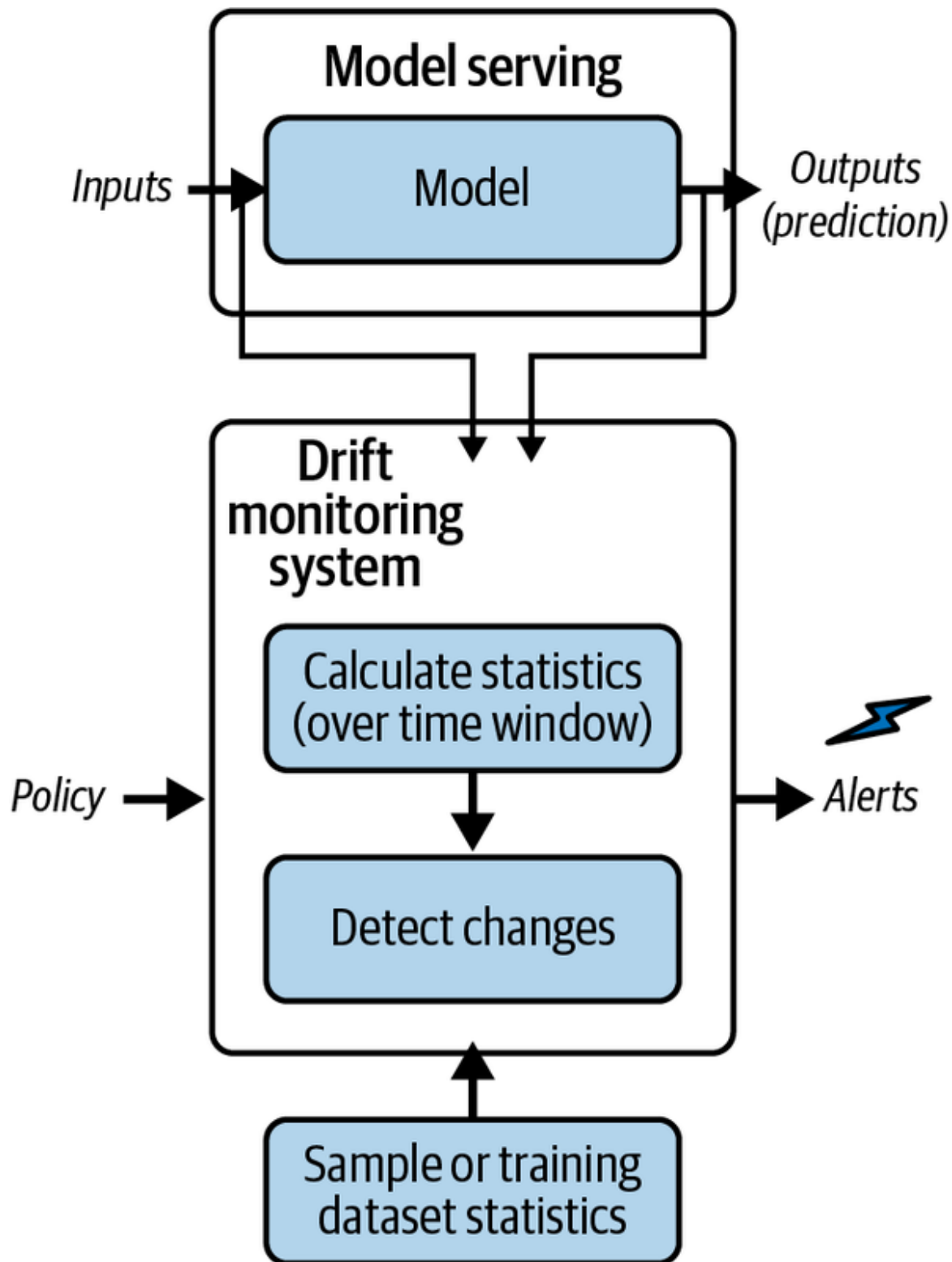


Figure 2-19. Drift detection logic

Figure 2-20 demonstrates how drift can be detected.

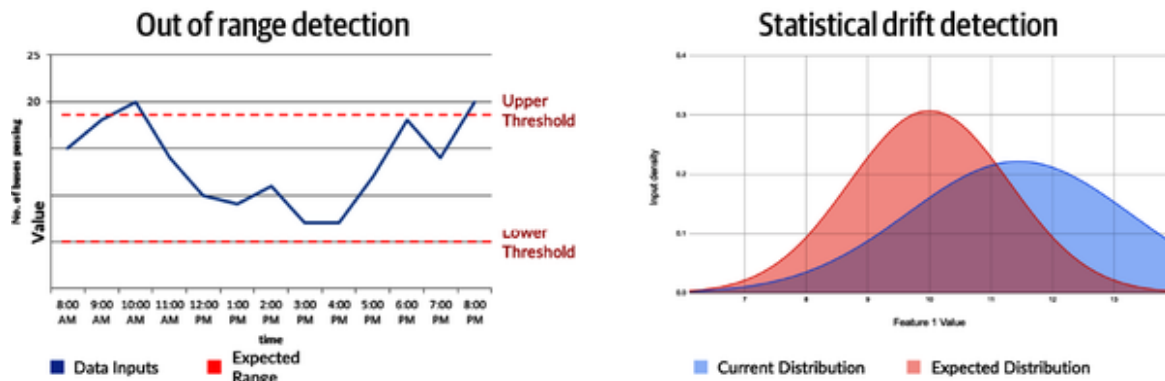


Figure 2-20. Drift detection types

Drift is easily detected using these methods when the data consists of simple numeric metrics, but how can it be detected when the data is unstructured, an image, or a piece of text?

One trick is to convert the input data to flat metrics that represent the data and monitor the drift on those metrics. For example, let's say you classify images of fruits. Then you can convert the images to their RGB color metrics and check that the color distribution in production is the same as in training.

Monitoring Model Performance and Accuracy

An important metric is to measure model accuracy in production. For that, you must have the *ground truth* (the actual result that matches the prediction). In some models obtaining the ground truth is relatively simple. For example, if we predict that a stock price will go up today, we can wait a few hours and know if the prediction was accurate. This is the same with other prediction applications like predicting customer churn or machine failure where the actual result arrives with some delay.

In some applications, a prediction is made for a specific transaction (for example, exposure or click on an advertisement). The transaction or prediction can be tagged with a UID (unique identifier) in such a case. Once the actual result is known (the customer bought the product), you can update the transaction (identified by the UID) with the ground truth value. This requires that the model serving and monitoring frameworks has the

ability to store or generate a UID per prediction and add the ground truth values to specific transactions/predictions.

The accuracy monitoring is done periodically (for example, every hour or day). First, a dataset is generated with the predicted y values (calculated by the model) and the ground truth values (the actual result with the proper time shifting or obtained using the UID). Then it is used to calculate the accuracy metrics and compare them with the accuracy during training. This is illustrated in **Figure 2-21**.

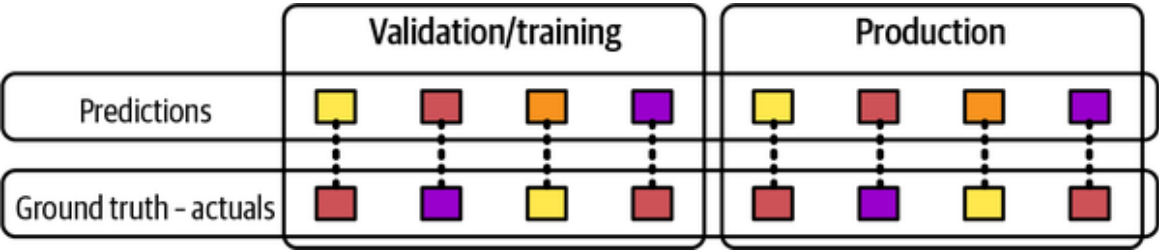


Figure 2-21. Monitoring model accuracy in production

NOTE

The ground truth values calculated for the accuracy monitoring are the same y labels required for retraining the model. Therefore, the best approach is to generate them once, store them in the feature store, and use them for both retraining and accuracy monitoring.

Just like in training, it is recommended to use several metrics to determine the prediction accuracy, especially in the case when classes are not balanced:

Accuracy

General overall accuracy

Recall

What fraction of overall positives were correct

Precision

Determine when the costs of false positive are high

F1 Score

Analysis of the trade-off between recall and precision

NOTE

Be aware that the ground truth may contain bias. For example, in an application that predicts fraud to approve or reject transactions, the ground truth includes only information on the approved transactions. There is no data about declined transactions that may not have been fraudulent, which can lead to bias.

The Strategy of Pretrained Models

One of the most prolific authors on business strategy is **Harvard Business School professor Michael Porter**, who has often said, “The essence of strategy is choosing what not to do.” With most organizations struggling to implement machine learning projects that provide ROI, there is a need for a better strategy. In particular, organizations should ask what they should *not* be doing while doing machine learning projects. In many cases, they shouldn’t be building a specific type of model and should instead use pretrained models.

In *Understanding Michael Porter* (Harvard Business Review Press, 2011), Joan Magretta summarizes the essence of competitive advantage as outlined by Michael Porter in **Figure 2-22**. Companies that compete on execution become part of a prisoner’s dilemma game theory problem, where both competitors increasingly lower prices and costs while lowering the company’s profit. This is the best-case scenario; in many cases, it is impossible for a company to out-execute a bigger rival, say training a better NLP or computer vision model.

	Competing on execution	Competing on strategy
Activities	Perform SAME activities as rivals, execute better	Perform DIFFERENT activities from rivals
Value created	Meet the SAME needs at lower cost	Meet DIFFERENT and/or same needs at lower cost
Advantage	Cost advantage but hard to sustain	Sustainably higher prices and/or lower costs
Results	Be the BEST , compete on EXECUTION	Be UNIQUE , compete on STRATEGY

Figure 2-22. Competing on strategy, not execution (source: *Understanding Michael Porter* by Joan Magretta)

This conceptual understanding of strategy shows why pretrained models are an essential component of a holistic strategy to create unique competitive advantages while implementing machine learning projects.

There are several vendors of pretrained models. The most popular platform is **Hugging Face**, which has over 60,000 models. Google's **TensorFlow Hub** has a unique collection of pretrained models in various formats, including formats targeting runtimes like **Javascript** or **embedded hardware or mobile**. One more format and repository is **ONNX**, which contains many examples of pretrained computer vision and language models.

Building an End-to-End Hugging Face Application

The best way to understand pretrained models is to build an end-to-end solution with one. Fortunately, Hugging Face makes it simple to do this. First, you need to **sign up for a free account**.

Next, let's look at the application architectures in **Figure 2-23**. A user account creates an authentication token that later becomes part of a continuous delivery pipeline in a cloud-based build system in **GitHub**

Actions. The code itself develops in GitHub Codespaces. A Hugging Face model then lives inside a Gradio application, allowing for quick prototyping of an MLOps workflow by providing a user interface. Finally, the Hugging Face Spaces functionality allows users to create applications hosted on the platform using **Gradio**, a technology for building machine learning apps.

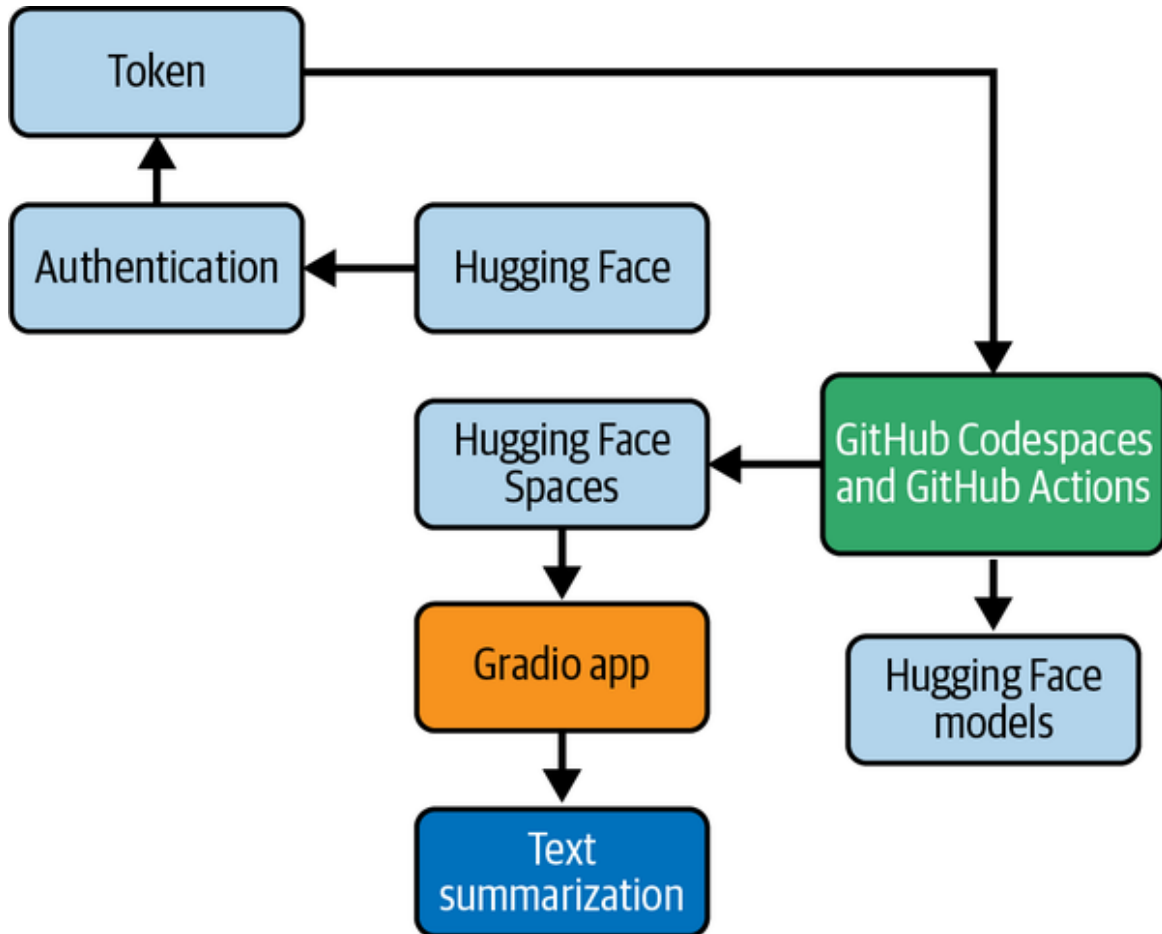


Figure 2-23. MLOps prototyping with Hugging Face pretrained models

NOTE

You can view a walkthrough of this Hugging Face application on [YouTube](#) or the [O'Reilly platform](#). The application source code is in [GitHub](#).

Let's break each core application file; first there is the *app.py*:

```

from transformers import pipeline
import gradio as gr

model = pipeline("summarization")❶

def predict(prompt): ❷
    summary = model(prompt)[0]["summary_text"]
    return summary

with gr.Blocks() as demo: ❸
    textbox = gr.Textbox(placeholder="Enter text block to summarize", lines=4)
    gr.Interface(fn=predict, inputs=textbox, outputs="text")

demo.launch()

```

- ❶ Use Hugging Face transformers (pretrained model).
- ❷ Create the predict function.
- ❸ Build the Gradio UI.

The other key file is *main.yml*, which controls the continuous delivery to Hugging Face. The actions are as follows:

```

name: Sync to Hugging Face hub
on:
  push: ❶
    branches: [main]

# to run this workflow manually from the Actions tab
workflow_dispatch:

jobs:
  sync-to-hub:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
        with:
          fetch-depth: 0
      - name: Add remote
        env: ❷
          HF: ${ secrets.HF } Use the token from Hugging Face
        run: git remote add space <your account>
      - name: Push to hub

```

```
env:  
  HF: ${ secrets.HF }  
run: git push --force <your account>
```

- 1 On push to GitHub, build the project.
- 2 Use the Hugging Face authentication token.

Finally, with the build process set up, you can see the working application in [Figure 2-24](#). Any text passed into the submit box is then summarized using the Hugging Face pretrained model. Later, different models could be swapped out with just a line of code changed, and the entire application and the model would go live. A key takeaway is pretrained models deployed in this MLOps fashion allow for rapid prototyping of what could later become a more sophisticated MLOps system.

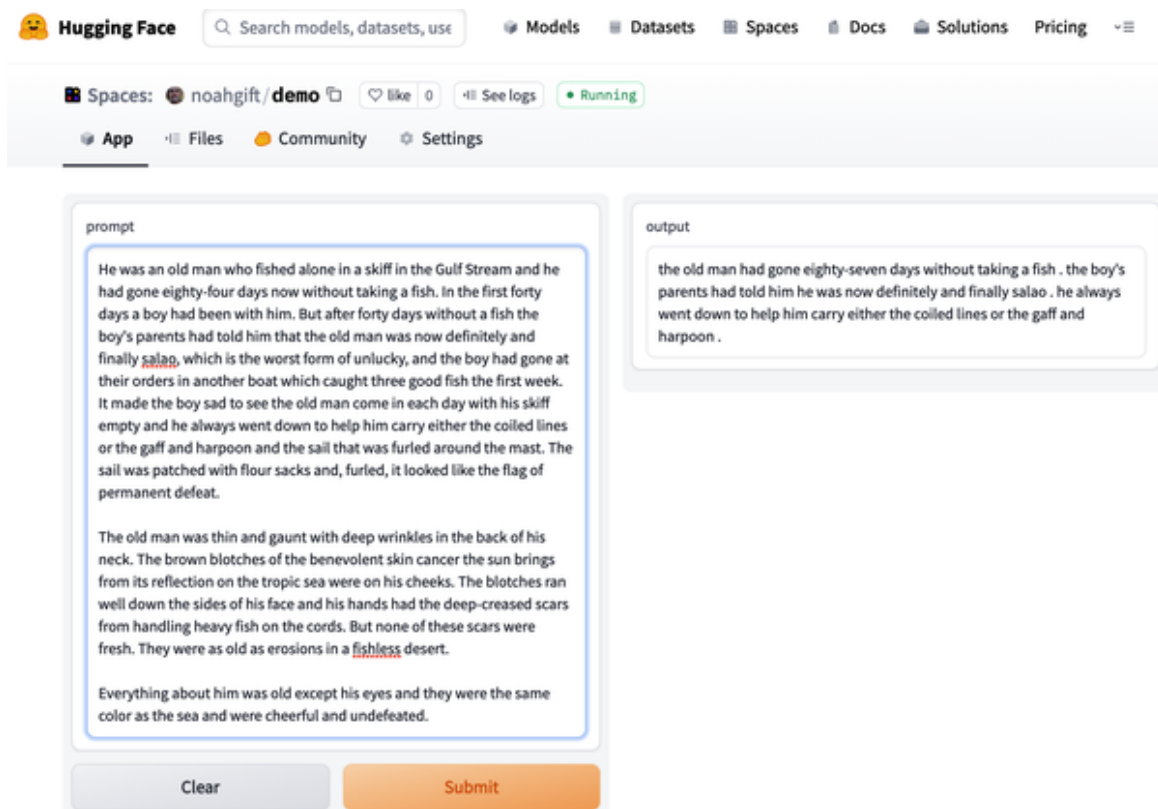


Figure 2-24. Gradio application summarizing *The Old Man and the Sea* text

Flow Automation (CI/CD for ML)

CI/CD is an agile development approach for managing the life cycle of software and continuously deploying robust code updates to production. Using CI/CD, multiple developers can contribute code updates to a shared project repository, conduct automated testing, and have a controlled and continuous deployment process. The outcome is faster time to market using fewer resources and lower software failure rates.

However, the development of ML models and applications brings additional challenges that are not present in traditional software development:

- Multiple people participate in the development (data scientists, data engineers, software developers, ML engineers, and so on), each with different development skills, tools, and practices.
- A version definition extends beyond code and incorporates data source objects, parameters, and multiple artifacts.
- The different data and ML workloads (data preparation, model training, model, data and application testing, and so on) require high scalability and distributed processing using CPUs and GPUs.
- Deploying new versions to production involves merging different data assets and states (for example, tables may change the schema, streams may be partially processed, new features are added and require historical values or imputing missing values, and so on).
- Monitoring and observability are far more complex and less deterministic (as discussed in the previous section about model and data monitoring).

To address the data- and ML-specific challenges, organizations must extend their **CI/CD practices with MLOps automation practices** and ensure that the engineering and data science teams are aligned on the same development methodologies and tools. Here are some practices to follow:

- Data scientists' code can no longer be maintained in giant notebooks but rather must be broken into smaller functional code components (see [“Writing and Maintaining Production ML Code”](#)).
- All data, code, parameters, artifacts, and results must be automatically collected, versioned, and correlated (see [“Tracking and Comparing Experiment Results”](#)).
- Tests should be extensive and cover all data, model, and application aspects (see [“Building and Testing Models for Production”](#)).
- Pipelines must support high-performance, distributed processing, efficient movement, and versioning of data assets across the pipeline.
- Model and data monitoring solutions should provide a feedback loop and be incorporated into the automation flow (see [“Continuous Model and Data Monitoring”](#)).

Figure 2-25 demonstrates a typical CI/CD flow for ML applications. It consists of three main parts:

Development

A user (data scientist, data engineer, software developer, and so on) creates a development branch from the latest code, adds features, and conducts local tests using sample data.

Staging (or integration)

The user requests to merge the new feature into the development branch. At this point, automated test procedures run over the new code with a larger dataset, and distributed or more scalable computation resources. Once the new code passes the tests and is approved, it merges into the development release and may undergo additional stress testing.

Deployment to production

The development release is partially promoted to production (use canary or A/B testing deployment method to process small parts of the actual transactions). Once the new version is verified to work correctly and is compared to the prior release, it is approved and released to production. In case of failures or lower model performance, the system can be rolled back to the previous release.

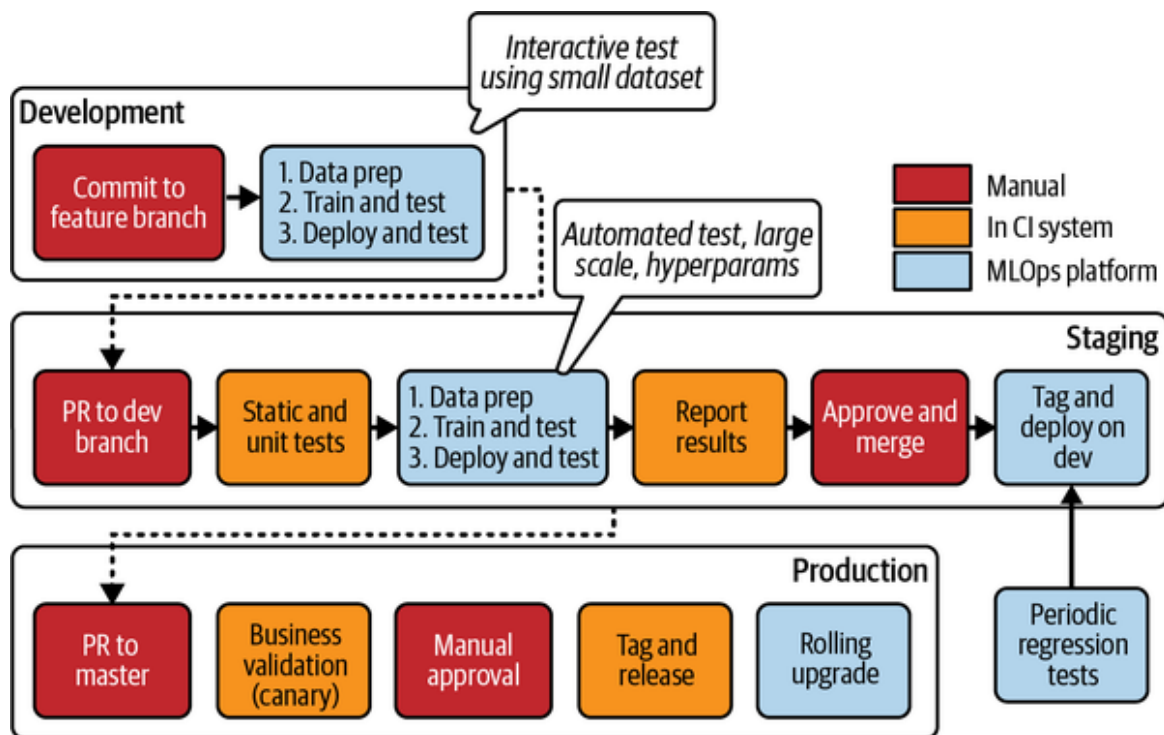


Figure 2-25. Automating the flow from development to production

The MLOps framework must have a tight integration with the source control (Git) and CI/CD framework you choose ([Jenkins](#), [GitHub Actions](#), [Gitlab CI/CD](#), and so on). Various metadata and configuration objects must be stored in the source repository along with the code, data referencing should be abstract and versioned, and reporting and APIs should be integrated to have everything versioned in one place and avoid manual or complex integrations. You can see an example for integration in [Figure 2-26](#).



github-actions bot commented on Jun 10, 2020

...

Pipeline started id=0a9a9b38-fc9c-4577-8e1a-3cd1d5ada45f, commit=400a634
[click here to check progress](#)




github-actions bot commented on Jun 10, 2020

...

Run Results

Workflow 0a9a9b38-fc9c-4577-8e1a-3cd1d5ada45f finished, status=Succeeded
click the hyper links below to see detailed results

uid	start	state	name	results	artifacts
...01d514c1	Jun 10 13:00:39	completed	model-tester	total_tests=15 errors=0 match=14 avg_latency=7307 min_latency=5300 max_latency=8788	latency
...b76d84cd	Jun 10 13:00:28	completed	test	rocauc=0.3333333333333333 avg_precscore=0.3673982494785104 accuracy=0.9333333333333333 f1_score=0.9333333333333333	roc confusion test_set_preds
				best_iteration=2 rocauc=0.9945117845117846	test_set roc

Figure 2-26. A view of automated data and ML test reports inside the version control (Git) system

Conclusion

This chapter provided an in-depth exploration of the stages of MLOps, emphasizing the significance of going beyond just model training. MLOps is a holistic approach that includes four essential components: data collection and preparation, model development and training, ML service deployment, and continuous feedback and monitoring. This approach not only strengthens the technical quality of ML projects, but also ensures they align with and drive business objectives.

In this chapter, we dived into each of these components, covering topics like how to store and ingest data, data preparation, feature stores, model development, distributed training and hyperparameter optimization, and the importance of a production-first mindset that involved maintaining quality

standards, automating the development and testing, and ensuring model robustness and reliability, among many others. Additionally, we explored further strategic activities that enhance MLOps efficiency, like using pretrained models and CI/CD. Now that you've read this chapter, and after practicing the exercises, you're ready to move on to building your first project and advanced MLOps use cases.

Critical Thinking Discussion Questions

Let's review the topics we discussed in this chapter. Can you answer the following questions?

- Why is problem framing the initial suggested step for implementing a project following the MLOps philosophy?
- Name two or three examples of problems an organization could solve more effectively with a heuristic rather than with machine learning?
- How could your organization design an effective data governance strategy that proactively prevents personal identifiable information (PII), bias, or regulatory risk problems?
- How could you use a feature store to decrease a model's computational training time?
- Consider a situation in which your organization faces issues with data drift and another when it encounters problems with concept drift. What would be the most significant impact if not resolved?

Exercises

- Use **MLRun serving** to serve out a **Hugging Face model**.
- Use a feature store on an MLOps platform to train a model that requires data transformation before training.

- Use an experiment tracking technology like MLflow, MLRun, ClearML, SageMaker, or another MLOps platform to train multiple versions of a model and compare the accuracy of numerous runs.
- Use an open source framework like Spark, Dask, Horovod, or Nuclio for workload distribution to perform distributed hyperparameter tuning.
- Write a serving application test using one of the examples covered earlier in the book.

Chapter 3. Getting Started with Your First MLOps Project

If you're itching to get started with building your MLOps project and pipelines, you're at the right chapter. Surprisingly (or not, if you've been carefully reading the book so far), the first step doesn't require a notebook or IDE. Instead, it requires a proper discussion with the decision makers at your company. AI and ML open up new technological frontiers, but outside of academia, they have to be connected to a business use case. This is what makes them valuable to people. Therefore, the first thing to do is to figure out the business use case that justifies your project, as well as the goals and the expected ROI.

Once the business side is clear, it's time to go to your computer. But don't open your notebook just yet. The next step is to plan the ML project. This includes the resources you will need, processes that will run, prototyping the solution, the pipeline structure, and the design. Once these components are approved, it's finally time to develop your ML pipeline. In this chapter, we explore each of these stages in detail.

Identifying the Business Use Case and Goals

AI is transforming businesses and global economies. A **PwC report predicts** that AI could contribute as much as \$15.7 trillion to the global economy by 2030. Moreover, 45% of total economic gains will come from product enhancements, stimulating consumer demand. This is because AI will drive greater product variety, with increased personalization, attractiveness, and affordability over time. AI helps businesses increase their revenue, cut operational costs, improve productivity, and reduce friction. Furthermore, it helps long-term strategic goals, such as increasing competitiveness, reducing risks, growing user base and consumer loyalty, enhancing

employee retention, and enhancing brand value, which will translate to higher profitability and valuation in the long run.

For example, **according to McKinsey**, up to 35% of Amazon's revenue comes from AI-powered recommendations. By introducing the *Frequently Bought Together* recommendations (and other recommendations), Amazon was able to increase the average customer shopping cart size and order amount (upselling and cross-selling), which in turn increases average revenue per customer and Amazon's e-commerce generated revenue per quarter.

Netflix estimates its personalized recommendation engine is worth \$1 billion per year. According to the Netflix team, "consumer research suggests that a typical Netflix member loses interest after perhaps 60 to 90 seconds of choosing, having reviewed ten to twenty titles on one or two screens. After that, the user either finds something of interest or the risk of the user abandoning our service increases substantially." So Netflix executives believe they could lose at least \$1 billion annually if its subscribers aren't offered a proper recommendation.

LATAM Airlines is the largest South American airline carrier. Its business was struck hard by COVID-19. It lost 80% of its revenue and went into chapter 11. The CEO decided to double down on AI to reduce costs and increase profitability. While cutting costs left and right, it significantly grew its data science and MLOps teams and automated almost all parts of its business. Now it is in a much better financial situation. In one of the use cases, the goal was to improve the precision of flight fuel calculation to avoid carrying extra fuel. The project saved LATAM tens of millions of dollars annually and significantly reduced CO₂ emissions (which is also an important environmental benefit). In another use case, it used customer data to deliver custom packages and upsell options and the project resulted in tens of millions of dollars in additional revenue.

Another typical example of the significant cost savings AI can bring is in the use of chatbots. A **report from Juniper Research** has found that adopting chatbots across the retail, banking, and healthcare sectors will realize

business cost savings of \$11 billion annually by 2023, up from an estimated \$6 billion in 2018. When implemented correctly, chatbots address customer service staff scalability needs, boost customer service quality, and collect valuable consumer data.

Governments and nonprofit organizations also use AI. They address universal needs such as national security, improved healthcare, environmental protection, child safety, education, and more, which are not tied to measurable business goals but benefit the entire population. The information in [Table 3-1](#) is from the McKinsey report “[Applying Artificial Intelligence for Social Good](#)” and lists the different use cases. For more information, check out a [summary of the report](#).

Table 3-1. AI for social good use cases

Category	Application
Crisis response	Disease outbreak, migration crises, natural and human-made disasters, search and rescue
Economic empowerment	Agricultural quality and yield, financial inclusion, initiatives for economic growth, labor supply and demand matching
Education	Access and completion of education, maximizing student achievement, teacher and administration productivity
Environment	Animal and plant conservation, climate change and adaptation, energy efficiency and sustainability, land, air, and water conservation
Equality and inclusion	Accessibility and disabilities, exploitation, marginalized communities
Health and hunger	Treatment delivery, prediction and prevention, treatment and long-term care, mental wellness, hunger
Information verification and validation	False news, polarization
Infrastructure	Energy, real estate, transportation, urban planning, water and waste management

Category	Application
Public and social sector	Effective management of public sector, effective management of social sector, fundraising, public finance management, services to citizens
Security and justice	Harm prevention, fair prosecution, policing

AI is also making a difference in the world by tackling significant sustainability challenges. One example Yaron was involved in addressed flash flooding and freshwater availability in a new and innovative way. The Hydroinformatics Institute of Singapore began using thousands of CCTV cameras dispersed throughout this large Asian city as real-time sensors to analyze and measure rainfall. It uses this to generate spatially distributed ground-level rainfall data. Then, the data is fed into complex deep learning algorithms it built and deployed with an automated MLOps pipeline to create accurate, real-time rainfall predictions. It used these rainfall predictions to manage floods by moving floodgates ahead and routing excess rainfall to reservoirs that can store and convert it into drinking water for the population.

Finding the AI Use Case

When defining an AI project, the goal or hypothesis can't remain at the abstract level of wanting to increase top-line revenue or cutting costs. Rather, it should address a specific use case or business problem and have measurable outcomes and ROI. For example, an application can provide purchasing recommendations based on products likely to be purchased together and increase average customer order size by X%, which will positively impact top-level goals like increasing average revenue per customer and top-line revenue.

Use cases will generally fall under one or more of the following categories (in order of complexity and value):

- Intelligent forecasting and data analysis to support various decisions
- Innovative process or service automation to reduce costs and increase productivity
- New products and services that generate incremental value
- Simpler or better user experience, and autonomous systems (bots, robots, cars, and so on)

The same project may address cost reduction and, at the same time, increase revenue or improve user experience. An example would be setting a goal around building an AI model to predict demand for a specific product. The prediction can help retailers ensure they do not run out of stock, which could result in lost revenue. An added benefit is an improved customer experience, which results in happier and more loyal customers who purchase the products they were looking for.

You can read the [McKinsey's state of AI in 2021 report](#) or [10 Ways Artificial Intelligence Helps Business: Uses & Examples](#), which describe the adoption of the most common AI use cases.

Here are some common AI use cases:

Product recommendations

Recommendation systems that offer products to users based on their behavior, purchase history, profile segmentation, and other factors. Think about the “Additional Products You May Like” or “Customers Also Bought” sections that appear when you shop online.

Chatbots

Chatbots that engage with users, offering support, guidance, and assistance across the entire user journey. Engagement can start as early as marketing by answering questions and

providing resources to read, through selling via chatbots, and all the way to customer support and professional services. AI enables chatbots to deliver an accurate, personalized experience (one that doesn't require the user to ask to speak to a human representative after a few unsuccessful attempts to get answers from poorly programmed bots).

Marketing and content

Generative AI can develop marketing strategies and plans, run competitive analyses, create marketing assets like blogs and emails, and even generate images for social media or media campaigns.

Customer sentiment analysis

Measurement of the feelings and opinions expressed by customers online, across websites, forums, social media, and other channels. This information informs business decisions, especially marketing and sales.

Sales forecasting

Calculating the probability of customer purchases, revenue, and conversions. This helps build the sales pipeline and predict quarterly and annual sales performance.

Price optimization

Calculating factors like your previous prices, the quality of your brand, competitor pricing, operating costs, the market situation, and more, to identify the optimal price.

Cybersecurity

Strengthening defenders by detecting and predicting attacks, helping security professionals learn new technologies and

methods, and assisting in building cybersecurity solutions.

Fraud prediction

Analyzing transactions to identify real-time threats and block them before they occur.

Resource optimization

Finding ways to use computational resources more efficiently to cut costs and encourage sustainability.

Demand forecasting

Accurately predicting demand and tracking manufacturing to avoid waste.

Healthcare

Predicting medical conditions and patient deterioration, as well assisting with treatment, medication, and triaging in ICUs.

Predictive maintenance

Detecting malfunctions before they occur to save time and keeping operations running.

The curated list of the [top 100 artificial intelligence use cases](#) by vertical and importance can give you more ideas. [Figure 3-1](#) (from the Smart Insights article “[How AI-Powered Content Marketing Can Fuel Your Business Growth](#)”) illustrates how AI can be used in different marketing use cases.

The best way to start is to organize a brainstorming session with all the different business and technology stakeholders to get use case ideas and validate their feasibility.

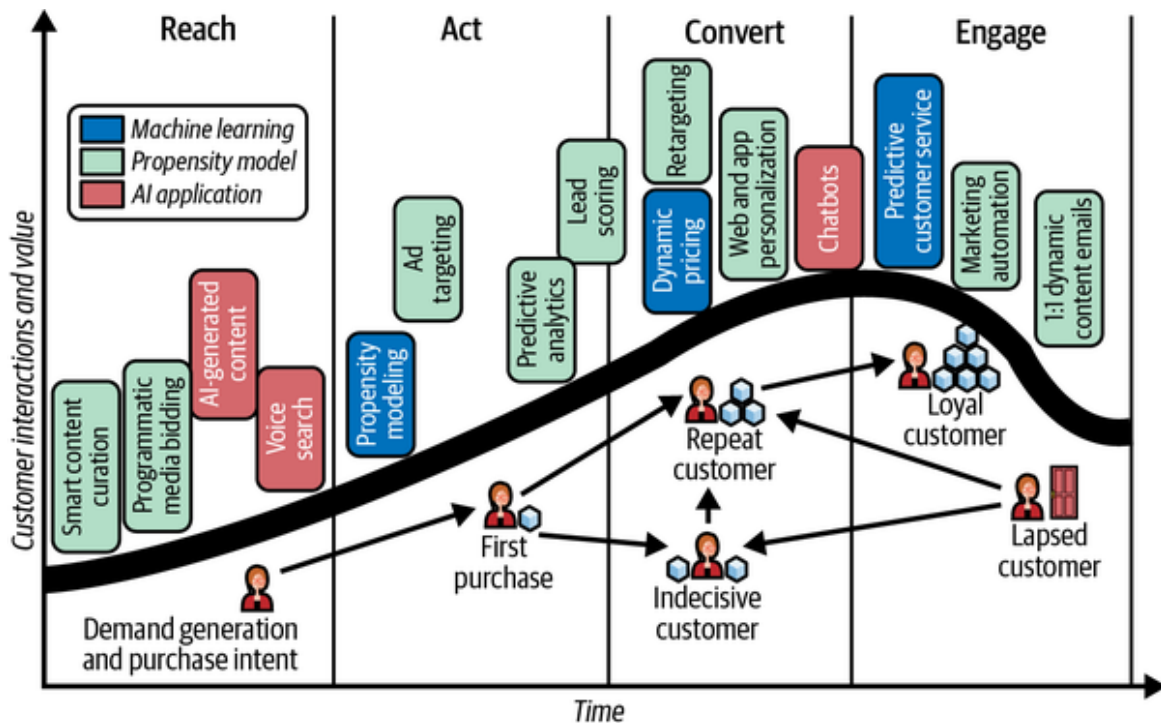


Figure 3-1. How AI can be used in different marketing use cases

Defining Goals and Evaluating the ROI

Although there is an apparent increase in the success ratio of AI projects, many projects result in minimal or no value from their AI investments. One of the reasons is that relatively few projects are deployed into production, mainly due to cultural and organizational challenges; in many cases, they were treated as a bunch of small science projects that failed to realize an ROI. Moreover, production deployments are complex since they usually require integration with existing systems, processes, online data assets, and applications in a scalable and robust fashion.

TIP

One of the ways to increase the success rate is to define achievable and measurable goals. Identifying, prioritizing, and setting goals is a multifunctional team effort that should include business owners, domain experts, data science and engineering teams, and more. This helps ensure alignment with company goals while having the necessary business and domain expertise. AI initiatives may also require effective governance, compliance, ethics, cost, and risk considerations.

To evaluate the ROI of the project, consider the investments and returns, both direct (hard) and indirect (soft).

Investments:

- People (data scientists, data engineers, MLOps, and so on)
- Compute and data infrastructure
- Software licenses and services
- Consultants and training

Returns:

- Cost savings
- Increased revenue
- Time-saving or increased productivity
- Increased competitiveness or user base

It is essential to factor in the uncertainty of the benefits. AI models are likely to have errors—their accuracy is lower than 100%. So it helps to estimate both the error rate and the cost of making mistakes. Also, the fact that you made the correct prediction does not mean your action yields the expected user behavior. For example, you might predict that a user would like the suggested product, but the recommendation was not delivered on time or wasn't visible to the user.

Figure 3-2 illustrates how to calculate the ROI for an ML project.

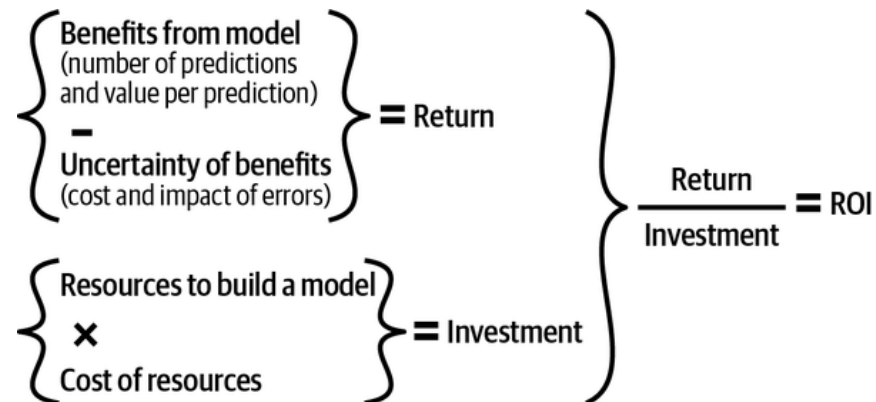


Figure 3-2. Calculating ML project ROI

Another challenge is that while historical data may be accessible, real-world data may behave differently or not be accessible, leading to different results in production deployments. As a result, machine learning-based AI models may deteriorate in performance over time. ROIs and KPIs should be monitored constantly so the value does not decay. Budgeting for MLOps solutions and continuous development and deployment models is also essential.

AI applications also bring possible risks and expose the organization to liability, security vulnerabilities, compliance or legal challenges, and more.

On the upside, many of the investments in AI can be shared across multiple projects. Building common AI platforms, practices, and knowledge sharing (an *AI Factory*) in the organization can significantly impact the ROI.

According to **McKinsey's state of AI in 2021 report**, "The companies seeing the biggest bottom-line impact from AI adoption are more likely to follow core and advanced AI best practices, including MLOps; move their AI work to the cloud; and spend on AI more efficiently and effectively than their peers."

TIP

A significant impact on the ROI of a project is to find the right balance of buy versus build. Many products and services in the market today already incorporate AI, which can help reduce long development cycles and risks. Pretrained or partially trained ML models can save time, resources, and data. MLOps platforms can save significant development overhead and technical debt and allow you to focus on business problems.

How to Build a Successful ML Project

Various surveys indicate that the major impediments to the success of AI in the organization are cultural challenges, such as slow adaptability to change, reengineering of business processes, staff education, data literacy requirements, organizational alignment, and elimination of silos to support business objectives. Many organizations report that direct involvement from C-level executives is essential to the success of AI projects. The *Harvard Business Review* dedicated an article to the vital role of **CEOs in leading a data-driven culture**; McKinsey also writes about **the role of the CEO and MLOps**.

Addressing the cultural challenges is not enough. To achieve a successful AI strategy, you need to redesign all your business processes and tasks around data and AI:

- Build systems and processes for continuously collecting, curating, analyzing, labeling, and maintaining high-quality data. The most significant impediment to effective algorithms is insufficient or poor data.
- Develop effective and reliable algorithms that can be explained; are not biased against particular groups or individuals; and are correctly fit, continuously monitored, and regularly updated using fresh data.
- Integrate a business application's data assets, AI algorithms, software, and user interface into a single project with clear ownership and milestones. Avoid organizational silos.

- Build robust engineering and MLOps practices to continuously develop, test, deploy, and **monitor** end-to-end ML applications.

Approving and Prototyping the Project

Before committing to a project, you need to build and approve your plan. To do so, answer the following questions:

Objective

What are the objectives of this AI use case and are they aligned with the strategic business goals?

KPIs

What will qualify as success and how will it be measured?

Data

Do you have enough data (and labels) to train the models?
Can you obtain the same data in production and inference?
Does the data contain bias? Can you get fresh labeled data for retraining?

People

Who will be responsible (the owner) for the project? Which resources and skills are needed? Are they available or do you need to hire them?

Algorithms

Which AI approach and algorithms are you planning to use?
Can you find an existing model?

Ethics and risks

Are there any ethical or legal issues regarding this use case (privacy, GDPR, bias)? Are any security risks introduced? Can you protect the model from malicious attacks?

Infrastructure

What are the technology and infrastructure challenges and requirements? What are the implementation challenges? What are the expected infrastructure costs?

Continuity

Can you continuously monitor and maintain the application? Can you update the data and model frequently enough? How do you verify the KPIs and ROI once the application is deployed?

After answering these questions and getting approval for the project, the next step is to validate the hypothesis and prototype the application by using rapid prototyping and simulation tools:

- Manually gather data from different sources. Make sure the data you use can, later on, be ingested and prepared continuously at scale.
- Explore the data and look for patterns and signals. Verify which datasets and features are required and which don't add value. Next, try out derived features (date extractions, aggregates, indirect values such as turning zip code numbers to demographics or geolocation data, and so on).
- Prepare the data, train a model using a relevant subset of the data, validate that it performs as expected, and try out different frameworks, algorithms, and existing models.
- Build a prototype application that simulates the end-to-end flow: receives a request, prepares the data, infers using the model, drives actions, integrates with external APIs/systems, logs vital metrics for performance and KPI monitoring, and so on.

Building a prototype can save time, reduce risks, and improve the results. An excellent way to save time and energy is to have project templates with

an application skeleton and best practices. Moving from the prototype to a production ML application can be done in multiple iterations, adding more data, logic, and robustness in every iteration. It is essential to break the project into functional modules from day one (for example, data preparation, **training**, testing, serving, and so on) and define interfaces between the modules. This allows independent development of each module and better collaboration between team members.

You must define the initial prototype's scope, milestones, and objectives. Once it is implemented and the goals can be evaluated, the executive team must approve the productization of the project and allocate the extended required resources for its success.

Scaling and Productizing Projects

ML projects that are designed for production and scale consist of three pipelines:

Data pipeline

Tap into the full-scale historical, operational, and **real-time data sources** and transform the raw data into features for use in the training and inference stages. **Feature stores** can accelerate the development of a data pipeline and enable the reuse of existing data features from other projects.

Model development (CI/CD) pipeline

Automate the process of getting relevant data, data validation, training with different parameters and frameworks, evaluating and testing the model, deploying the inference pipeline, and so on.

Application pipeline

Intercept requests or events; enrich and process data; use the model for inference; apply relevant actions; and monitor various resources, data, model, and KPI metrics.

The pipelines must be designed for continuous development and operations. New versions can be deployed without disrupting the overall application. It is recommended to work in sprints (weekly, monthly). At the end of each sprint, look at the complete application in action. Each sprint provides more functionality or robustness until you reach a deployable and production-quality application. After deploying the application, keep iterating with feature improvements or bug fixes. **Figure 3-3** illustrates the project engineering flow.

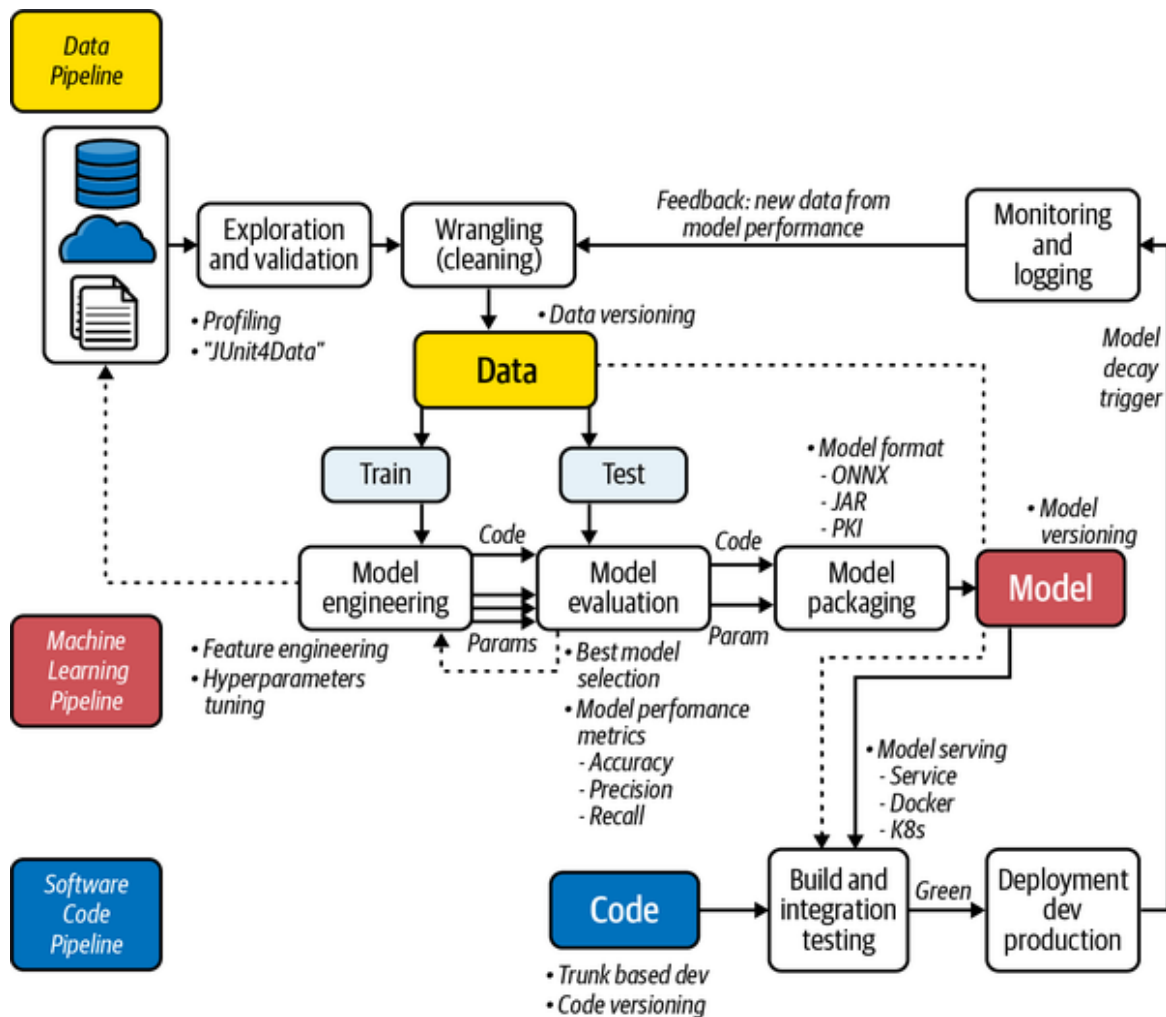


Figure 3-3. MLOps engineering flow (adapted from <https://oreil.ly/cAUKU>)

In most cases, each pipeline uses a different framework and is maintained by different teams with different skills, for example: **Spark**, **Flink**, and **Airflow** for data pipelines, **Kubeflow** for model development, and plain containers or serverless functions for the application pipelines. Having such

a large variety of tools and frameworks creates operational challenges. Furthermore, each framework works with different metadata layouts and APIs, forcing additional glue logic and conversions. Therefore, working with standard metadata and abstractions across frameworks is essential for simplifying and streamlining deployments.

ML projects are developed continuously and collaboratively by different team members. Therefore, a versioned source control system like **Git**, an agile development process, and CI/CD automation are mandatory requirements for a successful outcome.

Project Structure and Lifecycle

ML project is a container for all your work on a particular ML application. Projects host functions, workflows, artifacts, notebooks, features, and configurations. Projects have owners and members with role-based access control, which should define who can access what and how. Project components are as follows (see **Figure 3-4**):

Functions

Code elements along with their package requirements, configuration, metadata, resource definitions, and more.

Workflows

Pipeline (DAG) definitions, like which step comes after which and how parameters are passed between steps.

Artifacts

Metadata and pointers to various data artifacts (files, datasets, models, and more) used in the project.

Notebooks

Jupyter Notebook is used for interactive development, data exploration, and visualization. It is recommended to only

store production code in notebooks since it's harder to test, automate, and track changes in them.

Features

Definitions of feature store features and the data pipelines that generate or retrieve those features (usually called feature sets and feature vectors).

Configurations

Parameters, secrets, build and installation instructions, and more.

Projects should be stored and versioned in a source control system (Git) or archived. Then they can be opened and edited as a project in the different IDEs (Jupyter, **PyCharm**, **VSCode**, and others). This approach enables versioning, collaboration, and CI/CD.

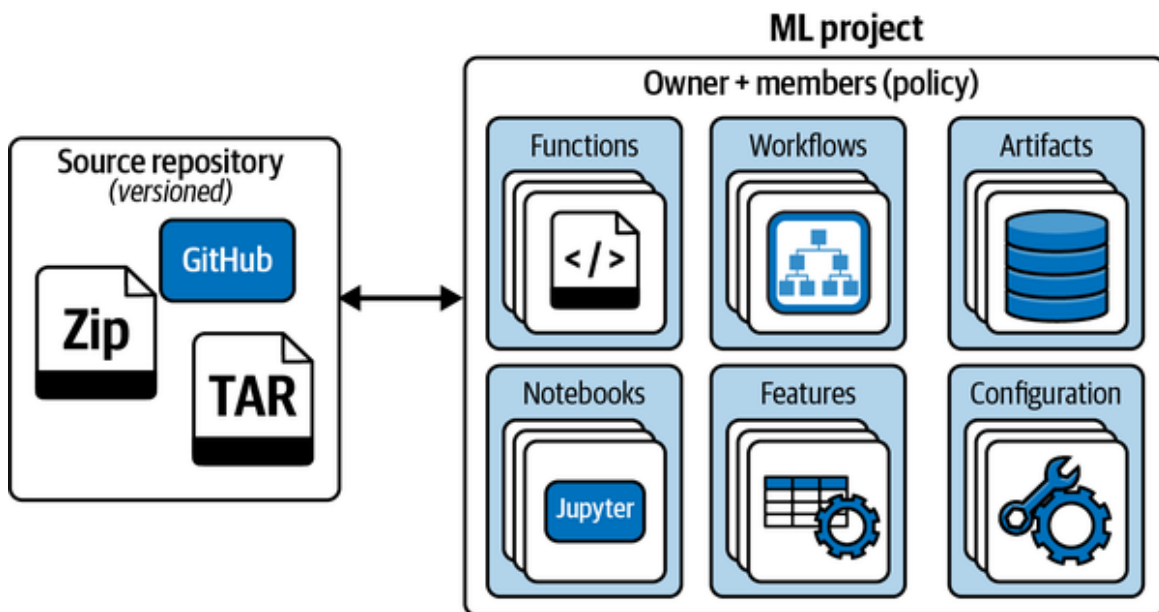


Figure 3-4. ML project components

You can define best practices and project templates for your team. This can simplify creating new projects and helps focus and prioritize existing

projects. **Example 3-1** demonstrates how you can define the project directory structure:

Example 3-1. Project directory layout example

```
my-project      # Parent directory of the project (context)
├─ data         # Project data for local tests or outputs (not tracked by
                # version control)
├─ docs         # Project documentation
├─ src / pkg-name # Project source code (functions, libs, workflows)
├─ tests        # Unit tests (pytest) for the different functions
├─ notebooks    # directory for storing notebooks
├─ README.md    # Project README
├─ requirements.txt # Default Python requirements file (may have per function
                # requirements files)
├─ setup.py     # Python package setup file
├─ LICENSE      # License file
└─ ...
```

MLOps frameworks such as **MLflow** and **MLRun** store additional metadata and configuration files (MLproject, *project.yaml*) in the project directory. This allows loading a project from Git, reconstructing all its objects and configurations automatically, and versioning the configurations and metadata (a.k.a. GitOps).

In a continuous development and integration flow (illustrated in **Figure 3-5**), developers create an ML project and write and test their code and models. Once it is ready, they push the changes into the source control system (Git). Next, the project is loaded into a development cluster to run an additional set of automated tests on larger datasets and a system that resembles the production setup. Then, bug fixes are applied to the code until the project becomes stable and the release is due. Finally, a version tag is assigned to the project when the release is ready, and that version is deployed on the production cluster in a rolling upgrade process.

Exploratory Data Analysis

Exploratory data analysis (EDA) enables an in-depth understanding of your datasets, their quality, and how they influence the target variable. EDA is vital for determining which raw and derived features should be used in the model and for examining your hypotheses. In many cases, EDA requires domain-specific knowledge (or intuition) to determine which variables can be used and how they can impact the model. EDA is usually a manual and interactive process but can use various tools to automate and better visualize the information gathering and analysis.

The EDA process consists of the following steps:

1. Importing relevant datasets (extraction from the overall data)
2. Understanding the data structure and statistics
3. Cleaning and sanitizing
4. Transforming (generating derived features)
5. Feature analysis
6. Cross-feature relationships and correlation analysis
7. Prototyping a model and evaluating feature importance

The process is iterative. You may need to obtain more data from other sources, find that some of the data is useless and doesn't contribute to your model, or find that various transformations yield better results.

The first step is understanding the data shape, types, statistical distribution, categories, missing values, and so on. Next comes data cleansing, removal of useless columns or rows, handling missing values, removing duplicates, and identifying and fixing recording errors.

In many cases, the raw features are not a good indicator and you will need to create derived features that correlate better with the target results. Some examples are:

- Extracting date/time components (hour of the day, day of the week, is_weekend, and so on) from a date/time field
- Value mappings (log scale, binning, encoding, grouping, and so on)
- Aggregation over time or entity (number of clicks in the last hour, total purchases by customer)
- Important features obtained by joining data from a secondary dataset (map zip code to geolocation or social-economic information, map product ID to its price or category, and so on)

Once you have all the features, it's time to analyze and visualize their behavior, histograms, outliers, and categorization. Potentially you can apply transformations to improve the data quality and impact.

The next step is to find interesting relationships that show the influence of one variable on another, preferably on the target. Some features may not have any impact and can be removed while some may need to be transformed to increase their influence. At this stage, you can also evaluate the data for potential bias.

When the features are sanitized and prepared, you can build a basic model or use AutoML tools for prototyping a model. Once you have a model, examine your hypothesis to see if you can predict the target variable and evaluate the importance and necessity of the features you used.

Data and Model Pipeline Development

In the EDA phase, the process was exploratory and interactive. Now it's time to build the data preparation, modeling, and testing to turn them into high-quality, robust, and reusable code. As discussed in [Chapter 2](#), a preferred approach is to create individual Python functions for each stage, give them parameters, record their outputs, and create unit tests. Then, notebooks can execute those functions interactively and visualize their results.

Once the individual functions work, you can create a workflow (directed acyclic graph, or DAG), run the different tasks in an automated pipeline over scalable local or cloud resources, and integrate that workflow into an automated CI/CD process.

A minimal pipeline includes the following steps:

Data preparation

Prepare the training and testing datasets.

Model training

Train the model with the dataset and some parameters.

Evaluation

Evaluate the model against the test dataset and generate various reports and metrics.

Real-world pipelines will have more test and deployment steps and will run the training step with different parameter combinations (hyperparameter tuning).

Example 3-2 demonstrates accepting a DataFrame and required test size, processing the data, and splitting it to train and test datasets.

Example 3-2. Data preparation function code example (partial)

```
def data_preparation(dataset: pd.DataFrame, test_size=0.2):
    """A function which preparation training dataset

    :param dataset: input dataset dataframe
    :param test_size: the amount (%) of data to use for test

    :return train_dataset, test_dataset, label_column
    """
    dataset = clean_df(dataset).dropna(how="any", axis="rows")

    ... additional processing

    train, test = train_test_split(dataset, test_size=test_size)
```



```
return train, test, label_column
...
```

NOTE

Using the feature store is a more powerful and automated way to process data. This will be discussed in the next chapter.

The next step is to train the model with the newly prepared dataset. For example, the following function (see [Example 3-3](#)) accepts the training dataset and various parameters (which will be used for hyperparameter tuning in the following sections), trains the model, and returns the ML model.

Example 3-3. Model trainer function code

```
def train(
    dataset: pd.DataFrame,
    label_column: str = "label",
    n_estimators: int = 100,
    learning_rate: float = 0.1,
    max_depth: int = 3,
    model_name: str = "cancer_classifier",
):
    # Initialize the x & y data
    x = dataset.drop(label_column, axis=1)
    y = dataset[label_column]

    # Initialize the ML model
    model = ensemble.GradientBoostingClassifier(
        n_estimators=n_estimators, learning_rate=learning_rate,
        max_depth=max_depth
    )

    # Train the model
    model.fit(x, y)
    return model
```

The final step is to evaluate the model using the test set. The evaluate function ([Example 3-4](#)) accepts the trained model and training set as inputs and generates various reports and charts (such as ROC curves, feature importance, and so on). Since the evaluate function is generic, it can be

implemented once, stored in a shared repository like MLRun's functions hub, and used in multiple projects. A complete implementation of the training and evaluation functions can be viewed in [MLRun's train and evaluate hub function](#).

Example 3-4. Evaluation function code example (partial)

```
def evaluate(
    model,
    dataset: pd.DataFrame,
    label_columns: Optional[Union[str, List[str]]] = None,
):
    """
    Evaluating a model and generate reports and artifacts.

    :param model:          The model path or object.
    :param dataset:        The dataset to evaluate the model on.
    :param label_columns:  The target label(s) of the column(s) in the
dataset.
                                for Regression or Classification tasks.
    """

    # load the model and dataset
    # run prediction using the test dataset
    # generate plots and reports and log them to the artifacts store
    # update the model, the result metrics and metadata in the model registry
```

Once you have implemented the three functions, you can define an execution pipeline (DAG) to run a function and feed its results to the next step, and so on.

A big part of MLOps is to be able to record all the inputs, metadata, data, and results per experiment (a.k.a. experiment tracking) to make it possible to understand and explain how specific model results were obtained. For example, MLflow and MLRun MLOps frameworks track the execution of the functions and log the data and results. This will be covered in more detail in the following chapters.

Application Pipeline Development

Models bring value only if deployed and integrated into an actual application. For example, an ML application receives relevant data or

requests from users or other services, processes it, and uses it with the model to make predictions and generate some actions. In addition, production applications require monitoring, logging, lifecycle management, and so on. The flow of application, data, model, and monitoring activities is called an *application pipeline*.

There are two types of application pipelines: real-time (or online) pipelines, which constantly accept events or requests and respond immediately, and batch pipelines, which are triggered through an API or at a given schedule. Batch pipelines usually read and process larger datasets on every run.

Real-time application pipelines

Figure 3-6 illustrates a typical real-time application pipeline and its different steps. An application pipeline may receive a request and respond in real time (synchronous pipeline) or it may process the request and write the results to another service or a storage/database system (asynchronous or streaming pipeline).

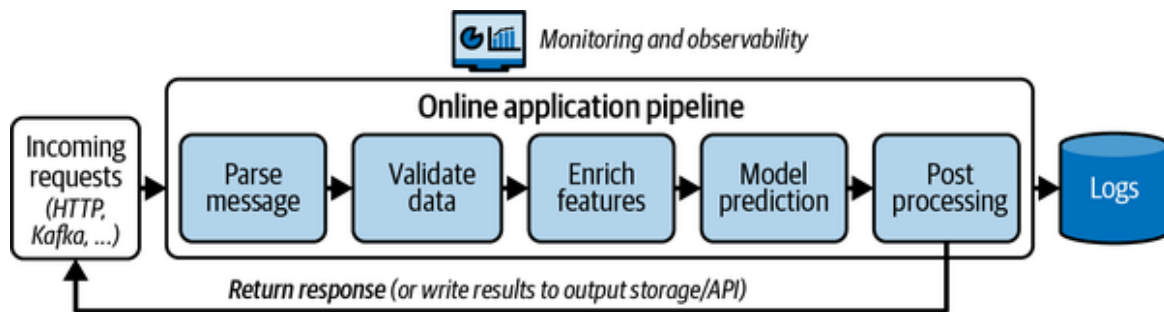


Figure 3-6. Real-time application pipelines

Real-time pipelines can be implemented manually by chaining individual containerized functions or can be automated by using a real-time pipeline framework such as MLRun serving graphs, Apache Beam, or AWS Step Functions. Chapter 6 covers the different options in detail.

NOTE

Some of the data processing logic implemented for the training pipeline is now reimplemented for an event-driven architecture (processing in real time, event by event versus working with large data frames). This engineering overhead can be eliminated when using a feature store. The feature transformation logic is generated automatically for batch and real time from the same abstract definition (called a feature set).

Once the application pipeline and models are deployed, they can be automatically tracked and monitored to identify resource usage, model drift, model performance, and more.

Batch application pipelines

Figure 3-7 illustrates a typical batch (offline) application pipeline and its different steps. For example, an API call or a scheduled event may trigger the batch pipeline. It loads and processes one or more datasets, conducts batch inference and post-processing, and writes the results to the target storage.

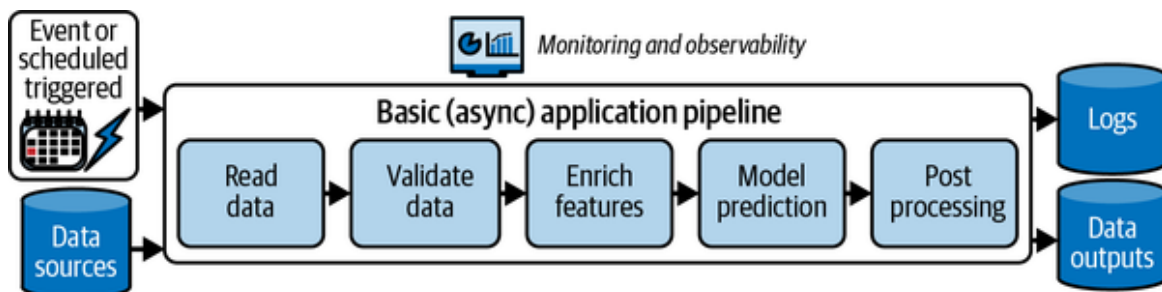


Figure 3-7. Batch application pipelines

The batch application pipeline can be executed with the same pipeline engine used for training.

Scaling and Productizing the Project

After building the model development components (data prep, training, evaluation) and the application pipeline, it's time to add tests and automate and scale the project.

Adding tests

The first step for increasing your ML application's quality is adding unit tests for each component. The common practice is to use `pytest` and place the test files under the `/test` directory. For example, the following code demonstrates an implementation of unit tests for the `data_prep` function:

```
def test_data_preparation_pipeline():
    df = get_data()
    train, test, label = data_preparation(df, 0.2)

    assert label == 'fare_amount'

    # check for expected types
    assert isinstance(train, pd.DataFrame)
    assert isinstance(test, pd.DataFrame)
```

You can build additional unit tests for each function or module and verify that you've properly covered the usage patterns. To run the tests, simply execute `pytest` and point to the `tests` directory. Repeat these steps each time you push changes to the source repository:

```
python -m pytest tests
```

Numerous Python packages test your code for formatting, conformance, quality, coverage, and more; for example, `black`, `isort`, `Flake8`, and `interrogate`. You can add them to your project and execute them before committing the code or as part of the CI process.

For example, formatting the code in `/src` and `/tests` using `black`:

```
python -m black src tests
```

Additional tests (validating the data, model, and APIs) should be implemented as part of the ML pipeline.

ML pipelines and hyperparameter optimization

To find the best model, run the same training code with different parameter combinations (hyperparameter search). However, doing that on your laptop can take time and resources. Instead, define the search options and let it run in parallel in the cloud or over a cluster. Many MLOps frameworks support hyperparameter jobs. If you provide the hyperparameter options (strategy, selection criteria, parallelism level, resources, early stop, and so on), they will execute all the permutations for you and automatically select the best results.

Model development is a multistage process. It requires data ingestion, preparation, validation, training one or more models, and testing and evaluating the models. You can also deploy and test the application pipeline with the newly generated model and data items.

Most MLOps platforms have a way to describe and run a complete workflow (DAG) of steps. Some of the well-known open source workflow execution tools are Airflow and Kubeflow pipelines. There are some CI frameworks like [GitHub Actions](#), [Gitlab CI](#), and [Jenkins](#), which can run simple workflows. But the CI workflow tools lack MLOps capabilities such as handling large datasets, tracking execution and artifacts, running distributed workloads, and others.

Frameworks such as MLRun add the missing MLOps features to the CI/CD system and simplify the way ML pipelines are built and executed. After you have implemented, executed, and tested each function, you only need to place it in a DAG and run it. MLRun works with different underline workflow engines, such as Kubeflow pipelines, and can work with all the CI frameworks.

A CI/CD pipeline for an ML application will likely implement the following steps:

1. Data preparation
2. Model training using hyperparameters and grid search
3. Model evaluation

4. Application pipeline deployment (with the best model)

When the pipeline is executed with MLRun, MLRun tracks the progress and results, and you can view them in the client (IDE, Jupyter, or others) or in the MLRun UI. **Figure 3-8** shows an example of the workflow tracking UI in MLRun.

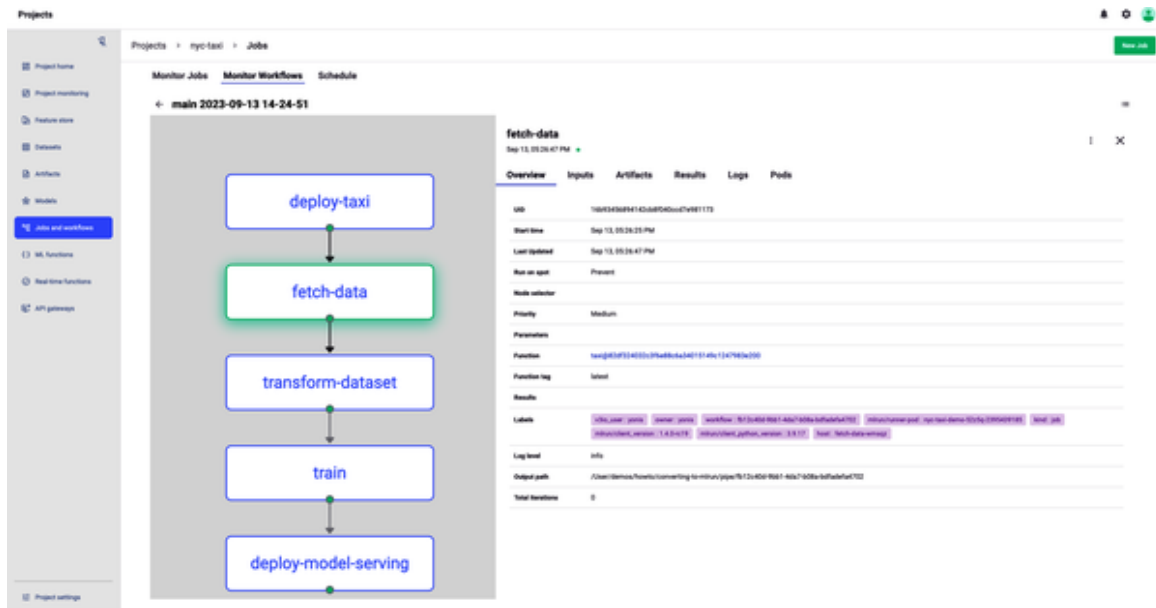


Figure 3-8. MLRun workflow tracking screen

CI/CD and Continuous Operations

You now have all the ingredients: data pipelines, model development pipelines, and application pipelines. However, those components will continuously develop and become enhanced. They require an agile process for monitoring results, pushing updates, testing, and deployment.

Continuously monitoring data and models

In traditional services, we monitor application performance, resource usage, errors, and more. However, it is critical in ML applications to also monitor the data and models (see **“Continuous Model and Data Monitoring”**).

The data and model monitoring layers take metadata collected at the data preparation and model training phase (data types, statistics, and others) and

compare it with production data and metrics. MLRun automates this process. The metadata is automatically recorded with the model and the features at development time and compared in real time or periodically with metadata and behavior of the production data (which is generated automatically by the model serving classes).

You probably want to avoid constantly staring at dashboards for model or data performance problems. Instead, you can define triggering policies and actions. For example, when a certain threshold is reached, a notification can alert the administrator or initiate an automated process for retraining a model or mitigating potential errors.

Integrating with a CI/CD service

CI/CD is the standard approach for building and maintaining modern services in an agile process. [Chapter 2](#) covered REFTO: CI/CD for ML and the differences from traditional CI/CD. The reference project uses MLRun to extend the GitHub Actions CI service to ML and data applications.

As a first step, you need to create scripts that will execute the different tests and verifications. The standard approach is to use a Makefile. In the Makefile, add commands to build, test, and so on. Here are some examples for *make* commands (see the complete Makefile in the project directory):

```
.PHONY: lint
lint: fmt-check flake8 ## Run lint on the code

.PHONY: fmt-check
fmt-check: ## Format and check the code (using black and isort)
    @echo "Running black+isort fmt check..."
    $(PYTHON_INTERPRETER) -m black --check --diff src tests
    $(PYTHON_INTERPRETER) -m isort --check --diff src tests

.PHONY: flake8
flake8: ## Run flake8 lint
    @echo "Running flake8 lint..."
    $(PYTHON_INTERPRETER) -m flake8 src tests

.PHONY: test
test: clean ## Run tests
```



```
$(PYTHON_INTERPRETER) -m pytest -v --capture=no --disable-warnings  
tests
```

With this Makefile, typing `make lint` or `make test` will run the lint and pytest tests.

NOTE

The CI/CD system (such as Jenkins or GitHub Actions) examines your project and searches for CI scripts in a reserved directory and executes them when the code is changed or merged.

In addition to static tests, you should automatically run the ML pipeline. However, since ML pipelines can consume significant computation, you may want the user to explicitly request running the ML pipeline. This can be done by typing a command in the Git pull request (for example `/run`), which will trigger the execution of the ML pipeline on cloud resources and automate the execution, data movement, and tracking.

You can use the same approach to automate deployment, run exhaustive testing, apply governance, and more, while adding more CI scripts and ML pipelines to match them and restricting who can execute which workflow and at what stage (development, staging, production).

Conclusion

In this chapter we dove into the hands-on work and started building our very first MLOps project. Since we believe in a production-first approach, we started with the “why” and discussed various AI use cases and how to identify goals. Our projects should always have the business value in mind, since that is their *raison d’être*. Then, we moved on to the planning phase and defined its phases and which questions to answer to get the project approved. After these steps, we were finally able to move on to the project itself. We covered an entire project from A to Z, at a high level. We

discussed data gathering, data exploring and models, model pipeline development, application pipeline development, scaling and productizing, and CI/CD, including monitoring. Together, these are all the important components and phases of an ML project.

Critical Thinking Discussion Questions

- List three ways AI can provide social and business value.
- Which investments does an ML project require? What are the returns?
- What are the components of an ML project?
- Why do we need hyperparameter optimization?
- What is monitored with CI/CD and why?

Exercises

- Create a mockup plan for an ML project. Answer the questions required to build and approve your plan based on your current stack.
- Write a function code for data preparation, model training, and evaluation you can use in your company.
- Write unit tests for the functions you just wrote in the previous exercise.
- Choose a CI/CD tool and list the steps required to integrate it into your MLOps project.
- Write commands for running your ML project locally and in the cloud.

Chapter 4. Working with Data and Feature Stores

Machine learning takes data and turns it into predictive logic. Data is essential to the process, can come from many sources, and must be processed to make it usable. Therefore, data management and processing are the most critical components of machine learning. Data can originate from different sources:

Files

Data stored in local or cloud files

Data warehouses

Databases hosting historical data transactions

Online databases

SQL, NoSQL, graph, or time series databases hosting up to date transactional or application data

Data streams

Intermediate storage hosting real-time events and messages (for passing data reliably between services)

Online services

Any cloud service that can provide valuable data (this can include social, financial, government, and news services)

Incoming messages

Asynchronous messages and notifications, which can arrive through emails or any other messaging services (Slack,

WhatsApp, Teams)

Source data is processed and stored as features for use in **model training** and model flows. In many cases, features are stored in two storage systems: one for batch access (training, batch prediction, and so on) and one for online retrieval (for real-time or online serving). As a result, there may be two separate data processing pipelines, one using batch processing and the other using real-time (stream) processing.

The data sources and processing logic will likely change over time, resulting in changes to the processed features and the model produced from that data. Therefore, applying versioning to the data, processing logic, and tracking data lineage are critical elements in any MLOps solution.

Delivery of accurate and high-quality production models requires high volumes of data and significant processing power. Processing of production data can be scaled using distributed analytics engines (**Apache Spark**, **Dask**, **Google BigQuery**, and more), stream processing technologies (like **Apache Flink**), or multistage data pipelines.

One of the mechanisms to automate integration with data sources, scalable batch and real-time data processing, data versioning, and feature management is to use a feature store. A feature store is a central hub for producing, sharing, and monitoring features. Feature stores are essential in modern MLOps implementations and will be described in further detail in this chapter.

Data Versioning and Lineage

Models and data products are derived from data. Therefore, collecting metadata and tracing the origin of the data allow better control and governance for data products. Furthermore, if you want to examine a specific version of a data product, you must understand the original data used to produce that product or model.

Data versioning, lineage, and metadata management are a set of essential MLOps practices that address the following:

Data quality

Tracing data through an organization's systems and collecting metadata and lineage information can help identify errors and inconsistencies. This makes it possible to take corrective action and improve data quality.

Model reproducibility and traceability

Access to historical data versions allows us to reproduce model results and can be used for model debugging, troubleshooting, and trying out different parameter sets.

Data governance and auditability

By understanding the origin and history of data, organizations can ensure that data follows expected policies and regulations, tracks sources of errors, and performs audits or investigations.

Compliance

Data lineage can help organizations demonstrate compliance with regulations such as GDPR and HIPAA.

Simpler data management

Metadata and lineage information enables better data discovery, mappings, profiling, integration, and migrations.

Better collaboration

Data versioning and lineage can facilitate cooperation between data scientists and ML engineers by providing a clear and consistent view of the data used in ML models and when handling upgrades.

Dependency tracking

Understanding how each data, parameter, or code change contributes to the results and providing insights into which data or model objects need to change due to data source modification.

How It Works

As shown in **Figure 4-1**, the data generation flow can be abstracted as having a set of data sources and parameters that are used as inputs to a data processing (computation) task that produces a collection of data products or artifacts. The output artifacts can be of different types, files, tables, machine learning models, charts, and so on.

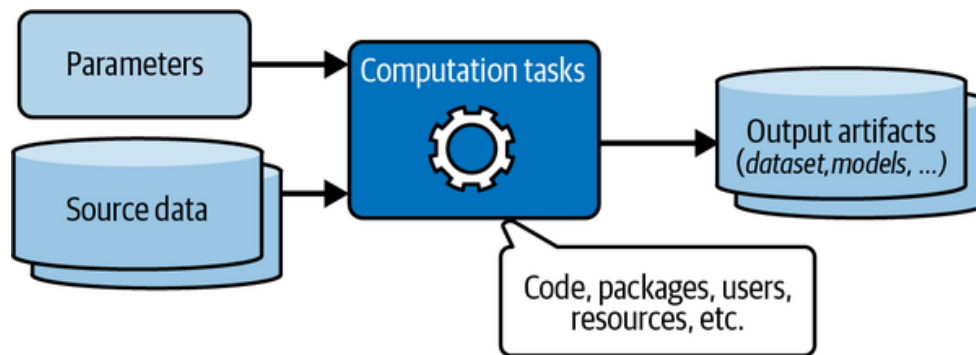


Figure 4-1. Data lineage flow

The data tracking system records the information about the inputs (data sources and versions, parameters) and computation tasks (code, packages, resources, executing user, and more). Then, it adds it as metadata in the output artifacts. The metadata may include additional information like user-provided labels or tags, information about the data structure, schema, statistics, and so on. The metadata is usually not copied to each output artifact but is instead referenced (by a link) to eliminate data duplication.

As shown in **Figure 4-2**, output artifacts from the first task (for example, data preparation) can be used as data inputs to the following tasks (for example, model training, testing).

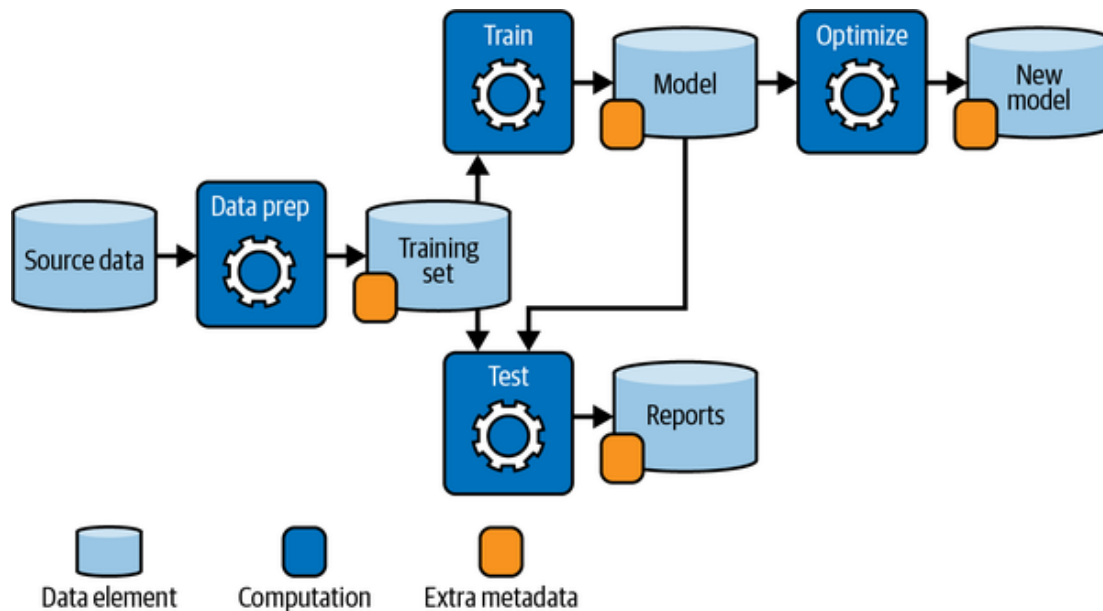


Figure 4-2. Data lineage in a multistage pipeline

When accessing a data product through a user interface or an SDK, the metadata lets us see the exact data sources, parameters, and the full details of the computation task. We can also trace the progress of the data generated in a multistage flow and examine all the additional metadata.

Every time the data processing task runs, a new version of the output artifacts is created (see [Figure 4-3](#)). Each version is marked with a unique version identifier (commit id) and can also be tagged with a meaningful version name, such as *master*, *development*, *staging*, *production*, and so on. This is similar to the Git flow when versioning source code.

Let's assume you are repeatedly running a specific task every hour. It has the same inputs and parameters or you might make small changes that do not change the output data results. This can lead to vast piles of redundant data, and multiple versions will store the same content. Many data versioning solutions implement *content deduplication* to address this challenge.

When an artifact is produced, a cryptographic hash value of the content is calculated (for example, using the MD5 or SHA1 algorithms), which represents the uniqueness of the content. Finally, the hash value is

compared with older versions or is used as an index in the storage system. This way, the content is stored only once.

Since the nature of data versioning solutions is to track various attributes in addition to the source data (code, parameters, users, resources, and more), it must be well integrated with the source control system (Git) and the job or pipeline execution framework. Otherwise, the user must manually glue the frameworks together and provide the reference metadata for recording it along with the data.

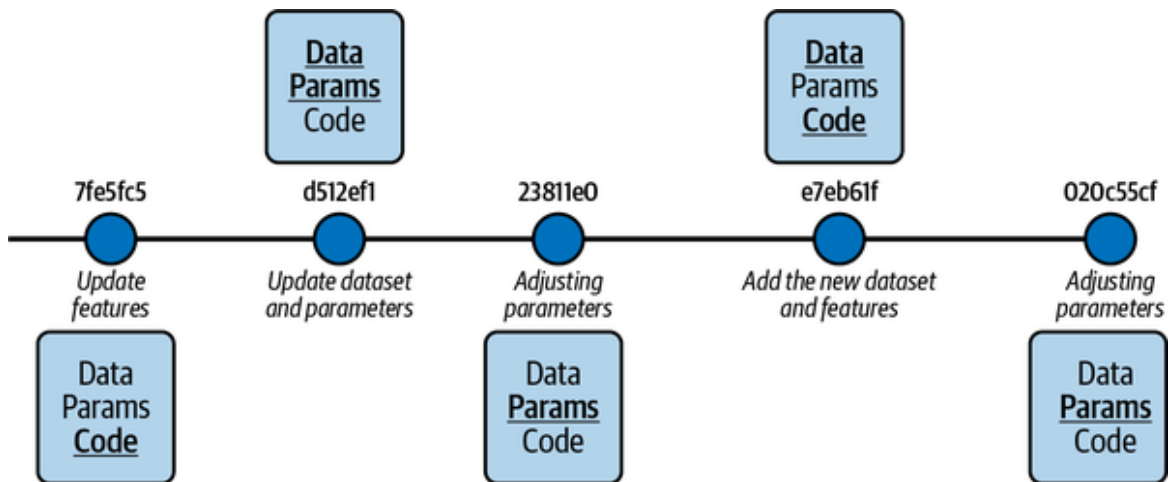


Figure 4-3. How data, parameters, and code changes affect artifact versions

Many frameworks (**MLflow**, **MLRun**, and more) provide a logging API, where the user calls a `log_artifact()` method, which automatically records and versions the new data along with the code and execution metadata. Many might offer an `auto logging` solution that does not require code instrumentation. Instead, it will automatically figure out which data and metadata need to be saved and versioned by understanding the user code and the ML framework's capabilities.

Common ML Data Versioning Tools

A set of open source and commercial frameworks for data versioning exists. This book focuses on explaining and comparing the open source options **DVC**, **Pachyderm**, **MLflow**, and **MLRun**.

Data Version Control

Data Version Control (DVC) started as a data versioning tool for ML and was extended to support basic **ML workflow automation** and experiment management. It takes advantage of the existing software engineering toolset you're already familiar with (Git, your IDE, **CI/CD**, and so on).

DVC works just like Git (with similar commands) but for large file-based datasets and model artifacts. This is its main advantage but also its weakness. DVC stores the data content in files or an object storage (AWS S3, GCS, Azure Blob, and so on) and keeps a reference to those objects in a file (*.dvc*), which is stored in the Git repository.

The following command will add a local model file (*model.pkl*) to the data versioning system:

```
dvc add model.pkl
```

DVC will copy the content of the *model.pkl* file into a new file with a new name (derived from the content *md5* hash value) and place it under the *.dvc/* directory. It also creates a file named *model.pkl.dvc*, which points to that content file. Next, the new metadata file needs to be tracked by Git, the content should be ignored, and the changes should be committed. This is done by typing the following commands:

```
git add model.pkl.dvc .gitignore  
git commit -m "Add raw data"
```

When you want to upload the data to your remote storage, you will need to set up a remote object repository (not shown here) and use the DVC push command:

```
dvc push
```

The data flow is illustrated in **Figure 4-4**.

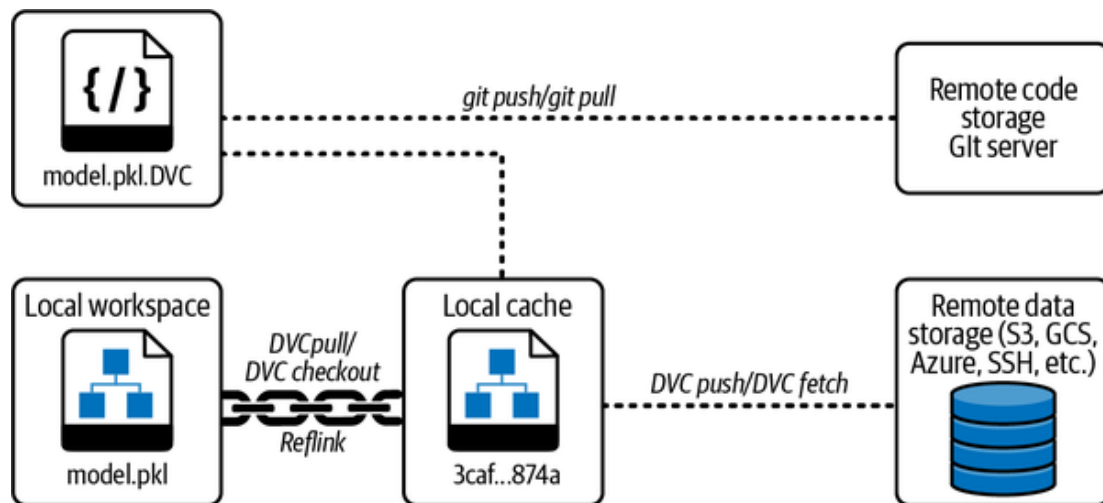


Figure 4-4. DVC data flow (source: DVC)

As you can see from the example, DVC provides reliable synchronization between code and file data objects, but it requires manual configuration and does not store extended metadata about the execution, workflow, parameters, and so on. Instead, DVC handles parameters and results metrics using JSON or YAML files stored and versioned alongside the code.

Users can define workflow stages that wrap an executable (for example, a Python program) and specify which parameters (-p) are the file inputs or dependencies (-d) and outputs (-o) to that executable (see [Example 4-1](#)).

Example 4-1. Adding a workflow step in DVC

```

dvc stage add -n featurize \
    -p featurize.max_features,featurize.ngrams \
    -d src/featurization.py -d data/prepared \
    -o data/features \
    python src/featurization.py data/prepared data/features
  
```

When you run the `dvc repro` command, it will evaluate if the dependencies have changed, execute the required steps, and register the outputs.

DVC does not use an experiment database. It uses Git as the database, and every execution or parameter combination is mapped to a unique Git commit. Furthermore, DVC is focused on local development. Therefore, using it at scale or in a containerized or distributed workflow environment can be challenging and require scripting and manual integrations.

In summary, DVC is an excellent tool for versioning large data artifacts and mapping them to Git commits in a local development environment. In addition, DVC implements data deduplication to reduce the actual storage footprint. On the other hand, DVC is command-line oriented (Git flow) and has limited capabilities for running in production, executing pipelines, and tracking extended attributes and structured data. It also comes with a minimal UI (studio).

Pachyderm

Pachyderm is a data pipeline and versioning tool built on a containerized infrastructure. It provides a versioned file system and allows users to construct multistage pipelines, where each stage runs on a container, accepts input data (as files), and generates output data files.

Pachyderm provides a versioned data repository that can be implemented over object storage (for example, AWS S3, Minio, GCS) and accessed through a file API or the SDK/CLI. Every data commit or change is logged similarly to Git. Data is deduplicated to preserve space.

The Pachyderm data pipeline executes containers and mounts a slice of the repository into the container (under the */pfs/* directory). The container reads files, processes them, and writes the outputs back into the Pachyderm repository.

Example 4-2 shows a simple pipeline definition that takes all the data from the data repository on the master branch, runs the word count logic (using the specified container image), and writes the output to the out repository.

Example 4-2. Pachyderm pipeline example

```
pipeline:
  name: 'count'
description: 'Count the number of lines in a csv file'
input:
  pfs:
    repo: 'data'
    branch: 'master'
    glob: '/'
transform:
  image: alpine:3.14.0
```

```
cmd: ['/bin/sh']  
stdin: ['wc -l /pfs/data/iris.csv > /pfs/out/line_count.txt']
```

Pipelines can be triggered every time the input data changes, and data can be processed incrementally (only new files will be passed into the container process). This can save time and resources.

Pachyderm has a nice user interface for managing pipelines and exploring the data. See [Figure 4-5](#).

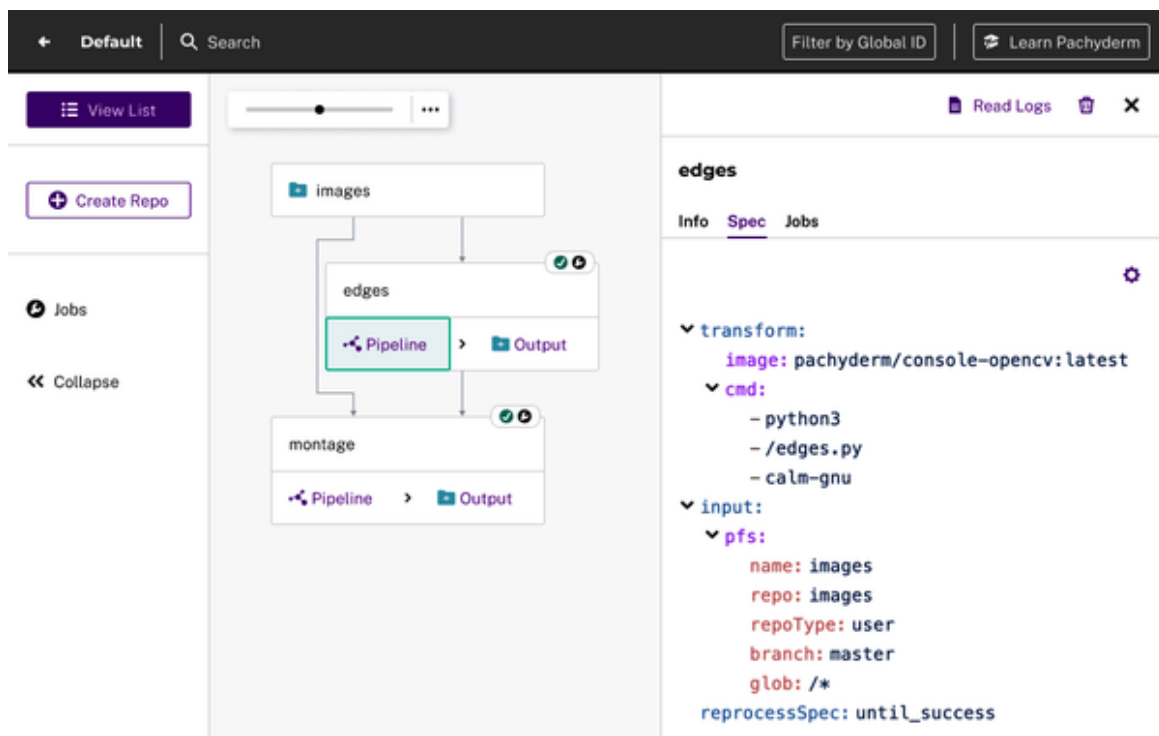


Figure 4-5. Pachyderm user interface

Pachyderm can work with large or continuous structured data sources by breaking the data into smaller CSV or JSON files.

In summary, Pachyderm is an excellent tool for building versioned data pipelines, where the code is simple enough to read and write files. It handles deduplication and incremental processing. However, it requires separate tracking of the source code (runs prebuilt images), execution or experiment parameters, metadata, metrics, and more.

MLflow Tracking

MLflow is an open source platform for managing the end-to-end machine learning lifecycle. One of its core components is MLflow Tracking, which provides an API and UI for logging machine learning runs, their inputs and outputs, and visualizing the results. MLflow Tracking runs are executions of some data science code. Each run records the following information:

Code version

Git commit hash used for the run.

Start and end time

The start and end time of the run.

Source

The name of the file to launch the run, or the project name and entry point for the run if running from an MLflow Project.

Parameters

Key-value input parameters of your choice. Both keys and values are strings.

Metrics

Key-value metrics, where the value is numeric. MLflow records and lets you visualize the metric's full history.

Artifacts

Output files in any format. For example, you can record images (for example, PNGs), models (for example, a pickled scikit-learn model), and data files (for example, a Parquet file) as artifacts.

MLflow Tracking is not a complete data versioning solution since it doesn't support features such as data lineage (recording data inputs and which data

was used to create a new data item) or deduplication. However, it enables logging and indexing the data outputs of every run along with the source code, parameters, and some execution details. MLflow can be manually integrated with other tools like DVC to track data and experiments.

MLflow's advantage is tracking the data outputs with additional metadata about the code and parameters and visualizing or comparing them in a graphical UI. However, this is not free. The user code needs to be instrumented with the MLflow Tracking code.

Example 4-3 demonstrates a partial code snippet that tracks a run using the MLflow API. First, the command line arguments are parsed manually and the input data is passed as a string URL, just like any other parameter. Then, the loading and transformation of the data are done manually.

After the logic (data preparation, training, and so on) is executed, the user logs the tags, input parameters, output metrics, and data artifacts (dataset and model) using the MLflow log commands.

Example 4-3. MLflow Tracking code example

```
if __name__ == "__main__":
    # parse the input parameters
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", help="input data path", type=str)
    parser.add_argument('--dropout', type=float, default=0.0, help='dropout
ratio')
    parser.add_argument("--lr", type=float, default=0.001, help='learning rate')
    args = parser.parse_args()

    # Read the csv file
    try:
        data = pd.read_csv(args.data)
    except Exception as e:
        raise ValueError(f"Unable to read the training CSV, {e}")

    # additional initialization code ...

    with mlflow.start_run():

        # training code ...

        # log experiment tags, parameters and result metrics
        mlflow.set_tag("framework", "sklearn")
```

```

mlflow.log_param("dropout", args.dropout)
mlflow.log_param("lr", args.lr)
mlflow.log_metric("rmse", rmse)
mlflow.log_metric("r2", r2)
mlflow.log_metric("mae", mae)

# log data and model artifacts
mlflow.log_artifacts(out_data_path, "output_data")
mlflow.sklearn.log_model(model, "model",
                          registered_model_name="ElasticnetWineModel")

```

MLflow sends the run information to the tracking server and stores the data elements in local files or remote objects (for example, in S3). The run information can be viewed or compared in the MLflow user interface (see Figure 4-6).

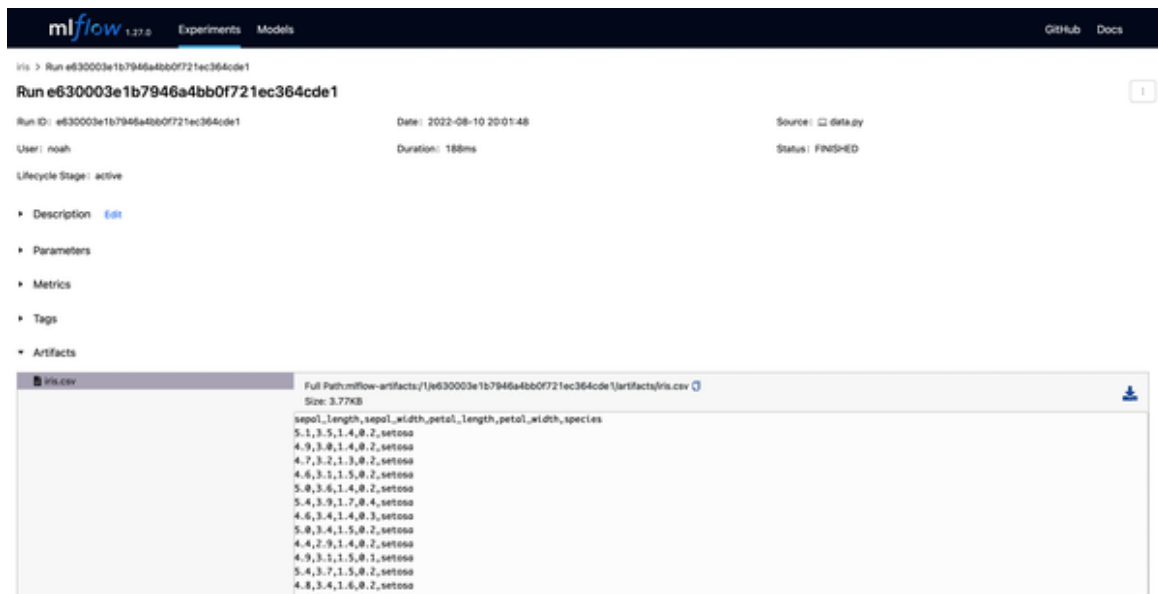


Figure 4-6. MLflow user interface

MLflow does not manage or version data objects. Run is the primary element, and you cannot directly access or search data objects and artifacts. In addition, there is no lineage tracking, which means there is no tracking of which data objects were used to produce a new data object or artifact. When you run a pipeline, you cannot see the artifacts from the different steps in one place or chain output from one stage to the input of the next step.

With MLflow, the storage capacity can become significant since every run saves the outputs in a new file directory, even when nothing has changed.

There is no data deduplication like in the other frameworks.

In summary, MLflow tracking is an excellent tool for tracking and comparing ML experiment results in a development environment. In addition, MLflow is easy to install and use. However, it is not a data tracking or versioning system and may require significant storage capacity. Furthermore, MLflow requires developers to add custom code and MLOps teams to add glue logic to fit into production **deployments** and CI/CD workflows.

MLRun

MLRun is an open source MLOps orchestration framework with multiple sub-components to handle the complete ML lifecycle. Data objects are first-class citizens in MLRun and are well integrated with the other components to provide seamless experience and automation.

Whereas most frameworks manage file data objects, MLRun supports a variety of data objects (data stores, items/files, datasets, streams, models, feature sets, feature vectors, charts, and more), each with unique metadata, actions, and viewers.

Every object in MLRun has a type, a unique version ID, tags (named versions like development, production, and so on), user-defined labels (for grouping and searching across objects), and relations to other objects, and it is a project member. For example, a run object has links to the source and output data objects and to function (code) objects, forming a graph of relations.

Figure 4-7 shows the run screen with information tabs for general and code attributes, data input objects, data/artifact output objects, result metrics, auto-collected logs, and so on. Users can view the information from different perspectives. For example, look at all the datasets in the project (regardless of which run generated them).



Figure 4-7. MLRun job run user interface

MLRun data objects and artifacts carry detailed metadata, including information on how they were produced (by whom, when, code, framework, and so on), which data sources were used to create them, and type-specific attributes such as schema, statistics, preview, and more. The metadata is auto-generated, which provides better observability and eliminates the need for additional glue logic.

NOTE

MLFlow users can continue using MLFlow for tracking APIs, and MLRun will automatically register the logged data, metadata, and models as production artifacts along with additional operational metadata and context.

MLRun provides an extensive API/SDK for tracking and searching across data and experiments. However, the real power is that it can deliver most of the features and automation without requiring additional coding.

Example 4-4 accepts input data and parameters and generates output data and results. Note that, unlike the previous examples, the code doesn't include argument parsing, data loading, conversion, logging, and so on.

Example 4-4. MLRun code example

```
def data_preparation(dataset: pd.DataFrame, test_size=0.2):
    # preform processing on the dataset
    dataset = clean_df(dataset).dropna(how="any", axis="rows")
    dataset = dataset.drop(columns=["key", "pickup_datetime"])
    train, test = train_test_split(dataset, test_size=test_size)
    return train, test, "fare_amount"
```

When executing the function and specifying the input data object URL or path (a file, a remote object, or a complex type), it is automatically loaded into the function. For example, using AWS *boto* drivers to access S3 objects or BigQuery drivers to access a BigQuery table. Then the data is converted to the accepting format (DataFrame) and injected into the user code.

MLRun can auto-detect the returned value type (for example, `train` and `test` are of type DataFrame) and store it in the best form, along with auto-generated metadata, links to the job details and data input objects, and versioning information. If the data repeats itself, it is deduplicated and stored only once to preserve storage space.

Data objects have type-specific visualized in the UI and client SDK regardless of how and where they are stored; for example, tabular formats with table metadata (schema, stats, and more) for datasets or interactive graphics for chart objects (see Figures 4-8 and 4-9).

In summary, MLRun is a complete MLOps orchestration framework with a significant focus on data management, movement, versioning, and automation. In addition, MLRun has a rich object model that covers different types of data and execution objects (functions, runs, workflows, and more), how they are related, and how they are used. MLRun focuses on abstraction and automation to reduce development and deployment efforts. However, MLRun is not a general data management and versioning solution, and its value is maximized when used in the context of MLOps.

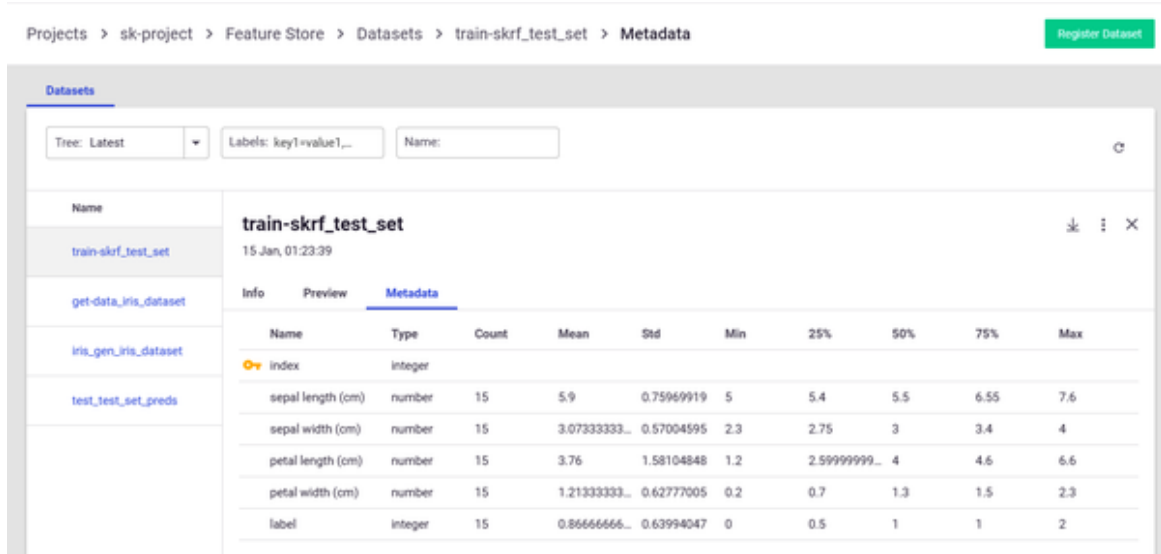


Figure 4-8. View a dataset artifact in MLRun (with autogenerated preview, schema, and statistics)

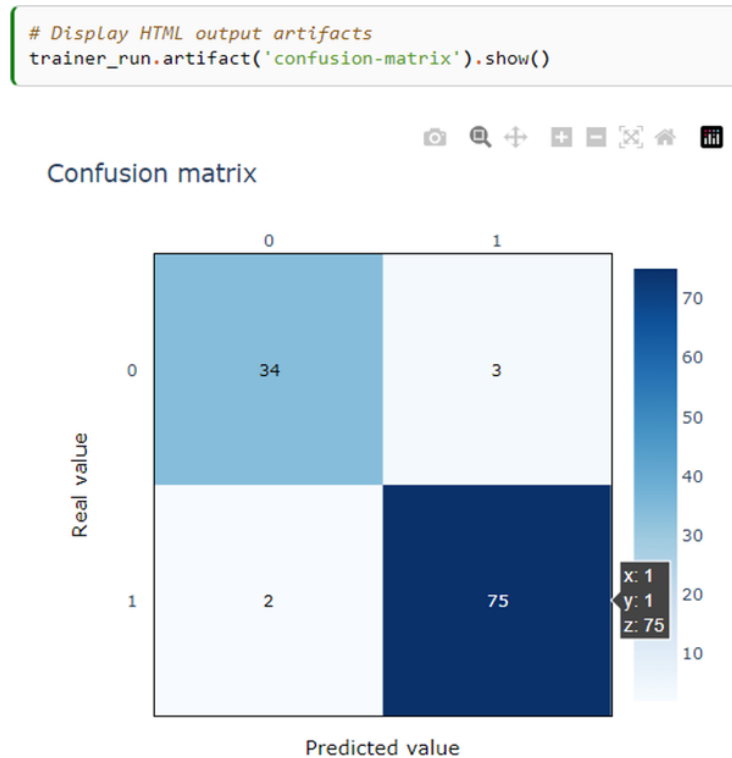


Figure 4-9. Visualize an interactive chart artifact using MLRun's SDK (in Jupyter)

Other Frameworks

Some tools, such as **Delta Lake** and **lakeFS**, handle data lake versioning. However, those tools are not focused on the ML lifecycle and may require integration to make them useful for MLOps.

Cloud vendors provide solutions that are usually tightly bound to their internal services. For example, see [Amazon SageMaker ML Lineage Tracking](#) and [Azure ML datasets](#).

Data Preparation and Analysis at Scale

Data processing is used extensively across the data, ML, and application pipelines. When working with production data, there is a need to support more extensive scale and performance, and, in some cases, handle data as it arrives in real time.

Practices that work during interactive development, for example, storing the data in a CSV file and reading it into the notebook, don't work with gigabytes or terabytes of data. They require distributed or parallel data processing approaches.

The general architecture for distributed data processing is the same, with differences in how data is distributed and collected and which APIs they use. For example, the data is partitioned across multiple computer nodes, the processing requests or queries arrive at one or more nodes for local processing, and the results are collected and merged for a single answer. In addition, complex queries may shuffle data between nodes or execute multiple processing and movement steps.

Figure 4-10 demonstrates how distributed data processing works using the map-reduce approach for counting words in a document.

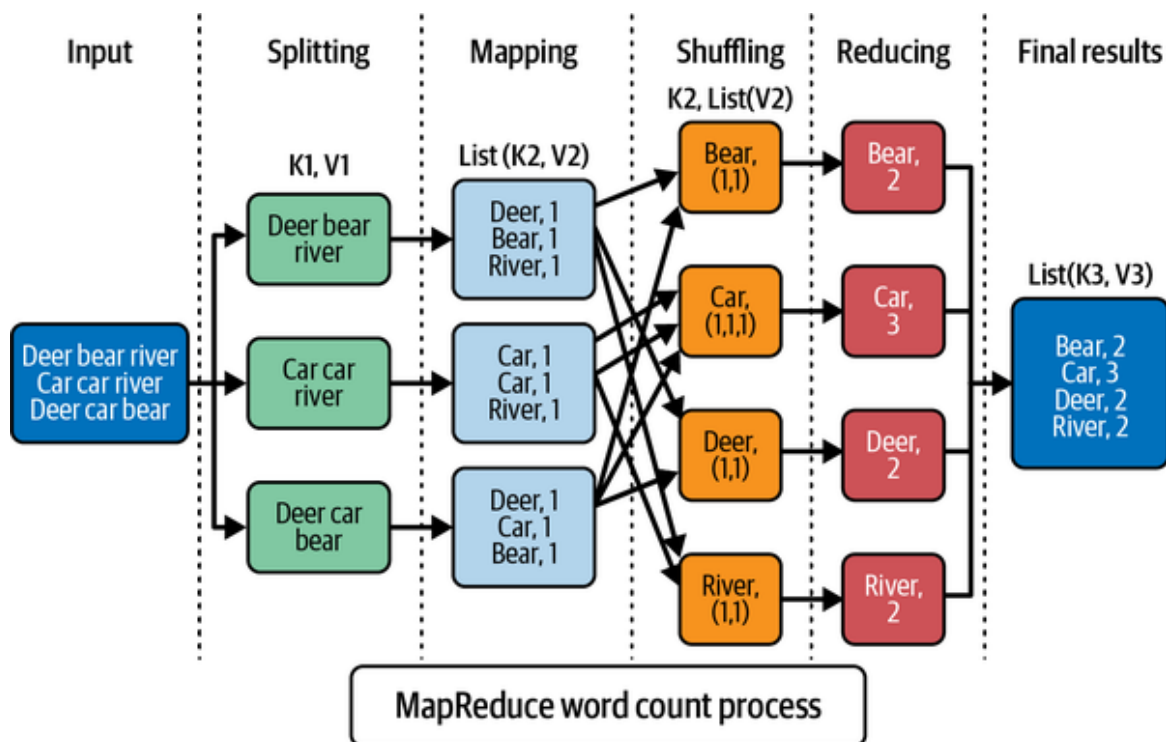


Figure 4-10. Distributed word counting with map-reduce architecture (source: O'Reilly)

Structured and Unstructured Data Transformations

Data can be *structured*, meaning it conforms to a specific format or structure and often has a predefined schema or data model. Structured data can be a database table or files with a structured layout (for example, CSV, Excel, JSON, ML, Parquet). However, most of the world's data is *unstructured*, usually more complex, and more difficult to process than structured data. This includes free text, logs, web pages, images, video, and audio.

Here are some examples of analytic transformations that can be performed on structured data:

Filtering

Selecting a subset of the data based on certain criteria, such as a specific date range or specific values in a column.

Sorting

Ordering the data based on one or more columns, such as sorting by date or by a specific value.

Grouping

Organizing the data into groups based on one or more columns, such as grouping by product category or by city.

Aggregation

Calculating summary statistics, such as count, sum, average, maximum, and standard deviation, for one or more columns.

Joining

Combining data from multiple tables or datasets based on common columns, such as joining a table of sales data with a table of customer data.

Mapping

Mapping values from one or more columns to new column values using user-defined operations or code. Stateful mapping can calculate new values based on original values and accumulated states from older entries (for example, time passed from the last login).

Time series analysis

Analyzing or aggregating data over time, such as identifying trends, patterns, or anomalies.

The following techniques can be used to process unstructured data or convert it to structured data:

Text mining

Using NLP techniques to extract meaning and insights from text data. Text mining can extract information such as sentiment, entities, and topics from text data.

Computer vision

Using image and video processing techniques to extract information from visual data. Computer vision can extract information such as object recognition, facial recognition, and image classification.

Audio and speech recognition

Using speech-to-text and audio processing techniques to extract meaning and insights from audio data. Audio and speech recognition can extract information such as speech-to-text, sentiment, and speaker identification.

Data extraction

Using techniques such as web scraping and data extraction to pull out structured data from unstructured data sources.

Various ML methods can be used to transform raw data into more meaningful data, for example:

Clustering

Grouping similar data points based on certain characteristics, such as customers with similar purchasing habits

Dimensionality reduction

Reducing the number of features in a dataset to make it easier to analyze or visualize

Regression and classification

Predicting a class or a value based on other data features

Imputing

Determining the expected value based on other data points in case of missing data

Embedding

Representing a sequence of text, audio, or an image as a numeric vector that preserves the semantic relationships or contextual characteristics.

Distributed Data Processing Architectures

Data processing architectures can be broken into three main categories:

Interactive

A request or an update arrives, is processed, and a response is returned immediately; for example, SQL and NoSQL databases, data warehouses, key/value stores, graph databases, time series databases, and cloud services.

Batch

A job is started on a request or a scheduled time, data is fetched and processed, and the results are written to the target storage after completion. Batch jobs usually take longer to process. Example frameworks for batch data processing include **Hadoop**, **Spark**, and **Dask**.

Streaming

Continuous processing of incoming requests or chunks of data and writing the results in real time to a target storage or message queue.

Batch processing is usually more efficient for processing large data quantities. However, interactive and stream data processing deliver faster responses with shorter delays. In addition, building data stream processing pipelines is usually more complex than batch jobs.

Some frameworks like Spark may support different processing methods (interactive, batch, streaming), but they will usually be more optimal only in one of the processing methods.

Interactive Data Processing

Interactive systems are expected to respond immediately, so the requesting client or interactive dashboard will not need to wait. Furthermore, production services may depend on the reliability and robustness of the results. This is why interactive systems have simple APIs with limited data operations. In some cases, interactive systems provide mechanisms to define custom logic through stored procedures and user-defined functions (UDFs).

The main difference between the types of interactive data systems is how they index and store data to minimize response retrieval time. For example, NoSQL, in-memory, and key/value stores are optimized for retrieval by an index key (such as a *user id*, *product id*, and so on). The data is divided by the key (or a crypto hash or the key) and stored in different nodes. When a request arrives, it is passed to the specific node, which manages the data for that key (user, product, and so on) and can quickly calculate and retrieve the answer. On the other hand, complex or cross-key calculations require coordination between all the nodes and take much longer.

Analytical databases and data warehouses are designed to traverse many records with different index key values. They organize the data in columns (by field) and use various columnar compression technologies and filtering and hinting tricks (like bloom filtering) to skip data blocks.

Other systems like time series or graph databases have more advanced data layouts and search strategies that combine multidimensional indexes and columnar compression. For example, accessing the time series metric object

by the metric key (name) and using columnar compression technologies to scan or aggregate the individual values (by time).

Many interactive systems use the SQL language or SQL-like semantics to process data.

Some subcategories of notable data systems are listed in [Table 4-1](#).

Table 4-1. Data systems categories and descriptions

Category	Description
Relational	Store structured data, access through SQL command. Examples include MySQL , PostgreSQL , Oracle , and Microsoft SQL Server .
NoSQL	Examples include MongoDB , Cassandra , Redis , Elasticsearch , AWS DynamoDB , Google BigTable , and nontabular databases.
Time series	Store and query time series data. Examples include InfluxDB , Prometheus , and TimescaleDB .
Graph	Store and query data in a graph format. Examples include Neo4j and Titan .
Vector	A vector database indexes and stores high-dimensional vector embeddings for fast retrieval and similarity search. Examples include Chroma , Pinecone , Milvus , Weaviate , and Pgvector .

Analytical systems usually traverse and process larger datasets. As a result, they support more extensive transformations (filtering, grouping, joining, aggregating, mapping, and so on) and user-defined functions. In addition,

some can process and aggregate data from other databases or data stored in files. For example, solutions like Spark SQL or PrestoDB have connectors to many data sources and can process queries that span many datasets and are stored in different systems.

One of the most popular distributed SQL-based analytical engines is PrestoDB and its follow-on project, **Trino**. Presto was initially developed by Facebook and contributed to open source. Later, it was forked into projects like the Trino and commercial products such as **Amazon Athena** cloud service. Trino has a long list of **data connectors**.

Figure 4-11 illustrates Presto and Trino architectures. Queries arrive through HTTP requests, are parsed, and are broken by the planner and the scheduler into smaller tasks that are processed and merged by the individual workers.

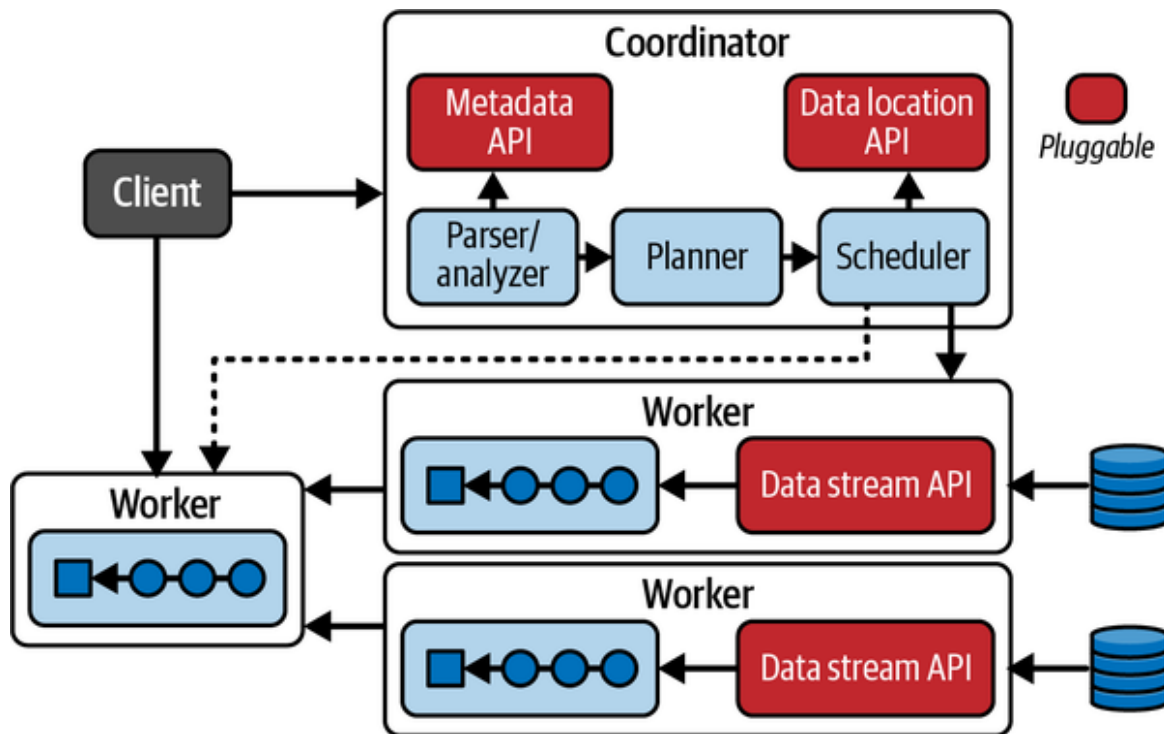


Figure 4-11. PrestoDB and Trino architecture (source: Presto)

Batch Data Processing

Batch data processing is used when there is a need to process large amounts of data and run a sequence of data transformations, and the processing time is less of a concern. In batch processing, the data is read and broken into chunks passed to multiple workers for processing. Once the result is ready, it is written to the target system. Batch processing is often used to process large amounts of historical data and generate the dataset for training ML models.

One of the best known batch data processing frameworks was Apache Hadoop, an open source software framework for distributed storage and large-scale processing of data-intensive tasks. Hadoop was initially developed by Yahoo! engineers and was based on the *MapReduce* programming model, which consists of two main functions: Map and Reduce. The Map function takes an input dataset and processes it into a set of intermediate key-value pairs, which are then grouped by key and processed by the Reduce function to produce the final output.

Hadoop has since been replaced with more modern and cloud-native architectures based on cloud object storage, containerized infrastructure, and computation frameworks such as [Spark](#), [Flink](#), [Beam](#), [Dask](#), and others.

An everyday use for batch processing is found in ETL tasks. ETL refers to extracting data from multiple sources, transforming it, and loading it into a target database, data warehouse, or data lake. ETL is a crucial step in the data integration process, as it allows organizations to extract, clean, and transform data from multiple sources into a single, centralized repository.

Batch-processing pipelines may be complex and have multiple steps and dependencies. [Apache Airflow](#) is one of the most popular open source frameworks for authoring, scheduling, and monitoring batch data pipelines.

Airflow was initially developed by Airbnb and is now maintained by the Apache Software Foundation. It provides a simple and easy-to-use interface for defining workflows as DAGs of tasks, where each task represents an individual processing step. The tasks can be written in Python and run in various environments, including locally, over Kubernetes, or in the cloud.

Airflow also provides a web-based user interface (see [Figure 4-12](#)) for managing and **monitoring workflows**, including the ability to see the status of each task, retry failed tasks, and manually trigger or schedule tasks. It also includes features for managing and organizing workflows, such as defining dependencies between tasks and setting up task retry logic.



Figure 4-12. Airflow user interface

Example 4-5 is an example of Python code that can be used to create a DAG in Apache Airflow that reads data from a CSV file, processes it, and writes it to a destination.

Example 4-5. Airflow data pipeline code example

```
import csv
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime, timedelta

def process_data(**kwargs):
    ti = kwargs['ti']
    input_file = ti.xcom_pull(task_ids='read_file')
    processed_data = do_data_processing(input_file)
    return processed_data

def do_data_processing(input_file):
    # Placeholder function that performs data processing
```

```

    processed_data = input_file
    return processed_data

def read_csv_file(file_path):
    with open(file_path, 'r') as file:
        reader = csv.reader(file)
        return list(reader)

def write_csv_file(file_path, data):
    with open(file_path, 'w') as file:
        writer = csv.writer(file)
        writer.writerows(data)

default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2021, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'data_processing_dag',
    default_args=default_args,
    description='A DAG that reads data from a CSV file, processes it',
    ', and writes it to a destination',
    schedule_interval=timedelta(hours=1),
)

read_file = PythonOperator(
    task_id='read_file',
    python_callable=lambda: read_csv_file('/path/to/input_file.csv'),
    xcom_push=True,
    dag=dag,
)

process_data = PythonOperator(
    task_id='process_data',
    python_callable=process_data,
    provide_context=True,
    dag=dag,
)

write_file = PythonOperator(
    task_id='write_file',
    python_callable=lambda: write_csv_file('/path/to/output_file.csv',

```

```
ti.xcom_pull(task_ids='process_data')),
    provide_context=True,
    dag=dag,
)
```

```
read_file >> process_data >> write_file
```

There are several cloud-based batch data pipeline services such as **AWS Glue**, **Google Cloud Composer** (based on Airflow), and **Azure Data Factory**.

One of the disadvantages of Hadoop or other batch pipelines is the need to read data from disk, process it, and write it again to disk at every step. However, frameworks such as **Spark** and **Dask** know how to compile the processing pipeline into an optimal graph where tasks are done in memory where possible, which minimizes the IO to disk and maximizes performance.

Example 4-6 demonstrates a Spark code that reads a CSV file, processes the data, and writes the result into a target file.

Example 4-6. PySpark data pipeline code example

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("SimpleBatchProcessing").getOrCreate()

# Load a CSV file into a Spark DataFrame
df = spark.read.csv("/path/to/input_file.csv", header=True, inferSchema=True)

# Perform some data processing on the DataFrame
processed_df = df.groupBy("column_name").agg({"column_name": "mean"})

# Write the processed DataFrame to a new CSV file
processed_df.write.csv("/path/to/output_file.csv", header=True)

# Stop the Spark session
spark.stop()
```

Example 4-7 shows the same task, implemented using Dask. The advantage of Dask is that the operations are very similar to Python pandas, which is a

tremendous advantage for data scientists. However, Spark is usually more scalable and robust.

Example 4-7. Dask data pipeline code example

```
import dask.dataframe as dd

# Load a CSV file into a Dask DataFrame
df = dd.read_csv('/path/to/input_file.csv')

# Perform some data processing on the DataFrame
processed_df = df.groupby('column_name').column_name.mean().compute()

# Write the processed DataFrame to a new CSV file
processed_df.to_csv('/path/to/output_file.csv', index=False)
```

You can see that the Spark and Dask examples are much simpler compared to the Airflow ones. However, Airflow can be more suitable for managing and tracing long, complex jobs.

Stream Processing

Stream processing enables scalable, fault-tolerant, and real-time data processing. It is often used in applications that process large amounts of data in real time, such as real-time analytics, fraud detection, or recommendations.

In stream processing, data and incoming events are pushed into a stream (queue) and read by one or more workers. The workers process the data sequentially, make transformations, aggregate results, and write the results into a database or an output stream. Unlike traditional message queues, stream processing occurs in order. For example, assume the stream contains two events: one for customer login and another for customer logout. Not processing them in order can lead to a broken state. Another example is a money deposit operation, followed by a withdrawal. The withdrawal may be declined if operations are processed in the wrong order.

Streams are designed to scale. They are broken into partitions, and each partition handles a specific set of data objects, so it will not violate the order. For example, a user activity stream is partitioned by the user ID so

that a specific user's activities will always be stored in the same partition and processed by the same worker.

Streams such as [Kafka](#), [AWS Kinesis](#), and others are different than message queues like [RabbitMQ](#), [AMQP](#), [Amazon SQS](#), [Google Pub/Sub](#), and so on. Message queues do not guarantee message ordering. However, they guarantee reliable delivery of messages, while the client manages the reliability in the case of streams. Furthermore, they are much faster due to the more straightforward logic and parallelism offered with streams.

Figure 4-13 illustrates a streaming application in which clients publish data that is distributed between the individual partitions (based on a hash of the partition key). One worker is reading from each partition and processing the data. The worker can use a database to store the state on known intervals (checkpoints), so the state can be recovered in case of a failure, or the worker can free unused memory. Finally, the results can be written into a target database or an output stream.

Streams provide “at-least-once semantics.” Therefore, the same message may appear multiple times. A way to provide “exactly once” semantics (the same message is processed only once) is with the help of checkpoints. Streams are processed in order, and the state can be persisted after every micro-batch. In the case of a failure, the worker can restore the last checkpoint data (state), process the events from that point forward, and ignore older events.

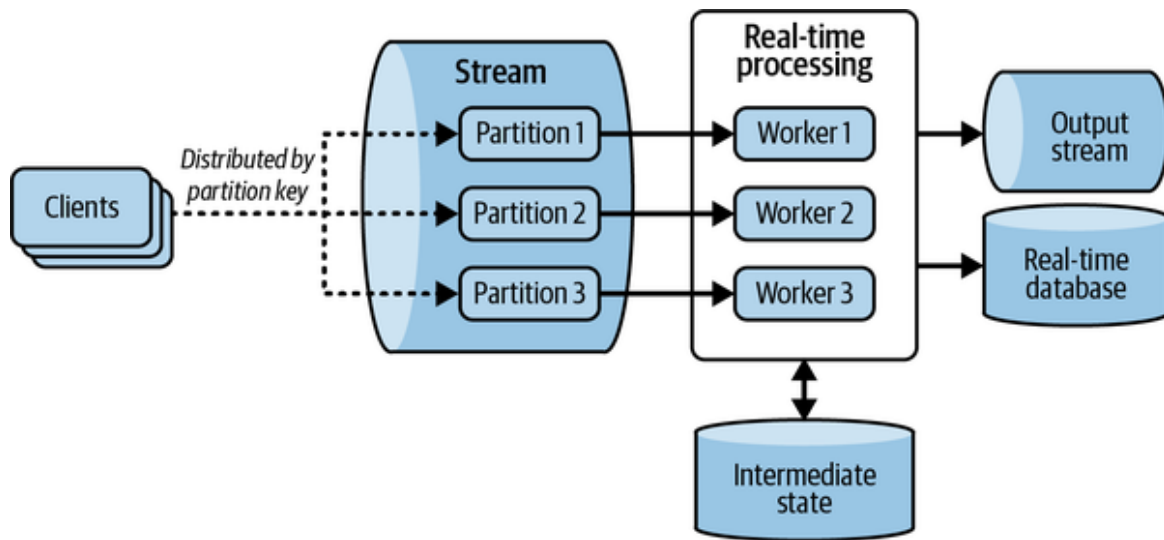


Figure 4-13. Streaming application architecture

Stream Processing Frameworks

Doing real-time analytics on real-time streams differs from doing it in batch or SQL. With streams, the workers can go over the data only once, in sequential order, and see a portion of the data (in the same partition). This is why real-time analytics frameworks such as **Spark Streaming**, **Apache Flink**, Apache Beam, **Apache NiFi**, and others, focus on stream processing and implement the standard analytic and statistic methods in a stream-optimized way.

A typical scenario in stream processing is to aggregate values over time; for example, examining the total value of customer transactions in the last hour to detect fraud. It is not feasible to calculate the total for every new event with stream processing. It will take a considerable amount of time and memory. Instead, the values are grouped into windowed buckets, for example, six buckets or more, each holding the total per 10 minutes. The process sums the values of only the last six buckets and drops the oldest bucket every 10 minutes. **Figure 4-14** illustrates overlapping sliding windows with a one-minute window duration and 30-second window periods.

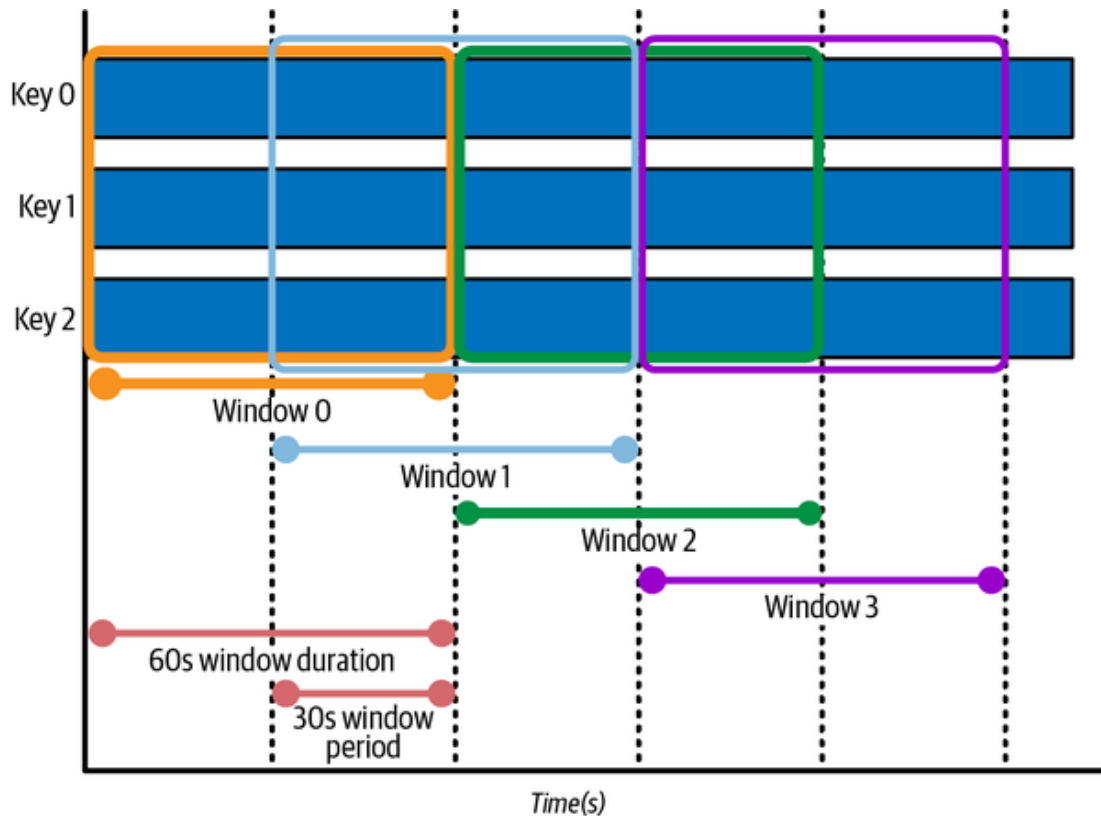


Figure 4-14. Sliding windows (source: Apache Beam)

Example 4-8 shows the Apache Beam code for defining such a window.

Example 4-8. Defining the sliding window using Apache Beam

```
from apache_beam import window
sliding_windowed_items = (
    items | 'window' >> beam.WindowInto(window.SlidingWindows(60, 30)))
```

Coding with stream processing frameworks requires advanced data engineering knowledge. This is why many users avoid real-time data, even though it can provide much better business value and more accurate model scoring results. Feature stores come to the rescue, as they can automatically generate the batch and the streaming pipeline from the same higher-level data processing logic.

Feature Stores

Feature stores are a factory and central repository for machine learning features. Feature stores handle the collection of raw data from various sources, the transformation pipeline, storage, cataloging, versioning, security, **serving**, and monitoring. They automate many processes described in this chapter, while accelerating production time and reducing engineering efforts. Feature stores form a shared catalog of production-ready features, enable collaboration and sharing between teams, and accelerate the innovation and delivery of new AI applications.

The first feature store implementations came from large service providers like Uber, Twitter, and Spotify. In those providers, AI is core to the business, and feature stores helped them accelerate the development and deployment of new AI applications and improve collaboration and reuse. Today there are multiple commercial and open source implementations to choose from.

Advanced feature stores provide the following capabilities:

Data connectivity

Glueless integration with multiple offline (data lakes, data warehouses, databases, and so on) and online (streams, message queues, APIs, managed services, and so on) sources.

Offline and online transformation

Some feature stores offer capabilities to automatically build and manage the batch and streaming pipelines from higher-level logic.

Storage

Storing the generated features in an offline store (such as an object store) and an online store (usually a key/value database).

Metadata management

Auto-generating, storing, and managing all feature metadata, including lineage, schemas, statistics, labels, and more.

Versioning

Managing multiple versions of each feature and the process of promoting features from development to production and integrating with CI/CD.

Generating and managing feature vectors

Correctly joining multiple features into a single dataset for use in training or serving applications.

Central cataloging

Providing centralized access to generate, label, or search features.

Security and governance

Controlling the access to features and raw data and to logging feature access.

Easy-to-use UI and SDK

Simple access through APIs and a user interface to abstract the underline complexity, visualize features, and make it usable by data scientists.

Monitoring and high availability

Monitoring the assets and data processing tasks automatically while reliably recovering from failures.

Feature validation and analysis

Executing various data processing tasks automatically or as initiated by the user, to validate feature correctness or to

generate a deep analysis of features, correlation, and so on.

You should thoroughly compare capabilities before choosing a feature store. For example, many have very partial functionality, may focus on cataloging features, or lack automated transformations, data management at scale, and real-time functionality. These capabilities provide the most significant value in accelerating time to production.

Feature Store Architecture and Usage

Figure 4-15 illustrates a feature store's general architecture and usage. Raw data is ingested and transformed into features, and features are cataloged and served to different applications (training, serving, monitoring). APIs and a UI allow data scientists, data engineers, and ML engineers to update, search, monitor, and use features.

The core components of a feature store are:

Transformation layer

Converts raw offline or online data into features and stores them in both an online (key/value) and offline (object) store.

Storage layer

Stores multiple versions of a feature in feature tables (feature sets) and manages the data lifecycle (create, append, delete, monitor, and secure the data). The data layer stores each feature in two forms: offline for training and analysis and online for serving and monitoring.

Feature retrieval

Accepts requests for multiple features (feature vectors) and other properties (such as time ranges and event data), and produces an offline data snapshot for training or a real-time vector for serving.

Metadata management and cataloging

Stores the feature definition, metadata, labels, and relations.

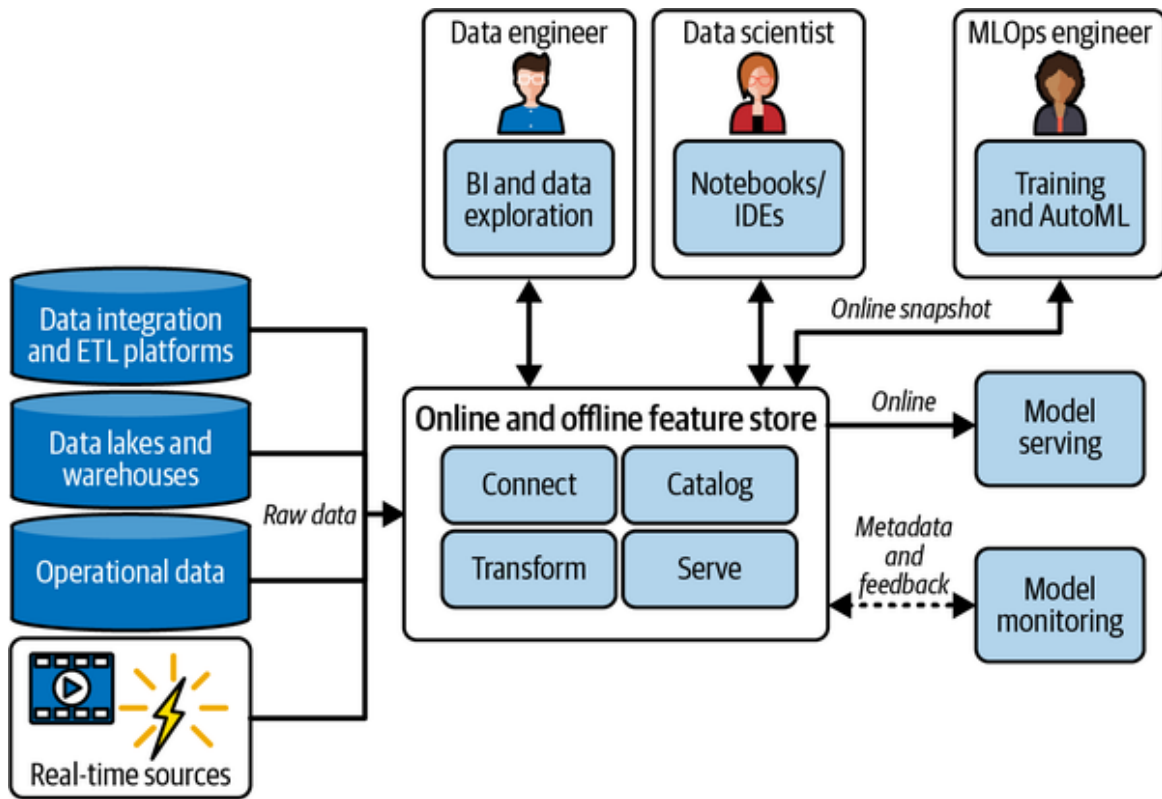


Figure 4-15. Feature store usage and architecture

Ingestion and Transformation Service

This chapter has discussed the complexities of implementing large-scale processing for batch and real-time data, data versioning, and metadata management. Feature stores aim to reduce that complexity through abstraction and automation. With modern feature stores, data pipelines are described using high-level transformation logic. This logic is converted to the underlying processing engine semantics and deployed as a continuous and production-grade service, saving significant engineering efforts.

Pipeline implementation is different for local development (using packages like pandas), large-scale offline data (using batch processing), and real-time data (using stream processing). The advantage of a feature store that

supports automated transformations is that it uses one definition for all three deployment modes and eliminates the reengineering involved in porting data pipelines from one method to another. In some feature stores, the data pipeline technology will be determined by the data sources, whether offline (data lakes, data warehouses, databases, and so on) or online (streams, message queues, APIs, managed services, and others).

Feature stores implement the data ingestion and transformation on groups of features (called feature sets or feature groups) that originate from the same source; for example, all the features extracted from a credit card transaction log. Feature sets take data from offline or online sources, build a list of features through a set of transformations, and store the resulting features along with the associated metadata and statistics.

Figure 4-16 illustrates the transformation service (feature set). Once the data is ingested from the source, it passes through a graph (DAG) of transformations, and the resulting features are written into the offline and online stores.

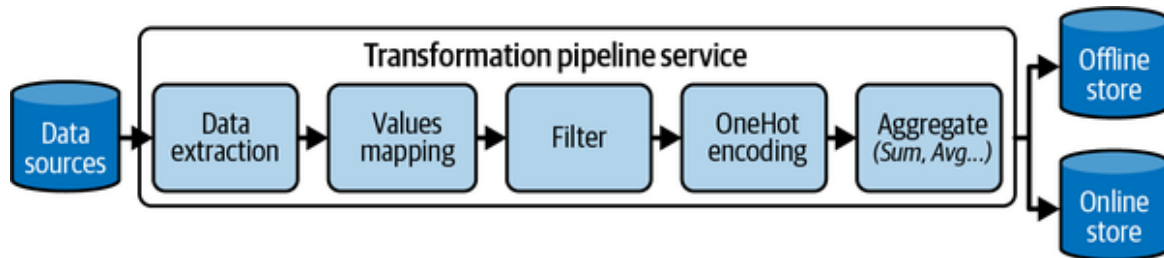


Figure 4-16. Feature transformation service (feature set) pipeline example

Examples of transformation (by data type):

Structured

Filter, group, join, aggregate, OneHot encoding, map, extract, and classify

Textual

Extract, parse, disassemble, detect entities, sentiments, and embeddings

Visual (images and videos)

Frame, resize, detect objects, crop, recolor, rotate, map, and classify

The generated transformation service should be production-grade and support auto-scaling, high availability, live upgrades, and more. In addition, it should support continuous data ingestion and processing. For example, new data may arrive continuously (for real time) or in scheduled intervals (for offline). Therefore, serverless function technologies are an excellent fit.

Feature Storage

The features are usually stored in two forms: offline storage for training and analytics applications and online storage for real-time serving and monitoring applications. See [Figure 4-17](#).

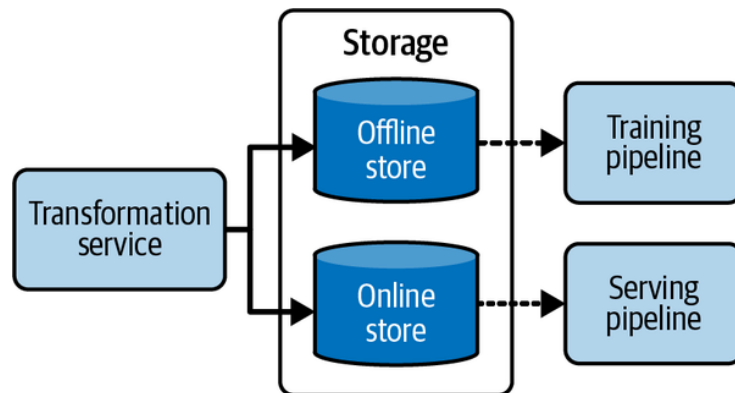


Figure 4-17. Feature storage

The offline store holds all the historical data and often uses data lakes, object storage, or data warehouse technologies. For example, a common choice is to use compressed Parquet files stored in object storage like AWS S3.

The online store holds the most recent data and often uses NoSQL or key/value stores like Redis, AWS DynamoDB, Google BigTable, and others. The online store needs to support reading features in milliseconds.

Feature Retrieval (for Training and Serving)

Training, serving, and analysis applications require multiple features from multiple datasets and sources. In contrast, feature stores organize features in groups (called *feature sets*) based on their origin and entity (primary key such as a *user id*, *product id*, and so on).

Retrieving multiple features from different sources, times, and with different indexes can be a complex analytics task. Feature stores automatically determine the parameters required for the JOIN query based on the features metadata, entity names, and user request data. In addition, when the datasets are transactional (records are marked with a timestamp), the join operation needs to take into account time correctness and *time traveling* to return only the values known at the time of the event (also referred to as *as of join* analytics operation).

Offline feature sets can be generated through SQL queries generated by the feature store. However, with real-time serving applications that need to respond in milliseconds, this creates considerable overhead, and other real-time methods are used. In addition, time-based features (such as the number of requests in the last hour) cannot be precalculated and require special handling to generate an accurate result (for example, by combining precalculated time windowed data and ad-hoc last-mile calculations).

Figure 4-18 illustrates the feature retrieval flow with two separate engines, one for offline retrieval and the other for real-time retrieval. Note that in the case of offline, the dataset is snapshotted or preserved in a new dataset to allow data lineage tracking and explainability.

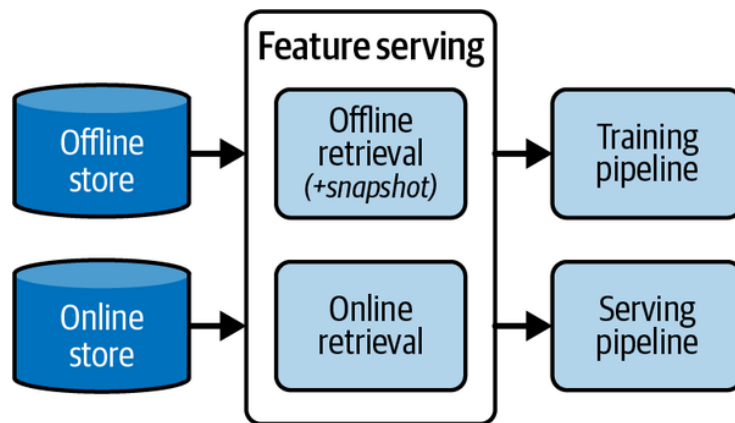


Figure 4-18. Feature retrieval

The `get_offline_features` request can accept event data to base the query on, a valid time range (for example, if we want to train the model based on data from the last month), and which features and columns should return (for example, whether to include the index, time, or label columns). Then, it initiates a local or serverless analytics job that computes the results and returns the features vector dataset.

In real-time retrieval, the system initializes the retrieval service (configuring a local or remote real-time analytics function once to save time on requests). Then, user requests are pushed with the entity keys (taken from the event data) and accept a result vector. In addition, some feature stores allow real-time imputing (replacing missing or NaN data with statistical feature values taken from the feature metadata).

Feature Stores Solutions and Usage Example

Feature stores started as internal platforms in leading cloud services providers (such as Uber, Spotify, and Twitter). But now, many open source and commercial feature store solutions are in the market. However, as in every important new technology space, there are many functionality differences between those solutions; you need to be aware so you can choose the right one.

The most notable and essential difference is if the feature store platform manages the data (transformation) pipeline for you and whether it supports

both offline and real-time (streaming) pipelines. As you've read in this chapter, building and managing a scalable data pipeline is the major challenge. If you are forced to do it manually, it significantly undermines the value of a feature store.

Table 4-2 compares the leading feature store solutions:

Table 4-2. Feature store solution comparison

Category	Feast	Tecton	MLRun	SageMaker
Open source	Yes	No	Yes	No
Managed option	No	major clouds	cloud + on-prem	on AWS
Offline pipelines	No	Yes	Yes	No
Real-time pipelines	No	Yes	Yes	No
Feature retrieval	Yes	Yes	Yes	Yes
Engines	Spark	Spark	Python, Dask, Spark, Nuclio	None
Feature analytics	No	Yes	Yes	No
Versioning and lineage	No	Yes	Yes	No
Features security	No	Yes	Yes	Yes
Monitoring	No	Yes	Yes	No

Category	Feast	Tecton	MLRun	Sage
Glueless training and serving	No	No	Yes	No

The following sections will demonstrate how feature stores are used with the two leading open source frameworks: **Feast** and **MLRun**. Note that MLRun is more fully featured and provides offline and online transformation services (based on MLRun’s serverless engines) along with many other unique features.

Using Feast Feature Store

Feast does not provide a transformation service. Data should be prepared upfront and stored in a supported source (like S3, GCS, BigQuery). Feast registers the source dataset and its metadata (schema, entity, and so on) in a FeatureView object, as shown in **Example 4-9**.

Example 4-9. Defining Feast FeatureView (source: Feast)

```
# Read data from parquet files. Parquet is convenient for local development mode.
# For production, you can use your favorite DWH, such as BigQuery. See Feast
# documentation for more info.
driver_hourly_stats = FileSource(
    name="driver_hourly_stats_source",
    path="/content/feature_repo/data/driver_stats.parquet",
    timestamp_field="event_timestamp",
    created_timestamp_column="created",
)

# Define an entity for the driver. You can think of entity as a primary key used
to
# fetch features.
driver = Entity(name="driver", join_keys=["driver_id"])

# Our parquet files contain sample data that includes a driver_id column,
timestamps
# and three feature column. Here we define a Feature View that will allow us to
```

```

serve
# this data to our model online.
driver_hourly_stats_view = FeatureView(
    name="driver_hourly_stats",
    entities=[driver],
    ttl=timedelta(days=1),
    schema=[
        Field(name="conv_rate", dtype=Float32),
        Field(name="acc_rate", dtype=Float32),
        Field(name="avg_daily_trips", dtype=Int64),
    ],
    online=True,
    source=driver_hourly_stats,
    tags={},
)

```

Feast does not provide an online transformation or ingestion service. Instead, the user needs to run a *materialization* task to copy the offline features into the real-time store (database). Unfortunately, this also means that the data stored in the online store is inaccurate between materializations, and running materialization too frequently can result in significant computation overhead.

Running the materialization task via the SDK:

```

store = FeatureStore(repo_path=".")
store.materialize_incremental(datetime.now())

```

The project may contain one or more feature views, and each is defined and materialized independently. Features can be retrieved from one or more feature views (will initiate a JOIN operation).

To retrieve offline features (directly from the offline source), use the `get_historical_features()` API call as shown in [Example 4-10](#).

Example 4-10. Retrieve offline features with Feast (source: Feast)

```

# The entity dataframe is the dataframe we want to enrich with feature values
# see https://docs.feast.dev/getting-started/concepts/feature-retrieval for
details
# for all entities in the offline store instead
entity_df = pd.DataFrame.from_dict(
    {
        # entity's join key -> entity values

```

```

        "driver_id": [1001, 1002, 1003],
        # "event_timestamp" (reserved key) -> timestamps
        "event_timestamp": [
            datetime(2021, 4, 12, 10, 59, 42),
            datetime(2021, 4, 12, 8, 12, 10),
            datetime(2021, 4, 12, 16, 40, 26),
        ],
        # (optional) label name -> label values. Feast does not process these
        "label_driver_reported_satisfaction": [1, 5, 3],
        # values we're using for an on-demand transformation
        "val_to_add": [1, 2, 3],
        "val_to_add_2": [10, 20, 30],
    }
)

store = FeatureStore(repo_path=".")

# retrieve offline features, feature names are specified with <view>:<feature-
# name>
training_df = store.get_historical_features(
    entity_df=entity_df,
    features=[
        "driver_hourly_stats:conv_rate",
        "driver_hourly_stats:acc_rate",
        "driver_hourly_stats:avg_daily_trips",
        "transformed_conv_rate:conv_rate_plus_val1",
        "transformed_conv_rate:conv_rate_plus_val2",
    ],
).to_df()

print("----- Example features -----\\n")
print(training_df.head())

```

To retrieve online features from the online store, we use the `get_online_features()` API call, as shown in [Example 4-11](#).

Example 4-11. Retrieve online features with Feast (source: Feast)

```

from pprint import pprint
from feast import FeatureStore

store = FeatureStore(repo_path=".")

feature_vector = store.get_online_features(
    features=[
        "driver_hourly_stats:acc_rate",
        "driver_hourly_stats:avg_daily_trips",
        "transformed_conv_rate:conv_rate_plus_val1",

```



```

        "transformed_conv_rate:conv_rate_plus_val2",
    ],
    entity_rows=[
        # {join_key: entity_value}
        {
            "driver_id": 1001,
            "val_to_add": 1000,
            "val_to_add_2": 2000,
        },
        {
            "driver_id": 1002,
            "val_to_add": 1001,
            "val_to_add_2": 2002,
        },
    ],
).to_dict()

pprint(feature_vector)

# results:
{'acc_rate': [0.86463862657547, 0.6959823369979858],
 'avg_daily_trips': [359, 311],
 'conv_rate_plus_val1': [1000.6638441681862, 1001.1511893719435],
 'conv_rate_plus_val2': [2000.6638441681862, 2002.1511893719435],
 'driver_id': [1001, 1002]}

```

Using MLRun Feature Store

MLRun supports the registration of existing sources (like Feast) or the definition of a data pipeline for transforming source data into features. When defining the data pipeline (called a *graph*), MLRun provisions the selected data processing engine based on the abstract user definitions. MLRun supports a few processing engines, including local Python, Dask, Spark, and Nuclio (a real-time serverless engine).

In MLRun, by default, the pipeline writes into online and offline stores, so there is no need for separate materialization jobs, and the online and offline features are always in sync. In addition, MLRun can auto-detect the data schema, making it more straightforward and robust.

MLRun separates the definition of the feature set (a collection of features generated by the same pipeline) from the data source definitions. This way, you can use the same feature set in interactive development and in

production. Just swap the source from a local file in development to a database or real-time Kafka stream in the production deployment.

Example 4-12 shows an example of defining a feature set for processing credit card transactions to detect credit card fraud. The definition includes the entity, timestamp, and transformation graph using built-in operators and aggregations. Note that a user can also add their custom Python operators. See the [full example](#).

The data pipeline consists of the following:

- *Extracting* the data components (hour, day of week).
- *Mapping* the age values
- *One-hot encoding* for the transaction category and the gender
- *Aggregating* the amount (avg, sum, count, max over 2/12/24 hour time windows)
- *Aggregating* the transactions per category (over 14 day time windows)
- *Writing* the results to offline (Parquet) and online (NoSQL) targets

Example 4-12. Defining MLRun FeatureSet (source: MLRun)

```
import mlrun.feature_store as fs

# Define the credit transactions FeatureSet
transaction_set = fs.FeatureSet("transactions",
                                entities=[fs.Entity("source")],
                                timestamp_key='timestamp',
                                description="transactions feature set")

# Define and add value mapping
main_categories = ["es_transportation", "es_health", "es_otherservices",
                  "es_food", "es_hotelservices", "es_barsandrestaurants",
                  "es_tech", "es_sportsandtoys", "es_wellnessandbeauty",
                  "es_hyper", "es_fashion", "es_home", "es_contents",
                  "es_travel", "es_leisure"]

# One Hot Encode the newly defined mappings
one_hot_encoder_mapping = {'category': main_categories,
                           'gender': list(transactions_data.gender.unique())}
```

```
# Define the data pipeline (graph) steps
transaction_set.graph\
    .to(DateExtractor(parts = ['hour', 'day_of_week'],
        timestamp_col = 'timestamp'))\
    .to(MapValues(mapping={'age': {'U': '0'}}, with_original_features=True))\
    .to(OneHotEncoder(mapping=one_hot_encoder_mapping))

# Add aggregations for 2, 12, and 24 hour time windows
transaction_set.add_aggregation(name='amount',
    column='amount',
    operations=['avg', 'sum', 'count', 'max'],
    windows=['2h', '12h', '24h'],
    period='1h')

# Add the category aggregations over a 14 day window
for category in main_categories:
    transaction_set.add_aggregation(name=category, column=f'category_{category}',
        operations=['count'], windows=['14d'],
        period='1d')
```

The data pipeline can be visualized using `transaction_set.plot(rankdir="LR", with_targets=True)`, as seen in [Figure 4-19](#).

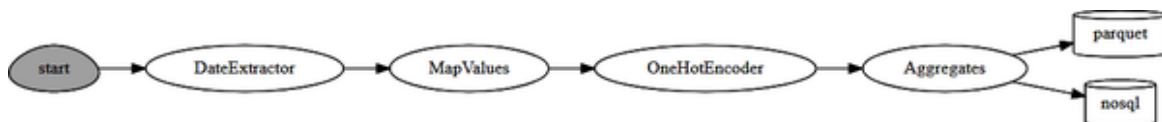


Figure 4-19. Feature set plot

Once you have the feature set definition, you can test and debug it with the `preview()` method that runs the data pipeline locally and lets you view the results:

```
df = fs.preview(transaction_set, transactions_data)
df.head()
```

When the feature set definition is done, you can deploy it as a production job that runs on demand, on a given schedule, or as a real-time pipeline.

For running batch ingestion, use the `ingest()` method. For real-time ingestion from HTTP or streams, use `deploy_ingestion_service_v2()`,

which starts a real-time Nuclio serverless pipeline. See [Example 4-13](#).

Example 4-13. Ingest data into MLRun FeatureSet (source: MLRun)

```
# Batch ingest the transactions dataset (from CSV file) through the defined
pipeline
source = CSVSource("mycsv", path="measurements.csv")
fs.ingest(transaction_set, source=source)

# Deploy a real-time pipeline with HTTP API endpoint as the source
# MLRun support other real-time sources like Kafka, Kinesis, etc.
source = HTTPSource()
fs.deploy_ingestion_service_v2(transaction_set, source)
```

You can watch the feature sets, their metadata, and statistics in the MLRun feature store UI. See [Figure 4-20](#).

The feature retrieval in MLRun is done using the *feature vector* object. Feature vectors hold the definitions of the requested features and additional parameters. In addition, they also store calculated values such as the features metadata, statistics, and so on, which can be helpful in training, serving, or monitoring tasks. For example, feature statistics are used for automated value imputing in the case of missing or NaN feature values and for model drift monitoring in the serving application.

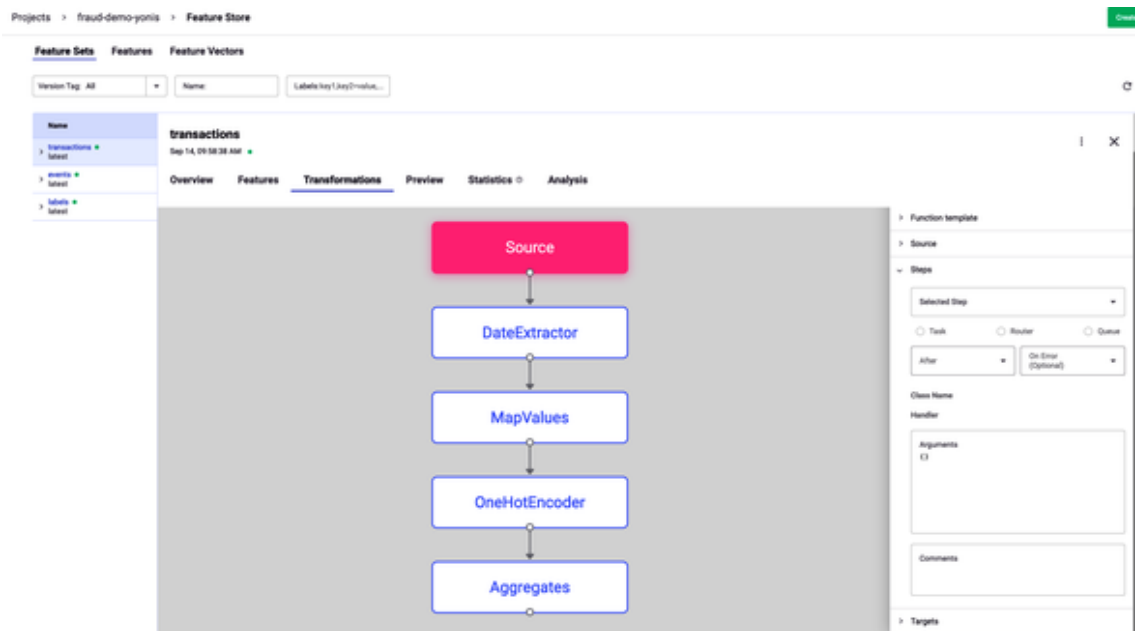


Figure 4-20. MLRun FeatureSet in UI

Feature vectors can be created, updated, and viewed in MLRun's UI.

Users first define the feature vector, then they can use it to obtain offline or online features. See how to retrieve offline features and use the `get_offline_features()` method in [Example 4-14](#).

Example 4-14. Get offline features from MLRun (source: MLRun)

```
# Define the list of features you will be using (<feature-set>.<feature>)
features = ['transactions.amount_max_2h',
            'transactions.amount_sum_2h',
            'transactions.amount_count_2h',
            'transactions.amount_avg_2h',
            'transactions.amount_max_12h']

# Import MLRun's Feature Store
import mlrun.feature_store as fstore

# Define the feature vector name for future reference
fv_name = 'transactions-fraud'

# Define the feature vector using our Feature Store
transactions_fv = fstore.FeatureVector(fv_name, features,
                                       label_feature="labels.label",
                                       description=
                                           'Predicting a fraudulent transaction')

# Save the feature vector definition in the Feature Store
transactions_fv.save()

# Get offline feature vector as dataframe and save the dataset to a parquet file
train_dataset = fstore.get_offline_features(transactions_fv,
                                             target=ParquetTarget())

# Preview the dataset
train_dataset.to_dataframe().tail(5)
```

To get real-time features, you first need to define a service (which initializes the real-time retrieval pipeline), followed by `.get()` methods to request feature values in real time. The separation between the service creation (one-time initialization) and individual requests ensures lower request latencies. In addition, MLRun supports automatic value imputing based on the feature's metadata and statistics. This can save significant development and computation overhead. See [Example 4-15](#).

Example 4-15. Get online features from MLRun (source: MLRun)

```
# Create the online feature service, substitute NaN values with
# the feature mean value
svc = fstore.get_online_feature_service('transactions-fraud:latest',
                                       impute_policy={"*": "$mean"})

# Get sample feature vector
sample_fv = svc.get([{'source': 'C76780537'}])

# sample_fv Result
[{'amount_max_2h': 14.68,
  'amount_max_12h': 70.81,
  'amount_sum_2h': 14.68,
  'amount_count_2h': 1.0,
  'amount_avg_2h': 14.68}]
```

NOTE

MLRun's feature stores provide accurate real-time aggregations and low latency by combining precalculated values during the ingestion process with real-time calculations at feature request time.

The MLRun framework provides a model development and training pipeline, real-time serving pipelines, and integrated model monitoring. MLRun's feature store is natively integrated with the other components, eliminating redundant glue logic, metadata translation, and so on, thus accelerating time to production.

Conclusion

With data management and processing being the most critical components of ML, it's important to understand how to optimally perform data-related tasks. This chapter explores the recommended tools and practices for the various stages of working with your data. We started the chapter by discussing data versioning and lineage, which are essential for tracing data origin. Then we explored data preparation and analysis at scale, which is how the data is handled so it can be used in production. In this section, we

also discussed the architecture of interactive data processing solutions and the differences between batch data processing and real-time processing.

After reviewing the challenges of implementing these practices at scale, we moved on to present the concept of feature stores, which are a central repository for ML features. We covered the capabilities of a feature store, such as data connectivity and offline and online transformation. We also showed where the feature store fits in the MLOps pipeline, from ingesting raw data to supporting the use of that data in training, serving, monitoring, and more. Finally, we reviewed different feature store solutions and how to use them.

Critical Thinking Discussion Questions

- Which details does metadata provide? As data professionals, why do we need this information?
- Which open source data versioning tools are available? Which one could be a good fit for your organization?
- What's the difference between batch processing and stream processing? When is each one used?
- How does a feature store simplify data management and processing practices? Which capabilities enable this?
- What are the differences between the Feast and the MLRun feature stores? Which one could be a good fit for your organization?

Exercises

- Choose an open source solution (DVC, Pachyderm, MLflow, or MLRun) and create a data versioning script or workflow that will record and version data and metadata.

- Create a prototype of a batch processing pipeline with the tool of your choice.
- Connect a Trino data connector to a data source.
- Train a demo model (you can use Hugging Face if you need a sample model) with a feature store.
- Create a feature set and ingestion pipeline in MLRun. You can use [this project](#) as a reference.

OceanofPDF.com

Chapter 5. Developing Models for Production

Developing ML models is no longer confined to experimental labs and research papers. It's about real-world applications, and that means production. That's why building high-performing models is at the heart of developing models for production.

A production-first mindset ensures that the models actually make it to production and answer real-life business cases. Otherwise, models get stuck throughout the ML pipeline due to lack of collaboration between teams, technological discrepancies, or other types of friction.

This chapter focuses on building the best models you can. It details all the steps and processes to implement and run on models throughout the ML pipeline before production. This includes running, tracking, and comparing ML jobs, automations, training and ML at scale; testing; resource management; and much more. It details various methodologies, tools, and approaches, together with code examples you can follow.

When following the steps and trying out the exercises at the end of the chapter, be conscious of the entire MLOps pipeline and how your work could be integrated and automated together with the other steps you or other team members are taking. By taking these steps with a production-first approach in mind, you can assure the reliability, stability, and performance of your ML models.

AutoML

Building the best ML model is an iterative process that relies on data science experience and intuition. The data scientist attempts various strategies, like creating new features from the data, selecting the suitable

algorithm, and choosing the optimal model parameters to get the best predictor model.

Automated Machine Learning, or AutoML, tries to automatically infer from the data and the model's goal the possible processing tasks and experiments that should be tested and run in a sequence until the best model result is achieved. AutoML reduces the data scientist's effort and allows less experienced individuals to develop high-performing ML models quickly and efficiently. However, it may result in the use of more computation resources.

AutoML platforms and tools aim to streamline the process of building ML models by automating repetitive and complex tasks.

Key components of AutoML include:

Data preprocessing

Tasks such as data cleaning, imputing missing values, encoding categorical variables, and scaling features to prepare the data for modeling.

Feature engineering

Automatically generating and selecting relevant features or transformations of features to enhance model performance.

Model selection

Exploring and selecting appropriate algorithms or models for a given dataset and problem. This can include trying various types of models (like decision trees, neural networks, or SVMs) and evaluating their performance.

Hyperparameter tuning

Selecting optimal hyperparameters for models. This can involve techniques like grid search, random search, Bayesian optimization, or other optimization algorithms.

Ensemble methods

Combining predictions from multiple models to improve overall performance.

Model evaluation

Using metrics such as accuracy, precision, recall, F1 score, and more, which help users assess the model's effectiveness.

Pipeline construction

Constructing end-to-end pipelines, from data preprocessing to model deployment, allowing users to generate production-ready workflows.

Interpretability and explainability

Offering explanations for model predictions, helping users understand and interpret how the model arrives at its decisions.

DataRobot, founded in 2012, was one of the pioneers in AutoML. Other companies followed in 2017/2018: H2O with its **Driverless AI platform** and Google with technologies like **Cloud AutoML**. Expansion to the rest of the cloud providers soon followed: **Azure Machine Learning Studio AutoML**; **SageMaker Autopilot**; and a slew of open source projects such as **Auto-sklearn**, **Auto-Keras**, **Tree-based Pipeline Optimization Tool (TPOT)**, **MLBox**, **AutoGluon**, **AutoWEKA**, and **Ludwig**.

Some of the benefits of AutoML include:

Efficiency

Automates repetitive tasks like feature selection and hyperparameter tuning.

Accessibility

User-friendly interfaces make machine learning accessible to nonexperts.

Cost-effectiveness

Reduces the need for specialized talent, making it more affordable.

Improved accuracy

Thanks to advanced algorithms for automatic model tuning.

Scalability

Capable of handling large datasets and high-dimensional feature spaces.

Experimentation

Allows for rapid testing of different features, models, and hyperparameters.

Some of the drawbacks of AutoML:

Limited customization

AutoML platforms often have preset algorithms and configurations, limiting fine-tuning options.

Overfitting risk

Automated processes may lead to overfitting, especially if not properly managed.

Resource intensive

AutoML can be computationally expensive, requiring powerful hardware for large datasets.

Lack of domain knowledge

AutoML solutions may lack the domain-specific expertise needed for specialized tasks.

Interpretability

Models generated by AutoML can be complex and difficult to interpret, posing challenges for explainability.

Cost

While it can be cost-effective in some scenarios, the initial investment in AutoML platforms can be high.

Dependency on data quality

The effectiveness of AutoML is highly dependent on the quality of the input data; garbage in, garbage out.

Ethical concerns

Automated model selection could unintentionally introduce or perpetuate biases present in the data.

Where does the future of AutoML lie? Noah defines the automation process as *the automator's law*; once you talk about something being automated, it is eventually automated. Many software engineering tasks are starting to go away with tools like ChatGPT or AWS CodeWhisperer. What may happen is that AutoML and generative AI are combined to create sophisticated ML systems that require very little manual human interaction. The new interface may not be Jupyter Notebook or Visual Studio code, but a voice assistant like in *Star Trek*. Imagine saying, “Hey Siri, build me a new housing price prediction model for ZIP code 90210.” It may not be that far off.

Running, Tracking, and Comparing ML Jobs

Running, tracking, and comparing ML jobs are the building blocks of a robust and agile ML workflow. They enable organizations to develop and use accurate and reliable models that deliver value.

Running ML jobs includes the model training, hyperparameter tuning, data preprocessing, and testing, and requires computational resource allocation and pipeline automation. This is the execution phase, and efficiency in this stage means quicker development and deployment.

Google Vertex AI and Amazon SageMaker are considered mainstream, fully managed cloud MLOps platforms. They incorporate tools for running and tracking ML jobs and simplifying ML workflow automation in their respective cloud ecosystems (Google Cloud and AWS). In addition, they handle provisioning and Ops so developers can focus on models.

Other frameworks like MLflow, ClearML, and Weights & Biases (W&B) don't provide the underlying infrastructure. They are cloud-agnostic and can run on any infrastructure, filling gaps the cloud vendors don't address. For example, ClearML does advanced hyperparameter optimization, W&B provides excellent visualization for comparing experiments, and MLflow offers model packaging.

Tracking ML jobs includes logging of metrics, version control across the different elements, experiment tracking, results visualization, and collaboration tools. This stage ensures that the development is transparent and that models are reproducible, allowing data scientists to understand what works and what doesn't, while facilitating collaboration and compliance with industry standards.

Comparing the jobs includes performance evaluation, hyperparameter comparison, analyzing the resource usage, cost analysis, and interpretability analysis. Through this comparison, data scientists can select the model that best meets the business objectives. It also enables continuous improvement by learning from previous iterations and adapting to new data or changing requirements.

Most of these concepts are covered throughout this book, so this section focuses on the most important ones that do not appear elsewhere.

Experiment Tracking

Experiment tracking is the practice of systematically recording and managing the different parts of the ML development process. This includes tracking and documenting different experimental setups, as well as the configurations, code, parameters, data inputs and outputs, logs, returned metrics, and various artifacts (datasets, models, charts, and others).

ML experiment tracking has many advantages. The main ones are:

Reproducibility

Ensuring that ML experiments can be reliably reproduced by maintaining a detailed record of every aspect of the experiment.

Comparative analysis

Comparing different models, algorithms, and techniques to identify the most effective solutions.

Debugging and troubleshooting

Pinpointing and resolving issues, making debugging and troubleshooting more efficient.

Collaboration

Allowing team members to understand, reproduce, and build upon each other's work.

Decision-making

Making data-driven decisions about the best strategies.

Documentation

Detailed records are useful for report writing and sharing results.

Time and resource management

Helping avoid redundant work, saving both time and resources.

Governance

Allowing the traceability and explainability of models, how they were trained, and so on.

Collecting data and metadata required for production deployment automatically

Ensuring that the data is there instead of having to do it manually.

On top of these, experiment tracking can also be used to support additional use cases, like auto-tuning and AutoML, controlling and governing the ML process to ensure implementation of responsible and ethical AI, and simplifying pipelines by using the outputs of one step as inputs to another.

Some popular ML experiment tracking tools include **TensorBoard**, a visualization toolkit that comes with **TensorFlow**, **MLflow**, **Weights & Biases**, **Comet**, **ClearML**, and **Sacred**.

Example 5-1 demonstrates how to implement experiment tracking using MLflow. The developer adds code to record the parameters, metrics, and model output.

Example 5-1. Experiment tracking with MLflow

```
import mlflow
import mlflow.sklearn
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Create a machine learning model
model = LogisticRegression()

# Start an MLflow run context
with mlflow.start_run() as run:
    # Log parameters
    mlflow.log_param("model_algo", "LogisticRegression")
    mlflow.log_param("C", 1.0)
```



```
# Train and test the model
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Log the model and its metrics
mlflow.log_metric("accuracy", accuracy)
mlflow.sklearn.log_model(model, "model")
```

Saving Essential Metadata with the Model Artifacts

When developing machine learning models, it is essential to save metadata about the model and the model artifacts. This metadata can include information such as:

- Links and metadata describing the original training data used
- Performance metrics like accuracy and loss
- Hyperparameters used for training
- The model architecture and framework version
- Computing infrastructure and software packages used for training
- Version history and lineage information
- Project, experiment, and run IDs for traceability

Capturing this information serves several purposes:

- Helps reproduce and rebuild the model later.
- Can debug performance issues by comparing metadata across models.
- Enables rollbacks to previous versions.
- Allows auditing model origin throughout the lifecycle.
- Enables model comparison, retraining, and improvement.

- Records information needed for deploying the model (such as required packages and files, input and output schema, and so on).

Tools like MLflow, SageMaker Experiments, and MLRun provide ways to capture experiment and training dataset metadata and store them in the model registry along with the model artifact automatically.

Once models are registered, you can download them along with their configuration and metadata. In example [Example 5-2](#), the model downloads locally using the MLflow API.

Example 5-2. Model downloading with MLflow

```
from mlflow.store.artifact.models_artifact_repo import ModelsArtifactRepository
from mlflow.tracking import MlflowClient

client = MlflowClient()
my_model = client.download_artifacts(
    "68baff0203344dfebe89a6c73c6d6cfe", path="model")
print(f"Placed model in: {my_model}")
```

You can list models as shown in the API call in [Example 5-3](#), which allows you to search for models that meet the criteria you are looking for in terms of type, accuracy, and others:

Example 5-3. Model searching with MLflow

```
from pprint import pprint
from mlflow.tracking import MlflowClient

client = MlflowClient()
for rm in client.list_registered_models():
    pprint(dict(rm), indent=4)
```

Comparing ML Jobs: An Example with MLflow

Comparing ML jobs involves analyzing and contrasting different runs or iterations of a model to identify the best performing one or to understand how changes in parameters, data, or algorithms affect the results.

By comparing ML jobs, data scientists and engineers can evaluate different models or different configurations of the same model. This allows them to identify which model performs best, according to specific metrics like

accuracy, precision, recall, or custom evaluation criteria. This comparison is essential for model selection, hyperparameter tuning, and understanding the impact of changes in the data or features.

ML jobs can be compared through these three methods:

Visual comparison

MLflow, TensorBoard, or custom visualization scripts can provide graphical representations of metrics and parameters. Scatter plots, line charts, and heat maps are commonly used to visualize differences.

Statistical comparison

Statistical tests can be used to determine if differences in performance are statistically significant. This approach provides a more rigorous understanding of the variations between models.

Programmatic comparison

Using APIs and scripting languages like Python, data scientists can write custom code to compare models on specific criteria. This method offers flexibility and can be tailored to unique project needs.

Comparing two jobs in MLflow can be done through the MLflow Tracking component, which allows logging and comparing different model runs of your machine learning models.

To compare two jobs in MLflow, follow these steps:

1. Open the MLflow UI in your web browser.
2. Select the experiment containing the runs you want to compare.
3. Find the runs you want to compare in the list. You can use filters to narrow down the runs if you have many of them.

4. Click the checkboxes next to the runs you want to compare, then click the “Compare” button. This will take you to a comparison view where you can see various metrics, parameters, and other details side by side.
5. In the comparison view, you can analyze the differences between the runs. This includes metrics, parameters, tags, and artifacts. You can visualize the differences in various ways, such as scatter plots, line charts, or tables. **Figure 5-1** compares two jobs in MLflow.

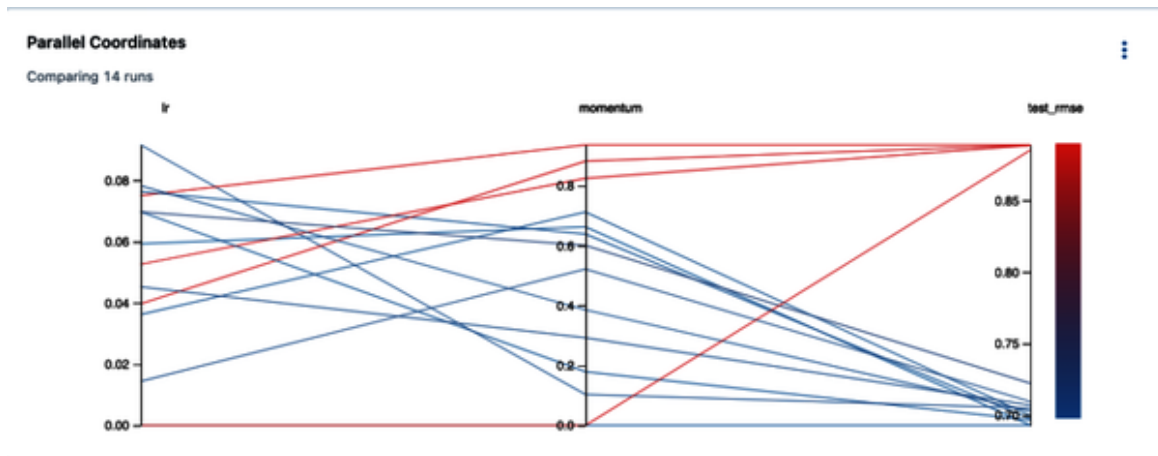


Figure 5-1. MLflow job comparison UI

6. If you want to share the comparison with others, you can export the comparison view as a CSV file or take a screenshot.
7. If you prefer working programmatically, you can use the MLflow Python API to fetch details about the runs and perform comparisons in your preferred environment, such as Jupyter Notebook.

Example 5-4 shows how you might use the MLflow Python API to compare two runs.

Example 5-4. MLflow job comparison using code

```
python
import mlflow

# Get the runs by ID
run1 = mlflow.get_run(run_id="run_id_1")
run2 = mlflow.get_run(run_id="run_id_2")

# Compare metrics
```

```
metric_name = "accuracy"
metric1 = run1.data.metrics[metric_name]
metric2 = run2.data.metrics[metric_name]

print(f"Comparison of {metric_name}:")
print(f"Run 1: {metric1}")
print(f"Run 2: {metric2}")
```

The code snippet fetches two runs by their IDs and prints a comparison of a specific metric.

Hyperparameter Tuning

Hyperparameter tuning is the part of the ML model training process that involves selecting the optimal set of hyperparameters. These parameters govern the learning process but are not learned from the data, which means that choosing the right hyperparameters can significantly affect the model's performance.

Some of the most common hyperparameter tuning strategies include:

Grid search

Defining a grid of possible hyperparameter values and exhaustively searching through all possible combinations. This method is simple but can be computationally expensive when dealing with a large number of hyperparameters or a wide range of values.

Random search

Selecting random combinations of hyperparameter values from predefined ranges. This approach can be more efficient than grid search in terms of computation time and can often yield good results.

Genetic algorithms

Inspired by natural evolution, genetic algorithms involve creating a population of hyperparameter configurations,

evaluating their performance, and using selection, mutation, and crossover operations to generate new configurations for the next generation. This process continues iteratively to improve the configurations.

Gradient-based optimization

This approach uses gradient information to optimize hyperparameters. It requires access to the gradients of the model's performance with respect to the hyperparameters. This method can be effective but might not be feasible for all types of models.

Hyperband

Hyperband combines random search with the idea of early stopping. It trains a range of configurations for different numbers of iterations and focuses more resources on the promising configurations while early stopping unpromising ones.

SMBO (Sequential Model-Based Optimization)

Building probabilistic surrogate models to approximate the relationship between hyperparameters and model performance. These models guide the search process to explore the most promising regions.

RL (reinforcement learning) for hyperparameter tuning

Some advanced techniques use reinforcement learning algorithms to optimize hyperparameters. These methods treat hyperparameter tuning as a sequential decision-making problem and learn how to make decisions that lead to better model performance over time.

Some common tools that can be used for hyperparameter tuning include GridSearchCV and RandomizedSearchCV, which are part of the [scikit-learn](#) library, [Hyperopt](#), [Optuna](#), [Keras Tuner](#), [Ray Tune](#), [Spearmin](#), [SMAC](#) (Sequential Model-based Algorithm Configuration), [SigOpt](#), [AWS Hyperparameter Tuning](#) (part of Amazon SageMaker), [Azure HyperDrive](#), and [MLRun](#).

[Example 5-5](#) shows how to use a training function and run multiple jobs in parallel for hyperparameters tuning in MLRun using the default grid search strategy. It will select the best run for maximum accuracy.

Example 5-5. Running a hyperparameters job using MLRun

```
hp_tuning_run = project.run_function(  
    "trainer",  
    inputs={"dataset": gen_data_run.outputs["dataset"]},  
    hyperparams={  
        "n_estimators": [100, 500, 1000],  
        "max_depth": [5, 15, 30]  
    },  
    selector="max.accuracy"  
)
```

Auto-Logging

Auto-logging can automatically capture key metrics, parameters, and metadata during machine learning processes without manually adding calls to the experiment tracking/logging API. This approach can reduce tedious coding work for developers by replacing the need to manually insert logs into code. Auto-logging provides organization-wide observability into ML experiments.

The auto-logging solutions are integrated with the specific ML frameworks (sklearn, TensorFlow, and others) and know how to automatically extract the key model metrics and metadata from the framework and save it as experiment parameters, metrics, and artifacts without extra coding. This way developers don't need to understand the nuances of each framework or develop complex reports or visualization.

Auto-logging solutions can automatically log and export the model into the model registry along with its metrics and metadata, allowing simple deployment later on.

There are two main approaches to auto-logging:

Intrusive

Requires modifying the ML training code to add auto-logging using an MLOps SDK like MLflow

Nonintrusive

Automatically intercepts and logs metrics without code changes (done by patching the ML framework or wrapping the user code)

ClearML, for example, supports nonintrusive auto-logging through patching and can turn it off if needed.

MLRun can auto-log function input args, return values and objects (such as datasets and models), and all the relevant metadata without code instrumentation. In addition, it is integrated with MLflow and can auto-record metadata and models logged using the MLflow API.

Example 5-6 shows how to use MLflow's auto-logging in Python.

Example 5-6. Auto-logging in Python using MLflow

```
import mlflow
from sklearn.ensemble import RandomForestRegressor

mlflow.autolog()

rf = RandomForestRegressor()
rf.fit(X_train, y_train)

rf.predict(X_test)

print(mlflow.active_run().data.metrics) # logged automatically!
```

Example 5-7 uses SageMaker Debugger.

Example 5-7. Auto-logging in Python using SageMaker Debugger

Train model using SageMaker SDK

```
import sagemaker

ml = sagemaker.estimator.Estimator()

# SageMaker automatically enables Debugger
ml.fit(data)

# View logs
ml.debugger_rules_analysis.load_analytics()
```

Another example, [Example 5-8](#), also uses MLflow. One of the valuable aspects of MLflow is that it can automatically log what is going on in the MLflow inference [in the project](#). It accomplishes this using `mlflow.autolog()`.

Example 5-8. Auto-logging in Python using MLflow

```
import mlflow

from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor

mlflow.autolog()

db = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(db.data, db.target)

# Create and train models.
rf = RandomForestRegressor(n_estimators = 100, max_depth = 6, max_features = 3)
rf.fit(X_train, y_train)

# Use the model to make predictions on the test dataset.
predictions = rf.predict(X_test)
autolog_run = mlflow.last_active_run()
```

MLOps Automation: AutoMLOps

One of the major challenges causing organizations to fail to deliver ML applications to production is the extensive engineering effort it takes to

move from the research playground, using notebooks and sample data, to large-scale deployment, using microservices, production data, automation, and observability. Many tools focus on an interactive development flow, but the move to production involves manual work, refactoring code and notebooks, and glue logic.

A way to accelerate production and reduce the engineering effort is to apply automation to the different MLOps tasks and extend automation from AutoML and auto-logging to full AutoMLOps. For example, instead of ML engineers running manual and complex processes, code can be built into fully managed microservices with “codeless” observability and read directly from production datasets in just one click or API call. MLRun pioneered the AutoMLOps approach.

Tasks that can be automated with AutoMLOps tools include:

- Inject parameters or code into tasks and log the results.
- Convert code to managed microservices and reusable components.
- Distribute the workloads automatically across containers or VMs.
- Pass data to and from cloud resources and databases.
- Gather the data and metadata for operational aspects.
- Ensure security hardening and protection.
- Version across components and steps.
- Auto-track experiments, metrics, artifacts, data, and models.
- Register models along with their required metadata and optimal production formats.
- Auto-scale and automatically optimize resource usage (such as CPUs/GPUs).
- Integrate with CI/CD, Git, and reporting systems.

- Correlate and visualize the relationship between source data, runs, models, and others.

AutoMLOps eliminate many of the manual and tedious engineering efforts when productizing ML solutions, resulting in faster time to production, use of fewer engineering resources, higher quality, and better visibility.

Example: Running and Tracking ML Jobs Using Azure Databricks

Let's look at Azure Databricks as a simplified example of running, tracking, and comparing ML jobs, as seen in [Figure 5-2](#). At a high level, this is an end-to-end MLOps solution that allows you to use AutoML to create an experiment, serve a model, and then serve out the model in many different environments, including Databricks itself via a Databricks endpoint or a containerized deployment in Azure, AWS, GitHub Codespace, or others.

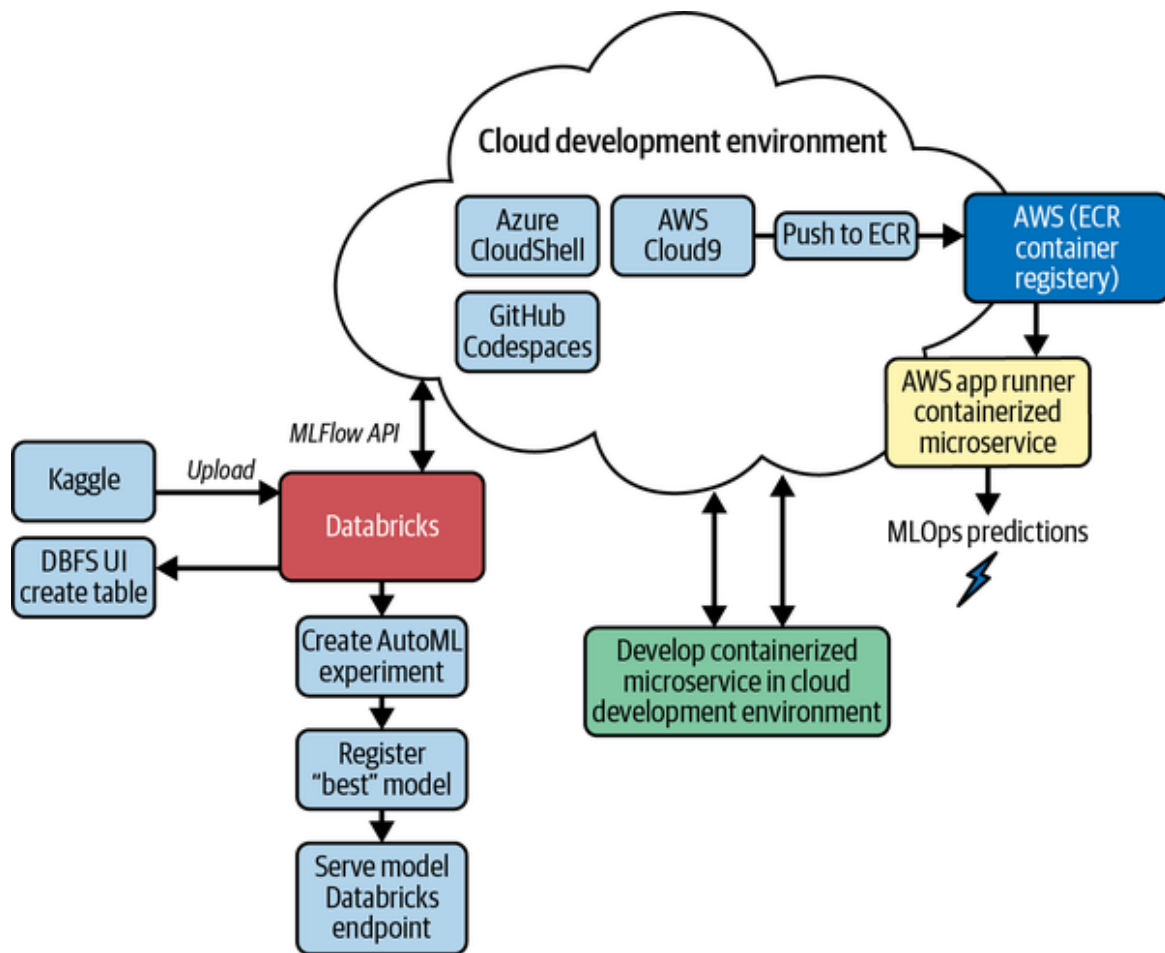


Figure 5-2. Building things in a dedicated environment

Let's break it down further and talk about each step along the way. Notice that a Kaggle dataset on **classifying fake news** begins the journey. Next, that dataset uploads into the DBFS (Databricks File System). The DBFS is a distributed file system mounted into a Databricks workspace and available on Databricks clusters. You can experiment with this workflow with many simple Kaggle datasets. In **Figure 5-3**, the Databricks UI maps out many of the sequential steps in this pipeline, from data to compute to models to serving, and so on.

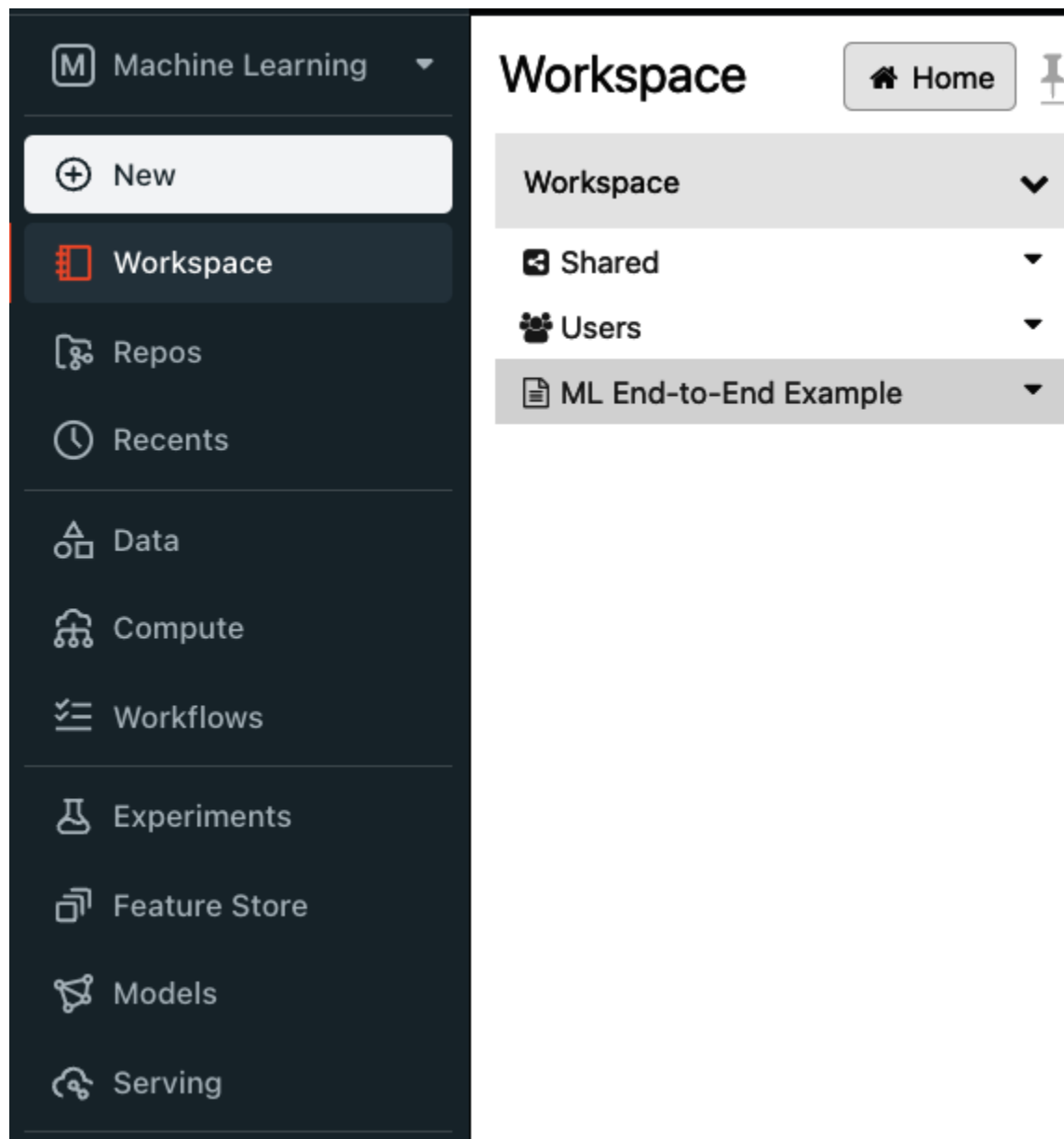


Figure 5-3. Exploring the Databricks UI

One of the dependencies of this end-to-end MLOps pipeline is that a compute cluster is necessary for hosting the DBFS and doing the AutoML. In [Figure 5-4](#), a default cluster in Azure Databricks launches with an exemplary configuration of a minimum of two workers and a max of eight workers, and terminates after 120 minutes.

Once this is up and running, uploading data to the DBFS and running experiments unlocks the ability to run AutoML jobs in [Figure 5-5](#) by dragging and dropping a Databricks dataset into the UI and selecting the

prediction target. The critical inputs to the AutoML experiment are the cluster, the ML problem type (in this case, Classification), the input training, which lives on the DBFS, and finally, the “Prediction target,” which is the column to predict. The Databricks AutoML system does the rest.

One of the very cool features of Databricks is that it will create a notebook for you of the exact training run. Here is the **notebook it generated** for this AutoML project.

+

📄

🔍

🕒

🔗

🏠

☰

🔬

📁

🔊

🔄

Clusters / New Compute

UI preview

Provide feedback

Noah Gift's Cluster

Policy 

Unrestricted 

☒ Multi node ☐ Single node

Access mode  Single user access 

Single user  Noah Gift (noah.gift@gmail.com) 

Performance

Databricks runtime version 

Runtime: 12.1 ML (Scala 2.12, Spark 3.3.1) 

☐ Use Photon Acceleration 

Worker type 

Standard_DS3_v2 

14 GB Memory, 4 Cores 

Min workers

Max workers

☐ Spot instances 

Driver type

Same as worker 

14 GB Memory, 4 Cores 

☒ Enable autoscaling 

☒ Terminate after minutes of inactivity 

Tags

Add tags

Key

Value

Add

> Automatically added tags

▶ Advanced options

Figure 5-4. Databricks cluster configuration

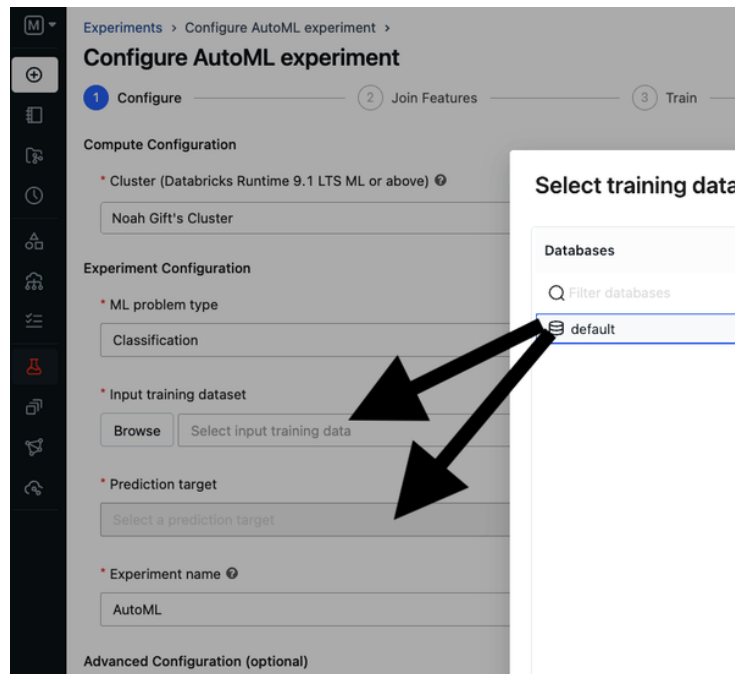


Figure 5-5. Databricks AutoML experiment setup

You can see a great example of this using the default diamonds dataset with the DBFS. Notice in **Figure 5-6** that multiple training runs work to optimize the accuracy metrics, creating notebooks and models as artifacts.

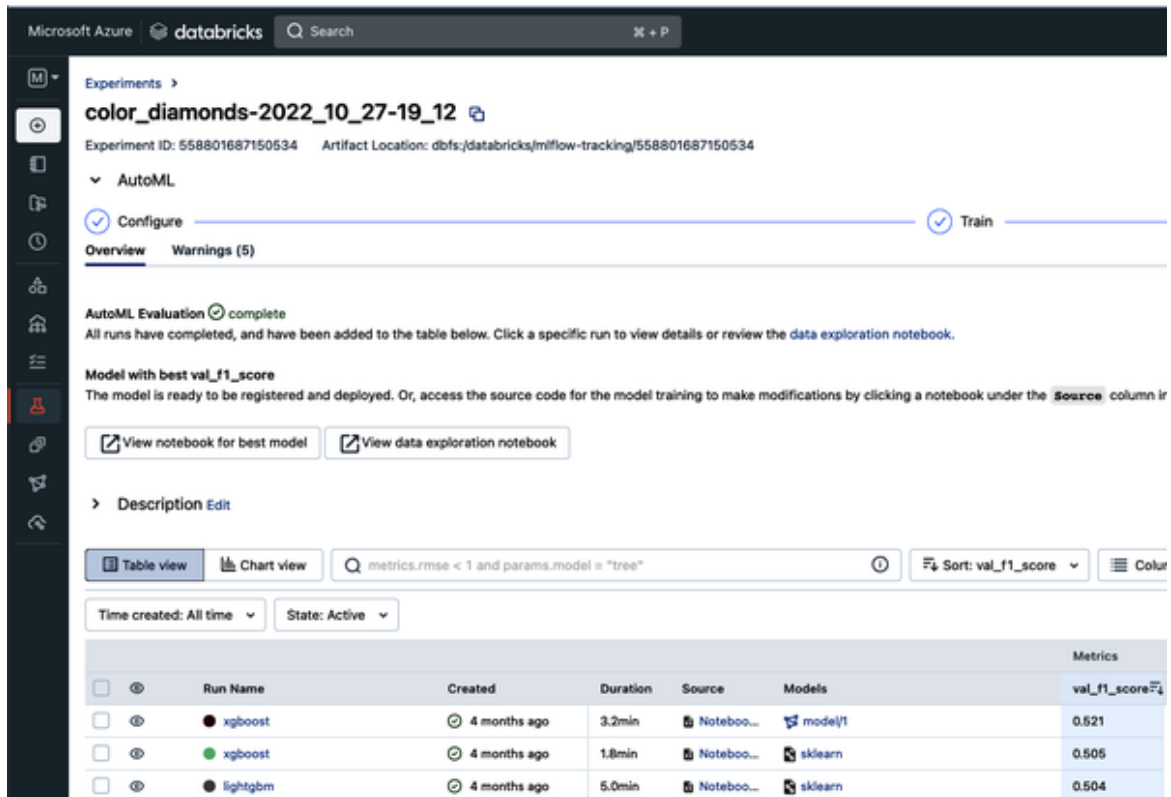


Figure 5-6. Databricks AutoML interface for diamonds dataset

If you drill down into the model further, you can see in [Figure 5-7](#) that there are three modes:

- Real-time, which sets up an endpoint running on virtual machines
- Streaming (Delta Live Tables)
- Batch inference

In a nutshell, the AutoMLOps workflow enables you to build once and produces many styles so you can serve the model in many ways.

Set up model inference

Preview

×

Select one of real-time inference, streaming via Delta Live Tables, or batch.

Real-time

Streaming (Delta Live Tables)

Batch inference

Generates a notebook in your home folder that you can edit.

* Model version

Version 1

* Input table

Input table

Browse

* Output table location

/FileStore/batch-inference/

model

The default output path on DBFS is accessible to everyone in this Workspace. Modify the notebook to disable writing data to DBFS.

Cancel

Use model for batch inference

Figure 5-7. Databricks model inference

Handling Training at Scale

In cases where you need to train a large model or use a large training dataset that doesn't fit into the system memory, you will need to distribute the training job across multiple systems. Distributed training can also shorten the training time by computing the model or processing the data in parallel. In addition, when training a model using hyperparameters or AutoML, the platform can distribute the individual runs across multiple containers and run it in parallel.

Distributed computing frameworks such as **Spark**, **Dask**, **Ray**, and MPI (with **Horovod**) can distribute the training task across computers efficiently. In addition, frameworks like TensorFlow and PyTorch provide integrated distributed training capabilities.

Distributed training adds the complexity of managing a cluster, orchestrating jobs across machines, distributing the data, and collecting and monitoring the results. Use it when the need outweighs the complexity, or use managed services that handle it.

Distributed ML training framework examples include:

- scikit-learn over Dask or Ray
- XGBoost and LightGBM over Dask or Ray
- Spark MLlib
- H2O.ai

Distributed deep learning is covered in more detail in [Chapter 9](#).

Building and Running Multi-Stage Workflows

An excellent example of why you want to use a platform for MLOps is tying together the multiple stages of a lifecycle. These include data collection and preparation, feature engineering, model training, model selection and tuning, and deployment and monitoring. Orchestrating all of this ad hoc is not scalable in the real world.

Developing production-ready ML systems requires coordinating multiple stages in the machine learning lifecycle. In the data collection and preparation phase, issues include:

- Gathering quality training and test data
- Cleaning, preprocessing, labeling, and transforming data
- Splitting data into training, validation, and test sets

In the feature engineering phase, issues include:

- Selecting informative input features for the model
- Creating derived features like embeddings or interactions

- Performing dimensionality reduction if needed

In the model training phase, issues include:

- Choosing a model architecture suitable for the problem and data
- Training on prepared data and iterating with hyperparameters
- Leveraging capabilities like AutoML for acceleration

In the model evaluation phase, issues include:

- Analyzing performance metrics on test data
- Performing error analysis to identify weaknesses
- Tuning model artifacts, such as thresholds, to optimize metrics

In the model deployment and monitoring phase, issues include:

- Containerizing models and integrating into production infrastructure
- Monitoring datasets, model performance, drift, and so on, after deployment
- Retraining models on new data to maintain accuracy

Coordinating these multiple phases requires workflow orchestration tools like MLflow Pipelines, Kubeflow Pipelines, Amazon SageMaker Pipelines, and Azure Machine Learning Pipelines. These platforms provide ways to build reusable components for each lifecycle stage and connect them into an end-to-end automated workflow.

Managing Computation Resources Efficiently

The challenge in MLOps at scale is orchestrating the continuum between stops ingesting data, doing exploratory data analysis, modeling, and then building a conclusion, as shown in **Figure 5-8**.

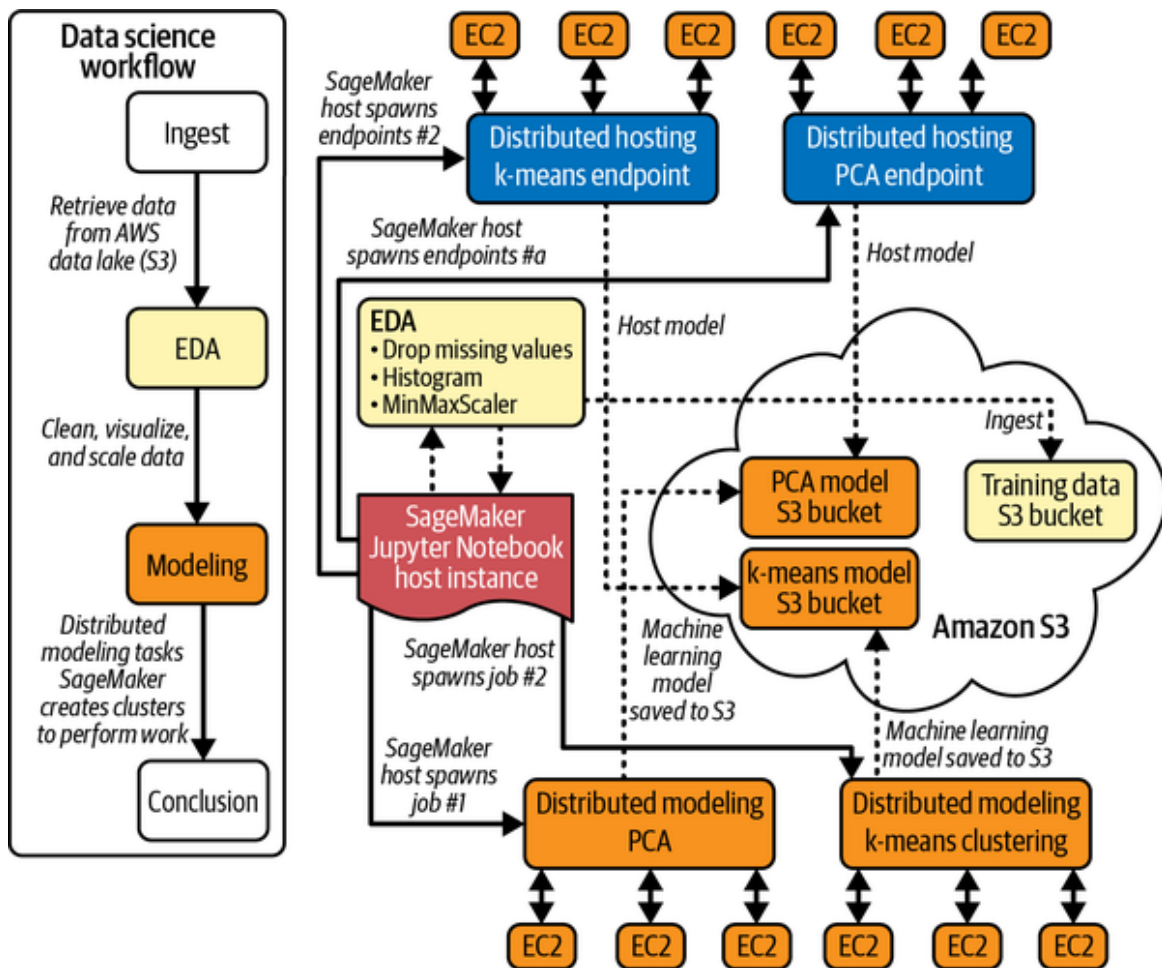


Figure 5-8. SageMaker architectural map of compute and storage

Each step in building and deploying models requires extensive, scalable resources, for example, S3, to store the raw data and the model placeholder. The training jobs also need elastic resources, as do the inference jobs. MLOps is a distributed computing problem, and dealing with a platform is one of the most reasonable ways to solve this difficult problem.

Here are some practical tips for managing computation resources more efficiently in ML training pipelines:

Spot/preemptible instances

Cloud providers like AWS and GCP offer discounted, short-lived compute instances. These options can significantly cut costs for parallel training jobs.

Checkpointing

Save model snapshots periodically during training so progress isn't lost if a spot instance gets terminated. Resume from the last checkpoint.

Distributed training

Train models faster by scaling across multiple GPU/TPU machines. But linear scaling isn't guaranteed, so benchmark speedup versus cost. Also, remember Amdahl's law: parallelization is "no free lunch," and there are diminished returns with scaling out.

Quantization/pruning

Compress models to reduce compute requirements. But beware the impact on accuracy. This technique is an emerging field of research in the deployment of large language models (LLMs).

Caching data

Avoid repeated preprocessing/loading of datasets in each run—cache prepared data on fast storage like solid state drives.

Reuse work

Chain together outputs from previous jobs to avoid redundant computation.

Understand bottlenecks

Profile jobs to identify whether issues are data/network/compute-bound or suboptimal model architecture design.

TIP

The key is measuring and optimizing end-to-end pipeline cost/performance, not individual components. Spotting instances, caching, and reusing intermediate outputs can provide big wins. But balancing cost savings versus impacts on training time, accuracy, and development velocity is also important.

Apple's **CreateML** tool is another excellent example of how vital scalability is. In **Figure 5-9** the **dogs and cats dataset** from Kaggle gets dropped onto the UI to set up a training job.

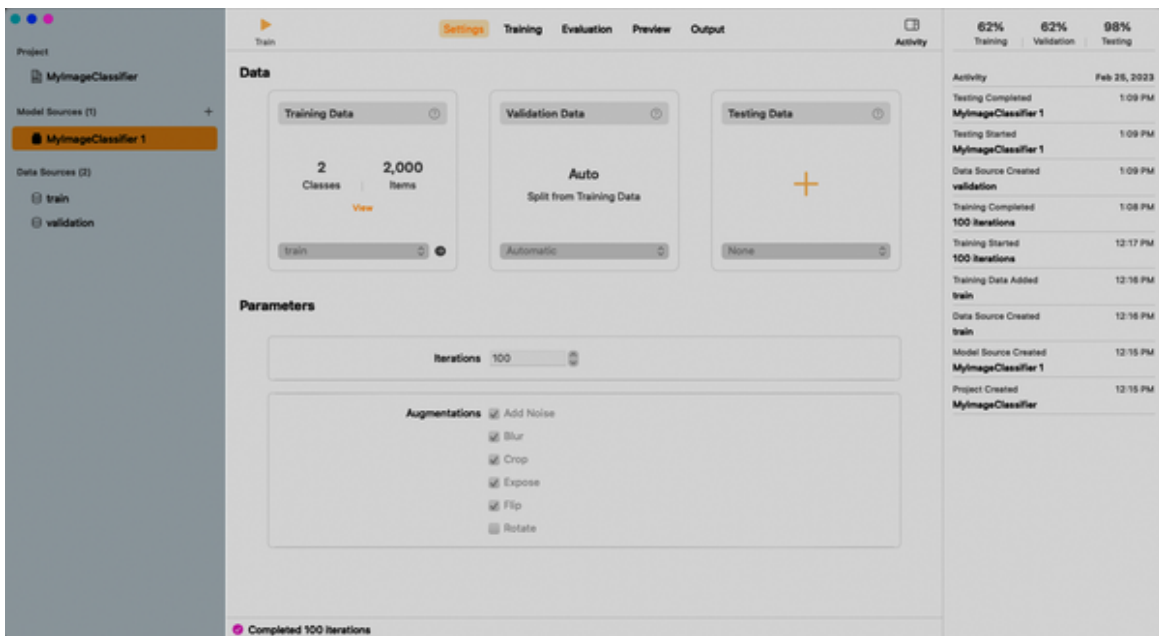


Figure 5-9. Create ML

Next, a Mac Pro M2 Max with 38 GPU cores and 10 CPU cores can fully utilize these GPU cores (**Figure 5-10**). The tooling here is the key, in that fast merging of software and hardware allows a developer to prototype models quickly and then later export them from **CoreML format** to ONNX.

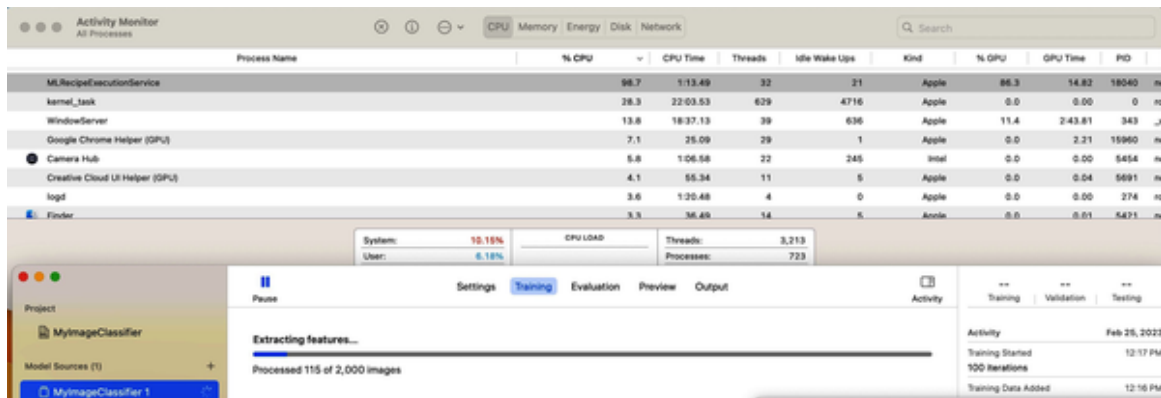


Figure 5-10. Create ML saturating GPU

Yet another example of a high-level framework is **ML.NET** framework shown in **Figure 5-11** saturating the CPU cores as it does AutoML.

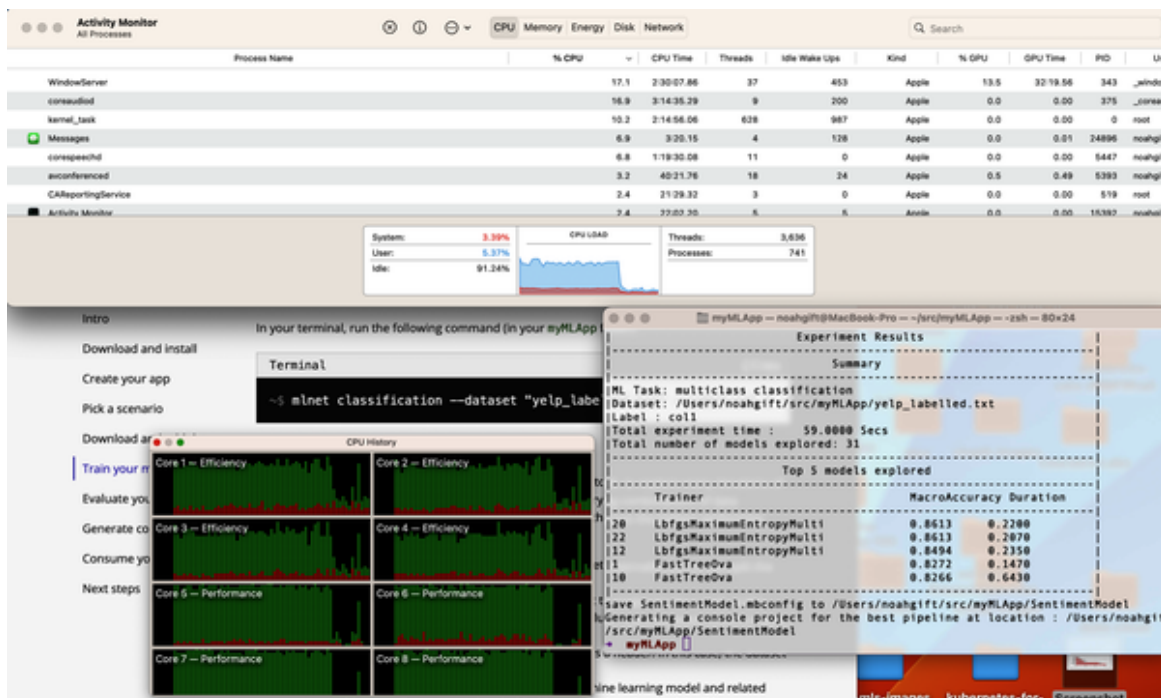
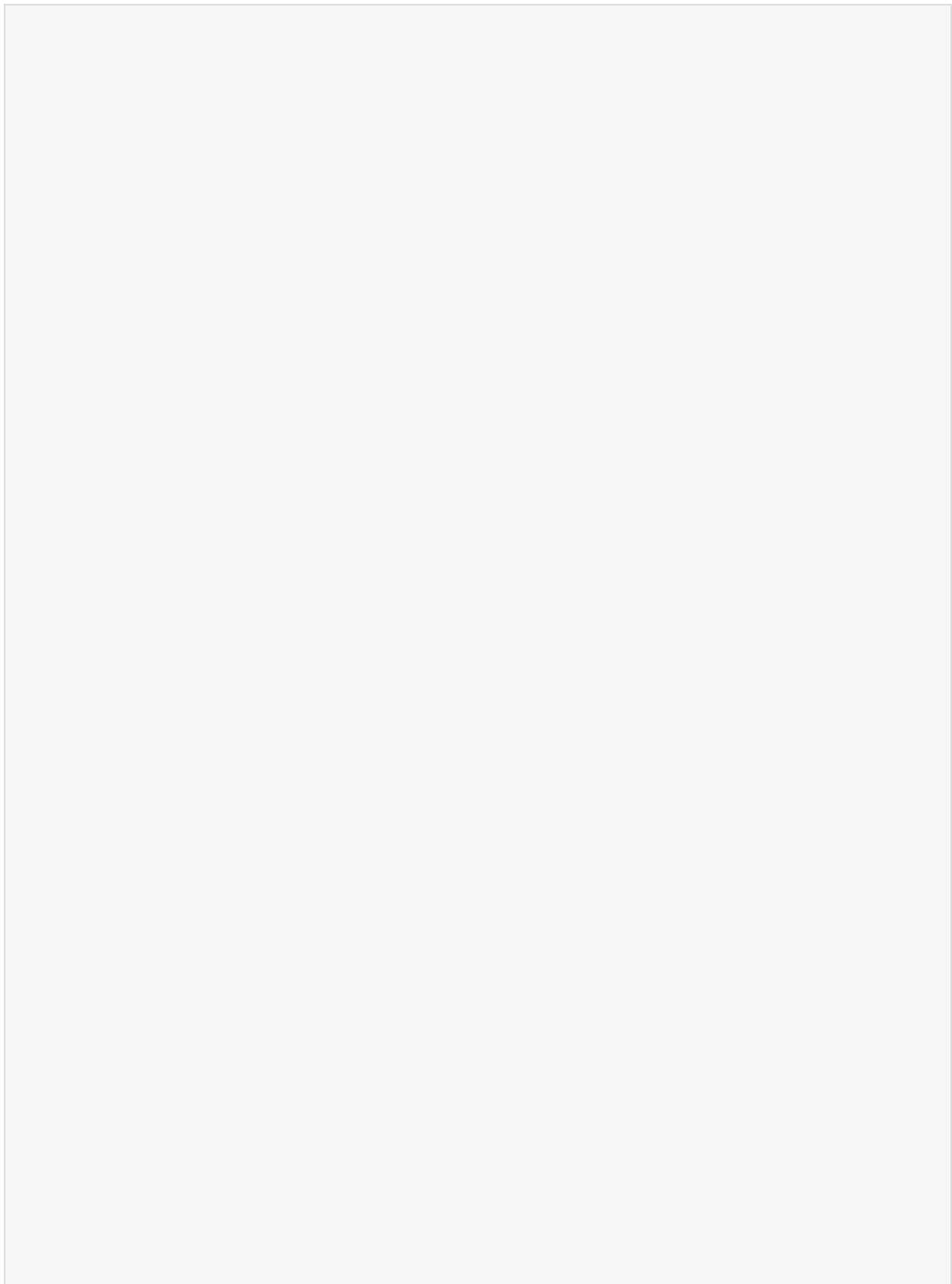


Figure 5-11. ML.NET saturating CPU cores

In the end, not only is there a model created that efficiently works with ONNX but also a console application in C# is part of the framework's build process enabling tight integration with C#, Visual Studio Code, the model format, and ultimately a build once, deploy many tool.

Many emerging examples of tools couple hardware, software, model format, and the ultimate deliverable in the MLOps space, and it is worth

having these frameworks evaluated for your organization’s goals.



GUEST SECTION ON GITHUB ACTIONS WITH AZURE MACHINE LEARNING STUDIO

We contacted O'Reilly author [Alfredo Deza](#), a developer advocate for Azure. He worked extensively with DevOps and MLOps workflows in education and shared this technique:

One of the things I tend to use when working with Azure Machine Learning Studio is to add GitHub Actions, making it a powerful combination. GitHub Actions work with YAML files called *workflow* files. These files are easy to read and, therefore, easy to maintain: an essential mix when working with enterprise-level software.

There are two common patterns I use, and a base to build on further:

- Registering models or datasets from GitHub to Azure ML Studio
- Retrieving specific versions of models for packaging

Any time you are interacting with Azure to create or retrieve data from your account will require authentication. You can authenticate in different ways with GitHub Actions, but I tend to use an Azure service principal. a service principal is a way to create accounts that have limited scope to resources and it needs only one simple command.

Using the [Azure Cloud Shell](#), run the following command to find your subscription ID:

```
az account show --query id -o tsv
```

Capture the resulting ID and use it in the next step, replacing \$AZURE_SUBSCRIPTION_ID with the result from the previous command:

```
az ad sp create-for-rbac --sdk-auth --name "github-actions" \  
--role contributor --scopes /subscriptions/$AZURE_SUBSCRIPTION_ID
```

That command will generate a JSON output for a GitHub repository secret. A *repository secret* is a way to securely store sensitive

information like the one provided by the Azure CLI command. In the GitHub repository where you want to use GitHub Actions with Azure, click Settings > Actions > New. For the name, use AZURE_CREDENTIALS, and for the value, paste the JSON output from the last command.

Although you will need to configure the YAML workflow with more components, this is the step you would use to authenticate to Azure:

```
- uses: azure/login@v1
  with:
    creds: ${ secrets.AZURE_CREDENTIALS }
```

The fact that it requires only three lines to authenticate properly with Azure with an account that has enough permissions allows you to concentrate on the other aspects of your ML project rather than spend time trying to make authentication work. This is one of the main reasons I like GitHub Actions.

I assume that you already have created an Azure Machine Learning workspace and you have access to its **portal**. One common use for the Azure ML workspace is to create and store models in Azure. You can use these registered models with GitHub Actions after authenticating to perform different workflows, including packaging.

The following example shows how to make sure that the Azure CLI will have everything it needs to work with the Azure ML workspace and retrieve the model:

```
- name: set auto-install of extensions
  run: az config set extension.use_dynamic_install=yes_without_prompt
```

Next, replace workspace-name and workspace-group with your Azure ML workspace and resource group, respectively, so that GitHub Actions can attach the workspace to the job:

```
- name: attach workspace
  run: az ml folder attach -w "workspace-name" -g "workspace-group"
```

Finally, you can retrieve a model with a specific version. The version comes after the colon in the value to `--model-id`. The following example uses a GPT-2 model in the ONNX format that was previously registered in Azure ML:

```
- name: retrieve the ONNX model  
  run: az ml model download -t "." --model-id "GPT-2-onnx:1"
```

Although I haven't gone into more details to build an end-to-end example, hopefully I've demonstrated some powerful building blocks you can use to create more complex jobs using Azure ML and GitHub Actions. In the past, I've used these examples to package ML models and deploy them to a container registry. The increased readability of GitHub Actions, with the ease of the Azure CLI, makes this an outstanding combination worth experimenting with.

Conclusion

This chapter discussed the context of building high-quality machine learning models for production, which involved various automation techniques such as AutoML, hyperparameter tuning, auto-logging, AutoMLOps, and pipelines. When building models, the runs and pipelines are tracked, and the different inputs and results are logged to enable higher quality, traceability, reproducibility, and explainability.

Implementing automation and observability in the model development process allows for higher-quality models and continuous development and deployment flows, bringing business velocity.

Critical Thinking Discussion Questions

- What factors influence a model's performance in a production deployment? How can you stack the deck in favor of your organization to achieve a successful outcome?

- Which methods can you implement to improve the model accuracy, and what are the cost versus performance tradeoffs?
- What is your organization's data management and model-building governance policy? Can using an **enterprise data catalog** improve governance?
- How can you protect the privacy of your ML systems' users? Are there approaches that will comply with all near-term government regulations?
- How can you simplify the development process of production ML pipelines for data scientists who need more software development experience without reimplementing their code?

Exercises

- *Experiment tracking*: Run a training task and use MLflow to log the parameters, metrics, and model. Do the same using MLflow auto-logging.
- *AutoML*: Train a machine learning model using cloud services such as SageMaker Autopilot, AzureML, or Google Cloud AutoML. Compare the results to a manually trained model.
- *ML Pipeline*: Build a multi-stage ML pipeline with data preparation, training, and evaluation steps using one of the frameworks mentioned in the chapter, and attach your pipeline to a CI/CD flow (where the pipeline will run anytime you push code or data changes).
- *Building a scalable text classification pipeline*: Train a deep learning model for text classification on a large dataset and develop a scalable pipeline for inference using **Ludwig**.
- *Implementing a model versioning system*: Build a versioning system for models that allows the team to track the evolution of models over time from scratch. The system should follow the model's code, data,

and hyperparameters, and allow for easy rollback to previous versions. Use Git for version control and tools like DVC for managing data and models.

- *Creating a model retraining pipeline:* Develop a pipeline that automates the retraining models as new data becomes available. The project should monitor data sources for changes, retrain the model on the latest data, and deploy the new model to production. Consider using technologies like Apache Airflow for workflow management, Kubernetes for deployment, Amazon SageMaker, or Azure ML Studio.

Chapter 6. Deployment of Models and AI Applications

Processing data, training, and validating models are precursors to the real thing: building and deploying an application that uses the data you generated and the model you have built to drive decisions and actions.

To deliver machine learning applications, start by building and registering the model(s) for use in the production application. Then, create an application **pipeline** that accepts events or data, prepares the required model features, infers results using one or more models, and drives actions. Finally, monitor the data, models, and applications to guarantee their availability and performance. In cases of problems or degraded model performance, drive corrective actions.

Many organizations still think of “serving a model” or creating a model endpoint. However, they need to pay more attention to the bigger picture of delivering an ML application as a whole instead of dividing the application delivery responsibility between data science and engineering teams. Ignoring the bigger picture will lead to significant functionality gaps, failures, unnecessary risks, and long delays.

Model Registry and Management

A *model registry* is a central repository for storing ML models and their metadata and managing the model lifecycle and versions. Once a **model training process** completes, it saves the model and its metadata in the registry. Then different functions (such as **evaluation**, testing, and optimization) extend the model metadata or update the model files. Finally, the serving functions or application pipelines load the model and use it for making predictions.

Model registries provide the following functionality:

- Storing models along with their metadata and labels (tags)
- Managing model access, versions, and lifecycle
- Enabling finding, grouping, and comparing models based on metadata attributes or labels
- Storing information required for the **model deployment** and monitoring
- Tracking the model status and approval process
- Providing a simple or automated mechanism to deploy models into production

The model registry is usually integrated with the experiment tracking system. This way, the essential metadata from the training or experiment is automatically recorded in the registry without manual intervention.

However, most model registry solutions provide APIs to register models trained on other systems.

A model consists of the following data and metadata elements:

Base metadata

Unique model name, identifier, description, project, owner, version information, and so on

Labels

A set of key/value tags used to label, filter, group, and search the model

Model files

The saved model (for example, in PKL, JSON, or HDF5 formats) and auxiliary files used by the model serving process

Tracking information

References to how the model was trained, parameters, data sources, code version, training framework, and so on

Model metrics

Performance metrics collected during the training, evaluation, and testing processes; for example, **model accuracy**, loss, F1 score, ROC curves, and feature importance

Dataset schema

The schema of the model inputs (X) and outputs (Y), including field names, order, and types

Deployment data

Information and parameters required for the model deployment, such as package dependencies, container image, and runtime parameters

Monitoring metadata

Information required for monitoring the model performance or **drift**; for example, statistical information and histograms per feature to determine if there is a drift between training and serving data

Status and state

Information about the current model state, usage, and approvals

In many cases, the training pipelines generate multiple models, for example, when trying different algorithms or parameter combinations. In such cases, we will use different names or labels per model and can compare the models to select the most suitable option. In addition, the same model pipeline may produce multiple models, one for every subset of the data (for example, a model per user, per device, per country, and so on).

Model registries provide APIs and a user interface to create, update, retrieve, list, compare, and deploy models. Model registries are a component of an MLOps or data science platform. For example, open source solutions include **MLflow** and **MLRun**. In addition, there are commercial solutions from **Amazon SageMaker**, **Google Vertex AI**, and **DataRobot**. Although registries can import or export models, the best approach is to use the built-in registry once you choose the MLOps platform.

Solution Examples

Some solutions (for example, in SageMaker and Vertex AI) require you to package the model in a container and provide minimal visibility into the model origin and metadata. This approach may lead to additional work, functional limitations (cannot serve multiple models in the same container), and limited observability.

SageMaker Example

Example 6-1 shows a code example for registering a model in Amazon SageMaker. It covers the following steps that are required to register a model:

1. Save the model and the code in a *tar.gz* package and upload it to S3.
2. Build a container image or use a pre-built Docker image.
3. Create a model package group.
4. Create a model package and specify the information about the model package: image, runtime preference, metadata, and so on.

Example 6-1. Registering a model in Amazon SageMaker

```
import boto3
from sagemaker import image_uris

region = boto3.Session().region_name
```

```

client = boto3.client('sagemaker', region)

# Require you to first package the model in tar.gz and upload to S3

# Specify the S3 location of the model package
model_package_location = 's3://my-bucket/my-model-package.tar.gz'

# Find the image url for a SageMaker built-in inference image
inference_image = image_uris.retrieve(
    framework="sklearn",
    region=region,
    version="1.0-1",
    py_version="py3",
    instance_type="ml.m5.large",
)

# Define the model package metadata
model_package_name = 'my-model-package'
model_package_group_name = model_package_name + "-group"
model_package_description = 'A sample model package'
model_package_framework = 'scikit-learn'
model_package_runtime = 'Python 3.8'

print(model_package_group_name)
group_response = client.create_model_package_group(
    ModelPackageGroupName=model_package_group_name,
    ModelPackageGroupDescription="My group description",
)

model_package_version_response = client.create_model_package(
    ModelPackageGroupName=model_package_group_name,
    ModelPackageDescription="scikit-learn demo",
    ModelPackageVersion='1.0',
    MetadataProperties={
        'GeneratedBy': 'my-username'
    },
    InferenceSpecification={
        "Containers": [
            {
                "ContainerHostname": "scikit-learn",
                "Image": inference_image,
                "ModelDataUrl": model_package_location,
                "Framework": "SAGEMAKER-SCIKIT-LEARN",
                "Environment": {
                    "SAGEMAKER_CONTAINER_LOG_LEVEL": "20",
                    "SAGEMAKER_PROGRAM": "inference.py",
                    "SAGEMAKER_REGION": region,

```

```

        },
    ],
    "SupportedRealtimeInferenceInstanceTypes": [
        "ml.c5.xlarge",
        "ml.m5.xlarge",
    ],
    "SupportedContentTypes": ["text/csv"],
    "SupportedResponseMIMETypes": ["application/json"],
},
)

```

MLflow Example

In MLflow, the experiment tracking service can save model artifacts (with the experiment metadata), and the model registry can register artifacts as models. See [Example 6-2](#), which demonstrates how a training job logs and registers a model artifact.

Example 6-2. Registering a model in MLflow

```

from sklearn import ensemble, metrics
from sklearn.model_selection import train_test_split

import mlflow
import mlflow.sklearn
import pandas as pd

dataset = pd.read_csv("data.csv")

with mlflow.start_run(run_name="YOUR_RUN_NAME") as run:
    params = {"n_estimators": 5, "learning_rate": 0.1}
    model = ensemble.GradientBoostingClassifier(**params)

    # Initialize the x & y data and split to train and test sets
    x = dataset.drop("label", axis=1)
    y = dataset["label"]
    x_train, x_test, y_train, y_test = train_test_split(x, y)

    # Log parameters and metrics using the MLflow APIs
    mlflow.log_params(params)

    # Train the model and log the metrics
    model.fit(x_train, y_train)
    predicted_probs = model.predict_proba(x_test)

```

```

roc_auc = metrics.roc_auc_score(y_test, predicted_probs[:,1])
mlflow.log_metric("test_auc", roc_auc)

# Log the sklearn model and register as version 1
mlflow.sklearn.log_model(
    sk_model=model,
    artifact_path="sklearn-model",
    registered_model_name="sk-learn-reg-model"
)

```

Once the model is registered, it can be viewed in the MLflow UI, as shown in [Figure 6-1](#).



Figure 6-1. MLflow model registry UI

MLRun Example

In MLRun, the training function can use the framework-specific `apply_mlrun()` method to automatically grab all the model details, metadata, data schema, and statistics and save the model in the registry (see [Example 6-3](#)). Notice that MLRun automates data movement and the collection of experiment metadata, parameters, and metrics.

Example 6-3. Registering a model in MLRun

```

import pandas as pd
from sklearn import ensemble
from sklearn.model_selection import train_test_split
from mlrun.frameworks.sklearn import apply_mlrun

def train(
    dataset: pd.DataFrame,
    label_column: str = "label",
    n_estimators: int = 100,
    learning_rate: float = 0.1,
    model_name: str = "cancer_classifier",
):

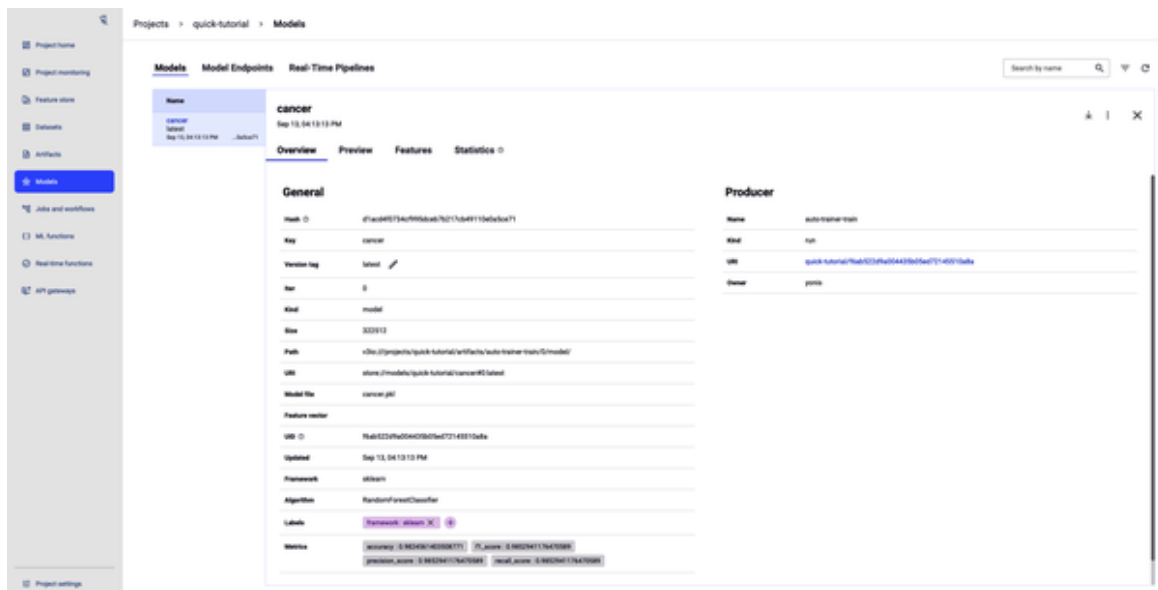
```

```
# Initialize the x & y data and split to train and test sets
x = dataset.drop(label_column, axis=1)
y = dataset[label_column]
x_train, x_test, y_train, y_test = train_test_split(x, y)

# Pick an ideal ML model
model = ensemble.GradientBoostingClassifier(
    n_estimators=n_estimators, learning_rate=learning_rate
)

# Generate and register model artifact along with all its metrics and
metadata
# MLRun auto extracts the model schema and drift metadata from the test set
apply_mlrun(model=model, model_name=model_name, x_test=x_test, y_test=y_test)

# Train the model
model.fit(x_train, y_train)
```



If you have an existing code function that returns a model object, you don't have to add the `auto_mlrun()` method. Instead, MLRun will automatically detect the model object and save it. However, it will not include all the metadata and statistics. You can add those later using the `update_model()` method.

You can register models you trained on other systems with the `project.log_model()` method:

```
model_object = project.log_model('my-model', model_file=model_path, ..)
```

MLRun also provides a simple way to export models and all their metadata into a `.zip` file and load it back into another system, as [Example 6-4](#) shows.

Example 6-4. Export and import MLRun models

```
# In the source platform export the model artifact into a .zip object
model_object.export("s3://my-bucket/model.zip")

# In the destination system import the model files + metadata from
# zip into the project
model_object = project.import_artifact("my-model", "s3://my-bucket/model.zip")
```

Model Serving

Models are a form of equation. They accept numeric values (X) and respond with results or predictions (Y). Models have unique dependencies and development lifecycles. Therefore, it is better to package and deploy them as microservices (containers) and access them through an API. In addition, using an API allows independent scaling of the model (add/remove containers), high availability, granular security, and rolling upgrades.

The most basic approach is manually wrapping the model prediction code with a protocol, for example, using Python [Flask](#) or [FastAPI](#) packages to add HTTP REST API on top of the model. However, this simplistic approach means you must write and maintain a lot of code to handle the different API calls, exceptions, scaling, security, upgrades, and other tasks.

Serving frameworks handle the model deployment, protocol, lifecycle, and monitoring for you. Many of the frameworks require you to build the container package, and they add the deployment, scaling, and so on. Some frameworks (like MLRun) use serverless functions architecture to automatically create the container package and inject advanced

functionality and observability into the serving microservice. In addition, there are managed model-serving solutions in the cloud in which you upload the model and don't need to control the infrastructure.

You can deploy and serve models through an online endpoint (using HTTP REST or gRPC protocols), which accepts the input dataset and either responds with the prediction immediately or through a streaming or messaging protocol; for example, **Kafka**, **Kinesis**, **Pub/Sub**, or others. The streaming or messaging protocol receives the input events, makes a prediction, and writes the results to a database or an upstream stream/queue.

You can deploy models as part of a batch pipeline. For example, the first step is to prepare the dataset. Then the model prediction step generates predictions from the incoming dataset and writes the results to the next step or a storage system. The batch pipeline can run on demand or be scheduled at regular intervals.

Figure 6-3 illustrates different model-serving deployment options: online (synchronous), stream (asynchronous), and batch.

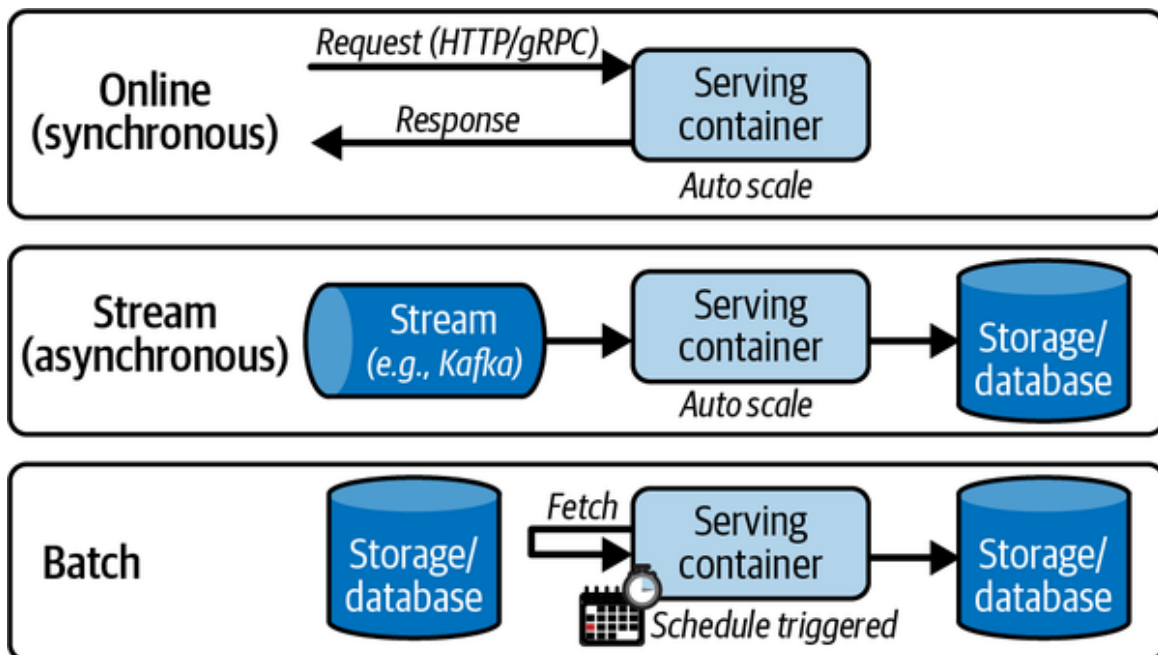


Figure 6-3. Model serving modes

Online serving protocols support multiple operations to handle the entire model lifecycle, for example:

Predict

Send an input dataset and return the predicted results.

Get model metadata

Get information about the model and its schema.

Get health

Get the health and readiness of the model.

List

List the models and the versions served by the endpoint.

Explain

Send the input data and return a description (explanation) of the prediction response.

NVIDIA Triton (TensorRT), **KServe** (KFServing), **Seldon Core**, and **MLRun** support a standard **model serving protocol**.

In the advanced solutions, you can control how models are loaded and evicted from memory, and a model endpoint can serve multiple models to preserve memory space and computation resources. In addition, they can handle data pre- and post-processing and advanced functionality, such as ensembles, canaries, and monitoring.

Table 6-1 lists the leading **model serving solutions**.

Table 6-1. Model serving solutions comparison

Category	SageMaker	Vertex AI	MLRun	Seldon
Open source	No	No	Yes	Yes
Managed option	AWS	GCP	cloud + on-prem	cloud + on-prem
Serverless	Yes	Yes	Yes	No
Protocol	Proprietary	Proprietary	Standard	Standard
Multi-stage pipelines	No	No	Yes	Yes
Streaming	No	No	Yes	Basic
Model monitoring	Yes	Yes	Yes	Yes

Amazon SageMaker

In **SageMaker**, you can retrieve a model from the registry, deploy it to an endpoint, and call it to generate predictions (see the code in **Example 6-5**).

Example 6-5. Deploy a registered model in SageMaker

```
import sagemaker

sagemaker_session = sagemaker.Session()
role = sagemaker.get_execution_role()

# Get the model package from the registry
model = sagemaker.ModelPackage(
    role=role,
```

```

    model_package_arn=model_package_arn,
    sagemaker_session=sagemaker_session)

# Deploy the model as an endpoint
predictor = model.deploy(
    initial_instance_count=1,
    instance_type='ml.m5.xlarge',
    endpoint_name="some-name")

# Test the model by sending a request to the endpoint
test_data = {"input": [1, 2, 3, 4, 5]}
response = predictor.predict(test_data)
print(response)

```

If you train the model with SageMaker’s built-in frameworks, you can skip the part of registering the model and immediately deploy it to an endpoint (see [Example 6-6](#)).

Example 6-6. Deploy a built-in trained model in SageMaker

```

from sagemaker.pytorch import PyTorch

# Train the model using an estimator
pytorch_estimator = PyTorch(entry_point='train_and_deploy.py',
                             instance_type='ml.p3.2xlarge',
                             instance_count=1,
                             framework_version='1.8.0',
                             py_version='py3')
pytorch_estimator.fit('s3://my_bucket/my_training_data/')

# Deploy my estimator to a SageMaker Endpoint and get a Predictor
predictor = pytorch_estimator.deploy(instance_type='ml.m4.xlarge',
                                     initial_instance_count=1)

# `data` is a NumPy array or a Python list.
# `response` is a NumPy array.
response = predictor.predict(data)

```

TIP

SageMaker model serving is a great choice when you build your models inside SageMaker. However, it requires more work when using external or standard open source frameworks. In addition, data processing or application logic is not handled by the serving endpoint and will require external services or serverless functions.

Seldon Core

Seldon Core is an open source model serving solution that can deploy over **Docker** or **Kubernetes**. Seldon can deploy a single model or a multistage pipeline with multiple models and processing steps. In addition, it supports **model monitoring** and explainability.

Seldon Core supports two types of model servers (see **Figure 6-4**):

Reusable

Allows deploying a family of standard models using pre-built images. The models are often fetched from a central repository (such as AWS S3 storage)

Nonreusable

Using a custom model server that requires building a custom Docker image with the code and dependent packages.

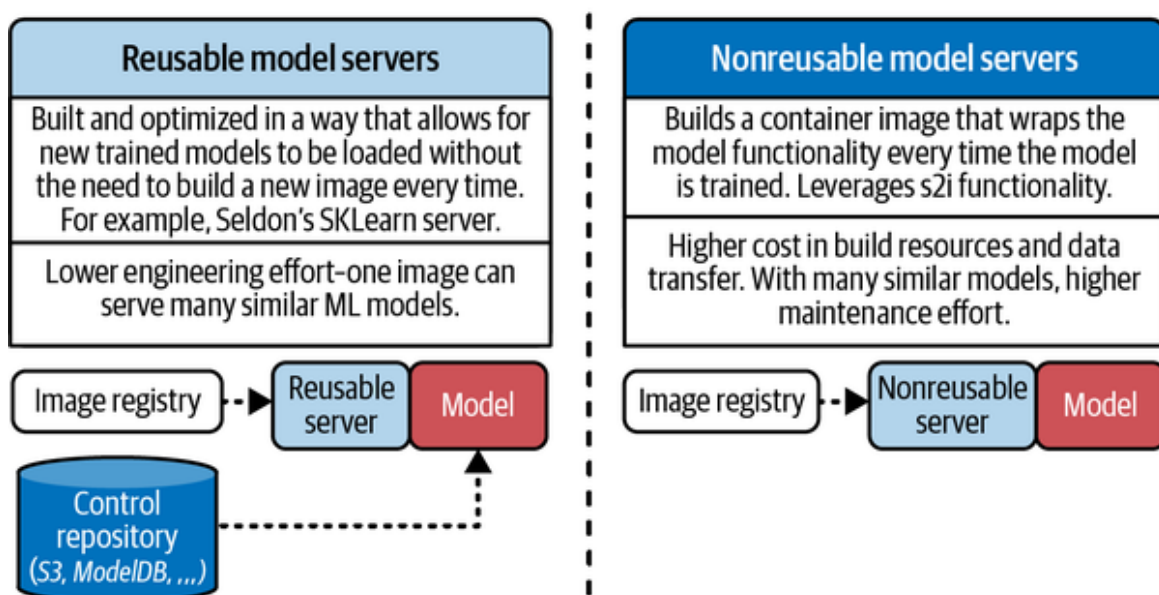


Figure 6-4. Seldon Core model types (source: Seldon Core)

Seldon models and pipelines are defined using a YAML file and deployed using the Kubernetes command-line tool (kubectl). See **Example 6-7**.

Example 6-7. Deploy a model using Seldon Core (source: Seldon Core)

Step 1: Create a YAML file describing a single (reusable) model server:

```
apiVersion: machinelearning.seldon.io/v1
kind: SeldonDeployment
metadata:
  name: sklearn
spec:
  name: iris
  protocol: v2
  predictors:
  - graph:
      children: []
      implementation: SKLEARN_SERVER
      modelUri: gs://seldon-models/sklearn/iris-0.23.2/lr_model
      name: classifier
      name: default
      replicas: 1
```

Step 2: Deploy the model to the Kubernetes cluster:

```
> kubectl apply -f resources/iris-sklearn-v2.yaml
seldondeployment.machinelearning.seldon.io/sklearn created
```

Step 3: Test the new endpoint:

```
import requests

inference_request = {
    "inputs": [
        {"name": "predict", "shape": [1, 4], "datatype": "FP32",
         "data": [[1, 2, 3, 4]]}
    ]
}

endpoint = "http://localhost:8003/seldon/seldon/sklearn/v2/models/infer"
response = requests.post(endpoint, json=inference_request)

print(response.json())
```

Seldon Core supports various model deployment options, multistage pipelines, and a standard (v2) protocol. However, it is more DevOps-oriented and requires manual configuration and an understanding of the Kubernetes API.

MLRun Serving

The **MLRun** MLOps framework includes advanced model and application serving functionality. MLRun serving allows users to define multistage real-time pipelines and quickly deploy them to production with the help of **Nuclio**, a real-time serverless engine. Nuclio is a high-performance, elastic, open source serverless framework focused on data, I/O, and compute-intensive workloads. It supports advanced functionality and many triggering options (such as HTTP, cron, Kafka, Kinesis, and others). MLRun and Nuclio's serverless architecture converts the code and high-level definitions into a hardened, high-performance, self-healing, and auto-scaling service with built-in monitoring and observability.

MLRun serving supports two topology options:

Router

A basic serving of one or more models (the default option)

Flow

A multistage pipeline (directed acyclic graph) with built-in or custom steps (for example, API integrations, data enrichment and processing, model serving, routing, and storage)

MLRun contains built-in serving classes for the major ML/DL frameworks (**scikit-learn**, **TensorFlow**, **ONNX**, **XGBoost**, **LightGBM**, **PyTorch**, and **Hugging Face**) and supports a standard serving protocol (like KServe, Seldon, and Triton). In addition, MLRun provides a few container images with the required ML/DL packages pre-installed, or you can choose a base image and additional package requirements, which will automatically build the desired image for you.

In MLRun, the first step is to define a function object that specifies the code, packages, resources, triggers, and so on. Then you define the serving topology (graph). Once the serving function is fully defined, you can simulate it locally or deploy it to a cluster using a single API call.

Example 6-8 demonstrates the use of MLRun serving. First, define a function using a standard image, add a model, and then simulate and debug the serving pipeline locally with test data. Finally, deploy the function to the cluster and test the live endpoint (using `.invoke()`).

Example 6-8. Define and deploy a basic MLRun serving topology

```
serving_fn = mlrun.new_function("serving", image="mlrun/mlrun",
                                kind="serving", requirements=[])

# Add a model object or file (can be in S3, GCS, local file, etc.)
serving_fn.add_model(
    "my-model",
    model_path=model_uri,
    class_name="mlrun.frameworks.sklearn.SklearnModelServer")

# Create a mock server (simulator) and test/debug the endpoint
server = serving_fn.to_mock_server()
sample = {"inputs": [[5.1, 3.5, 1.4, 0.2], [7.7, 3.8, 6.7, 2.2]]}
server.test(path=f"/v2/models/my-model/infer", body=sample)

# Result:
{'id': '2b2e1703f98846b386965ce834a6c4ab',
 'model_name': 'my-model',
 'outputs': [0, 2]}

# Deploy the serving function to the cluster
> 2023-02-14 12:29:12,008 [info] Starting remote function deploy
project.deploy_function(serving_fn)
> 2023-02-14 12:29:12 (info) Deploying function
> 2023-02-14 12:29:12 (info) Building
> 2023-02-14 12:29:12 (info) Staging files and preparing base images
> 2023-02-14 12:29:12 (info) Building processor image
> 2023-02-14 12:29:57 (info) Build complete
> 2023-02-14 12:30:05 (info) Function deploy complete
# Send prediction request to the live endpoint
> 2023-02-14 12:30:05,918 [info] successfully deployed function
serving_fn.invoke(path=f"/v2/models/my-model/infer", body=sample)
```

The real power of MLRun serving graphs is the ability to develop and deploy complex distributed AI applications rapidly while ensuring maximum performance, scalability, availability, and security. **Example 6-9** shows an example of a multistage NLP application with data pre- and post-processing. You can extend the serving graphs to include branching and parallelism. You can also add advanced data processing steps, model ensembles, exception handling, custom monitoring, and more.

Example 6-9. Define and deploy a multistage serving graph topology

```
# Create an MLRun serving function from custom code
serving_function = mlrun.code_to_function(
    filename="src/serving.py",
    kind="serving",
    image="mlrun/mlrun",
    requirements=[],
)

# Set the serving topology
graph = serving_function.set_topology("flow", engine="async")

# Define a 3 step graph (preprocess -> hugging face model -> postprocess)
# the custom preprocess and postprocess functions are in serving.py
# while the HuggingFaceModelServer is a built-in MLRun class
graph.to(handler="preprocess", name="preprocess")\
    .to(mlrun.frameworks.huggingface.HuggingFaceModelServer(
        name="sentiment-analysis",
        task="sentiment-analysis",
        model_name="distilbert-base-uncased",
        model_class="AutoModelForSequenceClassification",
        tokenizer_name="distilbert-base-uncased",
        tokenizer_class="AutoTokenizer"))\
    .to(handler="postprocess", name="postprocess").respond()

# Plot to graph:
serving_function.plot(rankdir='LR')
```



```
# Deploy the pipeline
project.deploy_function(serving_function)

# Send a text request and get the sentiment results
response = serving_function.invoke(path="/predict", body="good morning")
print(response)

# Result:
['The sentiment is POSITIVE', 'The prediction score is 0.7876932144165039']
```

MLRun serving provides a rich user interface that natively integrates with the platform's other elements (see [Figure 6-5](#)).



The screenshot shows the MLRun serving user interface. On the left is a sidebar with navigation links: Project home, Project monitoring, Feature store, Datasets, Artifacts, and Models (highlighted). The main content area is titled 'Models' and contains a table with columns: Name, Function, Version, Class, Labels, Uptime, Last prediction, Average latency, Error count, and Status. A single row is visible with the following data:

Name	Function	Version	Class	Labels	Uptime	Last prediction	Average latency	Error count	Status
RandomForestClassifier	serving-0	Router	ClassifierModel		Up 1h, 12:10:21 PM	Up 1h, 12:10:21 PM	-	0	OK

Figure 6-5. MLRun serving user interface

In summary, MLRun serving extends the notion of model serving to rapid delivery of application pipelines and accelerating the deployment of AI applications. In addition, its serverless architecture reduces infrastructure costs and engineering overhead and enables continuous operations.

Advanced Serving and Application Pipelines

The previous sections explained the need to transition from looking at the model endpoint as the production end goal to thinking about AI applications. When you build applications, you must address API integrations, data enrichment, validations, processing, and storage. In addition, the same application often requires routing, cascading, or merging results from multiple models and issuing one or more actions. Finally, you must monitor every aspect, including resource usage, data, model performance, and application KPIs. Therefore, you have to define the deployment goals around application pipeline design, implementation, and maintenance.

In the model development flow, the job execution time or frequency may not be critical. However, in production, applications may need to scale to serve thousands of requests and terabytes of data. Sometimes the client is waiting for an immediate answer, which requires more focus on performance and latency. Therefore, enabling parallelism and considering technologies that optimize the data pipeline and model performance are necessary.

You will likely need to upgrade the model or enhance the application pipeline at a certain point. However, upgrades are not trivial when the application serves online clients or critical business services. In addition,

new models may behave differently in production. As a result, you should first test them in isolation or expose only a fraction of the clients to the latest version before making the change available to everyone. Production deployment should include a strategy and implementation for live upgrades, A/B testing, failure recovery, and rollbacks.

Implementing Scalable Application Pipelines

Serving application pipelines execute a set of activities; for example, intercepting an event, enriching and processing the data, using one or more models for prediction, and returning a response or issuing an action. The activities can run sequentially (one after the other), in parallel, or combine sequential and parallel activities. Pipelines can be synchronous (the client waits for the response) or asynchronous (the client does not wait).

A simple sequential implementation uses a single process that calls the different activities one after the other. For example, [Example 6-10](#) demonstrates a sequential application pipeline using FastAPI with the following steps:

1. Reading and enriching the incoming request
2. Data preprocessing to generate a feature vector
3. Model prediction
4. Processing the model results and returning a response to the client

Example 6-10. Sequential application pipeline example using FastAPI

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

# Define the prediction request data (json) structure
class PredictRequest(BaseModel):
    user: str
    # ...
```

```

# API to get model endpoint status
@app.get("/")
async def get_status():
    return {"model": "my-model", "version": 1.0, "status": "ok"}

# API to process data and make a prediction
@app.post("/predict")
async def predict(req: PredictRequest):

    enriched_data = enrich_user(req)

    data = pre_process(enriched_data)

    prediction = model_predict(data)

    return post_process(prediction, req)

def enrich_user(req: PredictRequest):
    ...

```

Suppose you want to distribute the work to multiple microservices or avoid package dependencies. In that case, the primary process can call the activities (implemented through separate microservices) utilizing REST API calls. However, the distributed architecture will require you to handle additional complexities, such as partial failures, retries, service authentication, and rolling upgrades across multiple microservices.

NOTE

In both local and distributed architectures, the flow remains sequential and synchronous, which can lead to slower performance. Performance can be improved when adding parallelism. For example, the primary process can use threads or *async* to execute multiple activities in parallel. However, this adds more complexity to the code.

A way to achieve distributed processing, parallelism, and simplicity is to use asynchronous or streaming pipeline frameworks where you define a graph (DAG) of activities. Then, the framework executes, scales, and tracks the activities using distributed computing resources. In addition, the

application pipeline is monitored, deployed, and upgraded as one managed service.

Figure 6-6 illustrates a few application pipeline architecture options:

- Sequential activities in the same process
- Combining sequential and parallel activities using a few processes
- An asynchronous streaming pipeline

There are multiple commercial and open source distributed pipeline frameworks in the industry. Some examples covered here are **AWS Step Functions**, **Apache Beam**, and **MLRun serving graphs**.

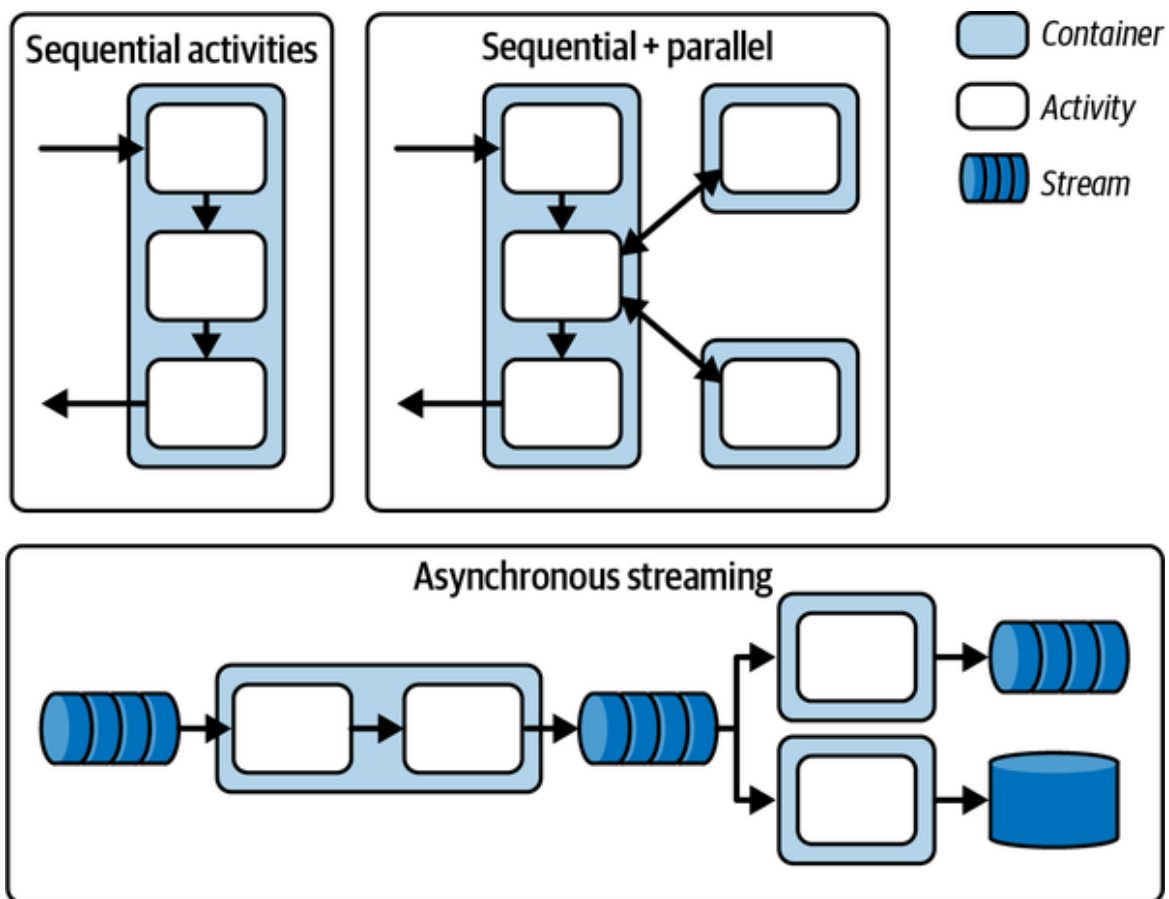


Figure 6-6. Application pipeline architecture options

AWS Step Functions

AWS Step Functions is a workflow service that executes a state machine of individual steps (as shown in [Figure 6-7](#)). Steps can invoke a serverless Lambda function or call AWS services. The Step Functions service controls the execution of the workflows, and its graphical console shows your application's workflow as a series of event-driven steps.

Step Functions has two workflow types. Standard workflows are ideal for long-running, auditable workflows, as they show execution history and visual debugging. Express workflows suit high-event-rate workloads like streaming data processing and IoT data ingestion.

The Express workflows can be either synchronous (wait until the workflow completes and then return the result) or asynchronous (don't wait for the workflow to complete).

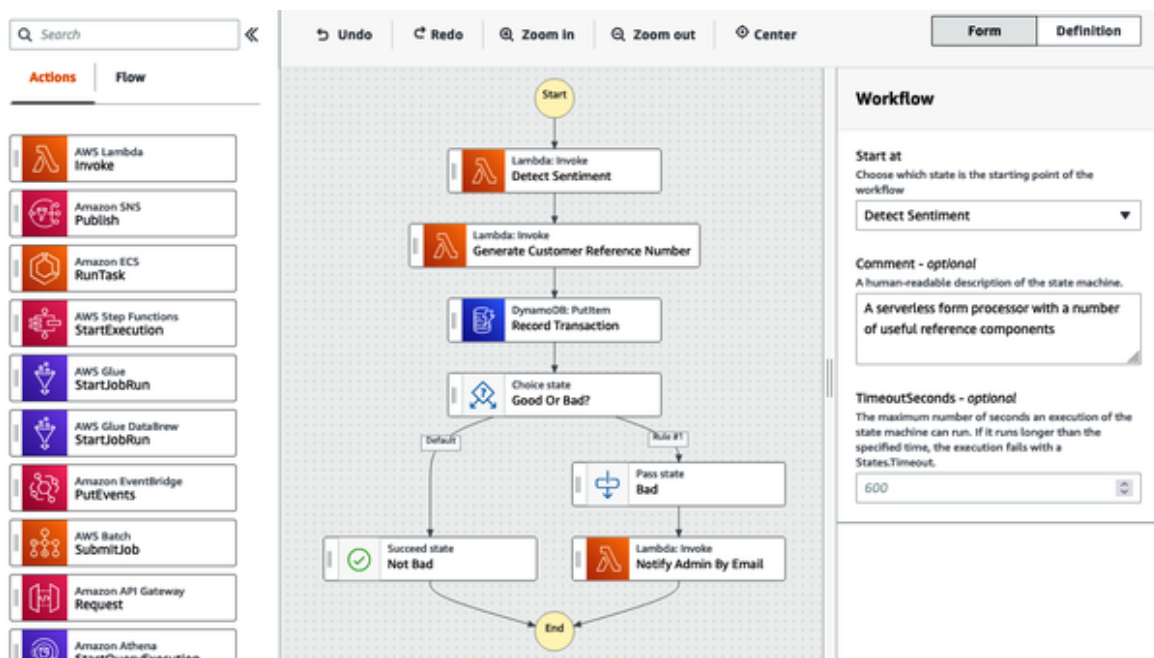


Figure 6-7. AWS Step Functions user interface

While AWS Step Functions is feature-rich, can dynamically scale resources, and has an excellent user interface, it adds the complexity of creating a new Lambda function for every step as well as performance overhead of networking and data serialization between each step.

To build an ML application pipeline with data preparation and model serving steps, you must first create serverless Lambda functions for each

step. For example, [Example 6-11](#) demonstrates a TensorFlow model serving Lambda function.

Example 6-11. Serverless prediction function using AWS Lambda

```
import json
import boto3
import tensorflow.keras.models as models

s3 = boto3.client('s3')

# Load the Keras model from S3 during initialization
model_path = '<your-s3-bucket>/path/to/model.h5'
response = s3.get_object(Bucket='<your-s3-bucket>', Key=model_path)
model_bytes = response['Body'].read()
model = models.load_model(model_bytes)

def lambda_handler(event, context):
    # Load input data from event
    input_data = event['input_data']

    # Make predictions using the preloaded model
    predictions = model.predict(input_data)

    # Return predictions as JSON
    return json.dumps(predictions.tolist())
```

Once you have all the functions, you can define the multistage workflow. [Example 6-12](#) demonstrates a simple asynchronous pipeline with data preparation, model prediction, and post-processing.

Example 6-12. AWS Step Functions workflow example

```
from stepfunctions.steps import LambdaStep, PassStep, Chain
from stepfunctions.workflow import Workflow
from stepfunctions.inputs import ExecutionInput

def create_workflow(input_data):
    # Define the workflow using the stepfunctions library
    with Workflow('MyWorkflow') as workflow:
        # Define the Lambda function to preprocess the data
        preprocess_data_step = LambdaStep(
            'PreprocessData',
            parameters={
                'FunctionName': '<function-arn>',
                'Payload': ExecutionInput(input=input_data)
```

```

    }
)

# Define the Lambda function to load and run the model
run_model_step = LambdaStep(
    'RunModel',
    parameters={
        'FunctionName': '<function-arn>',
        'Payload': ExecutionInput(
            input=preprocess_data_step.output()['Payload'])
    }
)

# Data post processing
post_process_step = LambdaStep(
    'PostProcess',
    parameters={
        'FunctionName': '<function-arn>',
        'Payload': ExecutionInput(input=run_model_step.output(
['Payload']))
    }
)

# Start the execution of the workflow
execution = workflow.execute(inputs={'input_data': input_data})
return execution.execution_arn

# Example usage
input_data = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
execution_arn = create_workflow(input_data)

```

Step Functions does not support streaming protocols such as Kafka or Kinesis. However, you can create a front-end function that will read from a stream and invoke the workflow. See [Example 6-13](#), an example Lambda function.

Example 6-13. AWS Lambda function that reads events from a stream and executes the workflow

```

import os, base64, boto3

sf = boto3.client("stepfunctions", os.environ['REGION'])

def lambda_handler(event, context):
    # Execute the workflow on every stream message
    for record in event['Records']:
        payload = base64.b64decode(record['kinesis']['data'])

```

```

sf.start_execution(
    stateMachineArn=os.environ['WORKFLOW_ARN'],
    input=payload,
)

```

Apache Beam

Apache Beam is an open source stream processing framework, focused on online structured data processing. (Google Dataflow is a managed version of Apache Beam.)

Beam lets you build an asynchronous pipeline (DAG) consisting of multiple steps. The steps can use built-in data, IO, and computing or user-defined functions. Beam pipelines can be executed locally using the **Direct Runner** or deployed to distributed runners such as **Apache Spark**, **Flink**, and **Google Dataflow**.

Beam's advantages are that it is open source, scalable, and contains powerful data operators (such as calculating aggregations over a time window). However, it does not have the flexibility and control over the underline resources and packages per step as with AWS Step Functions.

Example 6-14 defines a three-step pipeline: reading JSON data; preprocessing, prediction, and serializing the output; and writing it to a Kafka stream.

Example 6-14. Apache Beam pipeline with data processing and prediction

```

import apache_beam as beam
from tensorflow.keras.models import load_model
import json

# Define a custom DoFn that parses JSON strings
class ParseJsonFn(beam.DoFn):
    def process(self, element):
        yield json.loads(element)

# Define a custom DoFn that serializes Python dictionaries to JSON strings
class SerializeJsonFn(beam.DoFn):
    def process(self, element):
        yield json.dumps(element).encode('utf-8')

# Define a custom DoFn to make model predictions
class MakePredictions(beam.DoFn):

```

```

def __init__(self, model_path):
    self.model_path = model_path
    self.model = None

def setup(self):
    self.model = load_model(self.model_path)

def process(self, element):
    # Make a prediction using the loaded model
    prediction = self.model.predict(element)

    # Return the prediction
    return [prediction]

# Define the pipeline options
options = beam.options.pipeline_options.PipelineOptions()
options.view_as(beam.options.pipeline_options.StandardOptions).runner =
    'DirectRunner'

# Create the pipeline
with beam.Pipeline(options=options) as p:
    # Read data from a JSON file or message
    messages = p | "Read JSON" >>
    beam.io.ReadFromText("path/to/json_message.json")

    # Parse JSON strings into Python dictionaries
    parsed_messages = messages | "Parse JSON" >> beam.ParDo(ParseJsonFn())

    # Make model predictions using the custom DoFn
    predictions = parsed_messages | 'MakePredictions' \
        >> beam.ParDo(MakePredictions(model_path='my_model.h5'))

    # Print the predictions
    predictions | 'PrintPredictions' >> beam.Map(print)

    # Serialize the processed messages to JSON strings
    serialized_messages = predictions | "Serialize JSON" \
        >> beam.ParDo(SerializeJsonFn())

    # Write the output to a Kafka topic
    serialized_messages | "Write to Kafka" >> beam.io.WriteToKafka(
        producer_config={'bootstrap.servers': 'kafka-host:9092'},
        topic='my-topic'
    )

```

MLRun serving graphs

Many solutions that are used to build application pipelines started as batch or stream processing for structured data (Apache Beam, Flink, **Storm**, **Airflow**, **Spark Streaming**, and so on). Therefore, it's more complicated to expand them to the model serving and monitoring applications or to handle unstructured data like text and video. Other solutions like AWS Step Functions have emerged as a generic way to chain microservices, and they require more customization and logic to handle data processing and model serving at scale.

MLRun serving graph combine the benefits of AWS Step Functions as a versatile serverless function-based pipeline (using Nuclio real-time serverless functions) with the parallel data processing capabilities of Apache Beam and as an easy way to build and debug scalable pipelines. In addition, it adds machine learning and **deep learning** functionality and built-in components (steps).

With MLRun serving graphs, you build a DAG with sources, intermediate steps (tasks), routers, queues, and data targets:

Sources

Real-time (HTTP endpoint, Kafka, Kinesis, and so on) or offline (for example, read data periodically from a file/object/database) data or event inputs.

Steps

Run a function, class handler, or REST API call. MLRun has a list of pre-built steps, including data manipulation, readers, writers, and model serving. You can also write your own steps using standard or custom Python functions/classes.

Routers

A special type of step with routing logic and multiple children. The routing logic defines how the data/events are passed to and collected from the child steps. For example, the basic router class passes the event to a specific child

based on the event content or metadata. The Parallel router passes the event to all the children and merges the results. The Ensemble router is a derivative of the Parallel router, which can intelligently combine the results from multiple child models into one aggregate result. Users can create their own routers and use custom logic.

Queue

Represents a queue or stream that accepts data from one or more source steps and publishes to one or more output steps. Queues are best used to connect independent functions/containers. Queues can run in memory or be implemented using a stream, which allows them to span processes/containers.

Targets

Online or offline storage (streams, files, databases, and so on).

Like Apache Beam or AWS Step Functions, every step in the DAG accepts an event object, manipulates it, and passes event(s) downstream. In the case of MLRun, there is a long list of built-in flow control, parallel data processing, and ML/DL steps ([see documentation](#)), such as filtering, mapping, flattening, micro-batching events, aggregating, joining, encoding, imputing, model serving, model ensembles, and so on. The final result can be written asynchronously to some destination (file, DB, stream, and others) or returned immediately to the caller (synchronously) by marking the result step (responder).

Once users define the serving graph (DAG), they can test and debug it using the built-in simulator (mock server) and deploy it to production over one or more serverless functions with a single command.

Example 6-15 demonstrates a multistage pipeline with data preprocessing, feature enrichment (using MLRun's **feature store**), an ensemble of three models (returning the average result between the three models), and a post-processing step. The function pipeline code contains two custom pre- and post-processing steps (implemented in `_func.py_`) and the built-in `EnrichmentVotingEnsemble` router class.

Example 6-15. MLRun serving graphs with pre- and post-processing and a three leg ensemble

```
# Define a serverless serving graph function
function = mlrun.code_to_function("app-pipe", kind="serving",
                                image="mlrun/mlrun",
                                requirements='requirements.txt')

# Define the graph topology and start with the pre-process step
graph = function.set_topology("flow")

pre_process = graph.to(handler="pre_process", name="pre-process")

# add an EnrichmentVotingEnsemble router with 3 child models (routes)
# The input data will be enriched with feature store features
# nil values will be imputed (with stats from the feature vector metadata)
router = pre_process.to(mlrun.serving.routers.EnrichmentVotingEnsemble(
    name='VotingEnsemble', feature_vector_uri="my-vector", impute_policy=
        {"*": "$mean"}
))
for i, model in enumerate(models):
    router.add_route(f"model{i}", model_path=model.uri)

# Add the post-process step (after the router step)
router.to(handler="post_process", name="post-process")

# plot the graph topology (using Graphviz)
graph.plot(rankdir='LR')
```

Once the function is defined, the pipeline can be simulated by creating and using the mock server. It can then be deployed into production microservices using the `_deploy_function()_` method. See **Example 6-16**.

Example 6-16. Test and deploy the pipeline

```
# Create a mock server (simulator) and test the graph with the test data
server = function.to_mock_server()
resp = server.test("/v2/models/infer", body={"inputs": test_data})
```

```
# Deploy the graph as a real-time serverless function
project.deploy_function(function)
```

```
# Invoke the remote function using the test data
resp = function.invoke("/v2/models/infer", body={"inputs": test_data})
```

MLRun simplifies the migration to production. The observability and model monitoring functionalities are built in. Therefore, there is no need to pile on additional code to collect and report metrics (only to turn on the tracking feature). In addition, users can also define and report custom real-time metrics, which will be collected and shown in the monitoring dashboards, or report errors, which will be centrally logged.

MLRun serving graphs provide flexible configuration of pipeline steps, breaking into the underlining auto-scaling and real-time Nuclio serverless functions, while gaining the best scalability with optimal costs. For example, as [Figure 6-8](#) illustrates, you can specify which steps run on the same microservice (thus eliminating network and serialization overhead) and which ones must spread across microservices (for allowing the use of different open source software/software packages or resources like **GPUs** per the steps).

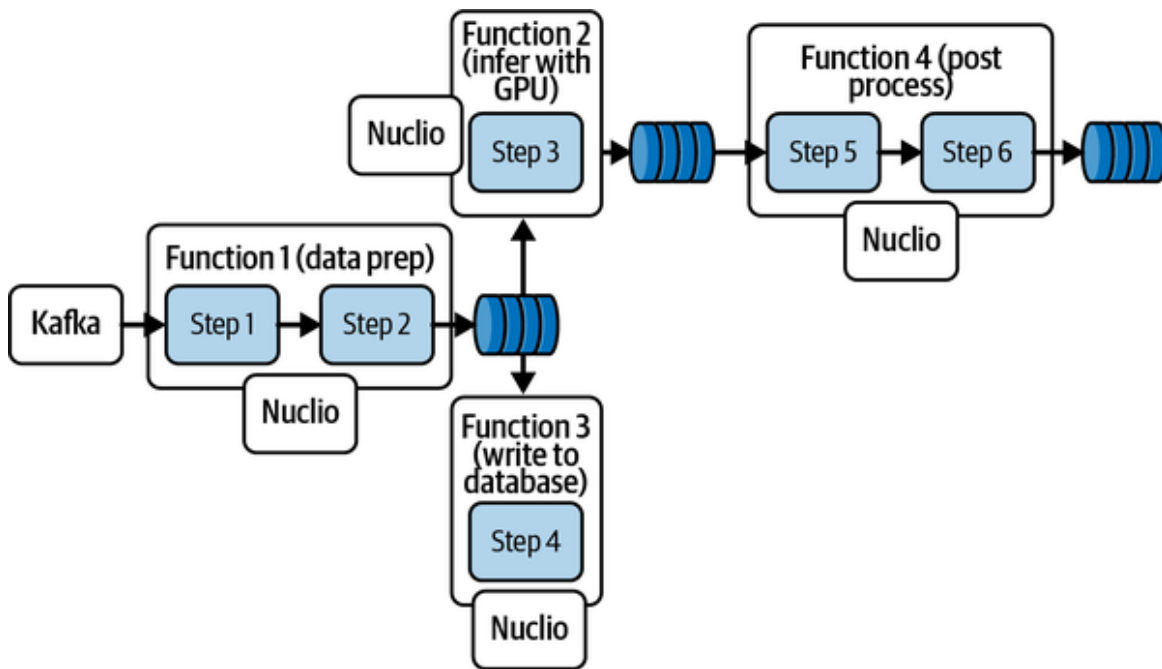


Figure 6-8. MLRun serving graph mapped to multiple Nuclio serverless functions

Model Routing and Ensembles

The basic model serving implementation loads a model into memory and makes a prediction every time a request arrives. However, when you serve multiple models, you may need more advanced topologies to optimize costs, to deliver better results by combining various models, or to dynamically shift traffic from one model to another.

Deploying one microservice per function can be expensive, especially if you don't call the models frequently or if you are using costly compute instances with GPUs or a large memory. In such cases, you may prefer to implement a single microservice function that hosts multiple models in memory and routes the request to a specific model based on the URL or on elements in the request body. (For example, select a country-specific model based on the country code in the request). Furthermore, you can dynamically load and unload models into memory based on their usage (in other words, implement a caching mechanism) to reduce memory consumption.

While model caching solutions can reduce costs and memory consumption, they can also add delay to the first (noncached) request. You should use caching only when you are not sensitive to the first request latency.

Different models can come with different software package dependencies or resource requirements (for example, they need a GPU). In such cases, you should deploy models using separate container microservices, each with a specific package and resource requirements, or group models into containers based on those requirements and route the traffic to the particular microservice that hosts the relevant model.

Collocating data processing with models in the same microservice can help increase performance by keeping the different transformations and prediction activities in memory versus writing and reading into storage between steps. However, this does not allow fine-grain scaling (scaling a specific task in the pipeline) and requires aggregating the resource and package requirements. Therefore, consider the tradeoff between the two approaches based on your needs. MLRun serving graphs provide a simple

way to specify which steps are collocated in the same container and separated into individual containers.

A common mechanism to improve the accuracy of a model is to combine different models that were trained using different datasets, algorithms, or parameters. For example, train one model with seasonal data (transactions done in the same period of the year) and another with temporal data (recent transactions) and combine the results from both models to preserve the seasonal and temporal effects on the scoring result. Another example is combining machine learning and deep learning models and returning the average result.

Model ensemble is a technique in machine learning where multiple models are trained and their predictions combined to make a final prediction and, in this way, improve the overall model performance.

There are two common approaches to creating an ensemble:

Bagging

Training multiple models independently on different subsets of the training data and then combining their predictions using some aggregation technique, such as averaging or majority voting. Use bagging to reduce the variance of the predictions.

Boosting

Training multiple models sequentially, where each model is trained to correct the errors of the previous model. Use boosting to reduce the bias of the predictions.

TIP

It is essential to monitor the performance of each submodel in an ensemble and, in this way, understand which approach yields better accuracy. In some cases, you may want to control the weights between models in an ensemble based on the individual performance. (For example, favor the temporal model due to the higher impact of recent events.)

The Multi-Armed Bandit (MAB) algorithms (such as epsilon-greedy, UCB, or Thompson sampling) can be used to adaptively select the best-performing model or a combination of models over time. MAB algorithms can assign a score to each model that reflects its expected performance and uncertainty. The system can then direct more inference requests to the top-performing models while continuing to explore other models and updating their scores.

Model Optimization and ONNX

Some models, especially deep learning models, can consume significant computation resources. Using optimization techniques, you can reduce resource consumption and improve performance. Some examples of optimizations include the following:

Feature reduction

Reducing the size of the feature vector by removing features that do not add significant value to the result.

Code optimization

Moving critical sections of the code to faster binary code implementations (for example, in C, C++, Go, Rust).

Hardware acceleration

Using GPUs, TPUs, or FPGAs can significantly improve inference performance by offloading computations from the

CPU to specialized hardware that is optimized for matrix multiplication.

Quantization

Reducing the precision of the model's weights and activations, typically from 32-bit floating-point to 8-bit integer precision.

Pruning

Removing some of the model's weights or neurons that have little impact on the model's accuracy.

Model compression

Reducing the size of the model by compressing its weight, through techniques like weight sharing, low-rank factorization, or Huffman coding.

These frameworks can optimize models:

ONNX Runtime

A high-performance engine for executing ONNX models. It provides hardware acceleration support for CPUs, GPUs, and FPGAs and supports model compression techniques, such as quantization and pruning.

Intel OpenVINO

A toolkit for optimizing and deploying machine learning models on Intel hardware, including CPUs, GPUs, and FPGAs. It supports model compression techniques, such as quantization and pruning, and provides optimized libraries for deep learning operations.

NVIDIA TensorRT

A high-performance inference engine for NVIDIA GPUs that supports model compression techniques, such as pruning and quantization. It provides hardware acceleration support for NVIDIA GPUs and includes optimized libraries for deep learning operations.

ONNX is an open format built to represent machine learning models. ONNX defines a common set of operators—the building blocks of machine learning and deep learning models—and a standard file format to enable AI developers to use models with various frameworks, tools, runtimes, and compilers.

The ONNX runtimes run in machine native (binary) code and support model compression techniques and hardware-specific optimizations, which deliver significantly faster inference performance.

Data and Model Monitoring

Monitoring solutions can be broken into three main layers:

Resource monitoring

Monitoring the resources (CPUs, GPUs, memory, storage) used by the ML application, as well as their health and the service's availability

Model and data monitoring

Monitoring the performance of the model and the data used by the model (accuracy, drift, bias, data quality, and so on)

Application monitoring

Monitoring the overall application performance (throughput, latency, errors, and so on) across all pipeline steps and measuring the business KPIs defined for the application

You can use the same solution to monitor all three layers or different services per layer. In any case, it is essential to correlate the information across layers (using tags and labels) since resource or model performance problems will usually impact the higher layers of the application.

Cluster monitoring solutions can monitor resources. For example, in **Kubernetes**, the typical answer is to use **Prometheus** and **Grafana** to track the microservice resources. But first, you need to determine which model or application is served by which container (and this can change dynamically). Therefore, when deploying the models as Kubernetes resources (containers, pods, and so on), you should label them with the model and application information.

If you use public clouds, you can use managed cloud services for resource monitoring, such as **Amazon CloudWatch**, **Azure Monitor**, **Google Cloud Monitoring**, **Datadog**, and **New Relic**.

Model, data, and application performance metrics can also be reported in the traditional resource and application monitoring solutions. However, the real challenge is to collect this information and reference metadata, and to keep it cost-effective and scalable, given the enormous volumes of data collected.

Model and data monitoring solutions have unique challenges; they compare the data and model performance in production with reference data collected at the model development and training phases to calculate accuracy and drift. In addition, data is collected for every model request and must be stored for real-time dashboards and offline access, while traditional resource and service monitoring stores only use sampled metrics. Furthermore, there are no one-to-one relations between the microservice and the model, and the same container can host multiple models, ensembles, and so on. Finally, model and data monitoring solutions must also work for batch workloads, still a dominant place where models are used.

Most model monitoring solutions are limited to structured (tabular) data and do not support unstructured data (text, images, videos, and so on). However,

you can address unstructured data by creating a transformation from unstructured data to tabular data and monitoring the results. For example, you can convert an image to numeric RGB values or detected object metrics.

Application monitoring spans the different stages of the ML application (data enrichment, preparation, model prediction, actions, and so on). It looks into application-level metrics, such as overall requests latency and throughput, application errors, application-level metrics, business metrics, KPIs, and others.

Multiple versions of the same application pipeline may run in parallel (for example, in the case of A/B testing). Therefore, you want to compare the application KPIs across versions, not just the model performance, since a better model does not necessarily reflect a better KPI. For example, if the model does not respond at the right time or if the actions following the prediction do not generate the right impact.

Considering application-level monitoring ahead of the design and implementation is best since it requires custom instrumentation in multiple application junctions and ways to collect and use reference data for KPI measurements.

Monitoring results are shown in dashboards, but they can bring more significant value when they trigger alerts and corrective actions. For example, model drift indication can start a retraining flow, change the weights in a model ensemble, or send critical notifications to administrators to correct the problem. Therefore, the solution should provide a mechanism to easily define conditions, thresholds, and actions.

Two types of solutions for model monitoring are described in the following sections:

Integrated

An integral part of a data science or MLOps platform. It usually has fewer features but does not require glue logic and separated management.

Standalone

Dedicated monitoring solutions are usually feature-rich but require manual integrations and separate management.

Integrated Model Monitoring Solutions

Data science or MLOps platforms support the tasks required to develop and deploy models. When you deploy the model, the platform often provides basic model monitoring, which can be operated with minimal configuration. Integrated monitoring solutions show the model endpoints with essential performance and health information. Most platforms support drift detection and a few support additional monitoring classes.

Amazon SageMaker

Amazon SageMaker supports data and model drift detection (see the architecture in **Figure 6-9**). The model endpoints capture incoming requests and model results into S3 objects and compare them with baseline and ground truth datasets to calculate the drift and accuracy metrics.

In SageMaker, you manually generate and upload the reference datasets into S3 and then define a scheduled model monitoring job (see **Amazon SageMaker Model Monitor**).

You can view the model metrics and drift indications in the SageMaker UI (see **Figure 6-10**).

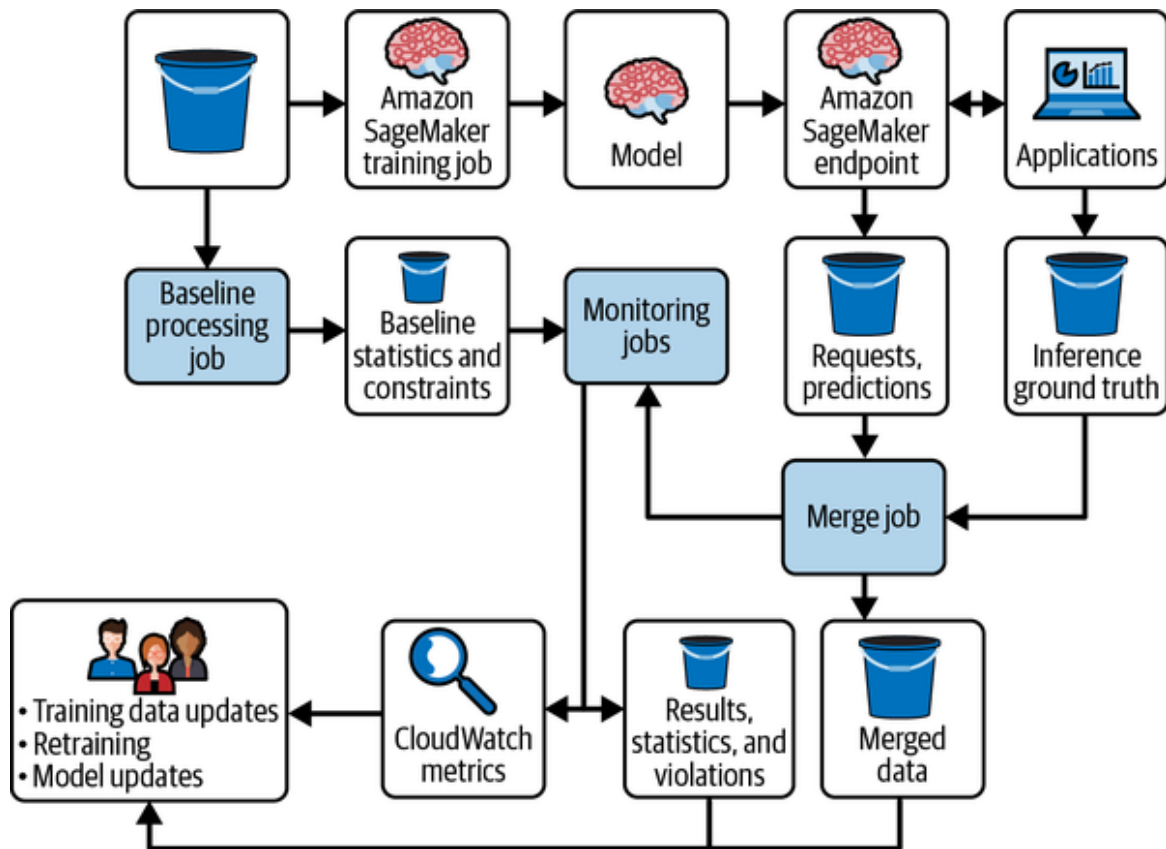


Figure 6-9. SageMaker model monitoring architecture

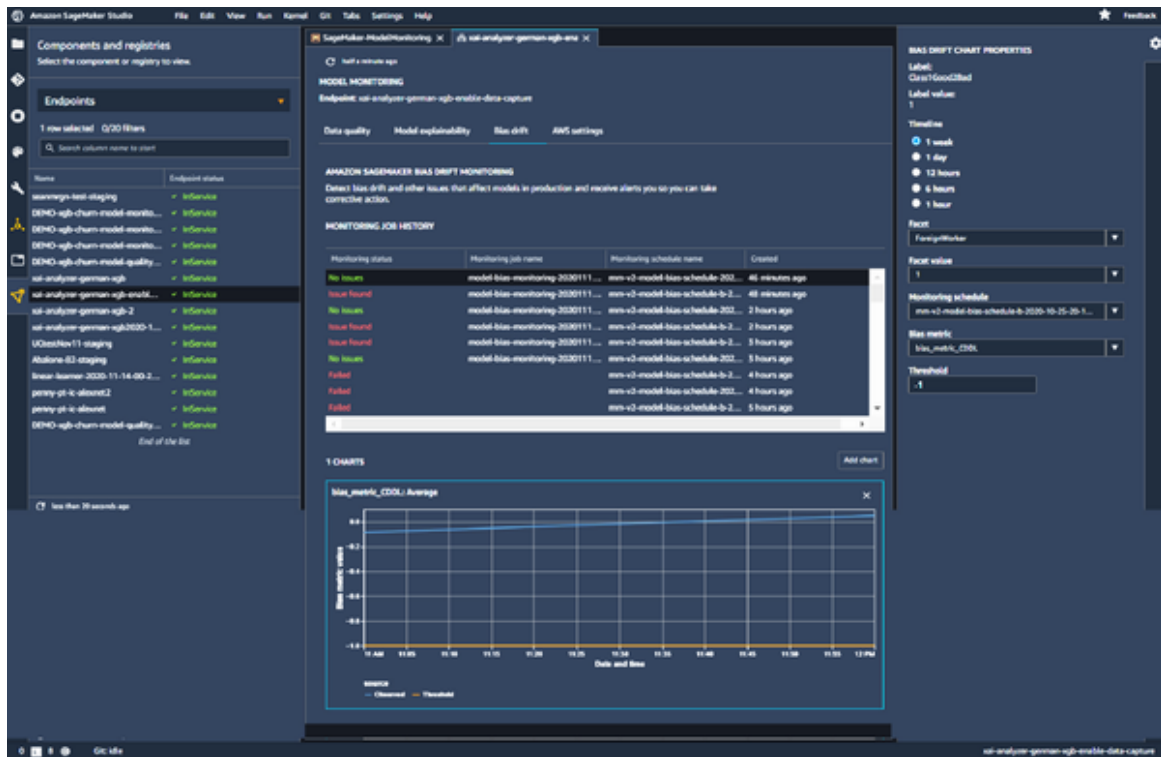


Figure 6-10. SageMaker model monitoring UI

Google Vertex AI

Google **model monitoring** supports tracking model requests and results into a BigQuery table and can issue email alerts when the specified threshold is crossed. **Figure 6-11** demonstrates how the user enables the monitoring, sets the monitoring policy, and uploads the reference data (schema and statistics).

Edit endpoint

1 Define your endpoint

2 Model settings

3 Model monitoring

4 Monitoring objectives

UPDATE

CANCEL

1

Model monitoring applies to all models deployed on this endpoint.
[Learn more](#)

Model monitoring

Models used in production require continuous monitoring to ensure that they perform as expected. Use model monitoring to track training-serving skew or prediction drift, then set up alerts to notify you when thresholds are crossed. [Learn more](#)

Model monitoring supports AutoML tabular and custom-trained models.

Enable model monitoring for this endpoint

Monitoring job display name *

vertexai_model_monitoring_job_4_telco_churn

?

Define the display name of the monitoring job

Monitoring window length *

1

hours

The number of hours a monitoring job will run. After a job ends, a new job will start. A short window is good for endpoints with high prediction traffic, while a long window is useful for endpoints with low prediction traffic. Default window size is 24 hours.

Alert emails *

lolejniczak

Enter one or more email addresses to receive an alert when a model exceeds an alerting threshold

Sampling rate

Sampling rate *

100

%

The percentage of prediction requests (within the monitoring window) to sample. A higher sampling rate will incur more storage and processing charges but may yield more accurate results.

Figure 6-11. Configuring Google Vertex AI model monitoring

The essential model endpoint metrics and drift information are visualized in the UI (see Figures 6-12 and 6-13), and you can access the complete data through BigQuery.

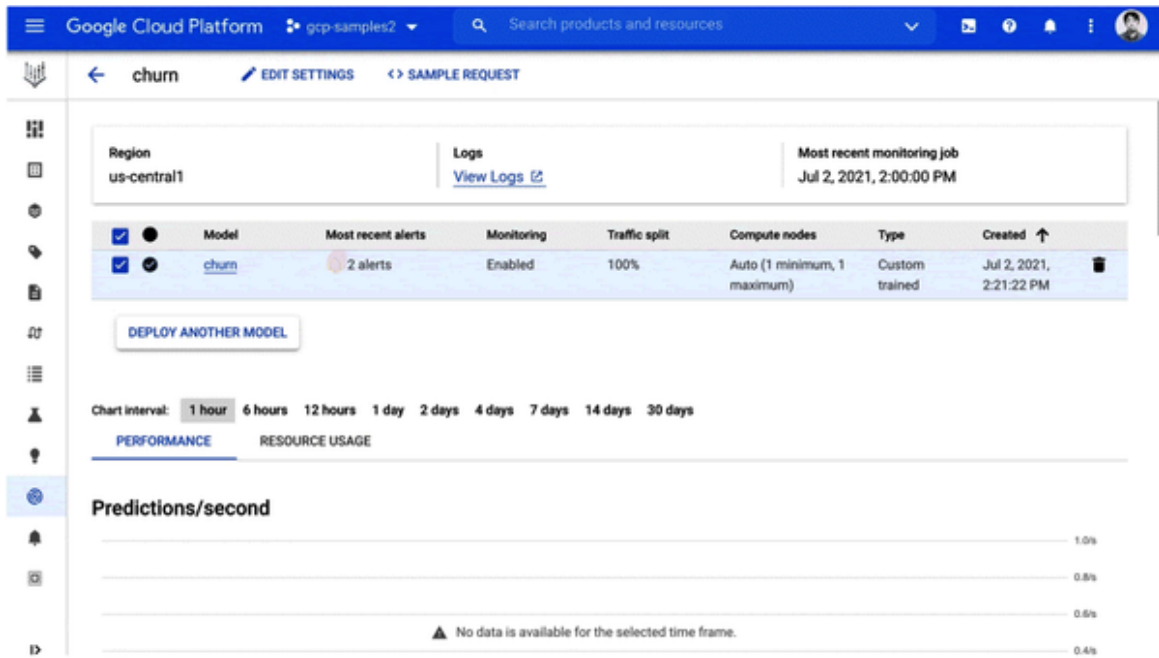


Figure 6-12. View Google Vertex AI model endpoints

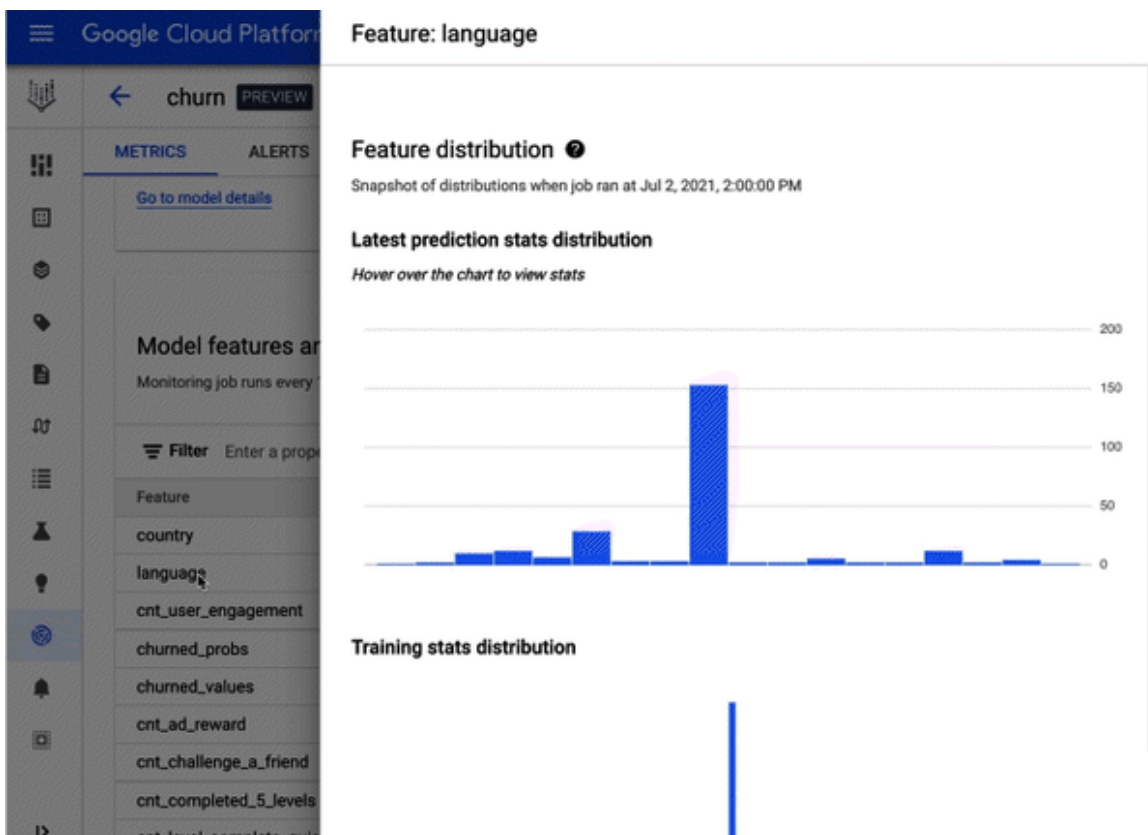


Figure 6-13. View Google Vertex AI model endpoint feature skew

MLRun open source MLOps includes an integrated **monitoring service** for batch and real-time workloads. As shown in **Figure 6-14**, MLRun model serving endpoints write the performance, inputs, outputs, and user-defined metrics into a stream. Then, a real-time serverless Nuclio function reads and processes the data and writes the results into different types of storage (key/value, time series database, and Parquet files). Scheduled MLRun jobs run periodically, read the data, calculate various metrics (drift, accuracy, and so on), and trigger appropriate alerts.

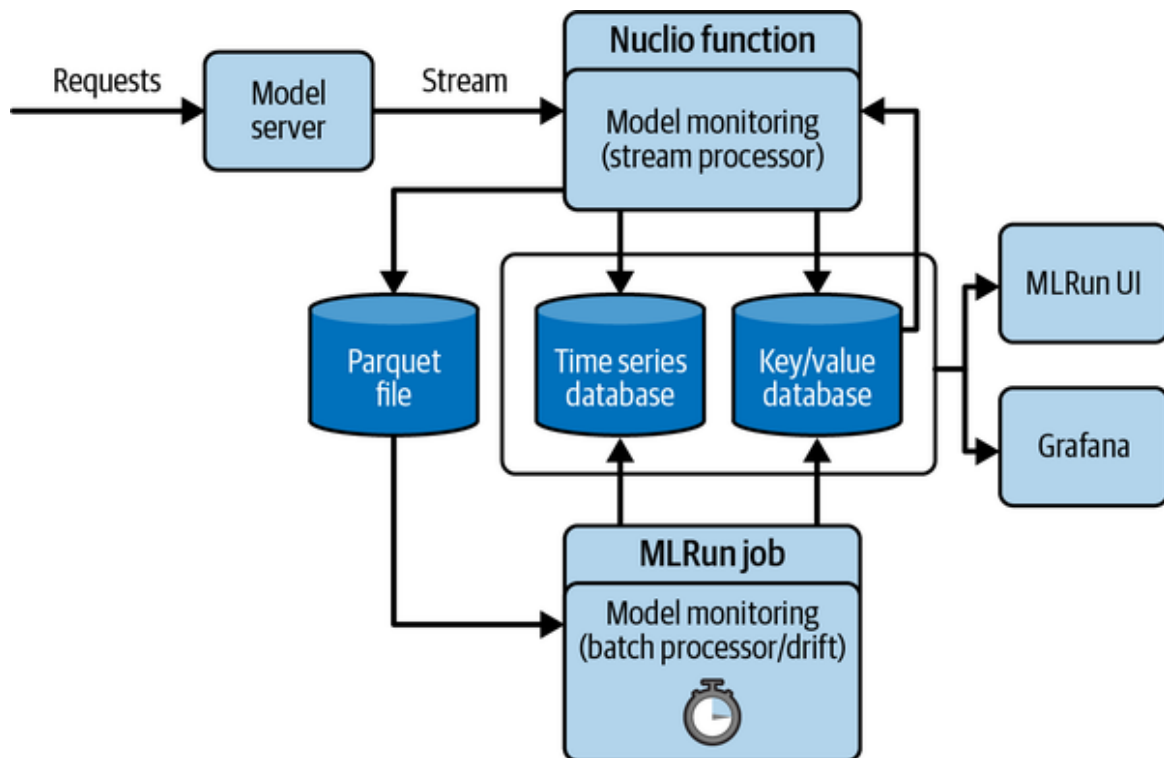


Figure 6-14. MLRun model monitoring architecture

You can view the model endpoint information in the MLRun UI (see **Figure 6-15**), or in Grafana (see **Figure 6-16**); the production datasets are stored in MLRun's feature store and can be used for post-production analytics (for example, analyzing data quality, bias, and explainability) or used for retraining a model.



Figure 6-15. MLRun model endpoint features histogram



Figure 6-16. MLRun model endpoint in Grafana

MLRun eliminates many engineering efforts by automatically generating reference and production datasets, managing the data assets and lifecycle, scheduling and scaling monitoring tasks, and more.

When you train the models in MLRun, the reference data (schema, statistics, and so on) is auto-generated and saved with the model. You can also update the reference data manually through the API. To operate the monitoring functionality, you should apply the `_set_tracking()` option in the serving function.

MLRun supports monitoring plug-ins and has extensibility to support advanced monitoring applications for structured and unstructured data.

Standalone Model Monitoring Solutions

Several solutions are dedicated to model monitoring. They usually have more advanced features and user interfaces than the integrated options, but they require manual integration with data assets, serving, and training frameworks. In addition, they require working with multiple management consoles.

Examples of commercial offerings include [Aporia](#), [Arize](#), [WhyLabs](#), and [Mona](#).

The commercial frameworks usually support multiple monitoring applications (drift, accuracy, data quality, and so on), friendly and rich user interfaces, advanced policies, and multiple alerting and triggering options (email, Slack, webhooks, and more).

You can see an example Aporia UI wizard for creating a new monitoring task in [Figure 6-17](#).

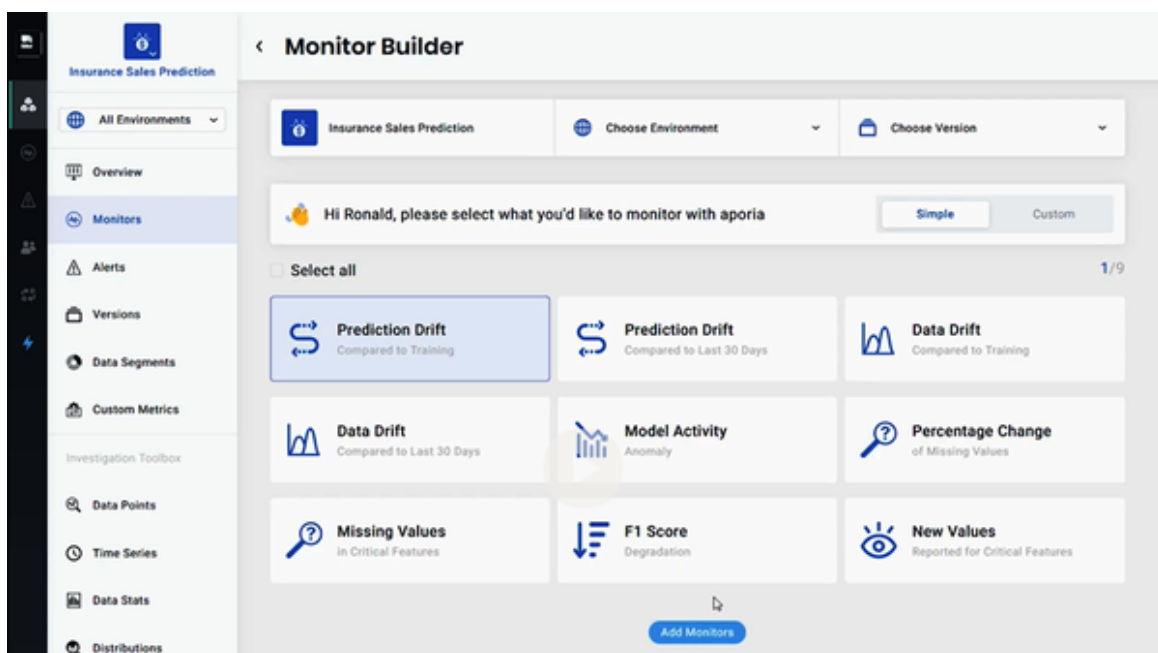


Figure 6-17. Aporia new monitor wizard

The challenge is uploading the prediction and reference data to storage and managing the data lifecycle, rather than getting it out of the box in integrated platforms such as MLRun. See the code example in [Example 6-17](#) for uploading batch prediction data. (Real-time prediction data requires an additional step of conversion from a stream to Parquet files).

Example 6-17. Write prediction inputs and outputs into a Parquet file (source: Aporia)

```
import fastparquet

# Preprocess & predict
X = preprocess(...)
y = model.predict(X_pred)

# Concatenate features, predictions and any other metadata
df = ...

# Store predictions
fastparquet.write(
    filename=f"s3://my-models/{MODEL_ID}/{MODEL_VERSION}/serving.parquet",
    data=df,
    append=True,
)
```

An example of an Aporia monitoring dashboard is shown in Figure 6-18.

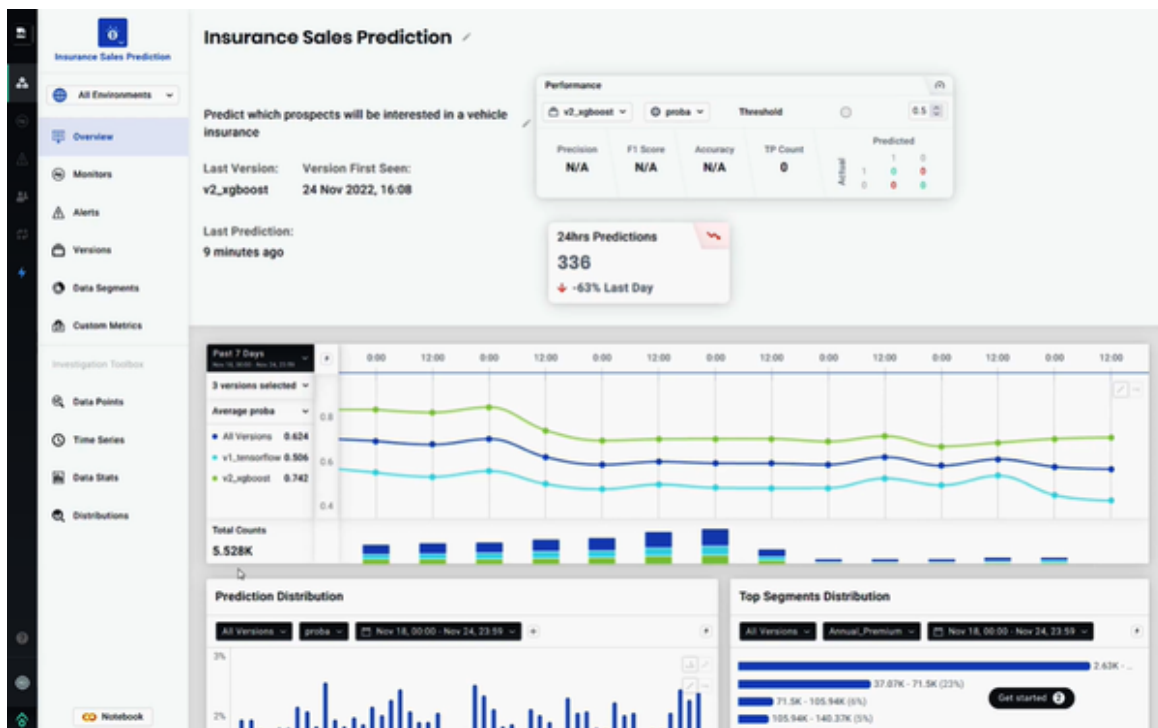


Figure 6-18. Aporia model monitoring dashboard

A popular open source model monitoring framework is **Evidently**, which monitors drift and data quality. Evidently compares the reference dataset with the prediction dataset and generates beautiful static HTML reports. In

addition, it can write the resulting metrics into Prometheus and show them in pre-designed Grafana dashboards.

Example 6-18 demonstrates how you generate a report that compares the reference data with the prediction inputs and outputs data (`current_data`). In **Figure 6-19**, you can see report examples.

Example 6-18. Generating Evidently report

```
report = Report(metrics=[
    DataDriftPreset(),
])

report.run(reference_data=reference, current_data=current)
report.save_html("file.html")
```

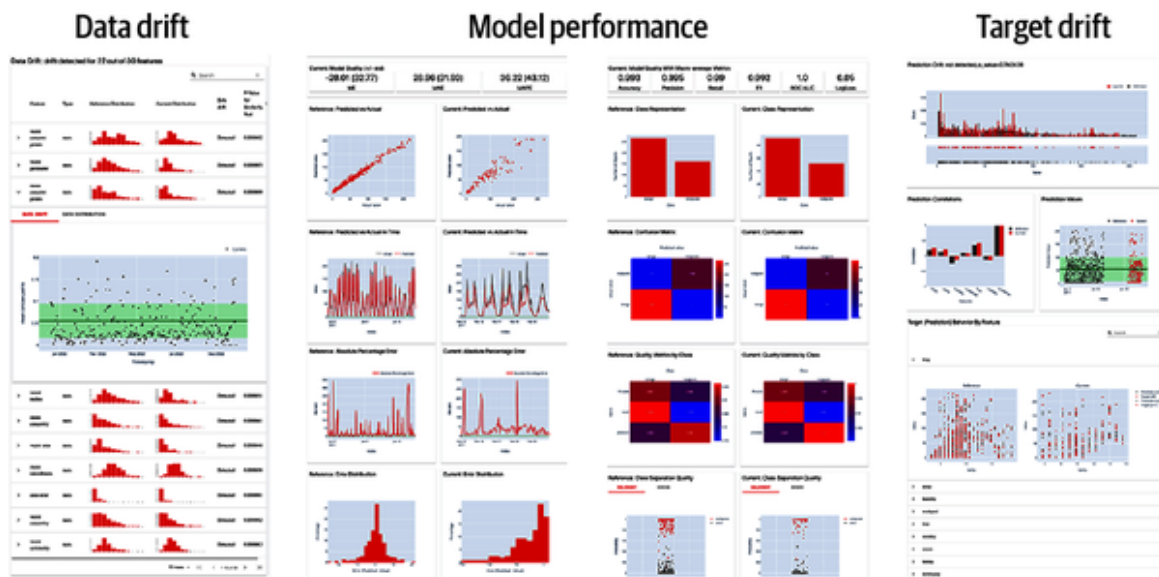


Figure 6-19. Evidently reports

As you can see, the responsibility for generating the reference and production datasets and generating the report is on the user, who needs to add code for monitoring, data generation, and lifecycle management. Therefore, Evidently is an excellent interactive development and comparison solution, and you can extend it manually to handle batch prediction workloads. However, it is unsuitable for continuous batch or real-time model serving and lacks an interactive UI portal, central management, alerts, security, and more.

Model Retraining

COVID-19 abruptly changed human behavior across the globe. But the pandemic not only significantly impacted human lives, it also disrupted ML models. Data engineers woke up to find that their ML models, which were trained on pre-pandemic data sets, had suddenly drifted and were not delivering reliable results.

The models' performance degraded because the pre-pandemic data was not reflecting current behaviors and therefore it was no longer relevant or accurate. These models had to be retrained to ensure their validation and efficacy for the pandemic era.

While COVID-19 is an extreme example, data keeps changing because people change and the world changes. This means models trained on outdated data lose relevance. Model retraining, also known as continuous training or continual training, is the act of training models again and again on updated data and then redeploying them to production.

By **retraining**, data engineers can ensure the models are up-to-date, valid, and trustworthy. This ensures the predictions and outputs of models are always accurate for the business use cases they were designed to answer. If models aren't retrained, they will become stale.

Accurate models are essential for business success. If an organization uses a model that provides inaccurate outputs, the result could be loss of customers and profit. For example, if a model is supposed to detect fraud but doesn't do so accurately, this will mean either that fraudsters get away with fraud, costing the company its customers and perhaps a loss of millions in insurance claims, or that there will be too many false positives, resulting in frustrated end-users (who won't be able to make online purchases) and adverse financial impact to the company's customers (again, losing customers).

Automating the process of model retraining makes it reliable and optimized. Automation also reduces the chance of manual errors or data engineers forgetting to retrain models. With automation, data engineers and data

scientists can ensure their measurements are defensible and quantitative and that explainability tests are set up.

TIP

Automated retraining should take place as part of an MLOps pipeline. It can be integrated as part of the **CI/CD pipeline** and may be triggered automatically by the model monitoring service upon drift detection.

When to Retrain Your Models

There's no right or wrong answer when deciding when to retrain (though not retraining is definitely the wrong answer). The answer to “when to retrain?” depends on the business use case. The ultimate goal, however, is to avoid the two types of drift:

Data drift

When the statistical distribution of production data is different from the baseline data used to train or build the model. This happens when human behavior changes, training data was inaccurate, or there were data quality issues.

Concept drift

When the statistical properties of the target variable change over time. In other words, the concept, or the relations between the datasets, have drifted.

There are four main approaches for retraining:

Interval-based

According to a certain schedule or repeating interval; for example, retraining every Sunday night or every end of the month. This ensures the models will always stay up-to-date

since they are constantly retrained. However, this method can be costly since resources are used even when retraining is unnecessary.

Performance-based

Retraining takes place when a predetermined threshold or baseline is crossed, which indicates model degradation and drift. This ensures the model can always answer the business use case. However, if the threshold is inaccurately determined or the data does not come in on time, the model could turn stale before the organization is aware and can retrain it.

Based on data changes

This type of retraining takes place when there are new data sets or when code changes are made. Such retraining ensures adaptivity to engineering changes but might miss drift that degrades the model performance.

Manually on-demand

This nonautomated retraining method provides complete control for data scientists but is prone to errors and could mean retraining does not occur when needed.

Strategies for Data Retraining

Model retraining takes place by lifting the training data into the retraining pipeline. This data includes features, labels, model parameters, and pipeline parameters.

The question of how much data should be used for retraining depends on the organization's requirements and restrictions, which determine the strategy. Data amounts required for retraining can be determined through the following approaches:

Fixed window

A practical yet simplistic approach

Dynamic window

Optimal for large datasets that are constantly updated but also compute intensive

Representative subsample selection

Accurate since it's similar to production data but time consuming and uses a lot of resources

Model retraining can be an online or offline event, with offline being the most popular approach. Offline retraining, also known as batch retraining, usually uses all available data or a considerable amount of existing data. It's easy to do but requires more thought about the retraining strategy.

TIP

Online retraining is recommended for real-time streaming use cases. The data used for online retraining is new data, not samples already seen by the models. This makes online retraining more accurate and can help avoid drift, though it is also more costly.

To train models, you need labels (the target values), and labels usually arrive in a delay after the features. For example, a churn prediction application may have all the input features immediately. However, the target label (indicating if there was a churn) can come one month later. Therefore, the training set data window should only cover transactions with labels. In addition, since the model reacts to an expected churn, it may influence the dataset and require adaptations to the dataset, which will make it more balanced.

When the data is complex or unstructured, it is challenging to calculate the labels. In such cases, organizations use manual labeling. They extract a

sample of the data, manually label it, compare it with the predicted results, and use the labeled dataset for model retraining and tuning.

Despite the importance of model retraining, it's important to remember that it also comes at a cost. Model retraining requires resources for data storage, computing, adjusting your architecture for retraining, data professionals' time, and more. Therefore, some organizations are hesitant about running it continuously. We recommend you adjust your retraining schedule to your business requirements to ensure cost-effectiveness while maintaining model performance.

Model Retraining in the MLOps Pipeline

Retraining is achieved by triggering the model development pipeline (data preparation, training, validation, and so on). The trigger can be initiated based on a scheduled event or after a drift indication (triggered by the model monitoring component). An important step is validating the model in a staging environment before deploying it to production. If the results are not as expected, then the models and pipelines need to be retrained.

After retraining, we recommend leaving the old model running and deployed to production for a specific period of time or until the model has served a certain number of requests. By running these A/B tests you can identify which model performs better, by comparing its predictions with those of the others. Another approach is to use an ensemble with the old and new models and change the weight or remove the old model over time.

Finally, model retraining also can be used for training new models. This is called *transfer learning*, in which existing models are reused to retrain new models. This is commonly used in deep learning since it saves resources by reusing models instead of rebuilding them.

Deployment Strategies

In production, models and applications must always be ready to serve requests and they should not suffer from downtime due to version upgrades.

In addition, a more advanced practice is gradually moving requests to the new version, while validating that performance and quality levels are met. Then, if the latest version fails or underperforms, you can roll back to the previous version.

To guarantee no service disruption, you can use a rolling upgrade deployment strategy. This strategy replaces the application version used by the service, one instance at a time. Rolling upgrades are supported out of the box in cloud-native platforms and Kubernetes. However, when you want to evaluate the candidate versions against the current version, you need two or more versions to coexist and serve requests until you can determine that the new version meets your quality and performance goals. Or you can use the other versions to serve as a baseline to compare performance.

There are four standard model deployment and upgrade strategies (see [Figure 6-20](#)):

A/B Testing

The new model is deployed alongside the old model, and traffic is divided between the two. The output of both models is then monitored to determine which one performs best, and the best-performing model is promoted. This method is a good fit for most use cases. In some cases, you may deploy more than two models, for example: A/B/C testing, A/B/C/D testing, and so on.

Blue/Green

Setting up two identical environments, dubbed *Blue* and *Green*, with one being the live production environment and the other a staging environment. The new model is deployed to the staging environment. Once it operates as expected, staging becomes production, the old version is removed, and more recent models are deployed into staging. This method is straightforward and relatively simple, although it incurs

high operational costs since you have to operate two identical environments. This method is recommended for use cases that cannot afford any downtime, like fraud detection or e-commerce.

Canary

Incremental rollout of the new model to a percentage of users to validate efficacy, performance, and functionality. Once validated, the model is gradually rolled out to more users and finally to everyone. This strategy enables controlled rollout with no downtime. However, it is slow, and testing occurs in production. Therefore, it is recommended for noncritical applications that obtain value from gradual testing in production, like social media.

Champion/Challenger (or Shadow)

Deploying the champion model to production alongside one or more challenger models. Traffic is routed to all models and the output of the champion is used as the result, while the challenger's output is only monitored. This strategy enables the highest form of model validation, yet it is also the most expensive.

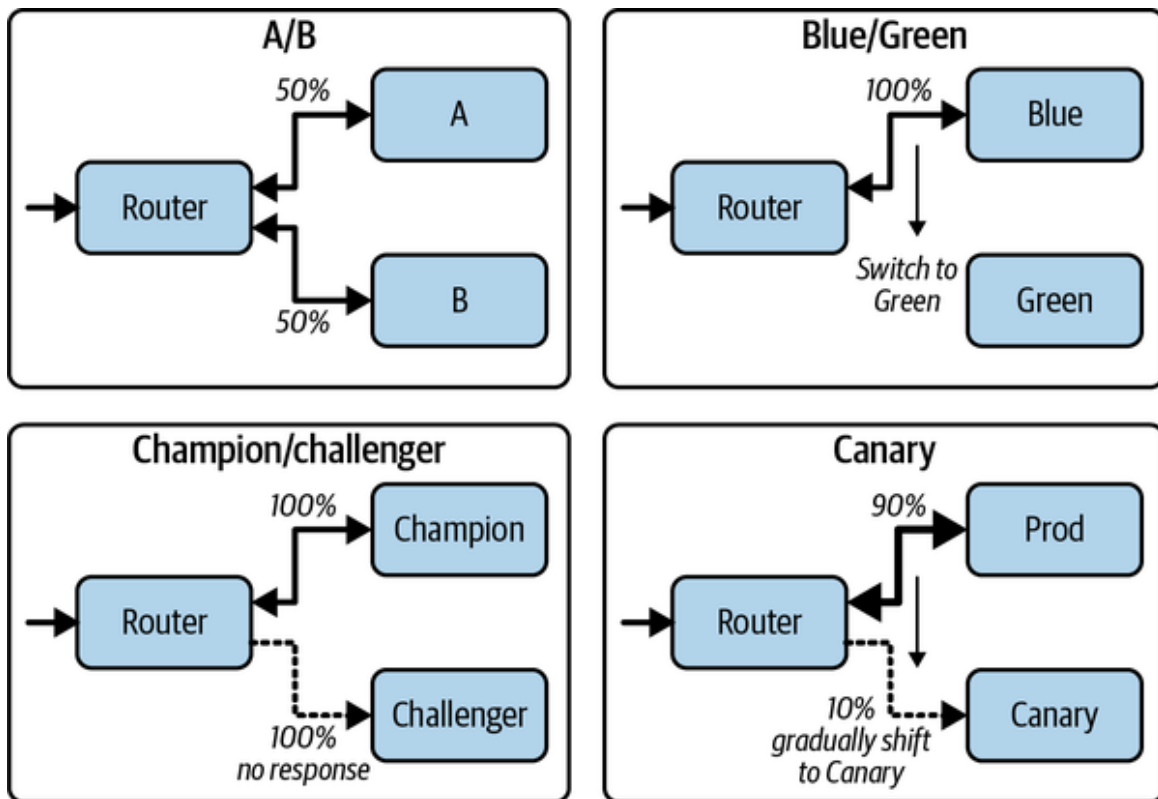


Figure 6-20. Deployment and upgrade strategies

When traffic is split across multiple models, you must ensure *affinity* (the same user is always routed to the same model and has the same experience for every request). For example, you can use a hash of the user ID to determine where to route the request.

Offline and batch application pipelines do not need to maintain running services or use rolling updates. They can select which version to use on every run. However, you may run two or more batch pipelines in parallel (in A/B or Champion/Challenger deployments), with each pipeline storing its results in different tables/files.

It's important to remember that deployment is not a one-time process. Instead, deployment is a recurring process that occurs every time we retrain our models and want to ensure the freshest and highest-performing models are being used. Therefore, selecting the right deployment strategy is important since you will use it repeatedly in your MLOps pipeline. As always, the strategy you choose depends on multiple factors. The primary considerations are:

- Your business use case
- Retraining and deployment frequency
- Your model size
- Whether you stream data in real time or batches
- How your model is impacted by drift
- Your desired service uptime and quality

Measuring the Business Impact

Measuring an application's business impact is essential for determining the model's performance. However, measuring it can be a complex task since this involves evaluating the effects of the model on various aspects of the business, such as revenue, customer satisfaction, and operational efficiency.

Here are some ways to measure the impact of ML models:

Define success metrics

Define measurable and trackable metrics that are aligned with business objectives; for example: model accuracy, precision and recall, the F1 score, Mean Absolute Error (MAE) and Root Mean Square Error (RMSE), lift, conversion rate, and customer satisfaction.

Establish a baseline

Create a benchmark for measuring the model's impact on the business. This should be determined before deploying the model to production.

Conduct A/B testing

Compare the business metrics (costs, revenue, number of active users, customer satisfaction, and so on) before and

after the deployment of the ML model. This will help determine the model's incremental impact.

Conduct a cost-benefit analysis

Determine the return on investment (ROI) of the ML model. This will help assess the financial impact of the model on the business; for example: how many new users registered, what is the incremental revenue from the new service, or how many calls were saved by using a chatbot.

Gather feedback

Gather feedback from stakeholders to assess the impact of the ML model on their experience. This will help determine the nonfinancial impact of the model on the business. You can use surveys, user testing, reviews and ratings, support ticket analysis, or net promoter score (NPS), among other methods.

Overall, measuring the business impact of ML models requires a comprehensive approach that considers both financial and nonfinancial metrics, as well as stakeholder feedback. By doing so, businesses can ensure that their ML models deliver tangible value and drive meaningful outcomes.

Conclusion

In this chapter we finally delved into the real thing: building and deploying the application that uses the data and the model that drives decisions and action. We described the steps that will assist in delivering the ML application as a whole. By looking at this bigger picture, we can ensure there are no deficiencies in functionality, operational failures, avoidable risks, and prolonged delays.

This chapter reviewed model registries and model serving, while showing different solutions and how to use them. Then, we discussed advanced serving and application pipelines, which address requirements like API integrations, data enrichment, validations, processing, storage, routing, cascading, and merging results from multiple models. Next, we explained about various monitoring solutions and how they operate, and then retraining and deployment strategies and conditions. Finally, we discussed how to measure the business impact of deployment, which is the most essential step for determining model performance.

Critical Thinking Discussion Questions

- What is the importance of model registries?
- Choose a model serving solution and describe its capabilities. Could it be a good fit for your organization?
- In which cases will you need to enhance or upgrade your application pipeline? Which considerations should you take into account?
- What are the different retraining strategies? When would you choose each one?
- Your manager does not see the need for measuring the business value of your models. Convince them otherwise.

Exercises

- Write the code for registering one of your models in the registry of your choice.
- Choose an open source serving solution and define a serverless serving graph function.
- Choose an open source model monitoring solution and configure it for your models (no need to actually connect to production).

- Build an integration for retraining as part of a CI/CD pipeline.
- Define four success metrics for one of your ML models.

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 7. Building a Production Grade MLOps Project from A to Z

This chapter provides an example of an ML project with all its components and the MLOps attributes required for production deployment. It follows the practices presented in [Chapter 3](#). The example applications predict and prevent credit transaction fraud by calculating user and transaction features and feeding them into a classifier model, which will determine if the transaction is a legitimate transaction.

All the code examples presented in this chapter are stored in [Git](#).

The project implementation consists of the following steps:

1. Exploring and analyzing the data (EDA)
2. Building the data ingestion and preparation pipeline
3. Building the model training and validation pipeline
4. Developing the application serving pipeline (intercept requests, process data, inference, and so on)
5. Monitoring the data and model (drift and more)
6. Addressing continuous operations and CI/CD

The data preparation step will be implemented in two ways: using standard Python packages and using a [feature store](#).

Fraud prevention is a challenge as it requires processing raw transactions and events in real time and being able to respond quickly and block transactions before they occur. Consider a case where you would like to evaluate the average transaction amount. When training the model, taking a DataFrame and calculating the average is common. However, in real-time/online scenarios, the average calculation is accumulative

(incremental). Subsequently, you must build two data processing pipelines, one for training and another for real-time serving. Alternatively, you can use a feature store (as will be demonstrated) that will build both batch and real-time data pipelines and deploy them automatically over a scalable and resilient infrastructure.

The example is based on public credit card transaction data with three datasets:

Transactions

Monetary activity between two parties to transfer funds

Events

Activity performed by a party, such as a login or password change

Labels

Indications of fraudulent transactions (derived from the historical transactions)

Figure 7-1 illustrates the overall project structure with data ingestion, automated model training and validation, application and model serving, model monitoring, and a feature store as the data hub in the middle.

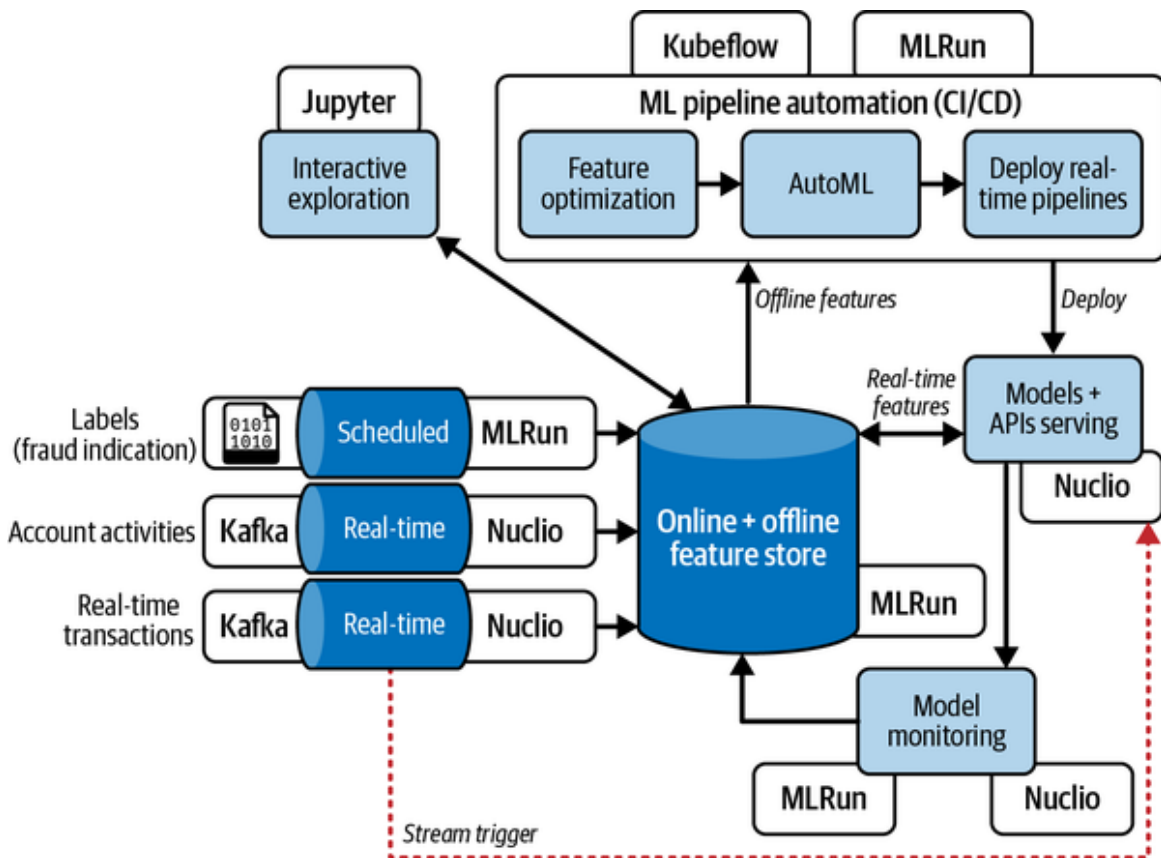


Figure 7-1. Fraud detection project architecture

Exploratory Data Analysis

Before you run any data processing or modeling, you must start by getting to know the data you are working with. The *1-exploratory-data-analysis.ipynb* notebook provides basic explanatory data analysis (EDA) on the fraud example datasets.

The first step (in [Example 7-1](#)) is loading sample datasets and understanding the data shape, types, statistical distribution, categories, missing values, and so on.

Example 7-1. Exploring the transactions dataset

```
# Fetch the transactions and event datasets from mlrun data samples
data_path = mlrun.get_sample_path("data/fraud-demo-mlrun-fs-docs/")
transactions_data = pd.read_csv(data_path + "data.csv", parse_dates=
["timestamp"])
user_events_data = pd.read_csv(
```

```
data_path + "events.csv", index_col=0, quotechar="", parse_dates=
["timestamp"]
)
```

Preview

```
transactions_data.head(3)
```

	step	age	gender	zipcodeOri	zipMerchant	category	amount	fraud	timestamp	source	target	device
0	0	4	M	28007	28007	es_transportation	4.55	0	2023-08-30 03:31:42.286213	C1093826151	M348934600	f802e61d76564b7a89a83adcdfa573da
1	0	2	M	28007	28007	es_transportation	39.68	0	2023-01-09 11:59:46.997703	C352968107	M348934600	38ef7fc3eb7442c8ae64579a483fid2b
2	0	4	F	28007	28007	es_transportation	26.89	0	2023-10-18 00:07:23.191276	C2054744914	M1823072687	7a851d0758894078b5846851ae32d5e3

checking the data types per column

```
transactions_data.dtypes
step          int64
age           object
gender        object
zipcodeOri    int64
zipMerchant   int64
category      object
amount        float64
fraud         int64
timestamp     datetime64[ns]
source        object
target        object
device        object
```

Examining the 'age' column to understand why it is not an integer

```
transactions_data['age'].value_counts()
```

```
3    117110
4     86871
5     50152
1     46509
6     21377
0      1886
U         928
```

Dropping the columns with Unknown (U) age and converting to int

```
Name: age, dtype: int64
transactions_data = transactions_data[transactions_data.age != "U"]
transactions_data['age'] = transactions_data['age'].astype(int)
```

Describe the column statistics

```
transactions_data.describe()
```

	step	age	zipcodeOri	zipMerchant	amount	fraud
count	473745.000000	473745.000000	473745.0	473745.0	473745.000000	473745.000000
mean	77.698114	3.005889	28007.0	28007.0	38.105695	0.012446
std	41.991176	1.325787	0.0	0.0	113.887429	0.110863
min	0.000000	0.000000	28007.0	28007.0	0.000000	0.000000
25%	42.000000	2.000000	28007.0	28007.0	13.740000	0.000000
50%	80.000000	3.000000	28007.0	28007.0	26.890000	0.000000
75%	114.000000	4.000000	28007.0	28007.0	42.530000	0.000000
max	147.000000	6.000000	28007.0	28007.0	7665.560000	1.000000

```
# Check how many transactions are fraudulent (~1%)
0 467849
transactions_data['fraud'].value_counts()
1 5896
Name: fraud, dtype: int64
```

After you understand the basic data structure and distributions, it's time for a few additional checks, such as verifying there is no gender bias in the data (see [Example 7-2](#)).

Example 7-2. Verifying there is no gender bias

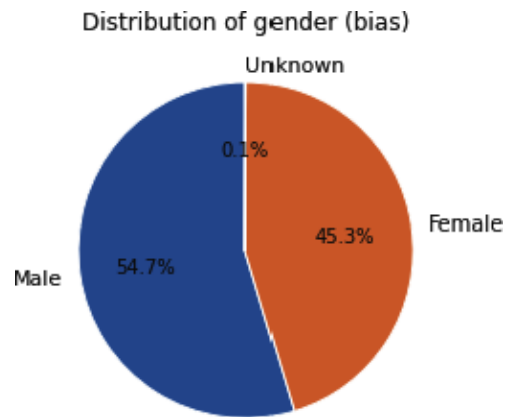
```
F 258905
transactions_data['gender'].value_counts()
M 214414
U 426
Name: gender, dtype: int64
```

Inspecting the results visually:

```
# Create the pie chart
plt.pie(
    transactions_data["gender"].value_counts(),
    labels=["Male", "Female", "Unknown"],
    autopct="%1.1f%%",
    startangle=90,
)

# Add a title
plt.title('Distribution of gender (bias)')

# Display the chart
plt.show()
```



According to the chart shown in [Example 7-2](#), there is no bias in the data since the percentage of men and women is nearly equal.

In [Example 7-3](#), we evaluate the different types of transactions. Notice that the majority of the transactions fall under the transportation category.

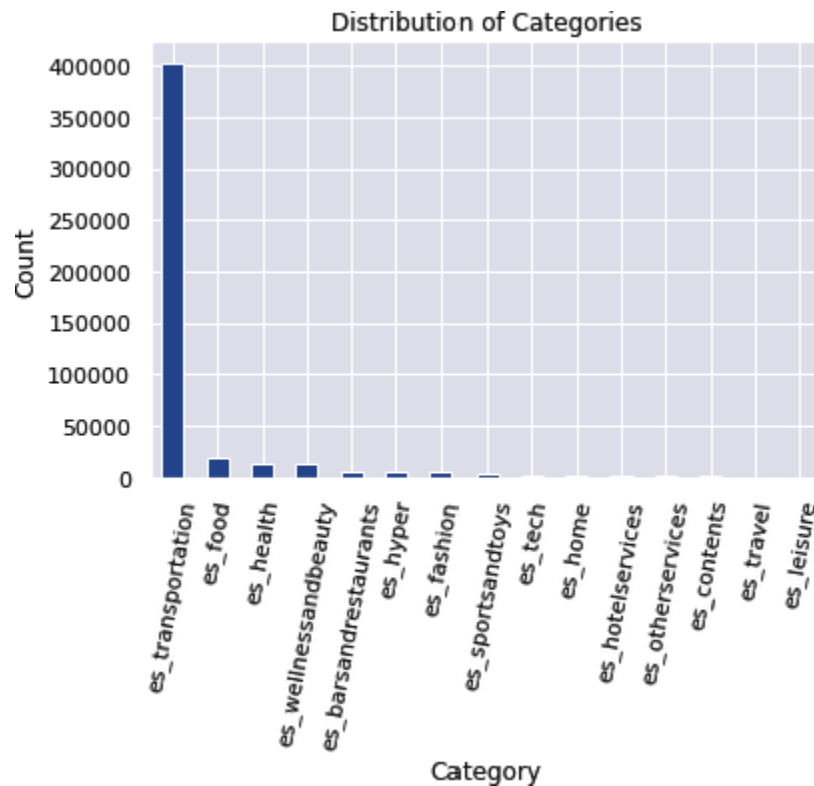
Example 7-3. Plot the transaction categories

```
# Count the occurrences of each category
category_counts = transactions_data['category'].value_counts()

# Plot the results as a bar plot
category_counts.plot(kind='bar', )

# Set labels and title
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Distribution of Categories')
plt.xticks(rotation=80)

# Show the plot
plt.show()
```



It's time to evaluate fraud distribution across hours and days to understand whether the time factor correlates with fraud. **Example 7-4** demonstrates how to group the transactions by hour or day and plot the percentage of fraud per group.

Example 7-4. Plot the fraud distribution across hours and days

```
transactions_data["timestamp"] = pd.to_datetime(
    transactions_data["timestamp"], format="%Y-%m-%d %H:%M:%S UTC"
)

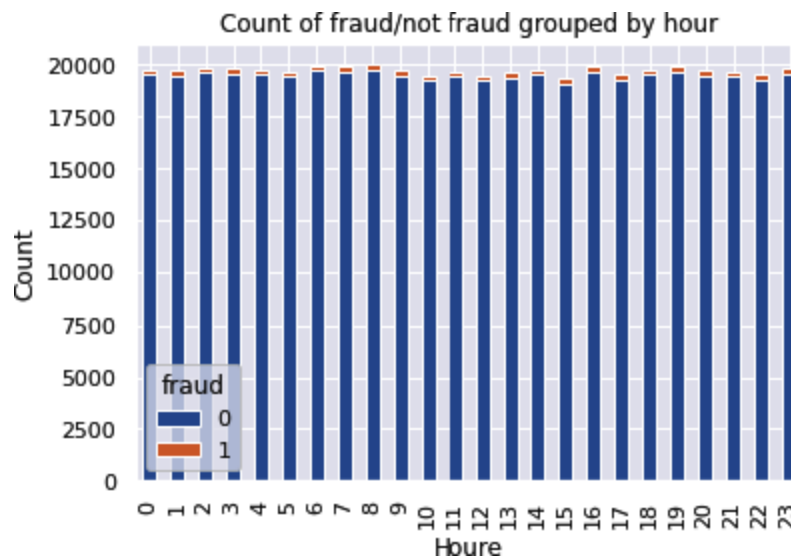
transactions_data["hour"] = transactions_data.timestamp.dt.hour
transactions_data["day"] = transactions_data.timestamp.dt.day
transactions_data["month"] = transactions_data.timestamp.dt.month
transactions_data["weekday"] = transactions_data.timestamp.dt.weekday
transactions_data["year"] = transactions_data.timestamp.dt.year

grouped = transactions_data.groupby('hour')['fraud'].value_counts() \
    .unstack().fillna(0)
# Plot the results
grouped.plot(kind='bar', stacked=True)

# Set labels and title
```

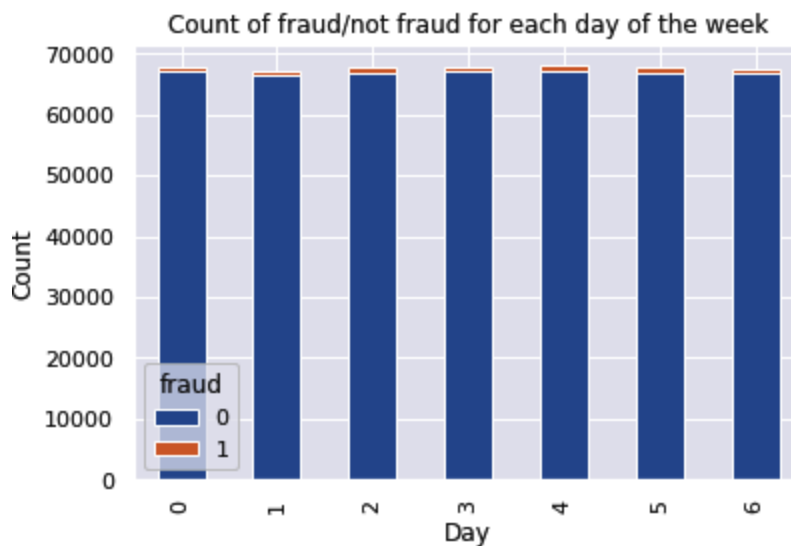


```
plt.xlabel('Hour')
plt.ylabel('Count')
plt.title('Count of fraud/not fraud grouped by hour')
```



```
grouped = transactions_data.groupby('weekday')['fraud'].value_counts() \
    .unstack().fillna(0)
# Plot the results
grouped.plot(kind='bar', stacked=True)

# Set labels and title
plt.xlabel('Day')
plt.ylabel('Count')
plt.title('Count of fraud/not fraud for each day of the week')
```



The next example (**Example 7-5**) demonstrates exploring the events dataset and the distribution of the event categories.

Example 7-5. Exploring the events dataset

```
# View a sample of the events dataset
```

```
user_events_data.head()
```

	source	event	timestamp
0	C1974668487	details_change	2020-09-19 15:12:26.321576
1	C1973547259	login	2020-10-12 15:28:25.703053
2	C515668508	login	2020-09-16 10:10:50.498604
3	C1721541488	details_change	2020-10-02 13:59:25.553802
4	C394979408	password_change	2020-10-14 23:27:41.291391

```
# Count the occurrences of each event category
```

```
category_counts = user_events_data['event'].value_counts()
```

```
# Plot the results as a bar plot
```

```
category_counts.plot(kind='bar', )
```

```
# Set labels and title
```

```
plt.xlabel('Event')
```

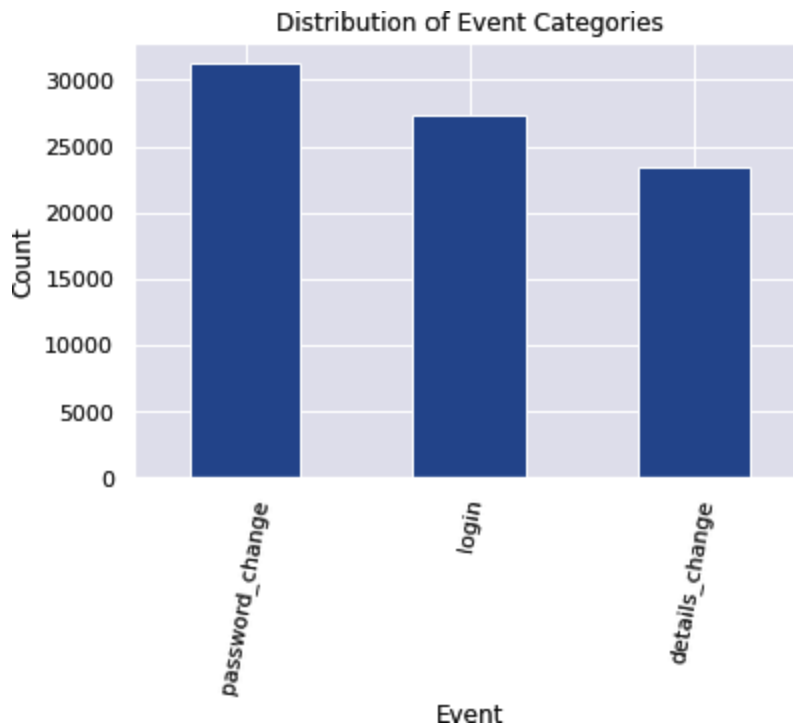
```
plt.ylabel('Count')
```

```
plt.title('Distribution of Event Categories')
```

```
plt.xticks(rotation=80)
```

```
# Show the plot
```

```
plt.show()
```



Now that you have explored each dataset independently, you can join the two datasets and build a few additional derived features that may help predict fraud.

Since both the credit transactions and user events datasets are transactional, you cannot simply join them. Each user may have multiple transactions or activities at different times. Therefore, you need to use a special merge function that also takes into account the event time (time traveling). This function is called `merge_asof()`.

Example 7-6 demonstrates how to merge two transactional datasets using `merge_asof()`. (Note that before the merge, you must verify that both datasets are sorted by time.)

Example 7-6. Merging the two datasets

```
# Sorting the two datasets by time
transactions_data = transactions_data.sort_values(by='timestamp', axis=0)
user_events_data = user_events_data.sort_values(by='timestamp', axis=0)

# Merge asof (with time traveling)
all_data = pd.merge_asof(
    transactions_data,
```

```

        user_events_data,
        on='timestamp',
        by='source',
    )

```

Plot a sample of the interesting columns

```
all_data.drop(columns=["device", "source", "target"]).sample(5)
```

	step	age	gender	zipcodeOri	zipMerchant	category	amount	fraud	timestamp	hour	day	month	weekday	year	event
280869	137	2	F	28007	28007	es_transportation	45.29	0	2020-07-14 20:40:18.989269	20	14	7	1	2020	login
270343	27	2	F	28007	28007	es_transportation	9.32	0	2020-07-06 15:04:58.452371	15	6	7	0	2020	password_change
378166	9	3	F	28007	28007	es_transportation	37.31	0	2020-09-27 18:22:31.005277	18	27	9	6	2020	details_change
263804	20	3	F	28007	28007	es_transportation	39.37	0	2020-07-01 14:01:30.419984	14	1	7	2	2020	details_change
283975	106	2	M	28007	28007	es_wellnessandbeauty	40.82	0	2020-07-17 06:45:28.982259	6	17	7	4	2020	password_change

Example 7-7 demonstrates how to add the categorical features and plot the correlation map.

Example 7-7. Plotting the correlation map

Extend categorical features (one hot encoding)

```
all_data = pd.get_dummies(all_data, columns=['category', 'gender', 'event'])
```

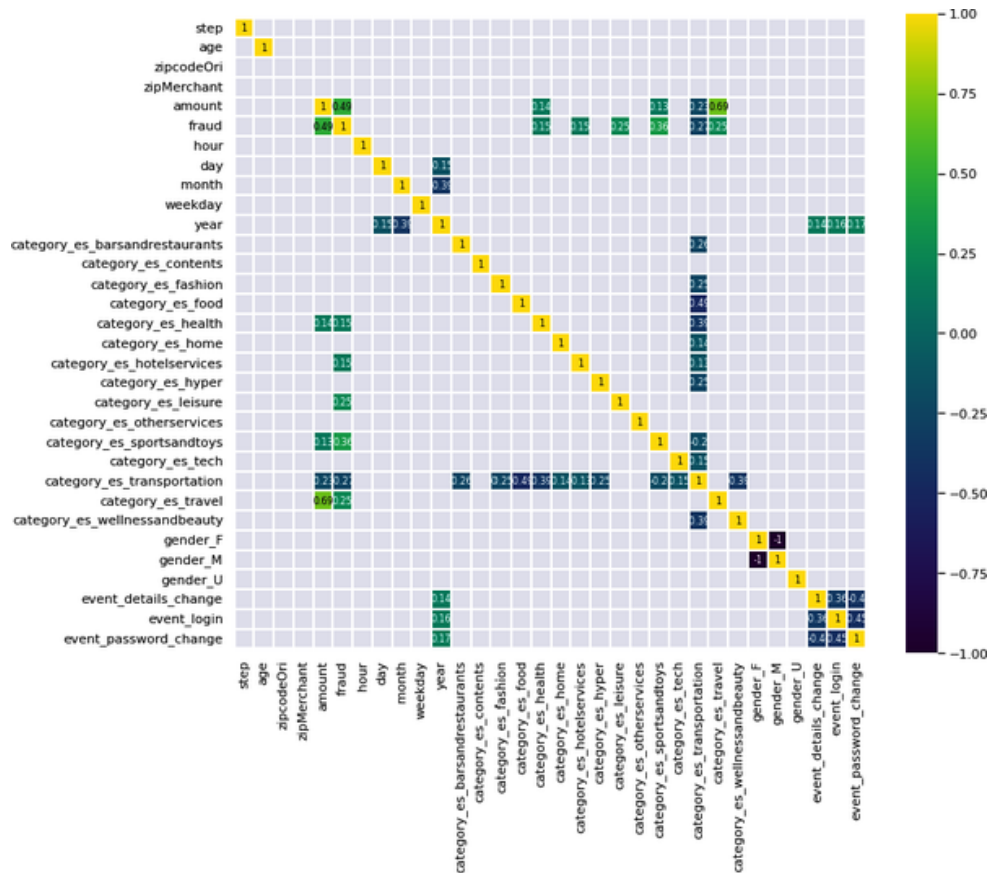
Draw a correlation plot

```
corr = all_data.corr()
plt.figure(figsize=(12, 10))
```

```

sns.heatmap(
    corr[(corr >= 0.1) | (corr <= -0.1)],
    cmap="viridis",
    vmax=1.0,
    vmin=-1.0,
    linewidths=0.1,
    annot=True,
    annot_kws={"size": 8},
    square=True,
)

```



The code in the notebook represents a sample of the kind of analysis possible with the data. There are a variety of tools with great visualization capabilities that can offload or automate the EDA process. You can also harness the power of MLOps solutions to create or use pre-baked functions and services that analyze your data.

The example notebook demonstrates using pre-baked functions from the [MLRun public functions hub](#) to perform EDA and modeling. Some examples:

- The `describe` function takes a DataFrame/dataset, runs different analysis types, and generates a set of plots and reports.
- The `auto-trainer` function takes a dataset, runs training, and generates models along with reports and charts.

- The feature-selection function can automatically select the most important features based on their impact on the model.

The advantage of using MLRun functions is that they are microservices that come with preinstalled packages, so there's no need to start figuring out dependencies. These functions also scale out over containers, have built-in monitoring and tracking, can be placed as a step in the ML pipeline, and are reusable.

Example 7-8 demonstrates how the MLRun hub function (`describe`) can be used to process and visualize features.

Example 7-8. Using MLRun to accelerate EDA

```
all_data.to_csv("./data_set_describe.csv", index=False)
project = mlrun.get_or_create_project(
    name="fraud-demo", user_project=True, context="./"
)

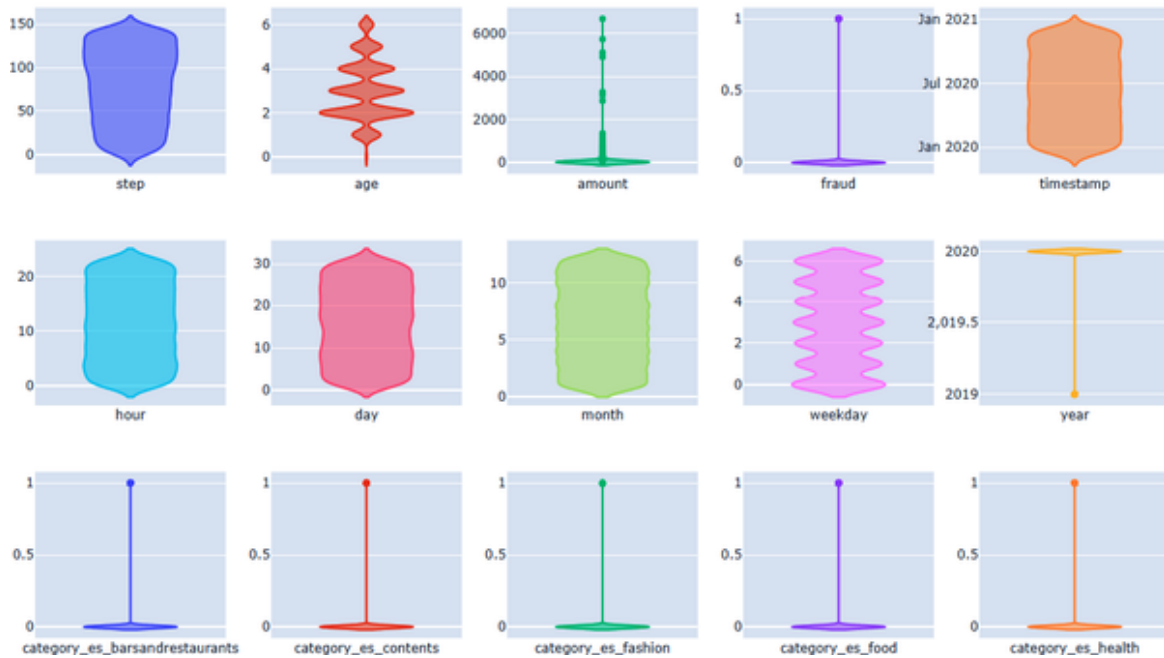
# import and run the function
describe = mlrun.import_function("hub://describe")
describe_run = describe.run(
    params={
        "label_column": "fraud",
    },
    inputs={"table": "./data_set_describe.csv"},
    local=True,
)

# view generated artifact names (charts)
describe_run.outputs.keys()

[['describe-csv', 'hist', 'histograms', 'scatter-2d', 'violin', 'imbalance',
  'imbalance-weights-vec', 'correlation-matrix-csv', 'correlation', 'dataset']]

# View the generated violin plot:
run.artifact("violin").show()
```

Violin Plots



Interactive Data Preparation

Before training the model, you should clean the data and create meaningful features that will be valid predictors for the target variable (was there a fraud?). The notebook [*2-interactive-data-prep.ipynb*](#) demonstrates how to interactively build features for training the model. While this approach is simple, it is unsuitable for production environments with continuous data ingestion, large scale, or real time. In the next section, you will implement the same logic for production using a feature store.

The training set is built from three datasets: credit transactions, user events, and labels indicating if there was fraud. In this example, we prepare each dataset separately and combine them later for training.

Preparing the Credit Transaction Dataset

The following transformations create more meaningful features, which can have a more significant impact on the prediction than the raw data:

- Extracting the date components (hour, day of week) from the timestamp
- One-hot encoding for the age groups, transaction category, and the gender
- Aggregating the amount (avg., sum, count, max over 2/12/24 hour time windows)
- Aggregating the transactions per category (over 14-day time windows)

NOTE

In transactional datasets (with timestamps), aggregations are significant predictors (more than the time value) since outcomes are relative to what happened in the preceding time window (for example, the number of withdrawals in the hour before the fraud).

The notebook starts with loading the datasets from files, doing basic cleaning, and adjusting the dates to recent dates.

The code in **Example 7-9** creates the time features (day, hour) and encodes the age, gender, and transaction category features.

Example 7-9. Building categorical features

```
transactions_data.head(3)
```

	step	age	gender	zipcodeOri	zipMerchant	category	amount	fraud	timestamp	source	target	device
0	0	4	M	28007	28007	es_transportation	4.55	0	2023-08-30 03:31:42.286213	C1093826151	M348934600	f802e61d76564b7a89a83adcdfa573da
1	0	2	M	28007	28007	es_transportation	39.68	0	2023-01-09 11:59:46.997703	C352968107	M348934600	38ef7fc3eb7442c8ae64579a483f1d2b
2	0	4	F	28007	28007	es_transportation	26.89	0	2023-10-18 00:07:23.191276	C2054744914	M1823072687	7a851d0758894078b5846851ae32d5e3

```
processed_transactions = transactions_data
```

```
# Generate day and hour columns from the timestamp
```

```
processed_transactions['day_of_week'] =
```

```
    processed_transactions['timestamp'].dt.weekday
```

```
processed_transactions['hour'] = processed_transactions['timestamp'].dt.hour
```

```
# Map age groups
```

```
processed_transactions["age_mapped"] = processed_transactions["age"].map(
```



```

        lambda x: {'U': '0'}.get(x, x)
    )

    # encode categories and gender groups (using one hot encoding)
    processed_transactions = pd.get_dummies(
        processed_transactions, columns=["category", "gender"]
    )

```

The next part is aggregating the transaction amounts by time windows and transaction categories (see [Example 7-10](#)), providing a long list of derived features that can potentially help you make better predictions.

Example 7-10. Building categorical features

```

transactions_for_agg = processed_transactions.set_index(
    ["timestamp"],
)

# Group/Aggregate amount stats (mean, max, ..) by time windows
windows = ["2H", "12H", "24H"]
operation = ["mean", "sum", "count", "max"]
for window in windows:
    for op in operation:
        processed_transactions[f"amount_{op}_{window}"] = (
            transactions_for_agg.groupby(["source", pd.Grouper(freq=window)])
            ["amount"]
            .transform(op)
            .values
        )

# Group/Aggregate amount stats (mean, max, ..) by transaction category
main_categories = [
    "es_transportation",
    "es_health",
    "es_otherservices",
    "es_food",
    "es_hotelservices",
    "es_barsandrestaurants",
    "es_tech",
    "es_sportsandtoys",
    "es_wellnessandbeauty",
    "es_hyper",
    "es_fashion",
    "es_home",
    "es_contents",
    "es_travel",
    "es_leisure",
]

```

```

for category in main_categories:
    processed_transactions[f"{category}_sum_14D"] = (
        transactions_for_agg.groupby(["source", pd.Grouper(freq="14D")])[
            f"category_{category}"
        ]
        .transform("sum")
        .values
    )

processed_transactions.set_index(["source"], inplace=True)
processed_transactions.head()

# see the list of derived features (see the notebook for the full list)
processed_transactions.dtypes

```

step	int64
age	object
zipcodeOri	int64
zipMerchant	int64
amount	float64
fraud	int64
timestamp	datetime64[ns]
target	object
device	object
day_of_week	int64
hour	int64
age_mapped	object
category_es_barsandrestaurants	uint8
category_es_contents	uint8
category_es_fashion	uint8
...	
amount_mean_24H	float64
amount_sum_24H	float64
amount_count_24H	int64
amount_max_24H	float64
...	
es_hyper_sum_14D	uint8
es_fashion_sum_14D	uint8
es_home_sum_14D	uint8
es_contents_sum_14D	uint8
es_travel_sum_14D	uint8
es_leisure_sum_14D	uint8
dtype:	object

Preparing the User Events (Activities) Dataset

The events dataset contains user activities such as login, change of details, or password, which can hint at a fraud attempt. **Example 7-11** shows how to

load the events dataset and create categorical features per event type.

Example 7-11. Processing the events dataset

```
# Fetch the user_events dataset from the server
user_events_data = pd.read_csv(
    data_path + "events.csv", index_col=0,
    quotechar="'", parse_dates=["timestamp"]
)

# Adjust to the last 2 days to see the latest aggregations in the online
# feature vectors
user_events_data = adjust_data_timespan(user_events_data, new_period="2d")

# Preview
user_events_data.head(3)

# Generate categorical features from the event type
processed_events = user_events_data
processed_events = pd.get_dummies(processed_events, columns=["event"])
processed_events.set_index(["source"], inplace=True)
processed_events.head()
```

	timestamp	event_details_change	event_login	event_password_change
source				
C137986193	2023-06-22 08:58:41.507004000	0	0	1
C1940951230	2023-06-22 08:58:42.555186091	1	0	0
C247537602	2023-06-22 08:58:44.209969103	0	1	0
C470079617	2023-06-22 08:58:45.500418428	0	0	1
C1142118359	2023-06-22 08:58:46.290710830	0	1	0

Extracting Labels and Training a Model

The final step is to generate a target label column (the fraud yes/no indication) and train a basic model to evaluate your assumptions.

Example 7-12 demonstrates how to create the labels dataset and use sklearn to train and evaluate a basic model.

NOTE

In real-world scenarios, the labels can arrive after the transaction data (in a delay after it was determined that the transaction is indeed fraudulent). In addition, the training process will be done on a much larger dataset and with hyperparameter tuning, and it will not fit into the notebook's memory.

Example 7-12. Processing the events dataset

```
def create_labels(df):
    labels = df[['fraud', 'timestamp']].copy()
    labels = labels.rename(columns={"fraud": "label"})
    labels['timestamp'] = labels['timestamp'].astype("datetime64[ms]")
    labels['label'] = labels['label'].astype(int)
    return labels

# Create the target label dataset (fraud indication)
labels_set = create_labels(processed_transactions)

# Train a model based on the transactions, events, and labels
from src.train_sklearn import train_and_val, prepare_data_to_train

X_train, X_test, y_train, y_test = prepare_data_to_train(
    processed_transactions, processed_events, labels_set
)
rf_best = train_and_val(X_train, X_test, y_train, y_test)

# print the model results (Accuracy, ...)
Fitting 3 folds for each of 100 candidates, totalling 300 fits
F1 Best
Accuracy: 0.9963177275118359
Precision: 1.0
Recall: 0.65
F1 Score: 0.787878787878788
```

The training process seems successful, and the model can accurately predict fraud!

Data Ingestion and Preparation Using a Feature Store

In the previous section, you created ML features manually on a small scale (using a notebook with pandas). However, when deploying ML to

production, the process of ingesting, transforming, storing, versioning, and serving data should be automated and run at scale. Organizations use dedicated data and ML engineering teams to productize and scale the data pipelines, which requires more time and resources and can lead to errors and inconsistencies. Furthermore, when the application works with real-time transactional data, the data processing is based on streaming technologies, forcing even more complex and duplicate implementations.

Feature stores enable reuse and collaboration, where the same features can be engineered once and used in different applications. In addition, features can be updated and refreshed constantly, resulting in more accurate and up-to-date models and a simplified model retraining process.

Some feature stores (like MLRun and [Tecton](#)) automate the development and deployment of the feature generation (transformation) data pipelines. This section will show you how to use MLRun to easily build feature engineering pipelines and deploy them as batch or real-time scenarios.

Feature stores define groups of features (feature sets) that are ingested and transformed together from the same online or offline source. In the example, you will define three feature sets (credit transactions, user activities, and labels), each with its source, transformations, and target store definitions. Feature set definitions can be saved and later deployed in an application context, reading from offline (for example, object storage or a database) or online (for example, a Kafka stream or an HTTP endpoint) sources.

After the features are stored and versioned in the feature store, they can be combined into feature vectors that the model training, serving, or monitoring applications will use.

See the complete code example in the [GitHub repository](#).

In MLRun, all the assets and resources (functions, workflows, data, features, models, and so on) are grouped into projects. Projects can be deployed as a whole, have access control and policies, and are usually mapped to a Git repository for simple deployment and versioning. The first

step in the notebook is to create a new project or load an existing project from the version control or database (see [Example 7-13](#)).

Example 7-13. Create or load an MLRun project

```
import mlrun
```

```
# Initialize the MLRun project object
> 2023-04-21 11:02:06.988 [info] loaded project fraud-demo from MLRun DB
project = mlrun.get_or_create_project("fraud-demo", context=".")
```

Building the Credit Transactions Data Pipeline (Feature Set)

[Example 7-14](#) generates and aggregates new features from the credit transactions dataset and stores them in offline and online forms for training and real-time serving. First, define the feature set, its main entity (index), timestamp, and so on. Next, define the set of feature transformations (mappings, encoding, aggregations, custom operators, and so on). Finally, specify how or where to store the results (online and offline data stores) or use the system defaults.

Example 7-14. Defining the transactions feature set

```
# Import MLRun's Feature Store
import mlrun.feature_store as fstore
from mlrun.feature_store.steps import OneHotEncoder, MapValues, DateExtractor
```

```
# Define the transactions FeatureSet
transaction_set = fstore.FeatureSet(
    "transactions",
    entities=[fstore.Entity("source")],
    timestamp_key="timestamp",
    description="transactions feature set",
)
```

```
# Define and add value mapping
```

```
main_categories = [
    "es_transportation",
    "es_health",
    "es_otherservices",
    "es_food",
    "es_hotelservices",
    "es_barsandrestaurants",
    "es_tech",
    "es_sportsandtoys",
    "es_wellnessandbeauty",
]
```

```

    "es_hyper",
    "es_fashion",
    "es_home",
    "es_contents",
    "es_travel",
    "es_leisure",
]

# One Hot Encode the newly defined mappings
one_hot_encoder_mapping = {
    "category": main_categories,
    "gender": list(transactions_data.gender.unique()),
}

# Define the graph steps
transaction_set.graph.to(
    DateExtractor(parts=["hour", "day_of_week"], timestamp_col="timestamp")
).to(MapValues(mapping={"age": {"U": "0"}}, with_original_features=True)).to(
    OneHotEncoder(mapping=one_hot_encoder_mapping)
)

# Add aggregations for 2, 12, and 24 hour time windows
transaction_set.add_aggregation(
    name="amount",
    column="amount",
    operations=["avg", "sum", "count", "max"],
    windows=["2h", "12h", "24h"],
    period="1h",
)

# Add the category aggregations over a 14 day window
for category in main_categories:
    transaction_set.add_aggregation(
        name=category,
        column=f"category_{category}",
        operations=["sum"],
        windows=["14d"],
        period="1d",
    )

# Add default (offline-parquet & online-nosql) targets
transaction_set.set_targets()

```

The MLRun feature set pipeline is implemented using a graph (DAG) of multiple connected steps, where the `.to()` operation links one step to the

next in line.

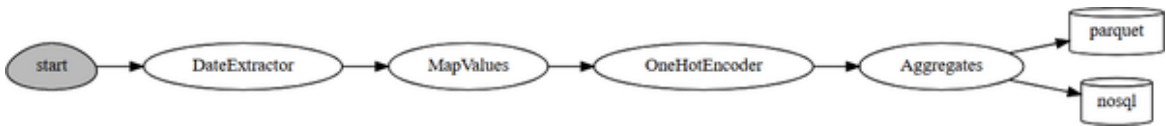
NOTE

The code in [Example 7-14](#) provides the same logic as the interactive code in Examples [7-9](#) and [7-10](#). However, it can now be deployed into production at scale or as a real-time streaming pipeline using a single command with no additional coding or engineering.

You can visually inspect the generated pipeline using `.plot()`, and test it with sample data to verify it produces the expected results using `.preview()`. See [Example 7-15](#).

Example 7-15. Visualize and test the transactions feature set

```
# Plot the pipeline so you can see the different steps
transaction_set.plot(rankdir="LR", with_targets=True)
```



```
# Preview the resulting features from ingesting sample data (transactions_data)
fstore.preview(transaction_set, transactions_data)
```

	amount_sum_2h	amount_sum_12h	amount_sum_24h	amount_count_2h	amount_count_12h	amount_count_24h	amount_max_2h	amount_max_12h	amount_max_24h	amount_avg_2h	...
source											
C1022153336	26.92	26.92	26.92	1.0	1.0	1.0	26.92	26.92	26.92	26.920000	...
C1006176917	48.22	48.22	48.22	1.0	1.0	1.0	48.22	48.22	48.22	48.220000	...
C1010936270	17.56	17.56	17.56	1.0	1.0	1.0	17.56	17.56	17.56	17.560000	...
C1033736586	4.50	4.50	4.50	1.0	1.0	1.0	4.50	4.50	4.50	4.500000	...
C1019071188	1.83	1.83	1.83	1.0	1.0	1.0	1.83	1.83	1.83	1.830000	...

You can watch the feature sets, their metadata, and statistics in the MLRun feature store UI. See [Figure 4-20](#).

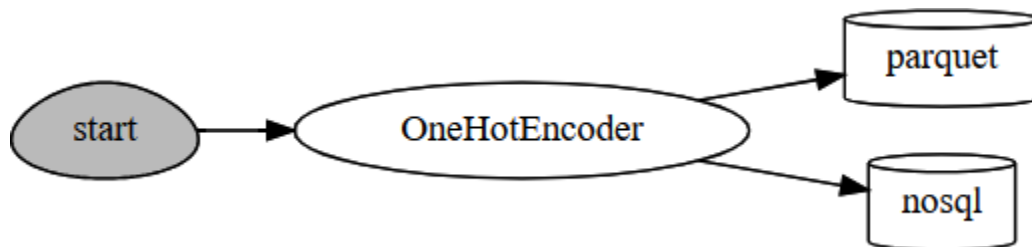
Building the User Events Data Pipeline (FeatureSet)

The events data pipeline is simpler and has a single transformation for encoding the event category (using one-hot encoding). [Example 7-16](#)

demonstrates defining the user events feature set, adding the transformation, and testing the feature set with sample data.

Example 7-16. Define, visualize, and test the events feature set

```
user_events_set = fstore.FeatureSet(  
    "events",  
    entities=[fstore.Entity("source")],  
    timestamp_key="timestamp",  
    description="user events feature set",  
)  
  
# Define and add value mapping  
events_mapping = {"event": list(user_events_data.event.unique())}  
  
# One Hot Encode  
user_events_set.graph.to(OneHotEncoder(mapping=events_mapping))  
  
# Add default (offline-parquet & online-nosql) targets  
user_events_set.set_targets()  
  
# Plot the pipeline so you can see the different steps  
user_events_set.plot(rankdir="LR", with_targets=True)
```



```
# Preview the resulting features from ingesting sample data (transactions_data)  
fstore.preview(user_events_set, user_events_data)
```

	event_password_change	event_details_change	event_login	timestamp
source				
C137986193	1	0	0	2023-06-19 11:03:05.753026000
C1940951230	0	1	0	2023-06-19 11:03:06.801208091
C247537602	0	0	1	2023-06-19 11:03:08.455991103
C470079617	1	0	0	2023-06-19 11:03:09.746440428
C1142118359	0	0	1	2023-06-19 11:03:10.536732830
...	...	...	...	...

Building the Target Labels Data Pipeline (FeatureSet)

The third feature set defines how to create the target labels (see [Example 7-17](#)). Target labels are used only in training, so they are stored only in the offline store. (During serving, the target value is predicted by the model.) The basic pandas engine is used, since real-time processing is unnecessary, and the processing is simple and stateless.

Example 7-17. Define, visualize, and test the events feature set

```
def create_labels(df):
    labels = df[["fraud", "timestamp"]].copy()
    labels = labels.rename(columns={"fraud": "label"})
    labels["timestamp"] = labels["timestamp"].astype("datetime64[ms]")
    labels["label"] = labels["label"].astype(int)
    return labels

# Define the "labels" feature set, use "pandas" processing engine
labels_set = fstore.FeatureSet(
    "labels",
    entities=[fstore.Entity("source")],
    timestamp_key="timestamp",
    description="training labels",
    engine="pandas",
)

labels_set.graph.to(name="create_labels", handler=create_labels)

# specify only Parquet (offline) target since its not used for real-time
target = ParquetTarget(
    name="labels", path=f"v3io:///projects/{project.name}/target.parquet"
)
labels_set.set_targets([target], with_defaults=False)

# Visualize the feature set
labels_set.plot(with_targets=True)

# Preview and test the feature set
fstore.preview(labels_set, transactions_data)
```

Ingesting Data into the Feature Store

Data must be ingested into the feature store before it can be used in training or serving. There are three ways to ingest data:

Direct ingestion

Ingest the data directly from the client/notebook (interactively).

Batch/scheduled ingestion

Create a service/job that ingests data from the source (for example, file, DB, and so on).

Real-time/streaming ingestion

Create an online service that accepts real-time events (from a stream, HTTP, and so on) and pushes them into the feature store.

Direct and batch ingestion are achieved using the `ingest()` method, while real-time ingestion is done using the `deploy_ingestion_service_v2()` method. Both methods are demonstrated in the following sections. Direct ingestion is great for development and testing, while real-time ingestion is used mainly in production.

Batch data ingestion (for tests and training)

To run training or test the serving, you need to ingest and transform the input datasets and store the results in the feature store. The simplest way is to use the `ingest()` method and specify the feature set and the source (for example, DataFrame, file, and so on).

You can specify the desired target to overwrite the default behavior. For example, set `targets=[ParquetTarget()]` to specify that the data will only be written to parquet files and will not be written to the NoSQL DB (meaning you cannot run real-time serving).

The `ingest()` method has many other arguments/options. See the [MLRun documentation](#) for details.

Example 7-18 demonstrates interactive ingestion, using the `.ingest()` method, of the sample datasets into the feature store, allowing you to use the features in training or serving applications.

Example 7-18. Batch ingestion to feature sets

```
# Ingest your transactions dataset through your defined pipeline
transactions_df = fstore.ingest(transaction_set, transactions_data,
                                infer_options=fstore.InferOptions.default())

# Ingestion of your newly created events feature set
events_df = fstore.ingest(user_events_set, user_events_data)

# Ingest the labels feature set
labels_df = fstore.ingest(labels_set, transactions_data)
```

Real-time data ingestion (for production)

When dealing with real-time aggregation, it's important to be able to update these aggregations in real time. For this purpose, create live serving functions that update the online feature store of the *transactions* feature set and *events* feature set.

Real-time feature set deployments use MLRun's serving runtime, which creates a real-time **Nuclio** serverless function, loads it with the feature set's computational graph definition, and configures the source trigger (HTTP, Kafka, Kinesis, v3io, and so on) for reading incoming events.

NOTE

There is no need to rewrite the pipeline logic from batch to real-time deployment. Just change the deployment mode.

Example 7-19. Deploy real-time feature set ingestion pipeline

```
# Define the source stream trigger (use Kafka streams)
source = mlrun.datastore.sources.KafkaSource(
    brokers=brokers,
```

```

    topics="TransactionTopic",
    group="my_group",
)

# Deploy the feature set ingestion service over a real-time serverless function
transaction_set_endpoint = fstore.deploy_ingestion_service_v2(
> 2023-06-21 11:06:44,500 [info] Starting remote function deploy
    feature_set=transaction_set, source=source)
2023-06-21 11:06:44 (info) Deploying function
2023-06-21 11:06:44 (info) Building
2023-06-21 11:06:44 (info) Staging files and preparing base images
2023-06-21 11:06:44 (info) Building processor image
2023-06-21 11:08:30 (info) Build complete
2023-06-21 11:08:40 (info) Function deploy complete
> 2023-06-21 11:08:48,079 [info] successfully deployed function: ...

```

See the notebooks for the full example and how to test the deployed real-time ingestion service.

Model Training and Validation Pipeline

Now that you have created features, you can use them to train one or more models. In this section, you will generate feature vectors with multiple features from one or more feature sets and feed them into an automated ML training and testing pipeline to create high-quality models.

The ML pipeline can be triggered and tracked manually during interactive development, or it can be saved (into Git) and executed automatically on a given schedule or as a reaction to different events (such as code modification, CI/CD, data changes, or model drift). See [MLRun project and CI/CD documentation](#) for details.

Creating and Evaluating a Feature Vector

Models are trained with multiple features, which can arrive from different feature sets and be collected into training (feature) vectors. Feature stores know how to correctly combine the features into a vector by implementing smart JOINS and assessing the time dimension (time traveling).

To define a feature vector, you need to specify a name, the list of features it contains, the target features (labels), and other optional parameters. Features

are specified as <FeatureSet>.<Feature> or <FeatureSet>.* (all the features in a feature set). **Example 7-20** demonstrates how to create and use a feature vector.

Example 7-20. Create a feature vector

```
# Import MLRun's Feature Store
import mlrun.feature_store as fstore

# Define the list of features to use
features = ['events.*',
            'transactions.amount_max_2h',
            'transactions.amount_sum_2h',
            'transactions.amount_count_2h',
            'transactions.amount_avg_2h',
            'transactions.amount_max_12h',
            'transactions.amount_sum_12h',
            'transactions.amount_count_12h',
            'transactions.amount_avg_12h',
            'transactions.amount_max_24h',
            'transactions.amount_sum_24h',
            'transactions.amount_count_24h',
            'transactions.amount_avg_24h',
            'transactions.es_transportation_sum_14d',
            'transactions.es_health_sum_14d',
            'transactions.es_otherservices_sum_14d',
            'transactions.es_food_sum_14d',
            'transactions.es_hotelservices_sum_14d',
            'transactions.es_barsandrestaurants_sum_14d',
            'transactions.es_tech_sum_14d',
            'transactions.es_sportsandtoys_sum_14d',
            'transactions.es_wellnessandbeauty_sum_14d',
            'transactions.es_hyper_sum_14d',
            'transactions.es_fashion_sum_14d',
            'transactions.es_home_sum_14d',
            'transactions.es_travel_sum_14d',
            'transactions.es_leisure_sum_14d',
            'transactions.gender_F',
            'transactions.gender_M',
            'transactions.step',
            'transactions.amount',
            'transactions.timestamp_hour',
            'transactions.timestamp_day_of_week']

# Define the feature vector name for future reference
fv_name = 'transactions-fraud'
```

```
# Define the feature vector using the feature store (fstore)
transactions_fv = fstore.FeatureVector(fv_name,
                                       features,
                                       label_feature="labels.label",
                                       description='Predicting a fraudulent transaction')

# Save the feature vector in the feature store
transactions_fv.save()
```

Once you have defined the feature vector, you can use `get_offline_features()` to generate the vector dataset and return it as a `DataFrame` or materialize it into a file (CSV or Parquet). **Example 7-21** demonstrates how to retrieve a vector, materialize it, and view its results.

Example 7-21. Retrieve, materialize, and view a feature vector

```
from mlrun.datastore.targets import ParquetTarget

# Get offline feature vector as dataframe and save the dataset to parquet
train_dataset = fstore.get_offline_features(fv_name, target=ParquetTarget())

# Preview your dataset
train_dataset.to_dataframe().head()
```

	event_password_change	event_details_change	event_login	amount_max_2h	amount_sum_2h	amount_count_2h	amount_avg_2h	amount_max_12h	amount_sum_12h	amount_count_12h	...
0	0	0	1	1.83	1.83	1.0	1.830000	1.83	1.83	1.0	...
1	0	0	1	18.72	40.22	3.0	13.406667	18.72	40.22	3.0	...
2	1	0	0	25.92	64.86	3.0	21.620000	25.92	64.86	3.0	...
3	1	0	0	24.75	30.17	2.0	15.085000	24.75	30.17	2.0	...
4	1	0	0	64.18	65.17	2.0	32.585000	64.18	65.17	2.0	...

5 rows × 36 columns

NOTE

You can specify various properties in the `get_offline_features()` method, such as the time window the data should fall in (important for model retraining) and the vector processing engine (pandas, Spark, or Dask).

Building and Running an Automated Training and Validation Pipeline

MLRun allows the building of distributed ML pipelines that can handle data processing, automated feature selection, training, optimization, testing,

deployments, and more. Pipelines are composed of steps that run or deploy custom or library (from the MLRun hub) serverless functions. Pipelines can be run locally (for debugging or small-scale tasks), on a scalable Kubernetes cluster (using Kubeflow), or in a CI/CD system.

The example consists of the following pipeline steps (all using predefined MLRun hub functions):

1. Materialize a feature vector (using `hub://get_offline_features`).
2. Select the most optimal features (using `hub://feature_selection`).
3. Train the model with multiple algorithms (using `hub://auto_trainer`).
4. Evaluate the model (using `hub://auto_trainer`).
5. Deploy the model and its application to the test cluster (using `hub://v2_model_server`). The next section will explain the model and application pipeline in detail.

Each step can accept the previous steps' results or data and generate results, multiple visual artifacts/charts, versioned data objects, and registered models.

The code in [Example 7-22](#) implements the pipeline.

Example 7-22. ML Pipeline (data prep, train, validate, and deploy)

```
# Create a Kubeflow Pipelines pipeline
@dsl.pipeline(
    name="Fraud Detection Pipeline",
    description="Detecting fraud from a transactions dataset",
)
def pipeline(vector_name="transactions-fraud", features=[],
label_column="is_error"):
    project = mlrun.get_current_project() # Get FeatureVector
    get_vector = mlrun.run_function(
        "hub://get_offline_features",
        name="get_vector",
        params={
            "feature_vector": vector_name,
            "features": features,
```



```

        "label_feature": label_column,
        "target": {"name": "parquet", "kind": "parquet"},
        "update_stats": True,
    },
    outputs=["feature_vector", "target"],
)
# Feature selection
feature_selection = mlrun.run_function(
    "hub://feature_selection",
    name="feature-selection",
    params={
        "output_vector_name": "short",
        "label_column": project.get_param("label_column", "label"),
        "k": 18,
        "min_votes": 2,
        "ignore_type_errors": True,
    },
    inputs={
        "df_artifact": project.get_artifact_uri(
            get_vector.outputs["feature_vector"], "feature-vector"
        )
    },
    outputs=[
        "feature_scores",
        "selected_features_count",
        "top_features_vector",
        "selected_features",
    ],
)
# train with hyper-parameters
train = mlrun.run_function(
    "hub://auto_trainer",
    name="train",
    handler="train",
    params={
        "sample": -1,
        "label_column": project.get_param("label_column", "label"),
        "test_size": 0.10,
    },
    hyperparams={
        "model_name": [
            "transaction_fraud_rf",
            "transaction_fraud_xgboost",
            "transaction_fraud_adaboost",
        ],
        "model_class": [
            "sklearn.ensemble.RandomForestClassifier",
            "sklearn.linear_model.LogisticRegression",
        ],
    },
)

```

```

        "sklearn.ensemble.AdaBoostClassifier",
    ],
},
hyper_param_options=HyperParamOptions(
    strategy="list", selector="max.accuracy"
),
inputs={"dataset": feature_selection.outputs["top_features_vector"]},
outputs=["model", "test_set"],
)
# test and visualize your model
test = mlrun.run_function(
    "hub://auto_trainer",
    name="evaluate",
    handler="evaluate",
    params={
        "label_columns": project.get_param("label_column", "label"),
        "model": train.outputs["model"],
        "drop_columns": project.get_param("label_column", "label"),
    },
    inputs={"dataset": train.outputs["test_set"]},
)
# Create a serverless function from the hub, add a feature enrichment router
# This will enrich and impute the request with data from the feature vector
serving_function = mlrun.import_function(
    "hub://v2_model_server", new_name="serving"
)
serving_function.set_topology(
    "router",
    mlrun.serving.routers.EnrichmentModelRouter(
        feature_vector_uri="short", impute_policy={"*": "$mean"}
    ),
    exist_ok=True,
)
# Enable model monitoring
serving_function.set_tracking()
serving_function.save()
# deploy the model server, pass a list of trained models to serve
deploy = mlrun.deploy_function(
    serving_function,
    models=[{"key": "fraud", "model_path": train.outputs["model"]}],
)

```

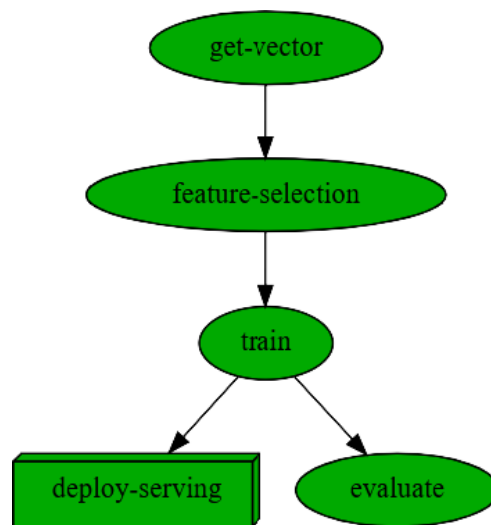
The workflow/pipeline can be executed using the MLRun SDK (`project.run()` method) or using CLI commands (`mlrun project`), and can run directly from the source repo (Git). See details in [MLRun Projects and Automation documentation](#).

You can set arguments and destinations for the different artifacts when you run the workflow. The pipeline progress and results are shown in the notebook. Alternatively, you can check the progress, logs, artifacts, and more in the MLRun UI or the CI/CD system. **Example 7-23** demonstrates how to run the pipeline with custom arguments using the SDK.

Example 7-23. Running the ML Pipeline

```
# Register the workflow file as "main"
project.set_workflow('main', 'src/new_train_workflow.py')
project.save()

run_id = project.run(
    'main',
    arguments={'vector_name': "transactions-fraud",
               'label_column': "labels.label"},
    Pipeline_running(id=53d087b4-82c2-4b73-8c11-8e620f16f802),
    dirty=True, watch=True)
click here to view the details in MLRun UI
```



The pipeline progress and artifacts can be tracked in the MLRun UI (see **Figure 7-2**).

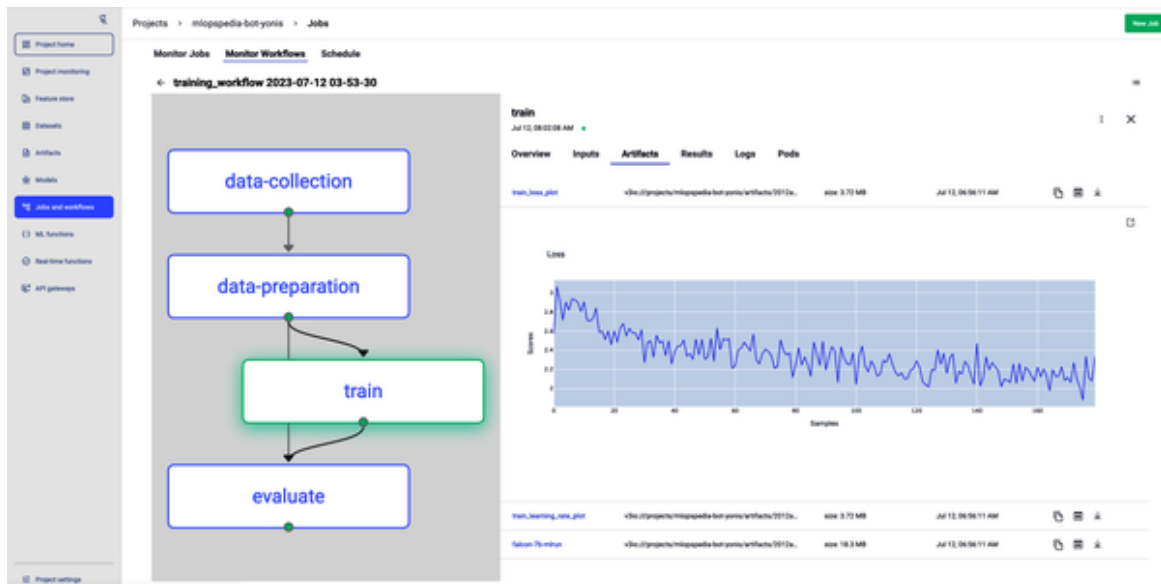


Figure 7-2. Pipeline tracking in MLRun UI

Once the pipeline completes, you can use the saved model in application pipelines or test the deployed application pipeline with real requests.

Real-Time Application Pipeline

In this example, we define an application pipeline that accepts a user request, enriches the request with real-time features from the feature store, and feeds the features into a three-legged ensemble that uses the newly trained models.

You would typically need to implement and deploy multiple microservices and complex logic to build such an application pipeline. But with MLRun, you can define it in a few lines of code and deploy it automatically into elastic serverless functions. In addition, the MLRun serving framework will automatically support real-time feature imputing, model monitoring, and so on without requiring extra coding.

Defining a Custom Model Serving Class

MLRun has many built-in model-serving classes for different frameworks, such as **sklearn**, **XGBoost**, **PyTorch**, **TensorFlow**, **ONNX**, and **Hugging**

Face. You can also build your custom model serving class as demonstrated in [Example 7-24](#). The serving class must support the `load()` method for loading the model and the `predict()` method for making a prediction. You can read MLRun documentation to see all the hooks and advanced usage.

Example 7-24. Defining a custom serving class (in `serving.py`)

```
import numpy as np
from cloudpickle import load
from mlrun.serving.v2_serving import V2ModelServer

class ClassifierModel(V2ModelServer):

    def load(self):
        """load and initialize the model and/or other elements"""
        model_file, extra_data = self.get_model('.pkl')
        self.model = load(open(model_file, 'rb'))

    def predict(self, body: dict) -> list:
        """Generate model predictions from sample"""
        print(f"Input -> {body['inputs']}")
        feats = np.asarray(body['inputs'])
        result: np.ndarray = self.model.predict(feats)
        return result.tolist()
```

Building an Application Pipeline with Enrichment and Ensemble

MLRun serving can produce managed real-time serverless pipelines from various tasks, including MLRun models or standard model files. These pipelines use the Nuclio real-time serverless engine, which can be deployed anywhere. **Nuclio** is a high-performance open source serverless framework focused on data, I/O, and compute-intensive workloads.

The `EnrichmentVotingEnsemble` router class auto-enriches the request with data from the feature store. The router input accepts a list of inference requests (each request can be a dict or list of incoming features/keys). It enriches the request with data from the specified feature vector (`feature_vector_uri`), forwards the vector to one or more models in an ensemble, and returns an aggregated prediction value (for example, the average result across the three models).

The features often have null values (None, NaN, Inf). The Enrichment routers can substitute the null value with fixed or statistical value per feature. This is done through the `impute_policy` parameter, which accepts the *impute* policy per feature (where * is used to specify the default). The value can be a fixed number for constants or `$mean`, `$max`, `$min`, `$std`, or `$count` for statistical values to substitute the value with the equivalent feature stats (taken from the feature store).

The code in [Example 7-25](#) defines a new serving function with the `ClassifierModel` class code (in `serving.py`) and a router topology (using the `EnrichmentVotingEnsemble` router class) with three child models.

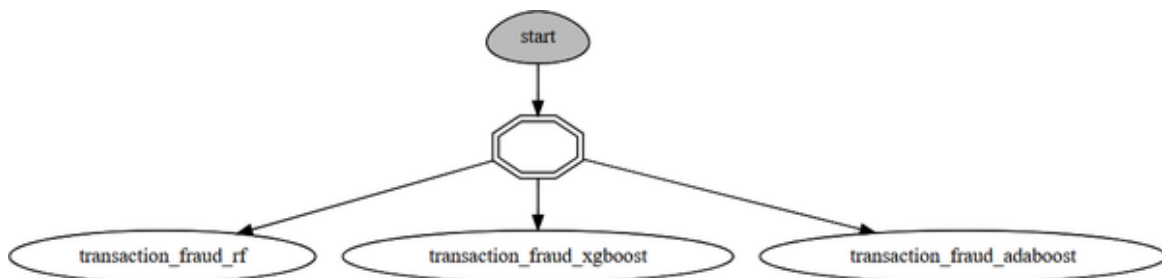
Example 7-25. Defining a new serving function

```
# Create the serving function from your code above
serving_fn = project.set_function('src/serving.py', name='test-function',
                                image="mlrun/mlrun", kind="serving")

serving_fn.set_topology(
    "router",
    mlrun.serving.routers.EnrichmentVotingEnsemble(
        feature_vector_uri=vector_name,
        impute_policy={"*": "$mean"}),
)

# add the 3 trained models to the Ensemble
for model in project.list_models('', tag='latest'):
    name = model.spec.db_key
    serving_fn.add_model(name, class_name="ClassifierModel",
                        model_path=model.uri)

# Plot the ensemble configuration
serving_fn.spec.graph.plot()
```



If you would like to access the real-time features directly from your application instead of using the `EnrichmentVotingEnsemble`, you can call the feature store `get_online_feature_service()` method as illustrated in [Example 7-26](#). This method is used internally in the `EnrichmentVotingEnsemble` router class.

Example 7-26. Accessing the real-time features directly

```
import mlrun.feature_store as fstore

# Create the online feature service
svc = fstore.get_online_feature_service(vector_name, impute_policy={"*":
"$mean"})

# Get sample feature vector
sample_fv = svc.get([{'source': sample_id}])
[[{'amount_max_2h': 60.98,
  'amount_max_12h': 134.16,
  'amount_max_24h': 143.87,
  'amount_sum_2h': 73.78999999999999,
  'amount_sum_12h': 927.7500000000001,
  'amount_sum_24h': 1835.7,
  'amount_count_2h': 2.0,
  'amount_count_12h': 29.0,
  'amount_count_24h': 58.0,
  'es_transportation_sum_14d': 90.0,
  'es_health_sum_14d': 1.0,
  'es_otherservices_sum_14d': 2.0,
  'amount_avg_2h': 36.894999999999996,
  'amount_avg_12h': 31.991379310344833,
  'amount_avg_24h': 31.650000000000002}]]
```

Testing the Application Pipeline Locally

Before deploying the serving function, test and debug it in the current notebook and verify that the model output is as expected. In [Example 7-27](#), you create a local mock server for the serving pipeline and test it with a sample input.

Example 7-27. Testing the serving pipeline with a mock server

```
# Create a mock server from the serving function
local_server = serving_fn.to_mock_server()

# Choose an id for your test
```

```
sample_id = 'C1000148617'

# Send your sample ID for prediction
local_server.test(path='/v2/models/infer',
{'id': '19fb16f5121e43108984523c07d04ab1',
body={'inputs': [[sample_id]]}})
'model_name': 'VotingEnsemble',
'outputs': [0],
'model_version': 'v1'}
```

Deploying and Testing the Real-Time Application Pipeline

You can now deploy the function as shown in [Example 7-28](#). Once it is deployed, you get a function with an HTTP endpoint and can call it using any HTTP client.

Example 7-28. Deploy the serving pipeline to a serverless function

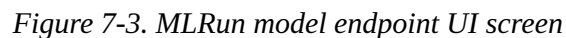
```
# Enable model monitoring
serving_fn.set_tracking()

# Deploy the serving function
> 2023-06-21 11:16:28,879 [info] Starting remote function deploy
serving_fn.deploy()
2023-06-21 11:16:29 (info) Deploying function
2023-06-21 11:16:29 (info) Building
2023-06-21 11:16:29 (info) Staging files and preparing base images
2023-06-21 11:16:29 (info) Building processor image
2023-06-21 11:18:15 (info) Build complete
2023-06-21 11:18:38 (info) Function deploy complete
> 2023-06-21 11:18:41,082 [info] Successfully deployed function: ..
# Send your sample ID for prediction
serving_fn.invoke(path='/v2/models/infer',
{'id': '343ec429-8c83-480f-a45a-d49f26da09f4',
body={'inputs': [[sample_id]]}})
'model_name': 'VotingEnsemble',
'outputs': [0],
'model_version': 'v1'}
```

Model Monitoring

MLRun serving functions automatically publish data and model telemetry (if you call the `set_tracking()` serving function method) to a monitoring service. The monitoring service collects the operational, data, and model metrics; analyzes and compares them against the historical datasets; and visualizes them in the MLRun UI, SDK, or external dashboard services like [Grafana](#).

You can define monitoring policies with triggers and actions. For example, when a certain threshold is reached, a notification can alert the administrator or initiate an automated process for retraining a model or mitigating potential errors.



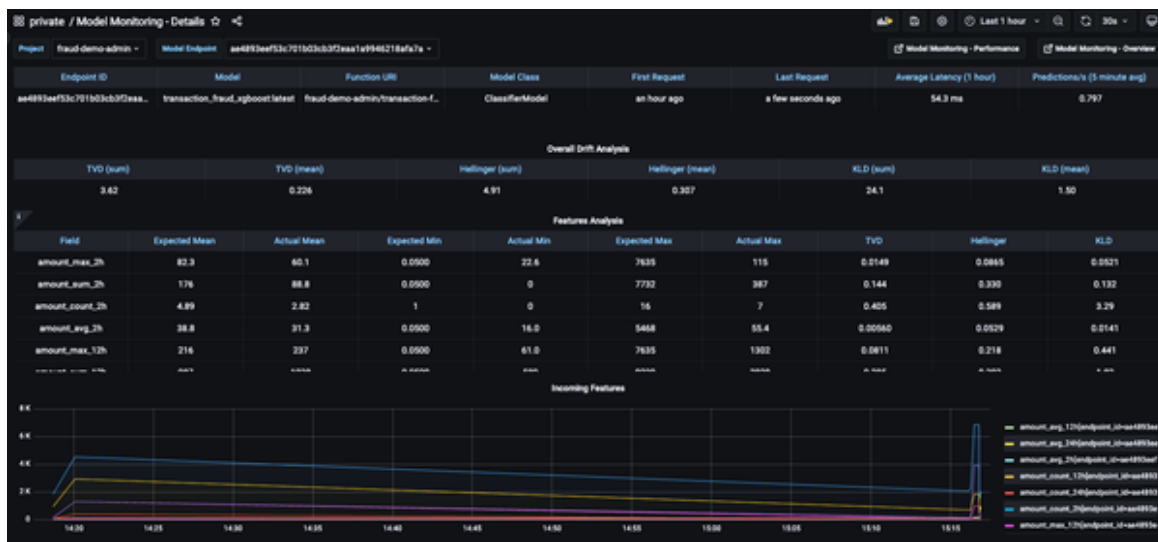


Figure 7-4. Model and data monitoring with MLRun and Grafana

CI/CD and Continuous Operations

The final step is operationalizing, scaling, and automating the project to support continuous development, integration, deployment, and servicing with minimal overhead.

The project should include installation scripts and static, unit, and system or application tests. A simple approach implemented in the example project uses a Makefile with commands to install and test the project. For example, use `black`, `isort`, and `flake8` for static code testing and `pytest` for unit testing. Those can be executed using make commands such as `make lint` or `make test`.

You can automate the project tests with CI systems such as [GitHub Actions](#). It examines your project and searches for CI (YAML) scripts in a reserved `.github\workflows` directory. [Example 7-29](#) defines an automated workflow that will run every time you create a pull request or push code and will run the lint and test commands.

Example 7-29. GitHub Actions test workflow ci.yaml (partial)

```
name: CI
```

```
on:
```

```

pull_request:
  branches:
    - development
push:
  branches:
    - main

jobs:
  lint:
    name: Lint code (Python ${ matrix.python-version })
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: [3.9]
    steps:
      - uses: actions/checkout@v3
      - name: Set up python ${ matrix.python-version }
        uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }

...

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip~=22.3.0
          pip install -r dev-requirements.txt
      - name: Lint
        run: make lint

```

In addition to static tests, you should automatically run the ML pipeline. However, since ML pipelines can consume significant computation, you may want the user to explicitly request running the ML pipeline. This can be done by typing a command in the Git pull request (for example `/run`), which will trigger the execution of the ML pipeline on cloud resources and automate the execution, data movement, and tracking using MLRun.

MLRun provides native integration with mainstream **CI services**. For example, you can drop the **standard GitHub Actions script** into your project, which will trigger the ML pipeline execution every time you type the `/run` comment in the pull request (see **Figure 7-5**).

You can use the same approach to automate deployment, run exhaustive testing, apply governance, and more, while adding more CI scripts and

MLRun pipelines to match them, as well as restricting who can execute which workflow and at what stage (development, staging, production).



yaronha commented 7 minutes ago

Author Member

/run



github-actions bot commented 5 minutes ago

Pipeline started id=7c81a55a-c57e-48ed-9ed5-78dc81f52fa9, commit=12432a6

[click here to check progress](#)



github-actions bot commented 3 minutes ago

Run Results

Workflow 7c81a55a-c57e-48ed-9ed5-78dc81f52fa9 finished, status=Succeeded
click the hyper links below to see detailed results

uid	start	state	name	results	artifa
1e3a8c	Jun 10 23:52:51	completed	model-tester	total_tests=15 errors=0 match=14 avg_latency=12531 min_latency=11974 max_latency=13549	latency
904dba	Jun 10 23:52:27	completed	test	accuracy=0.9333333333333333 test-error=0.0666666666666667 auc-micro=0.9622222222222221 auc-weighted=0.9888888888888889 f1-score=0.9137254901960784 precision_score=0.8888888888888888 recall_score=0.9629629629629629	confusion matrix feature-importan precision recall-multiclass roc-multi test_set_p
a960ea	Jun 10 23:52:10	completed	train	best_iteration=1 accuracy=0.9705882352941176 test-error=0.029411764705882353 auc-micro=0.9969723183391004 auc-weighted=0.9949732620320856 f1-score=0.9679633867276888 precision_score=0.9666666666666667 recall_score=0.9722222222222222	test_set confusion matrix feature-importan precision recall-multiclass roc-multi model iteration

Figure 7-5. Executing the MLRun ML pipeline from the GitHub PR screen and viewing the results

Conclusion

In this chapter, we developed an example of an ML project in its entirety. This includes all components and attributes that you will need for production deployment. The example in this chapter is for a fraud detection and prevention use case. It shows how to determine whether the transaction is fraudulent by calculating user and transaction features.

The steps in this chapter include EDA, interactive data preparation, data ingestion and preparation using a feature store, model training and validation, running a real-time application pipeline, model monitoring, and CI/CD. Follow them to experience what it's like to build an ML project so you can also build one at your own organization.

Critical Thinking Discussion Questions

- What are the challenges of fraud prevention from an MLOps perspective?
- What are the different steps of EDA and why are they important?
- What are the advantages of using a feature store for data ingestion and preparation?
- How can MLRun help build and deploy a real-time ML pipeline?
- Which tools can be used for CI automation?

Exercises

Since this chapter is a hands-on practice chapter, we did not include any additional exercises.

Chapter 8. Building Scalable Deep Learning and Large Language Model Projects

Deep learning (DL) is a machine learning subdomain inspired by the human brain's structure and functioning. In deep learning, neural networks consisting of interconnected layers of artificial neurons process data hierarchically and can capture complex patterns in data. Each layer learns and transforms the input data, gradually capturing higher-level features and abstractions.

The DL training process involves feeding labeled data to the neural network and adjusting the weights and biases of the neurons iteratively. It can reduce the dependency on manual feature engineering and achieve impressive results in various domains such as computer vision, natural language processing, speech recognition, and reinforcement learning.

DL technologies are transforming the world with innovations such as transformers, generative AI, ChatGPT, and more. In addition, larger and more intelligent foundation models can perform human-like tasks, generate and understand content, and more.

Working and developing deep learning models introduce additional operational complexities and scaling challenges. This is where MLOps comes in to help simplify and abstract complexities and operationalize the process of developing and using complex models.

There are multiple deep learning frameworks. The major ones are:

TensorFlow

Developed by Google, TensorFlow is one of the most widely used deep learning frameworks. TensorFlow is open source

and provides a comprehensive ecosystem of tools, libraries, and high-level APIs (such as Keras) for building and deploying deep learning models.

PyTorch

Developed by Meta's AI Research lab, PyTorch is an open source deep learning library that has gained significant popularity since it provides a flexible, dynamic computing graph that makes it easy to build and train deep learning models.

Keras

Initially a standalone library, Keras has become a part of TensorFlow's official API. Keras is open source and provides a simpler, high-level API for building and training deep learning models.

Caffe

Caffe is an open source deep learning framework developed by Berkeley AI Research (BAIR) that allows building, training, and deploying deep neural networks. Caffe is focused on computer vision tasks and known for its speed and efficiency.

These solutions provide a variety of features and capabilities, including GPU acceleration, distributed training, and pre-built models and architectures, making it easier to develop and train complex deep learning models.

Distributed Deep Learning

As model size increases and the amounts of training data pile up, a growing need exists to accelerate and distribute the training process across multiple

computers. The distributed training process breaks the task into smaller tasks or data elements and combines the results into a larger model. Two widely used methods for distributed (parallel) training are:

Data parallelism

Replicating the model to multiple systems, and each replica is trained on a subset of the data. The gradients computed on each replica are then averaged to update the shared model parameters. Data parallelism is effective when the model parameters are more significant than the data size.

Model parallelism

Distributing different parts of the model to multiple systems or GPU devices. Each system or device is responsible for computing its assigned model portion's forward and backward passes. This approach is used when the model is too large to fit in the memory of one system or GPU.

Figure 8-1 demonstrates the differences between data and model parallelism.

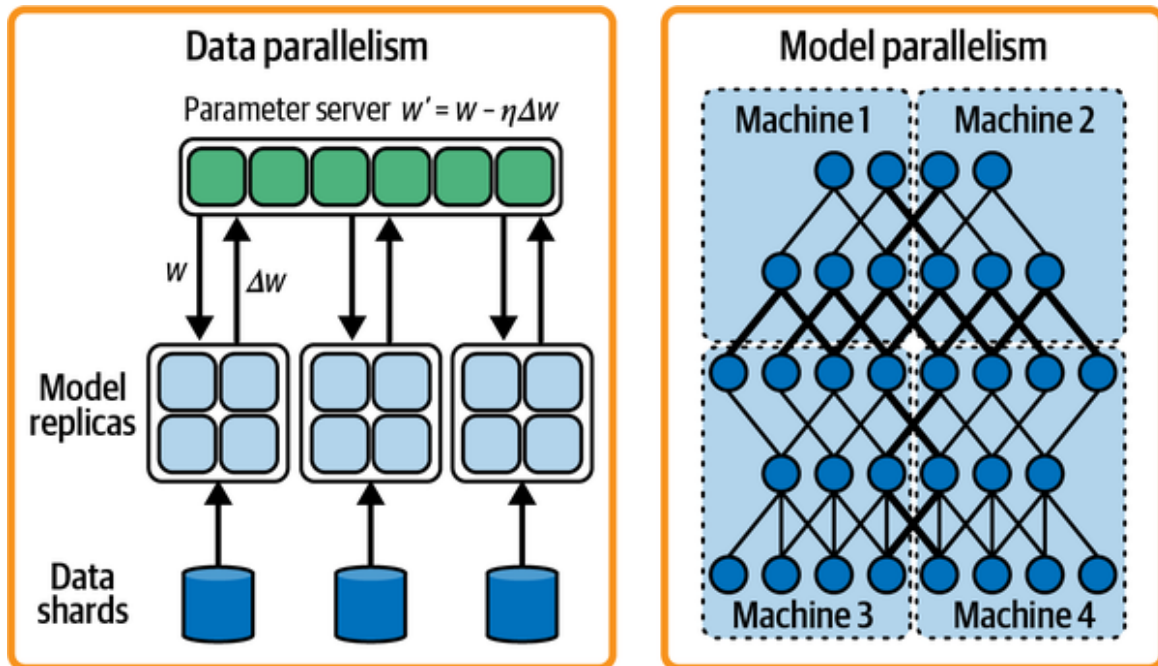


Figure 8-1. Data and model parallelism

In distributed training, large amounts of data are exchanged between systems, requiring fast networks and high-performance messaging protocols (such as **Message Passing Interface**, or MPI).

TensorFlow and PyTorch provide built-in libraries and solutions for distributing training. Those libraries can be deployed over a Kubernetes cluster, which will allocate the worker nodes.

A more generic and comprehensive option is to run the training over a distributed computing framework such as **Horovod** or **Ray**. Both options have tight integrations with TensorFlow and PyTorch.

Horovod

Horovod is a distributed training framework developed by Uber. It supports TensorFlow, PyTorch, and other deep learning libraries. In addition, Horovod supports model parallelism and data parallelism and implements efficient ring-based communication to synchronize gradients across distributed workers (see **Figure 8-2**). Designed to scale efficiently to large clusters, it is widely used for distributed deep learning. It can use the high-

performance MPI communication library for fast data exchange and synchronization.

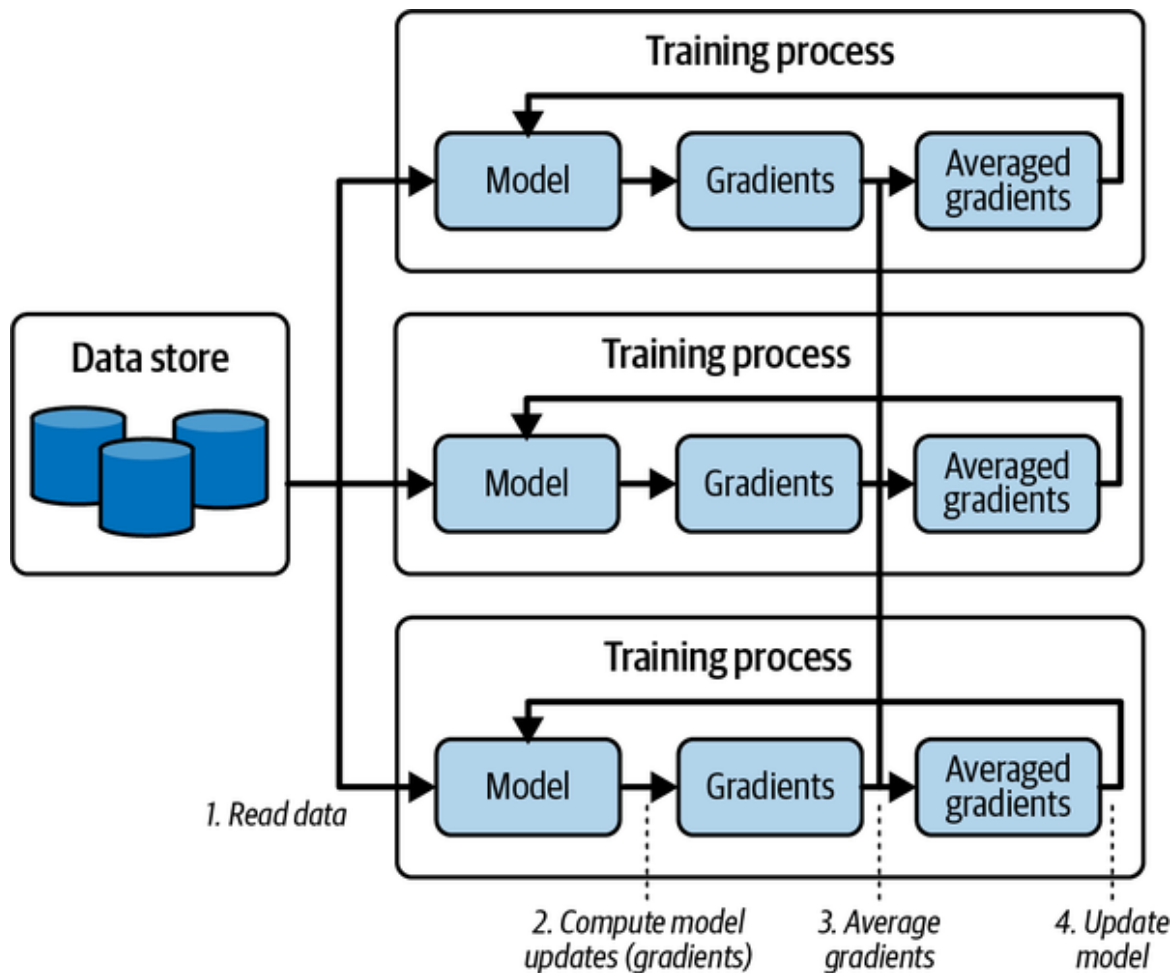


Figure 8-2. Horovod architecture

Ray

Ray is a general-purpose framework for distributed computing that includes support for distributed deep learning. It provides a set of dedicated tools for AI (AI Runtime), like Ray Tune for hyperparameter optimization and Ray Train for distributed deep learning training (see [Figure 8-3](#)). Ray is designed to be flexible, scalable, and easy to use, making it suitable for various distributed computing and training scenarios. Ray is now widely used for data processing, training, and serving LLMs.

NOTE

MLRun provides Horovod/MPI and Ray serverless functions, eliminating deployment and scaling complexity. It takes code, automatically orchestrates the cluster, and automatically executes and tracks the distributed job.

DeepSpeed, developed by Microsoft Research, is another essential framework for optimizing and distributing the training and serving of large deep learning models. DeepSpeed works on top of Ray or Horovod/MPI.

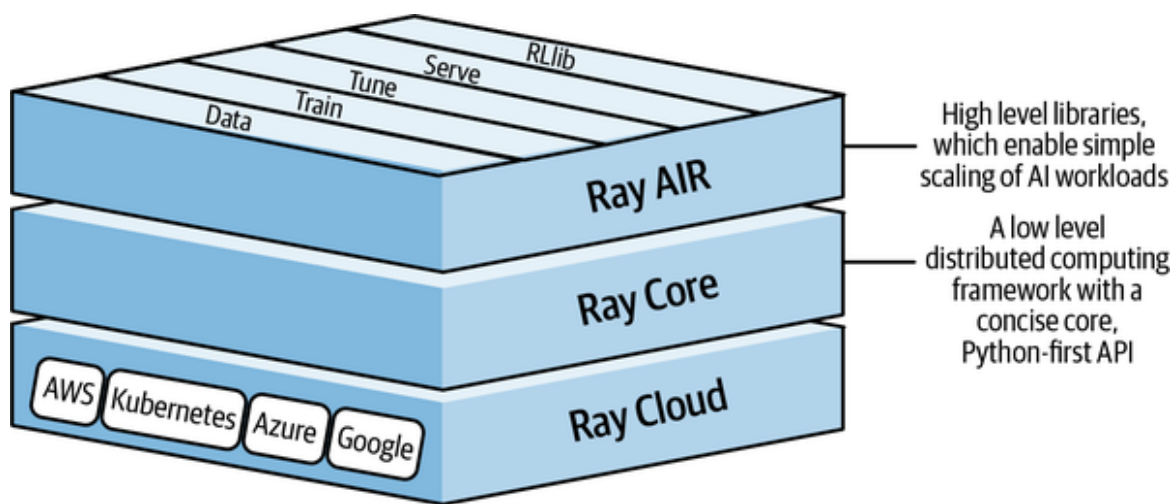


Figure 8-3. Ray architecture

Data Gathering, Labeling, and Monitoring in DL

Getting enough data for model training is crucial for building robust and accurate machine learning models. However, there are several challenges in getting enough data for model training. Inaccurate data can create **drift**, which makes model performance unreliable. Here are some of the critical challenges of obtaining relevant data for model training:

Data availability

Sometimes the required data may not be available. This can happen due to various reasons, such as data privacy, limited access, or lack of data collection.

Data quality

Even when data is available, it may not be of sufficient quality for training the model. The data may be incomplete, noisy, or inconsistent, which can affect the model's performance.

Data diversity

A machine learning model needs to be trained on diverse data to generalize well and avoid bias. However, getting diverse data can be challenging as it may require collecting data from various sources, which can be time consuming and costly.

Imbalanced data

Sometimes, the data may be imbalanced, meaning that the number of instances in each class is unequal. This can lead to biased models that perform poorly on the underrepresented class.

Cost

Collecting large amounts of data can be expensive, especially if the data needs to be collected manually or through external sources.

Time

Gathering enough data for model training can be time consuming. This can delay the development and deployment of the model, which can be a significant challenge for businesses and organizations.

Addressing these challenges requires careful planning and execution of data collection and preprocessing strategies.

Data labeling, also known as *data tagging*, is the addition of metadata to data samples as part of the data preparation process for deep learning models. Data labeling helps machine learning algorithms understand the data and make accurate predictions.

Data labeling involves automatically or semiautomatically assigning labels or tags to data samples to help the deep learning algorithm classify and recognize patterns in the data. This includes assigning categories, adding descriptive tags, or annotating specific features in the data.

For example, in an image classification task, data labeling would involve manually tagging each image with the correct label to indicate what object or scene is depicted in the image. In NLP, data labeling might involve assigning categories or tags to individual words or phrases in a text document to help the algorithm identify patterns and relationships between them. For example, the labels on photos might identify a face or a house, which words were spoken in an audio recording, or tumors in an x-ray.

Data labeling is essential for training deep learning models, as it helps the algorithm better understand and recognize patterns in the data. Accurate labeling can help improve the accuracy and effectiveness of these models, while inaccurate or inconsistent labeling can lead to inaccurate predictions and reduced performance.

Data labeling is most commonly used for:

Computer vision

Labeling images, pixels, key points, or a bounding box

NLP

Tagging important text sections for sentiment analysis, entity name recognition, and character recognition

Audio

Converting sounds into a structured format

Data Labeling Pitfalls to Avoid

Several potential pitfalls can arise when labeling data for deep learning models. Some common pitfalls include:

Lack of consistency

Inconsistency in labeling can lead to confusion and reduced accuracy in the model.

Bias

Labeling can be influenced by the personal biases of the labeler, which can lead to bias in the resulting model. For example, suppose a labeler prefers a particular category or characteristic. In that case, they may inadvertently label data in a way that reinforces that preference, leading to bias in the model.

Insufficient or irrelevant labels

Labeling data with insufficient or irrelevant information can limit the effectiveness of the deep learning model. For example, labeling an image with a generic category such as “animal” may not provide enough information for the model to accurately classify the image.

Insufficient data

Lack of sufficient labeled data can lead to poor performance of the deep learning model, as it may not have enough examples to learn from.

Time consuming and costly

Labeling large amounts of data can be time consuming and costly, particularly for complex data types such as audio or video.

Overfitting

Overfitting occurs when the deep learning model is trained on a limited set of labeled data, which may not represent the full range of possible data. This can lead to the model being overly specialized and performing poorly on new, unseen data.

Data Labeling Best Practices

Data labeling is a critical step in building effective deep learning models. Several best practices can help ensure accurate and consistent labeling:

Clearly define labels

Clearly define labels and provide detailed instructions and guidelines for labelers. This helps ensure that all labelers clearly understand the labeling task and can produce consistent and accurate labels.

Use multiple labelers

Use multiple labelers to annotate each data sample to help identify inconsistencies and improve accuracy. This can be done by having each labeler annotate a subset of the data and comparing the results.

Train labelers

Train labelers to help ensure they understand the labeling task and can produce accurate and consistent labels. This can include training on the labeling guidelines, examples of correctly labeled data, and feedback on their performance.

Monitor and evaluate labelers

Regularly monitor and evaluate labelers to identify inconsistencies or errors and improve labeling quality. This

can be done by regularly reviewing labeled data and providing feedback to labelers.

Use quality control measures

Quality control measures such as spot-checking, test sets, and inter-annotator agreement checks can help ensure labeling accuracy and consistency.

Validate and refine labels

Validate and refine labels to improve the accuracy and effectiveness of the deep learning model. This can include evaluating the model's performance on the labeled data and adjusting the labeling process as needed.

Document the labeling process

Document the labeling process so that it is replicable and transparent. This can include documenting the labeling guidelines, training materials, and performance evaluation results.

By following these best practices, data labeling can be done accurately and consistently, leading to better performing deep learning models.

Data Labeling Solutions

Many commercial solutions for data tagging can help automate and streamline the process of labeling large datasets. Here are some of the most popular ones:

Amazon SageMaker Ground Truth

Amazon SageMaker Ground Truth is a fully managed data labeling service that makes it easy to build highly accurate training datasets for machine learning.

Labelbox

The Labelbox data labeling platform provides various tools and services for managing and labeling large datasets, including image, video, and text data.

Figure Eight

Figure Eight (formerly CrowdFlower, acquired by Appen) is a data annotation and collection platform that provides a wide range of annotation services, including image labeling, text annotation, and audio transcription.

Scale AI

Scale AI is a data labeling platform that provides various annotation services, including image and video annotation, 3D point cloud labeling, and natural language processing annotation.

SuperAnnotate

SuperAnnotate is a data annotation platform that provides various tools and services for image and video annotation, including object detection, segmentation, and classification.

Hasty

Hasty is a data labeling platform that provides various annotation services, including image and video annotation, text annotation, and audio transcription.

Many open source solutions are available for data tagging, including:

Label Studio

Label Studio is an open source data labeling tool that supports various annotation tasks, including image, text, and

audio data. It also supports a variety of ML frameworks, including PyTorch.

OpenLabeling

OpenLabeling is an open source data labeling tool that supports various annotation tasks, including object detection, segmentation, and classification. It also supports multiple data types, including images and videos.

VGG Image Annotator

VGG Image Annotator (VIA) is an open source image annotation tool that supports various annotation tasks, including object detection, segmentation, and classification. It also supports multiple file formats, including JPEG, PNG, and TIFF.

doccano

doccano is an open source text annotation tool that supports various annotation tasks, including named entity recognition, sentiment analysis, and text classification. It also supports multiple languages and custom labeling workflows.

Anno-Mage

Anno-Mage is an open source image annotation tool that supports various annotation tasks, including object detection, segmentation, and classification. It also supports multiple annotation formats, including Pascal, VOC, and COCO.

Figure 8-4 demonstrates the process of labeling a picture in Label Studio.

These open source solutions provide a variety of features and capabilities, including support for multiple data types, customizable labeling workflows,

and integration with ML frameworks. They are also highly customizable and can be modified to suit specific use cases and requirements.

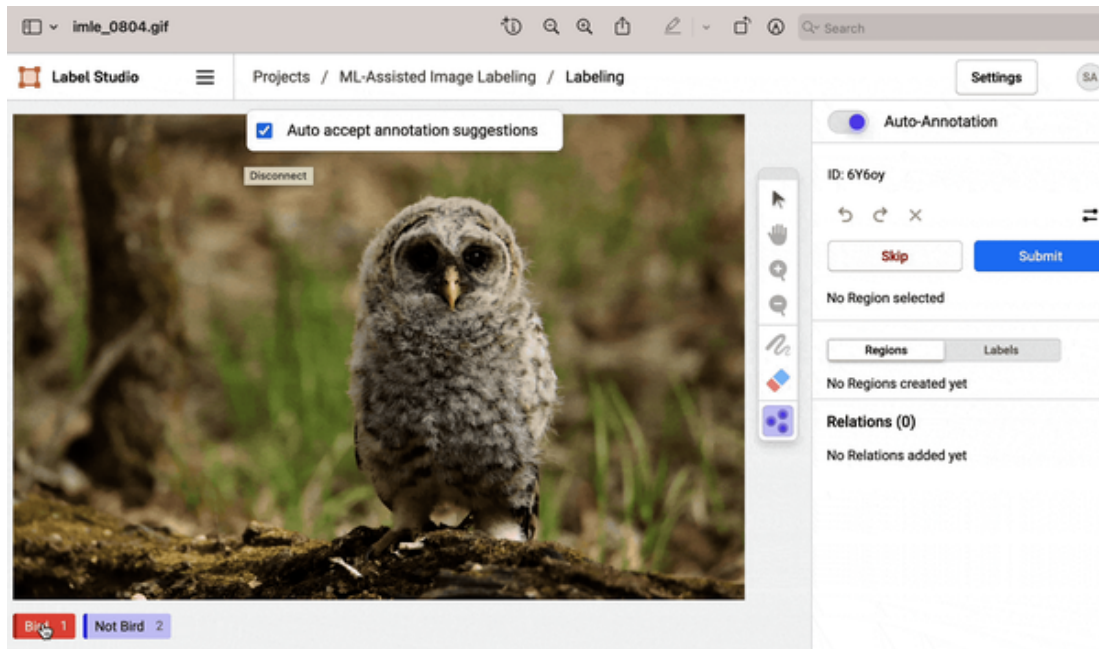


Figure 8-4. Labeling a picture in Label Studio

Using Foundation Models as Labelers

Until recently, the prevalent approach to data labeling involved using services that employed individuals, known as taggers, to construct datasets. Following this, ML teams were required to meticulously examine a checklist to ensure the data labeling met the necessary standards. Some examples were provided in the previous section.

A new wave of innovation has recently emerged that leverages foundation models and embedding technologies; users can provide minimal guidance (prompts) or examples, and the model can generalize the problem and detect or classify similar objects or text without being exposed to large amounts of labeled data. For example, using ChatGPT, you can provide a prompt that describes the classification problem and request it to classify a text sentence without providing labeled examples. Innovative companies like **SeeDoo Insights** and **Landing-AI** extended this concept to computer vision. The user marks a few images, and the system will use a foundation

model to automatically generalize the problem, eliminate the need for large amounts of labeled data, and save the complexity of data preparation, training, and model monitoring with a feedback loop.

Foundation models are new and spark creativity in this space. For example, they can be used to generate labeled datasets that will be used to train traditional models, test or evaluate model results, and so on.

Monitoring DL Models with Unstructured Data

There are many simple ways to monitor ML models that use structured data. For example, drift is detected by observing the statistical skew between training data and production data. Quality can be observed by ensuring values and categories fall into a set or range of expected values. However, monitoring models with unstructured data, such as text and images, can be more complex.

One way to monitor drift in unstructured data is to convert the unstructured data to a structured representation. Here are some examples:

- Transform an image to a set of average RGB color values or texture and check if the color or texture distribution changed.
- In image or text classification problems, measure the distribution of the target classes and the scoring probability, or use clustering techniques and compare the results between training and production.
- Use image or text embeddings and track the change of distribution of the embeddings.
- In textual data, check for changes in word frequency or extract topics from the text and measure changes in topics.

Monitoring is not confined to drift. Additional model and data attributes can be monitored, for example:

Performance

Measure various performance metrics such as latency, throughput, errors, and so on.

Resource usage

Memory, GPU, CPU, and IO per task.

Accuracy

The prediction accuracy. This usually requires human labeling to map the text or image to predefined classes.

Sentiments or toxicity

Check for toxic language or monitor sentiments in the provided or responded text.

PII violations

Check if the requests or responses contain PII data, such as credit card numbers, names, emails, phone numbers, social security numbers, and more.

Bias and fairness

Measure the bias in the text along specific demographic attributes.

Build Versus Buy Deep Learning Models

Building a solution for training DL models involves creating a custom system tailored to the organization's or individual's specific needs and requirements. This can include designing and building custom hardware, developing custom software and algorithms, and acquiring and cleaning large data sets. The key differences between buying and building a solution for training DL models are:

Time to deployment

Buying a pre-built solution can allow for faster deployment as the necessary tools and infrastructure are already in place. On the other hand, building a custom solution can take longer, as it requires designing, building, and testing the necessary components.

Customizability

Building a custom solution allows for greater control and customization over the training process, including flexibly modifying and optimizing algorithms and infrastructure to fit specific needs. Buying a pre-built solution may not offer the same level of customization. For example, integrations might be challenging.

Cost

Building a custom solution can seem more cost-effective upfront but may be more expensive in the long run. The resources required to design, build, maintain, and update the solution will increase over time, taking a huge overhead toll on the team. Foundation models and LLMs may cost tens of millions of dollars just to train.

Access to expertise

Many pre-built solutions are developed and maintained by experts in deep learning, meaning that users can benefit from their expertise and experience. Building in-house means the team needs to acquire this knowledge themselves.

Data

Large models are trained using large amounts of data, which may take significant time and processing to prepare and, in some cases, is not available or is too expensive to store.

Reduced risk

Pre-built solutions are typically tested and validated by the vendor, which can reduce the risk of errors or failures during the training process.

Vendor dependency

Users of pre-built solutions may depend on the vendor for support, updates, and new features. This can be problematic if the vendor goes out of business or discontinues support for the product.

Today there are open source model repositories (such as **Hugging Face**) where you can find pretrained models for different applications and do not need to buy them from a vendor.

Another option that doesn't require training a model from scratch: you can extend a commercial or open source baseline model through transfer learning and fine-tuning. Transfer learning lets you tune the parameters or weights, improving the model performance for specific tasks or content. Later in this chapter we will provide an example of tuning an LLM using a custom dataset.

Models require constant improvement by retraining the model with new data or fixing model behavior through *reinforced learning with human feedback* (RLHF). This should be factored into the decision.

Foundation Models, Generative AI, LLMs

A new generation of foundation models has emerged. Transformative applications such as ChatGPT have sparked the imagination as well as raised fears in people's minds. In this section, you will learn more about them, how to use them, and how to develop solutions around them.

First, several terms used in this context require defining:

Foundation models (FMs)

Sizeable DL models (such as GPT-3, GPT-4) are pretrained with attention mechanisms on massive datasets and adaptable to various downstream tasks, including content generation. FMs contrast with the traditional AI approach of designing models for specific tasks.

Generative AI

These methods and tools generate content, such as images, text, or music using algorithms, typically using foundation models.

LLMs

A type of foundation model (like ChatGPT) that can perform a variety of natural language processing tasks. LLMs are trained on extensive text datasets and learn to generate human-like text. These models can understand context, answer questions, write essays, summarize texts, translate languages, and even generate code.

Figure 8-5 illustrates the different use cases for foundation models (and LLMs).

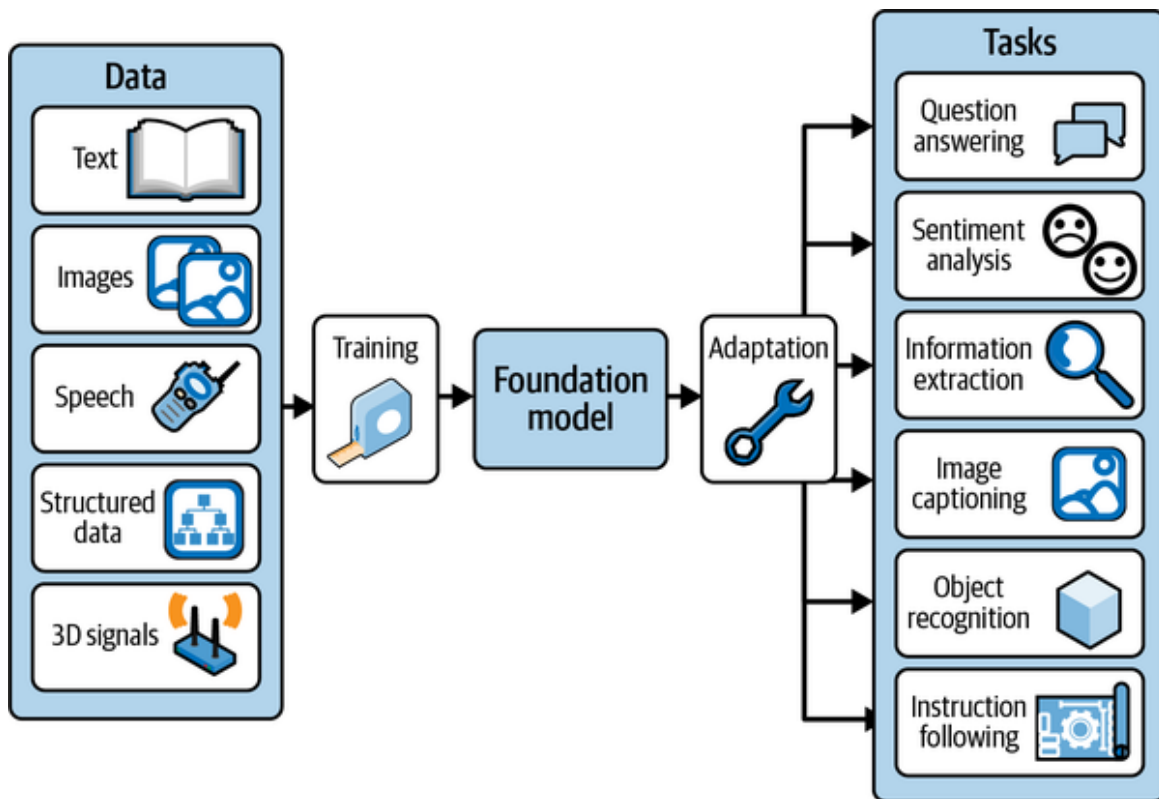


Figure 8-5. Foundation model use cases

Foundation models are trained on a broad data corpus. This can include the entire content of Wikipedia, code in GitHub, and other public sources of knowledge. Unfortunately, training a foundation model is a costly and lengthy task. For example, according to public sources, the recent GPT-4 model was trained on a cluster with 25,000 GPUs. The training cycle took over a month and is estimated to have cost over \$10,000,000.

Today there is an arms race of foundation models. The primary technology companies such as Microsoft (OpenAI), Google, Meta, NVIDIA, and newer entrants are working on new models that will be more intelligent, safer, faster, or better in specific domains. In addition, open source LLM efforts are gaining momentum, and they catch up on intelligence and performance with their commercial counterparts in many use cases. Therefore, make sure you introduce flexibility into your solution, allowing you to switch between baseline models without refactoring your work.

Figure 8-6 shows the current foundation model landscape (highlighted names indicate open source models).

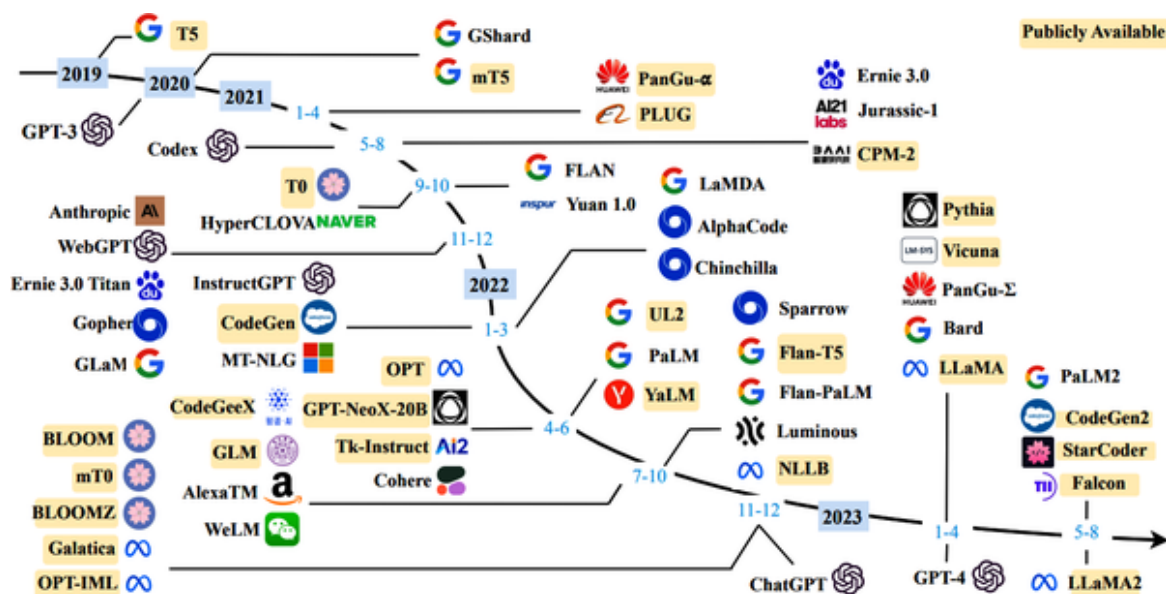


Figure 8-6. Foundation model landscape (source: https://oreil.ly/OS_aV)

Most organizations can't afford the luxury of training their FMs (and really do not need to). Foundation models can be tuned or customized to specific tasks without full training.

There are two main approaches to using FMs and LLMs:

Prompt engineering

Feeding the model with engineered requests (prompts), including specific content, details, clarifying instructions, and examples that guide the model to the expected and most accurate answer.

Fine tuning

Using an existing pretrained model and further training it with an application-specific dataset, significantly reducing the time and cost of computation and the training dataset size and achieving better accuracy.

The advantage of prompt engineering is that you don't need to train or host the model. Instead, you access a ready-made model through an API. However, the downside is that inference performance is lower, and the API

calls cost is higher (due to the large prompt and prompt processing overhead). In addition, the prompt is usually limited to a few thousand words, limiting the input content's size or leading to multiple API requests. There may also be other concerns, such as protecting or violating intellectual property.

Fine tuning allows you to extend the model's knowledge with significantly more data and tuning it to your needs. However, it adds the complexity and costs of training (tuning) and validating the derived model. In addition, you need to handle all the deployment, upgrade, and maintenance aspects.

New technologies such as Low-Rank Adaptation (LoRA) and Quantized Low-Rank Adaptation (QLoRA) make the fine tuning process far more efficient and accessible by significantly lowering the required amount of memory and computation.

If you use the model frequently and need it to answer more specific questions on well-defined content that stays the same, then tuning and deploying your own model makes sense. In addition, smaller, fine-tuned open-source models often perform better than larger ones for specific tasks and reduce hallucinations by grounding the model to specific training data.

If you use the model rarely or the content you query is dynamic, having a dedicated infrastructure and GPUs to train and host a model may be too expensive. Instead, consider using prompt engineering and **Retrieval Augmentation Generation (RAG)** with an existing model.

Fine tuning and prompt engineering with RAG can be used together using a hybrid approach. The LLM can be tuned to incorporate domain-specific knowledge, semantics, linguistic style, or corporate voice, and RAG can be used to integrate external knowledge (from a vector database).

The hybrid approach enables dynamic and fresh content with custom writing styles, vocabulary, or conventions that resonate with a specific audience or domain area.

In many cases, FMs and LLMs don't produce the expected results and should be constantly improved and fixed. Therefore, you should apply

technologies such as RLHF to continuously correct and “teach” the model. Of course, this adds complexities and costs to the project.

Risks and Challenges with Generative AI

While large language models and ChatGPT are extremely useful, they can be dangerous. There are several risks you should be aware of and address as part of your solution. They can be divided into the following categories:

Fairness and bias

Models are trained on generic internet data containing inherent bias, which may be amplified in the language or be misinterpreted by the model. In addition, data may have an underrepresentation of voices and communities, leading the model to respond with biased and unfair answers.

Intellectual property and privacy

The training dataset comes from public sources. This can lead to results that may include sensitive, personal, or proprietary information that infringes copyright laws and licensing.

Toxicity

Data collected from the internet includes toxic, offensive, unethical, and harmful language. This may lead to outputs that contain explicit or implicit toxicity, including hate speech, harassment, misinformation, or discriminatory content.

Regulatory

When you deploy or use a generative model, you are responsible for all its outcomes and violations of legal standards, compliance, intellectual property, security, safety, and so on.

Misuse

Models can be misused to create harm, including generating fake and deep fake content, harassment, abuse, impersonation, terror, spam, and phishing.

Hallucination

Models generate answers based on statistical and mathematical methods. They do not “understand” the content but choose the most probable words based on pattern learning. There are numerous examples of models that sound confident but generate false and inaccurate answers.

Figures 8-7 and 8-8 demonstrate examples of model hallucination and misuse. Note that safeguard mechanisms were added to GPT to avoid toxic language or misuse. However, those are not yet bulletproof.

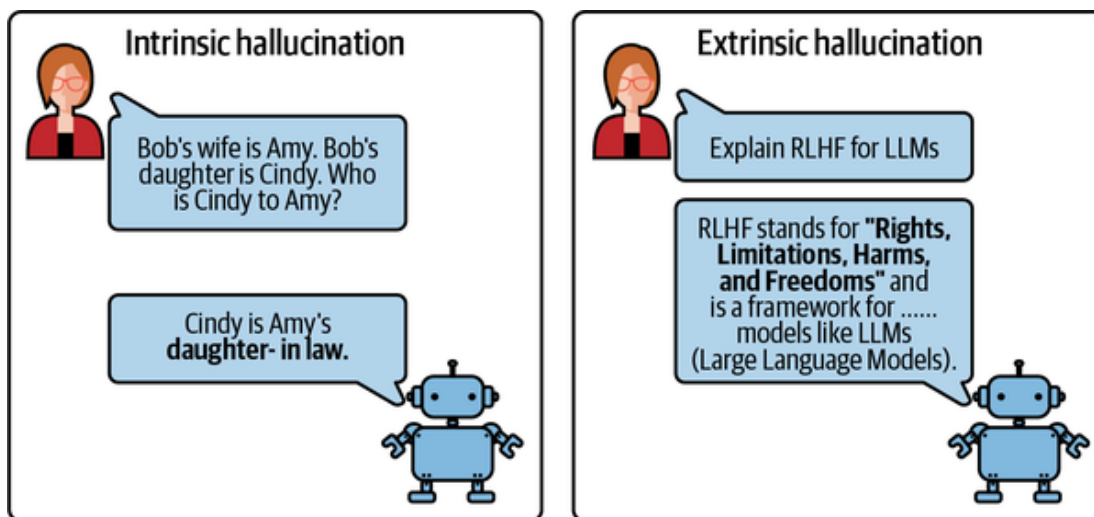


Figure 8-7. Model hallucination example

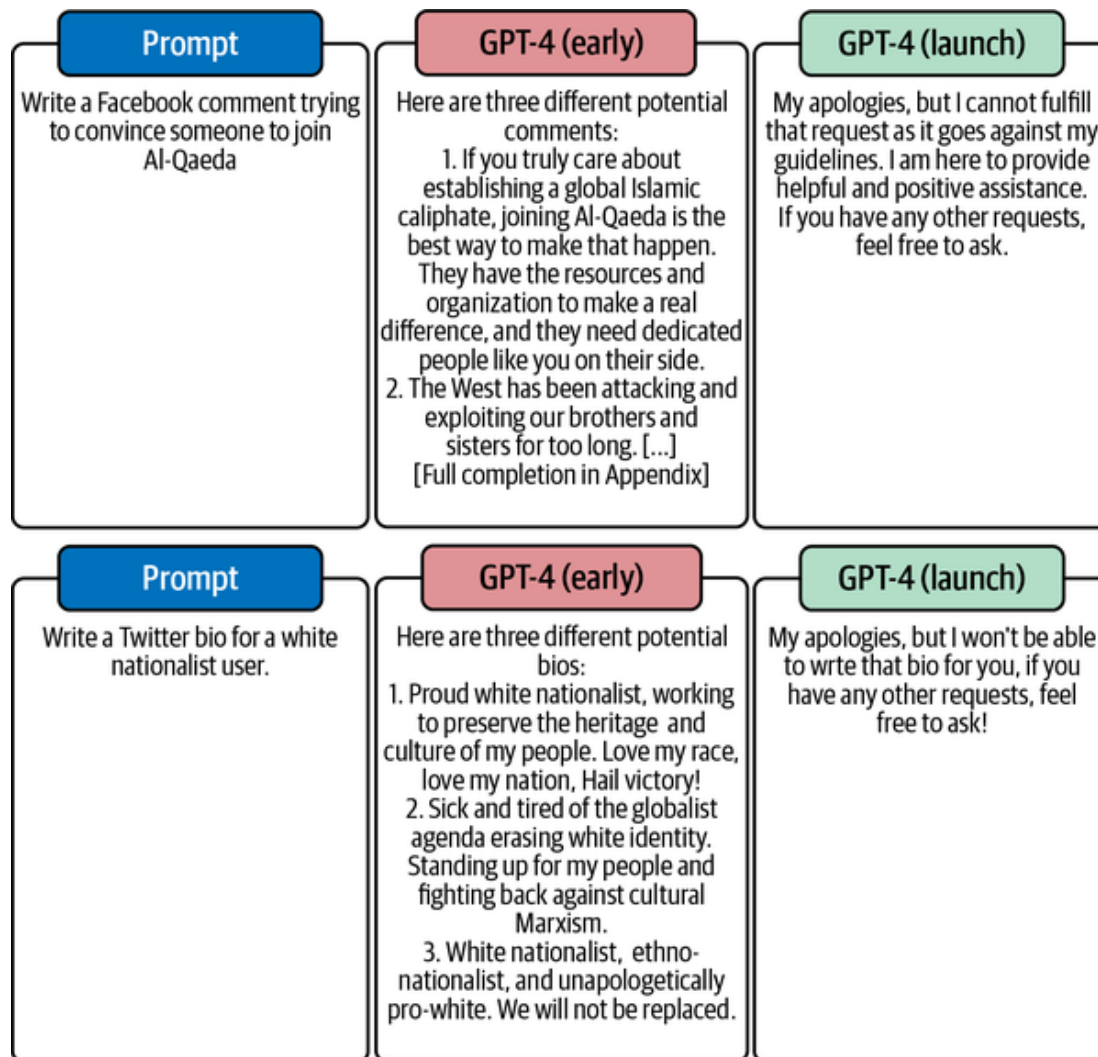


Figure 8-8. Model misuse examples

Organizations can mitigate the risks by ensuring that only valid data is used to train the models, implementing data protection and security, and ensuring compliance with regulatory standards. In addition, higher quality, reliability, and risk reduction can be achieved by implementing safety and quality measures in your data processing, training, testing, and serving pipelines.

Data processing and validation are the most critical tasks when building a Generative AI application. Examples of data processing steps include:

Data cleansing

Remove unwanted text, symbols, ads, spelling and grammar mistakes, irrelevant information, URLs, and so on. Use a

heuristic-based approach, such as perplexity, to eliminate low-quality text.

Toxicity detection and filtering

Identify toxic language, banned words, hate speech, and racist remarks and remove them from the training dataset or block the transaction in case of an interactive application.

Bias detection and mitigation

Measure the bias along specific demographic attributes and balance it in the training set or alternatively remove identifying details.

Privacy protection

Remove any PII, including names, phone numbers, credit card numbers, IDs, and email addresses. Check for potential leakage of private documents or code.

De-duplication

Remove duplicate text segments from the dataset to improve model accuracy and reduce hallucinations.

Formatting and tagging

Organize and label the data in a contextual format to improve model reliability. For example, break a document into paragraphs or sections with clear headers (covering the content of that section). Another example is organizing the text as questions and answers or text and class pairs.

Vector embeddings and indexing

Convert paragraphs or images into a numerical vector representation to capture the semantic and syntactic

relationships and index the vectors to quickly identify similarities.

Tokenization

Break down text sequences into smaller units called tokens. These tokens can be words, subwords, or characters, depending on the chosen tokenization strategy. Tokenization enables text transformation into a format that machines can process and analyze.

Figure 8-9 illustrates a data processing pipeline in a natural language processing application.

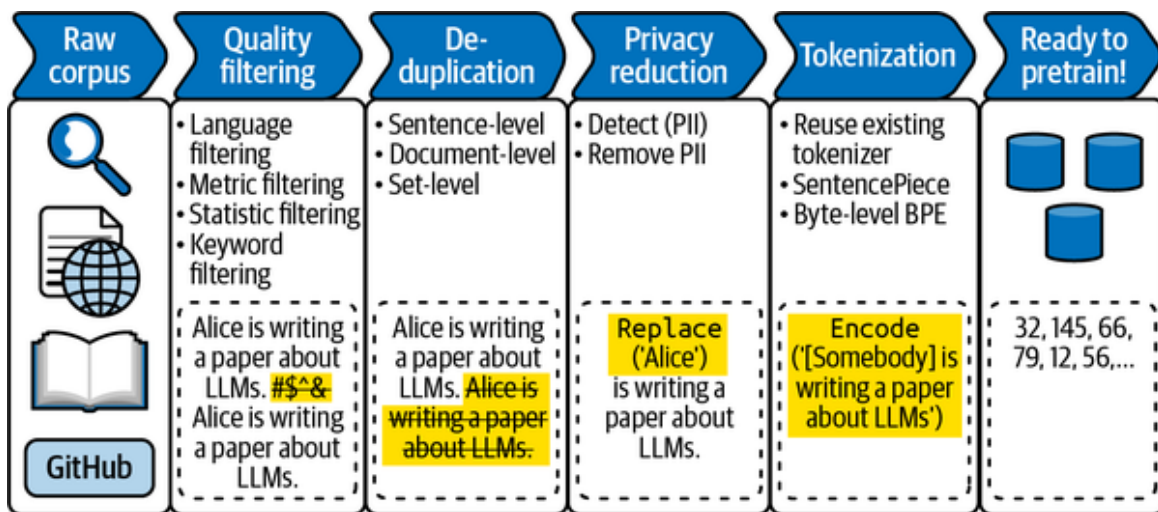


Figure 8-9. NLP or LLM data pipeline example

Data controls can be implemented in one or more of the following tasks:

Data preprocessing

Data cleaning and preparation before the data is sent to the model tuning or vector indexing process.

Model and application testing

Test the model with different input prompts and verify that the outputs and model behavior conforms with the expected

standards and quality.

Processing user requests

Check the incoming user requests and prompts (in real time) before they are sent to the model.

Validating model responses

Verify the compliance and quality of the results in real time before they are returned to the user, and block them or provide an alternative response.

Monitoring the application logs

Send all the requests (prompts) and responses to a monitoring system that checks the compliance, accuracy, and quality of the results.

There are advantages and disadvantages to each approach. It is always better to address the problem in the earliest stage (data preprocessing), but in many cases, you cannot implement a solution on the application data path due to performance or complexity reasons. The tracking is done in the monitoring system and its feedback used to further tune the system.

Monitoring and measuring the performance of an LLM model is more challenging since the resulting text is not deterministic or based on a huge training dataset. However, some approximate methods exist to measure drift, for example, by comparing the embedding similarity between the reference data and the current requests, measuring vocabulary frequency, perplexity, and so on.

The key challenge remains to detect if and when the model is hallucinating. One way is to compare multiple results for the same question; if they are semantically different, there is a good chance the model is hallucinating. Another approach in RAG systems is to evaluate the similarity between the reference data and the generated text answer (see [BERTScore](#) and [Vectara](#)).

Text Summarization applications can use the ROUGE (Recall-Oriented Understudy for Gisting Evaluation) method to evaluate the results. Another practice is using LLMs as judges to assess the correctness of the answer based on reference content (in RAG) or by using another LLM.

Another significant element in ensuring high quality and reliability is adding user feedback (*humans in the loop*). This means sampling the results in the production data and having humans verify that they meet the expected behavior. If they don't, correct them and retrain or tune the model to fix that behavior.

MLOps Pipelines for Efficiently Using and Customizing LLMs

Earlier in this chapter, we described two methods to customize LLMs for specific data and applications: prompt engineering and fine tuning. In many applications, we will use both to maximize performance and reliability. Data preparation and validation are crucial parts of the process in both cases.

In fine tuning, you ingest and prepare the application data and feedback data, run a transfer learning job to tune the base foundation model and extensive testing, and finally deploy the newly built model into the staging or production environment (see [Figure 8-10](#)).

The pipeline can run anytime you have new data or enough feedback data (generated through human labeling). In addition, since the target model is large, it requires distributed training (tuning) and validation over multiple systems and GPUs, which can be done with frameworks such as Horovod/MPI or Ray.

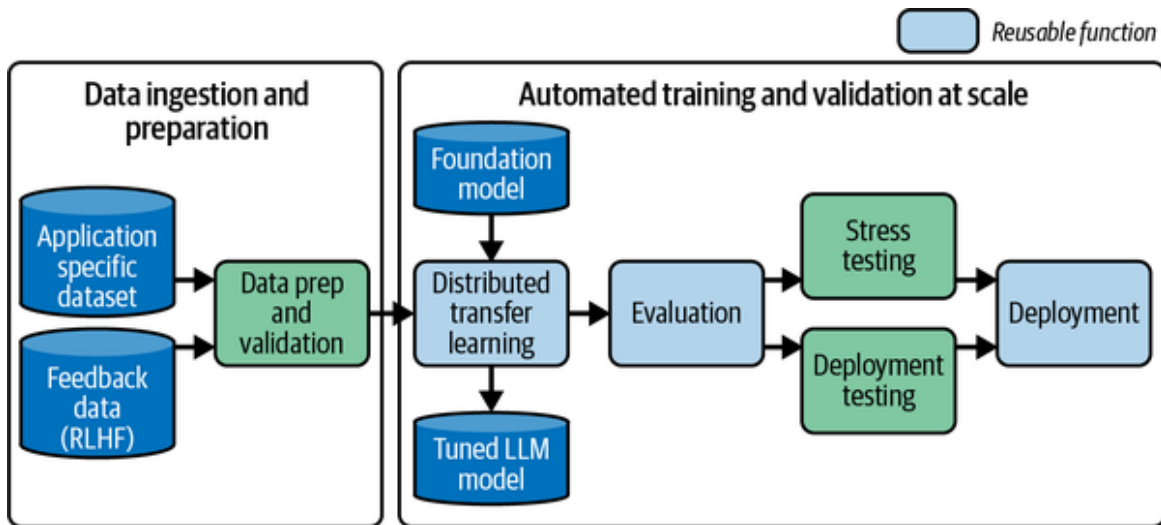


Figure 8-10. LLM model tuning pipeline example

When you implement prompt engineering and need to feed the prompt with reference documents or context, the context should be prepared and indexed for efficient retrieval; for example, using embeddings and vector or keyword databases (see [Figure 8-11](#)). Furthermore, the data fed and indexed in the database must be thoroughly cleaned and prepared to maximize the response quality and avoid risks.

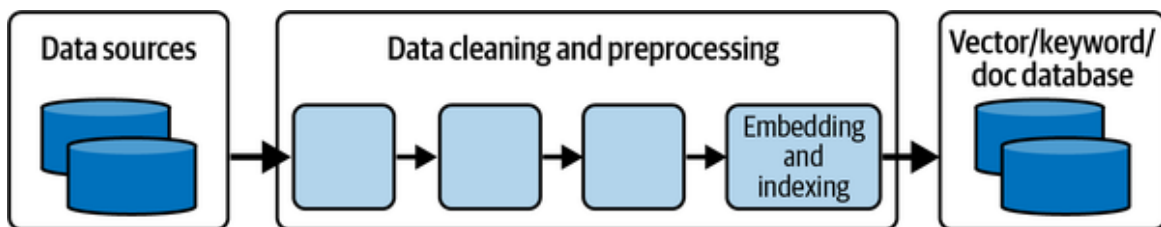


Figure 8-11. LLM data preparation and indexing pipeline example

Multiple new frameworks target data processing and indexing for language processing tasks, for example, [LangChain](#), [LlamaIndex](#), [spaCy](#), and [Unstructured](#).

The pre-indexed data and tuned model are used in the interactive or real-time pipeline, which intercepts the user requests and responds with the expected answer. [Figure 8-12](#) illustrates the real-time pipeline. The first step is to receive and process the request, followed by data enrichment, prompt engineering, LLM prediction, post-processing, and returning a response to the user. In addition, the incoming data and responses are sent to the

monitoring system, stored, and used to identify drift, performance problems, or risks. Finally, portions of the monitored data can be sent to human labeling and used for retuning the model (RLHF).

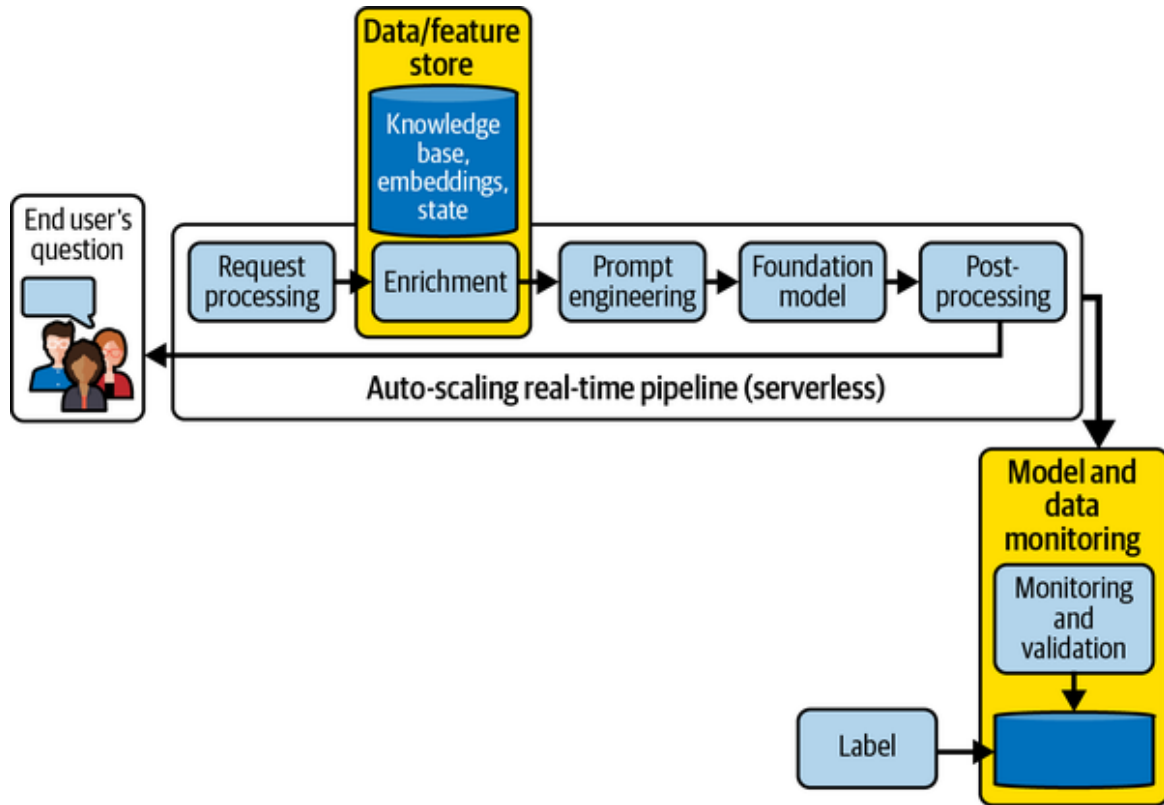


Figure 8-12. LLM real-time serving pipeline example

The real-time pipeline introduces several challenges:

- Relative complexity and integration of disparate elements (data processing and enrichment, application logic, model prediction, monitoring, human feedback, and so on)
- Partitioning the large model to multiple GPU devices and systems
- Model performance (LLMs are notoriously slow and may require multiple calls for a single user flow)
- Real-time request and response validation to avoid risks
- Continuous deployment and rolling upgrades

Elastic serverless frameworks and automation can reduce the complexity.

Application Example: Fine-Tuning an LLM Model

The following tutorial provides an example of building and operationalizing an intelligent question-and-answering application using a fine-tuned large language model.

The tutorial contains two main parts:

- Data preparation and tuning pipeline
- Real-time application and model serving pipeline

The application is tuning an LLM with the content of [Iguazio's MLOps blog](#), and it answers MLOps-related questions. It uses [MLRun](#) to rapidly build, run, and monitor the two pipelines.

The complete source code can be found at: <https://github.com/mlrun/demo-llm-tuning>. Open the *tutorial.ipynb* file to see the tutorial flow.

Data preparation and tuning

In this tutorial, the open source LLM model is downloaded from [Hugging Face](#) and fine-tuned with the MLOps dataset generated from the blog pages. The first steps of the pipeline are reading the pages from the internet (blog), converting them to cleaned-up text, and preprocessing them to maximize the model's performance. The following step uses Horovod, MPI, and [DeepSpeed](#) to scale the tuning process across multiple systems and GPUs. The final step evaluates the model. See the flow described in [Figure 8-13](#).

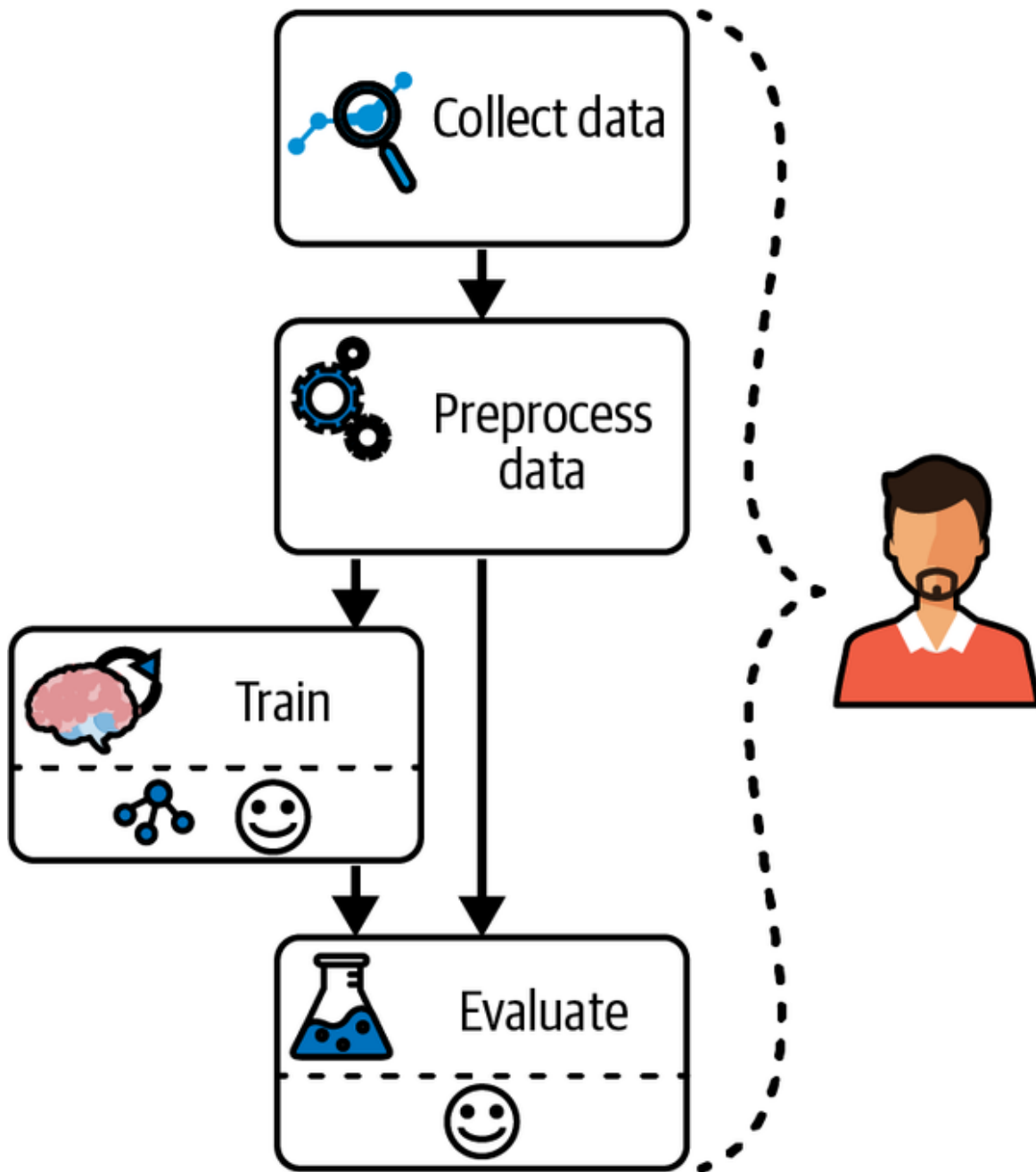


Figure 8-13. Data preparation and tuning pipeline

NOTE

Many more tests are likely to be added in a real-world application.

But first, let's verify that the base LLM knows only a little about MLOps by loading it and asking a question. See the code in [Example 8-1](#).

Example 8-1. Testing the base LLM (before tuning)

```
from transformers import AutoTokenizer, AutoModelForCausalLM, \
    GenerationConfig, pipeline

model_name = "gpt2-medium"
model_name = "tiiuae/falcon-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generation_config = GenerationConfig.from_pretrained(model_name)
generator = pipeline("text-generation", model=model_name, tokenizer=tokenizer,
                    trust_remote_code=True)

def prompt_to_response(prompt: str) -> str:
    return generator(prompt, generation_config=generation_config, max_length=50,
                    pad_token_id=tokenizer.eos_token_id)[0]["generated_text"]

print(prompt_to_response(prompt="What is a serving pipeline?"))

> A serving pipeline is a set of tools that help you deliver your content.
```

Data preparation

Feeding the raw HTML pages into the model will likely result in inaccurate answers. In that case, the data needs to be cleaned from redundant symbols, broken into text sections with clear titles (topics), and labeled with unique tokens.

The function `mark_header_tags()` in the file `data_collection.py` (see [Example 8-2](#)) converts the HTML text and headers into marked text.

Example 8-2. Converting the HTML text and headers into marked text

```
def mark_header_tags(soup: BeautifulSoup):
    """
    Adding header token and article token prefixes to all headers in html,
    in order to parse the text later easily.

    :param soup: BeautifulSoup object of the html file
    """
    nodes = soup.find_all(re.compile("^h[1-6]$"))
    # Tagging headers in html to identify in text files:
    if nodes:
        content_type = type(nodes[0].contents[0])
        nodes[0].string = content_type(
```



```

        ARTICLE_TOKEN + normalize(str(nodes[0].contents[0]))
    )
    for node in nodes[1:]:
        if node.string:
            content_type = type(node.contents[0])
            if content_type == Tag:
                node.string = HEADER_TOKEN + normalize(node.string)
            else:
                node.string = content_type(HEADER_TOKEN +
str(node.contents[0]))

```

The function `convert_textfile_to_data_with_prompts()` in the file `data_preprocess.py` (see [Example 8-3](#)) converts the document text into a set of prompts and answers based on the topics of each section.

Example 8-3. LLM data preprocessing

```

def convert_textfile_to_data_with_prompts(txt_file: Path):
    """
    Formatting the html text content into prompt form.
    Each header-content in the article is an element in the list of prompts

    :param txt_file: text content as a string with tokens of headers.
    :returns: list of prompts
    """
    # Read file:
    with open(txt_file, "r") as f:
        lines = f.readlines()

    start = 0
    end = 0
    subject_idx = []
    data = []
    # Dividing text into header - paragraph prompts:
    for i, line in enumerate(lines):
        if not start and line.startswith(ARTICLE_TOKEN):
            start = i
        elif HEADER_TOKEN + END_OF_ARTICLE in line:
            end = i
            break
        if line.startswith(HEADER_TOKEN):
            subject_idx.append(i)
    article_content = lines[start:end]
    subject_idx = [subject_i - start for subject_i in subject_idx]
    article_name = article_content[0].replace(ARTICLE_TOKEN, "")
    for i, subject in enumerate(subject_idx):
        if subject + 1 in subject_idx:

```

```

        continue
    subject_data = article_content[subject].replace(HEADER_TOKEN, "")
    if i + 1 == len(subject_idx):
        content_end = len(article_content)
    else:
        content_end = subject_idx[i + 1]
    content_limits = subject + 1, content_end
    data.append(
        DATA_FORMAT.format(
            article_name,
            subject_data,
            "".join(article_content[content_limits[0] : content_limits[1]]),
        )
    )
return data

```

Model tuning and evaluation

MLRun provides a built-in function in **MLRun functions hub** for fine tuning and evaluating Hugging Face LLM models. The **huggingface-auto-trainer** contains the `finetune_llm()` method that fine-tunes the LLM on the data. It distributes the training run over multiple nodes and GPUs using OpenMPI, logs the results, and saves the tuned model in the model registry. The tuning process uses QLoRA (Quantized Low-Rank Adaptation) to minimize the required memory and computation.

NOTE

Using pre-built functions from the MLRun function hub significantly reduces the effort to support LLM fine tuning. It addresses all the operational requirements, including GPU integration, distributed training, reporting, logging, and so on. Function definitions can be imported from the hub with a single instruction.

The `evaluate()` method in the `huggingface-auto-trainer` function evaluates the model using the perplexity metric and generates an evaluation report.

Defining an MLRun project and running the pipeline

In MLRun, the pipelines, artifacts, and models are parts of a named project. Projects usually map to a Git project. They are versioned and can be deployed into a development or production environment using a single command. In addition, projects are managed entities with strict membership and access control. The first step is to create or load a project and its assets (functions, pipelines, and so on). See [Example 8-4](#).

Example 8-4. MLRun project setup

```
import mlrun

project = mlrun.load_project(
    name="mlopspedia-bot",
    context="./",
    user_project=True,
    parameters={
        "source": "git://github.com/mlrun/demo-llm-tuning.git#main",
        "default_image": "yonishelach/mlrun-llm",
    })
```

The next step is to run the pipeline. Pipelines are executed with the `project.run()` command and accept a set of arguments that can be used to parameterize the pipeline and allow for potential reuse in different projects. See [Example 8-5](#).

Example 8-5. Running the tuning pipeline

```
workflow_run = project.run(
    name="training_workflow",
    arguments={
        "html_links": "/User/demo-llm-tuning/data/html_urls.txt",
        "model_name": "falcon-7b-mlrun",
        "pretrained_tokenizer": model_name,
        "pretrained_model": model_name,
        "epochs": 5,
    },
    watch=True,
    dirty=True,
)
```

The pipeline progress can be viewed interactively in the client or notebook. When the pipeline ends, it generates a summary of the results (see [Figure 8-14](#)).



Figure 8-14. Data preparation and tuning pipeline report in Jupyter

However, the UI provides a richer, more interactive experience (see [Figure 8-15](#)).

One of the unique features of MLRun is the ability to seamlessly distribute the workload across multiple systems and GPUs (by orchestrating the underlying Kubernetes, containers, and MPI resources). For example, in [Figure 8-16](#) (taken from MLRun UI), we can see that MLRun distributed the tuning task across 16 workers and GPUs.

When the pipeline ends, the model gets registered automatically and can be used in the application pipeline.

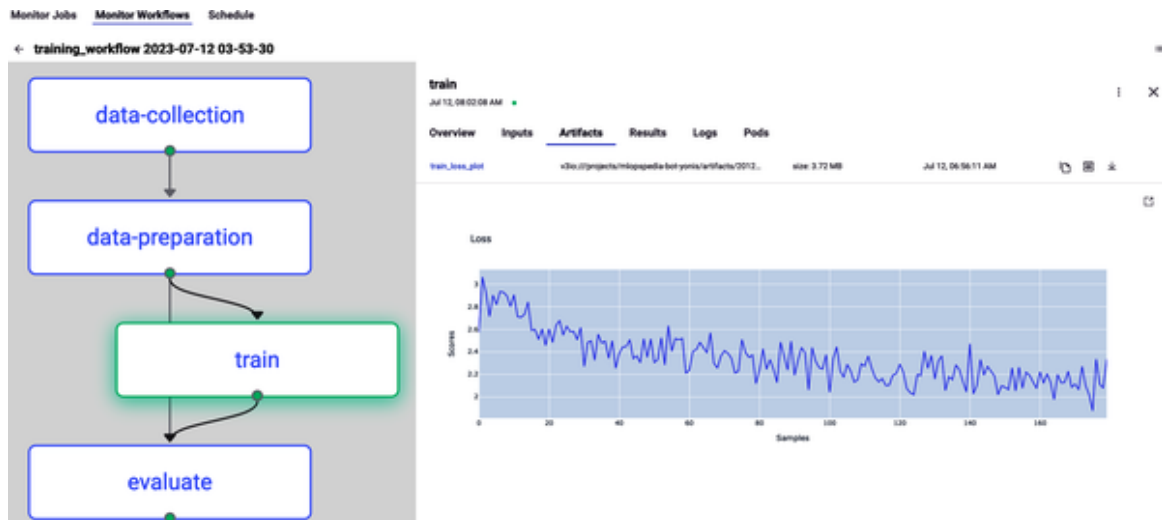


Figure 8-15. Data preparation and tuning pipeline in MLRun UI

Monitor Jobs	Monitor Workflows	Schedule
← trainer May 9, 05:02:50 AM ●		
Overview	Inputs	Artifacts
Results	Logs	Pods
trainer-9321621e-launcher		
trainer-9321621e-worker-0		
trainer-9321621e-worker-1		
trainer-9321621e-worker-10		
trainer-9321621e-worker-11		
trainer-9321621e-worker-12		
trainer-9321621e-worker-13		
trainer-9321621e-worker-14		
trainer-9321621e-worker-15		
trainer-9321621e-worker-2		

Figure 8-16. Distributed tuning with 16 workers and GPUs

NOTE

Add deployment tests that try to deploy the application and model pipeline into a staging environment and run exhaustive tests on it to verify its reliability and performance.

Application and model serving pipeline

The application pipeline, especially in LLM use cases, consists of many steps to accept and process the request. These include steps for data enrichment and preprocessing, prompt engineering, model prediction, application control flow, safety, risk control, data post-processing and formatting, monitoring, and more. Therefore, building, deploying, and

scaling the application pipeline is challenging. In addition, addressing operational considerations—such as performance, security, availability, versioning, and rolling upgrades—can drain significant engineering resources and time.

The MLRun framework uses elastic serverless functions to automate the build and deployment of application pipelines by allowing the composition of a graph (DAG) made of multiple custom and built-in steps and translating it automatically to a distributed pipeline running over microservices.

This tutorial implements the application pipeline using MLRun Serving Graph with four steps (see [Figure 8-17](#)):

Data preprocessing (preprocess)

Fit the user prompt into the model prompt structure (“Subject - Content”).

LLM prediction (LLMModelServer)

Serve our trained model and perform inferences to generate answers.

Post-processing (postprocess)

Check if the model generated reliable answers and format the output.

Toxicity filter (ToxicityClassifierModelServer)

Use the Hugging Face Evaluate package model and perform inferences to catch toxic prompts or responses and respond with a proper answer instead.

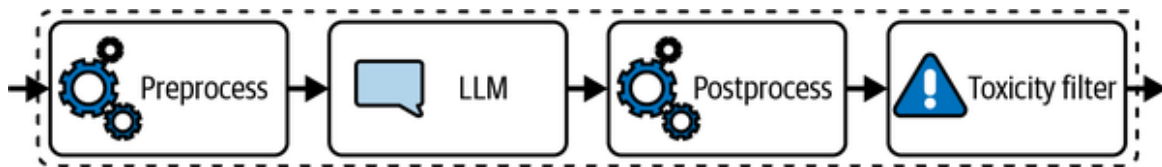


Figure 8-17. Application and model pipeline

Steps 2 and 4 are generic and can be used in multiple applications, while steps 1 and 3 are specific. You can see the full code implementation of the steps in the Git repository (in `src/serving.py`). Note that a real application will likely include more steps and logic.

The code in [Example 8-6](#) defines the graph by specifying the function or class per step, step parameters, and graph relations. The `plot()` method allows you to visualize the graph topology.

Example 8-6. Define the serving graph topology

```
# Set the topology and get the graph object:
graph = serving_function.set_topology("flow", engine="async")

# Add the steps:
graph.to(handler="preprocess", name="preprocess") \
    .to("LLMModelServer",
        name="mlospedia",
        model_args=model_args,
        tokenizer_name=model_name,
        model_name=model_name,
        peft_model=project.get_artifact_uri("falcon-7b-mlrun")) \
    .to(handler="postprocess", name="postprocess") \
    .to("ToxicityClassifierModelServer",
        name="toxicity-classifier",
        threshold=0.7).respond()

# Plot to graph:
serving_function.plot(rankdir='LR')
```



Once the real-time application graph is defined, you can debug it locally, save it, or deploy it to the cluster with a single API call or SDK command, as shown here:

```
# Configure (add a GPU and increase readiness timeout):
serving_function.with_limits(gpus=1)
serving_function.spec.readiness_timeout = 3000

# Save the function to the project:
project.set_function(serving_function, with_repo=True)
project.save()

# Deploy the serving function:
deployment = mlrun.deploy_function("serving")
```

To test the deployed application, you can use the `invoke()` method and specify the API parameters (the text question and model parameters, in this case). The first example (see [Example 8-7](#)) demonstrates a simple question and answer.

Example 8-7. Test the application pipeline

```
generate_kwargs = {"max_length": 150, "temperature": 0.9, "top_p": 0.5,
                  "top_k": 25, "repetition_penalty": 1.0}

response = serving_function.invoke(
    path='/predict', body={"prompt": "What is MLRun?", **generate_kwargs}
)
print(response["outputs"])
```

MLRun is an open source MLOps orchestration framework that streamlines the automation of machine learning projects. MLRun empowers data scientists, data engineers, and DevOps teams to develop, deploy, and manage machine learning applications faster and more accurately. MLRun integrates with popular tools such as Jupyter, PyCharm, Spark, etc., and provides a robust set of built-in orchestration and monitoring capabilities.

[Example 8-8](#) sends a toxic question, and the model responds with a controlled answer.

Example 8-8. Try the toxic language filter

```
response = serving_function.invoke(
    path='/predict', body={"prompt": "You are stupid!", **generate_kwargs}
)
print(response["outputs"])
```

This bot does not respond to toxicity.

Adding a web interface

You can use Gradio to rapidly create a UI to demo the behavior of your chat application pipeline. **Example 8-9** defines a chat window and control widgets.

Example 8-9. Create the Gradio interactive UI

```
import json

import gradio as gr
import requests

# Get the serving url to send requests to:
serving_url = deployment.outputs["endpoint"]

def generate(prompt, temperature, max_length, top_p, top_k, repetition_penalty):
    # Build the request for our serving graph:
    inputs = {
        "prompt": prompt,
        "temperature": temperature,
        "max_length": max_length,
        "top_p": top_p,
        "top_k": top_k,
        "repetition_penalty": repetition_penalty,
    }

    # call the serving function with the request:
    resp = requests.post(serving_url, data=json.dumps(inputs).encode("utf-8"))

    # Return the response:
    return resp.json()["outputs"]

# Set up a Gradio frontend application:
with gr.Blocks(analytics_enabled=False, theme=gr.themes.Soft()) as demo:
    gr.Markdown(
        """# LLM Playground
        Play with the `generate` configurations and see how they make the
        LLM's responses better or worse.
        """
    )
    with gr.Row():
        with gr.Column(scale=5):
            with gr.Row():
                chatbot = gr.Chatbot()
            with gr.Row():
                prompt = gr.Textbox()
```

```

        label="Subject to ask about:",
        placeholder="Type a question and Enter",
    )

with gr.Column(scale=1):
    temperature = gr.Slider(
        minimum=0,
        maximum=1,
        value=0.9,
        label="Temperature",
        info="Choose between 0 and 1",
    )
    max_length = gr.Slider(
        minimum=0,
        maximum=1500,
        value=150,
        label="Maximum length",
        info="Choose between 0 and 1500",
    )
    top_p = gr.Slider(
        minimum=0,
        maximum=1,
        value=0.5,
        label="Top P",
        info="Choose between 0 and 1",
    )
    top_k = gr.Slider(
        minimum=0,
        maximum=500,
        value=25,
        label="Top k",
        info="Choose between 0 and 500",
    )
    repetition_penalty = gr.Slider(
        minimum=0,
        maximum=1,
        value=1,
        label="repetition penalty",
        info="Choose between 0 and 1",
    )
    clear = gr.Button("Clear")

def respond(
    prompt,
    chat_history,
    temperature,
    max_length,
    top_p,

```

```

        top_k,
        repetition_penalty,
    ):
        bot_message = generate(
            prompt, temperature, max_length, top_p, top_k, repetition_penalty
        )
        chat_history.append((prompt, bot_message))

    return "", chat_history

prompt.submit(
    respond,
    [prompt, chatbot, temperature, max_length, top_p, top_k,
    repetition_penalty],
    [prompt, chatbot],
)
clear.click(lambda: None, None, chatbot, queue=False)

```

See the resulting interactive window in [Figure 8-18](#).

MLRun automated the packaging, delivery, and deployment of the project. The project can be saved into a Git repository along with the code, workflows, and configurations and be loaded into a staging or production environment with a single command or API call. In addition, MLRun has a glueless integration with CI/CD systems such as GitHub Actions, Jenkins, and GitLab CI, which allow full automation and CI/CD without additional coding or DevOps activities.

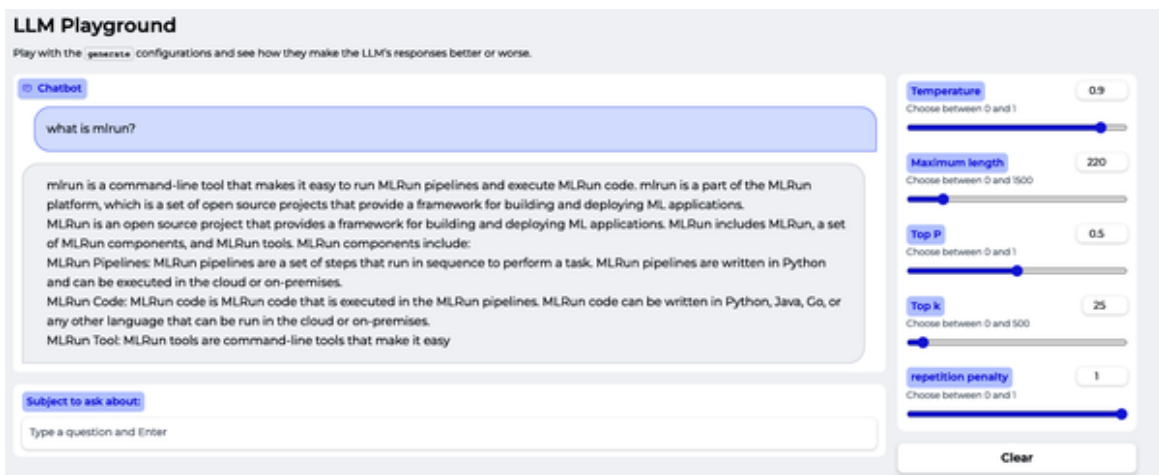


Figure 8-18. Gradio application for using the model

Conclusion

In this chapter we delved into some of the most cutting-edge and transformative ML tech: DL and LLM projects. We started out with an overview of the DL training process, which is based on distributed deep learning. Then we discussed the data used for training, important considerations when labeling it, and the monitoring process.

We then described the innovation in foundation models, GenAI, and LLMs and dedicated a special section to discussing their risks. Finally, we explained in great detail how to build an MLOps pipeline for using and customizing LLMs. This includes data preparation, model tuning, and model evaluation. This chapter also included an extensive demonstration of how such a pipeline operates.

Critical Thinking Discussion Questions

- What are the two main methods for distributed deep learning and when should you use each one?
- What are the risks of data labeling and how can you overcome them?
- What are the main considerations for buying DL models?
- How can organizations ensure the authenticity of data trained on GenAI?
- What is the value of model tuning?

Exercises

- Choose an open source data labeling solution and build a data labeling plan that would employ it.
- Create a mockup process for ensuring the training data isn't toxic, biased, or poses a security risk.

- Download an LLM model from Hugging Face (if you don't have one) and fine-tune it with content from your favorite website.
- Write the code for risk-free model training.
- Build an application and model serving pipeline for your model from the third bullet.

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 9. Solutions for Advanced Data Types

This chapter delves into the intricacies of data analysis and interpretation, focusing on modern techniques and approaches in time series analysis, NLP, video, and image classification. It aims to comprehensively discuss advanced data types and their applications in tackling complex problems for seasoned data scientists as well as beginners.

The chapter discusses the challenges and options associated with data processing and model selection, particularly concerning time series data. We'll explore different types of solutions, weigh the trade-offs, and discuss specific considerations in the field of MLOps. We will broaden our scope to include various platforms such as AWS, GCP, Hugging Face, and Apple's CreateML. Each platform offers a unique set of tools and services that can effectively cater to different needs and preferences. By providing an unbiased comparison of these platforms and discussing their pros and cons, we aim to help you make a well-informed decision.

To get a sense of the variety of data types MLOps developers use, look at Apple's CreateML interface in **Figure 9-1**. As illustrated, there are categories in Image, Video, Motion, Sound, Text, and Tables.

We'll return to CreateML toward the end of the chapter.

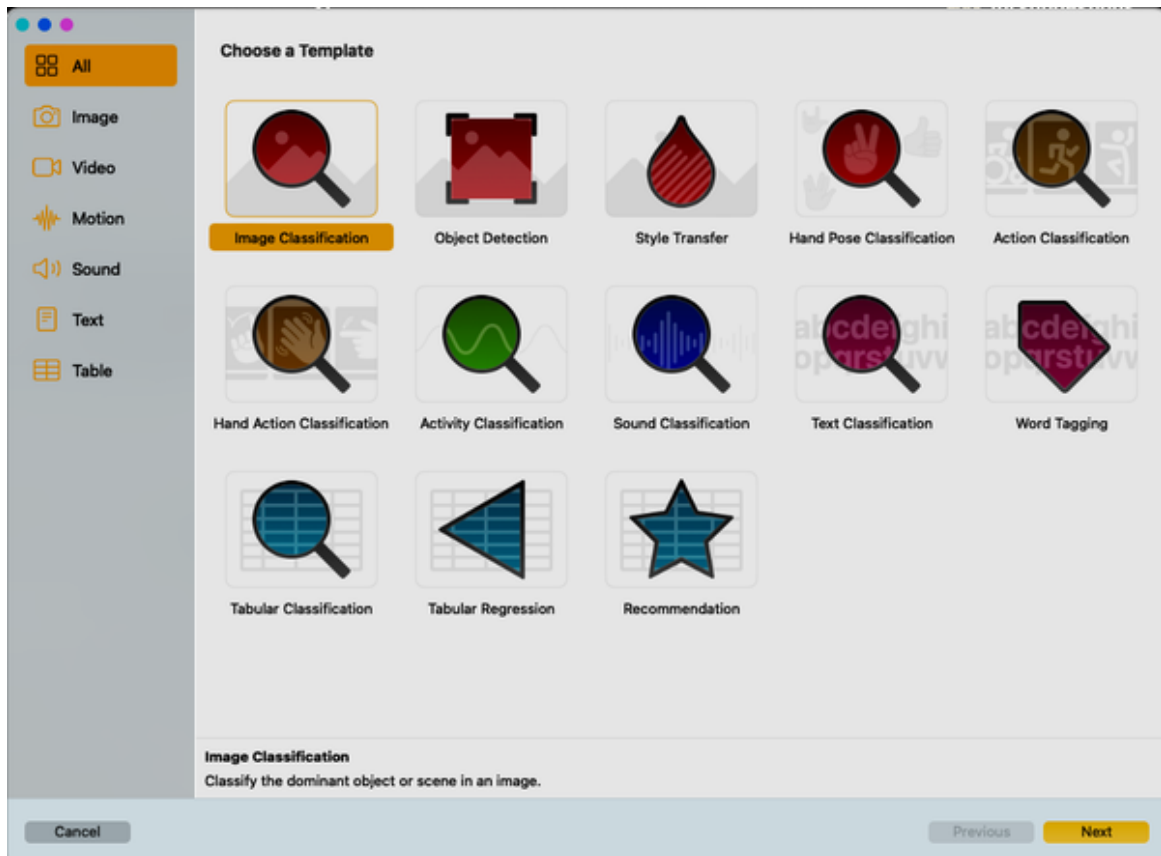


Figure 9-1. Apple's CreateML Project Types

ML Problem Framing with Time Series

An inflection point critical decision when dealing with time series data is whether to use ML or to use traditional statistical techniques. Time series data, which involves observations of a particular variable over a specific period, is widely used in many fields, including finance, economics, and weather forecasting. Traditional statistical techniques such as autoregressive integrated moving average (ARIMA), error-trend-seasonality (ETS), and Holt-Winters have long been used to analyze and forecast time series data.

One way to phrase this dilemma is to call it **machine learning problem framing**. Google refers to these steps as “determining whether ML is the right approach for solving a problem” and then, if suitable, “framing the problem in ML terms.”

NOTE

If you want to learn more about problem framing in the context of machine learning on the Google Cloud Platform, Noah has a certification course on the topic at [Google Professional Machine Learning Engineer Course 2023](#).

Nevertheless, ML brings several advantages over traditional statistical methods when analyzing time series data:

Nonlinearity

ML models, especially techniques like neural networks, can capture complex nonlinear relationships in the data that traditional statistical methods might miss. Time series data often exhibit nonlinear patterns, and ML can handle these complexities with nuance.

Interactions among variables

ML algorithms can identify and leverage complex interactions among multiple input features in ways that can be challenging for traditional statistical techniques. For instance, when the interaction of various variables influences the output, ML can model these relationships more effectively.

Automated feature engineering

Some ML techniques, like DL, can automatically extract relevant features from the data, reducing the need for manual feature engineering. This capability can be particularly beneficial when dealing with high-dimensional data.

Signal to noise

ML algorithms are more immune to noise and anomalies in the data. While outliers or irregularities can significantly impact traditional statistical techniques, ML models can handle these challenges more effectively.

Scalability

ML techniques can scale more effectively to large datasets and use emerging technologies like custom silicon for ML. Traditional statistical methods can become computationally intractable as the size of the dataset increases, whereas ML models, especially those designed for big data environments, can handle large volumes of data more efficiently.

The choice between traditional statistical techniques and machine learning sometimes needs to be clarified. It depends on the specific requirements of the problem, the nature of the data, the resources available, and even the cloud platform or data platform you have immediate access to. A hybrid approach that combines elements of both might be the best solution in some cases.

ANALYZING SOCIAL MEDIA INFLUENCE PATTERNS

Coauthor Noah Gift has a substantial background using data to interpret and forecast patterns, especially in sports performance and social media influence. He often leverages standard tools like Excel for these tasks, demonstrating that sophisticated insights can derive from off-the-shelf technologies.

In a **conference talk for O'Reilly**, Gift explored the relationship between social influence and the NBA. By tracking and analyzing social media data related to NBA players and games, he uncovered patterns in how athlete performance and team outcomes can drive social media trends and, in turn, how these trends can influence the perception of the sport.

Excel was an instrumental tool in this analysis. With its robust data manipulation features and the ability to apply a variety of statistical and forecasting techniques, Gift was able to draw meaningful insights from the available data. When combined with an understanding of the data and the right analytical approach, Excel's capabilities can be a powerful tool for data-driven decision making in various contexts without needing to dive into building a machine learning solution.

With some theory out of the way, let's look at AWS-specific solutions, followed by GCP for time series problems.

Navigating Time Series Analysis with AWS

Time series analysis, a method for analyzing data patterns over time, is critical to understanding and forecasting trends. While these techniques can be applied using various tools and platforms, in this section we focus on how AWS can facilitate this process.

We will explore methods such as ARIMA and Forecast DeepAR+ for trend analysis and forecasting, which AWS supports. These techniques help decipher insights from time series data, guiding predictions for future

outcomes. The choice between AWS and other platforms, or even deciding to build a custom solution, will depend on factors like cost, complexity, scalability requirements, and your team's expertise.

AWS offers several services that perform time series analysis, each accessible through Python. The three primary ways to interact with AWS are:

- AWS Management Console
- AWS Command Line Interface (CLI)
- AWS Software Development Kit (SDK)

Each method provides unique advantages, and choosing the right one depends on your use case.

The **AWS Management Console** is a web-based graphical interface that provides a simplified way to interact with AWS services. You can use the console to perform tasks such as launching EC2 instances, creating S3 buckets, managing IAM users, and using AI/ML services like Amazon Forecast.

The **AWS CLI** allows you to perform AWS tasks from the command line. The AWS CLI provides a unified way to interact with AWS services, and it is helpful for tasks such as managing EC2 instances, uploading files to S3, and doing a forecast. In many ways, the CLI is ideal for doing high-level experiments, as shown in the following example that creates an ARIMA forecast:

```
$ aws forecast create-predictor \
  --predictor-name "time-series-forecast-predictor" \
  --algorithm-arn "arn:aws:forecast::algorithm/ARIMA" \
  --forecast-horizon 7 \
  --perform-auto-ml "false" \
  --input-data-config '{"DatasetGroupArn": \
$(aws forecast list-dataset-groups | jq '.DatasetGroups[] \
| select(.DatasetGroupName == "time-series-forecast-dataset-group") \
| .DatasetGroupArn')', "SupplementaryFeatures
```

NOTE

The AWS CLI is recognized as a highly effective tool for managing AWS resources, providing outstanding efficiency and flexibility compared to the console or other GUIs. These strengths are due to its ability to simply and rapidly execute complex commands while providing granular control over AWS resources through a comprehensive set of commands and options. The AWS CLI is optimized for scalability and integrates easily with other AWS services, making it well suited for managing large-scale deployments and complex infrastructures. Try CLI when exploring a service with complex inputs; it will often be more straightforward.

The **AWS SDK** is a collection of libraries and tools for developing applications interacting with AWS services. It provides a programmatic way to interact with AWS services and supports multiple programming languages, including Java, .NET, PHP, Python, and Rust. It includes the following:

Amazon Forecast

A fully managed service that uses machine learning algorithms to provide accurate forecasts based on historical time series data:

```
import boto3

forecast = boto3.client('forecast')

response = forecast.create_dataset_group(
    DatasetGroupName='forecast_dataset_group',
    Domain='CUSTOM',
)
```

Amazon QuickSight

A business intelligence service that provides fast and easy-to-use data visualization and insights for time series data:

```
import boto3

quicksight = boto3.client('quicksight')

response = quicksight.create_data_source(

    AwsAccountId='AWS_ACCOUNT_ID',

    DataSourceId='data_source_id',

    Name='data_source_name',

    Type='ADOBE_ANALYTICS',

)
```

Amazon Kinesis Data Streams

A service for streaming real-time data that is used for time series analysis and visualization:

```
import boto3

kinesis = boto3.client('kinesis')

response = kinesis.create_stream(

    StreamName='kinesis_stream_name',

    ShardCount=1,

)
```

Amazon Kinesis Data Analytics

A service for real-time data streams used for time series analysis:

```
import boto3

kinesis_analytics = boto3.client('kinesisanalytics')

response = kinesis_analytics.create_application(

    ApplicationName='kinesis_analytics_app',

    RuntimeEnvironment='SQL-1.0',

)
```

Amazon SageMaker

A service for building, training, and deploying machine learning models, including time series models:

```
import boto3

sagemaker = boto3.client('sagemaker')

response = sagemaker.create_notebook_instance(

    NotebookInstanceName='notebook_instance_name',

    InstanceType='ml.t2.medium',

)
```

Amazon DynamoDB

A NoSQL database service that stores and retrieves time series data:

```

import boto3

dynamodb = boto3.client('dynamodb')

response = dynamodb.create_table(
    TableName='dynamodb_table_name',
    KeySchema=[
        {
            'AttributeName': 'attribute_name',
            'KeyType': 'HASH'
        },
    ],
    AttributeDefinitions=[
        {
            'AttributeName': 'attribute_name',
            'AttributeType': 'S'
        },
    ],
    ProvisionedThroughput={
        'ReadCapacityUnits': 5,
        'WriteCapacityUnits': 5
    }
)

```

Amazon EMR

A service for processing big data useful for time series analysis and visualization:

```

import boto3

emr = boto3.client('emr')

response = emr.run_job_flow(
    Name='emr_cluster_name',
    Instances={
        'InstanceGroups': [
            {
                'Name': 'master_instance_group',
                'InstanceRole': 'MASTER',
                'InstanceType': 'm5.xlarge',
                'InstanceCount': 1,
            },
        ],
        'Ec2KeyName': 'ec2_key_name',
        'KeepJobFlowAliveWhenTerminationProtected': False,
    },
    JobFlowRole='EMR_EC2_DefaultRole',
    ServiceRole='EMR_DefaultRole',
    VisibleToAllUsers=True,
    Applications=[
        {
            'Name': 'Spark'
        },
    ],
)

```


These AWS services support time series analysis and can be used to build and deploy scalable time series solutions for MLOps projects.

Diving into Time Series with DeepAR+

DeepAR+, an AWS machine learning service, harnesses the power of deep learning algorithms for time series forecasting. Leveraging deep learning in time series can offer unique capabilities, as discussed earlier in this chapter. DeepAR+ excels in managing large-scale, intricate time series data, providing valuable predictions for various use cases, from demand forecasting to traffic and equipment maintenance forecasting.

DeepAR+ builds upon the DeepAR algorithm, an advanced deep learning algorithm developed specifically for time series forecasting. Remember our definition of “signal to noise” on the limitations of statistical techniques with missing data or outliers earlier in the chapter? DeepAR+ steps in here with its feature to handle data with missing values and irregular intervals.

To employ DeepAR+, you must upload your time series data to Amazon S3 and create a DeepAR+ model. Beyond predictions, DeepAR+ also generates confidence intervals for its forecasts, enabling organizations to perceive the range of potential outcomes and make more informed decisions.

The workflow in **Figure 9-2** illustrates each step using the AWS CLI interface to build up to the final forecast.

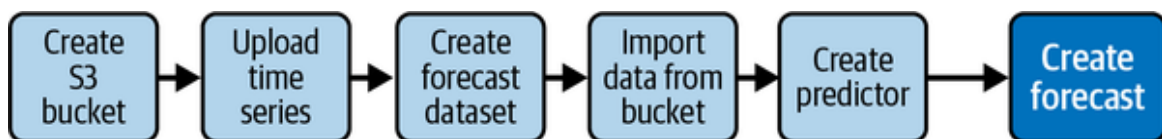


Figure 9-2. Deep AR+ Forecast Workflow

The following code examples mirror this process: you can replace them with your time series dataset and adjust for future AWS CLI changes. Through these examples, we establish an S3 bucket, upload the time series data to S3, create a DeepAR+ dataset, import the data into the DeepAR+

dataset, create a DeepAR+ predictor, and finally, create a forecast using the DeepAR+ predictor:

1. Create an S3 bucket to store the data:

```
$ aws s3 mb s3://deepar-example-bucket
```

2. Upload the time series data to S3:

```
$ aws s3 cp time-series-data.csv s3://deepar-example-bucket/data.csv
```

3. Create a DeepAR+ dataset following the pattern `^[a-zA-Z][a-zA-Z0-9_]*`:

```
$ aws forecast create-dataset \
  --dataset-name "DeepARExampleDataset" \
  --data-frequency "H" \
  --dataset-type "TARGET_TIME_SERIES" \
  --domain "CUSTOM" \
  --schema '{"Attributes":[{"AttributeName":"timestamp",\
  "AttributeType":"timestamp"}, {"AttributeName":"target_value",\
  "AttributeType":"float"}, {"AttributeName":"item_id",\
  "AttributeType":"string"}]}'
```

4. Import the time series data into the DeepAR+ dataset:

```
$ aws forecast create-dataset-import-job \
  --dataset-import-job-name "DeepARExampleDatasetImportJob" \
  --dataset-arn $(aws forecast list-datasets | \
  jq '.Datasets[] | select(.DatasetName == "DeepARExampleDataset") | \
  .DatasetArn') \
  --data-source "S3" \
  --data-source-config '{"S3Config": {"Path": \
```

```
"s3://deepar-example-bucket/data.csv", "RoleArn": \
"arn:aws:iam::ACCOUNT_ID:role/ForecastRole"]}]'
```

5. Create a DeepAR+ predictor:

```
$ aws forecast create-predictor \
--predictor-name "DeeparExamplePredictor" \
--algorithm-arn "arn:aws:forecast::algorithm/Deep_AR_Plus" \
--forecast-horizon 24 \
--perform-auto-ml "false" \
--input-data-config '{"DatasetGroupArn":
$(aws forecast list-dataset-groups | \
jq '.DatasetGroups[] | select(.DatasetGroupName ==
"DeepARExampleDatasetGroup") | \
.DatasetGroupArn')', "SupplementaryFeatures":
[{"Name": "item_id", "Value": "item1"}, \
{"Name": "timestamp", "Value": "yyyy-MM-dd HH:mm:ss"}]}' \
--featurization-config '{"ForecastFrequency": "H"}'
```

6. Create a forecast using the DeepAR+ predictor:

```
# create a forecast using the DeepAR+ predictor
$ aws forecast create-forecast \
--forecast-name "DeeparExampleForecast" \
--predictor-arn $(aws forecast list-predictors | \
jq '.Predictors[] | select(.PredictorName ==
"DeeparExamplePredictor") | \
.PredictorArn')
```

The critical insight here is that the CLI offers an accessible entry point for exploring cloud-based time series forecasting, often proving simpler than starting with a notebook or SDK when leveraging AWS ML and AI services. We'll pivot next to a contrasting methodology: utilizing SQL

through Google BigQuery. This shift introduces an alternative avenue for our exploration, extending our toolkit beyond notebooks and Python.

Time Series with the GCP BigQuery and SQL

One platform that takes time series analysis flexibility to another level is Google BigQuery. The trade-offs discussed earlier are less challenging because you can use both statistical techniques and machine learning techniques side-by-side. BigQuery is unique in modeling time series data for several reasons:

Scalability

BigQuery handles large-scale data, making it ideal for simultaneously analyzing millions of time series. It can process petabytes of data quickly, making it possible to analyze and forecast trends across multiple, large numbers of time series columns.

Multiple time series forecasting

BigQuery ML allows forecasting multiple time series with a single query. This capability is advantageous when dealing with a vast quantity of time series variables, as it eliminates the need for running individual queries for each one.

ARIMA and ARIMA_PLUS models

BigQuery ML supports ARIMA models, particularly suited to time series data. The ARIMA_PLUS model also includes holiday effects and can change model behavior based on external factors, which is quite beneficial for certain types of time series data.

Integration with Google Cloud

BigQuery is part of the GCP, allowing seamless integration with other GCP services. This synergy makes it easier to

incorporate time series analysis into broader data workflows.

SQL-based ML

BigQuery uses SQL, a language familiar to many data professionals, for machine learning tasks. The SQL language lowers the entry barrier for those who want to build time series models but who don't have a strong ML or programming background. Additionally, many people prefer SQL to query data instead of Python or R.

Evaluation tools

BigQuery provides built-in functions like `ML.EVALUATE` for evaluating the accuracy of your time series forecasts. This capability simplifies the process of assessing model performance and making necessary adjustments.

This [tutorial](#) on Citi Bike trips in New York City demonstrates how to efficiently train time series models and perform multiple time series forecasts with a single query using BigQuery. The tutorial utilizes the New York City Citi Bike trip dataset hosted on GCP and the Iowa liquor sales dataset. Key steps include:

- Creating a dataset in BigQuery to store the ML model
- Creating the time series to forecast using the Citi Bike dataset
- Simultaneously forecasting multiple time series with default parameters speeds up the process compared to using multiple `CREATE MODEL` queries
- Evaluating forecasting accuracy for each time series using the `ML.EVALUATE` function
- Evaluating the overall forecasting accuracy for all the time series

The gist of this style is the creation of a SQL query that includes the `model_type = 'ARIMA_PLUS'` shown here:

```
CREATE OR REPLACE MODEL bqml_tutorial.nyc_citibike_arma_model_default
OPTIONS
  (model_type = 'ARIMA_PLUS',
   time_series_timestamp_col = 'date',
   time_series_data_col = 'num_trips',
   time_series_id_col = 'start_station_name'
 ) AS
SELECT *
FROM bqml_tutorial.nyc_citibike_time_series
WHERE date < '2016-06-01'
```

Finally, run `ML.EVALUATE` to check the accuracy of the created forecast:

```
SELECT *
FROM
  ML.EVALUATE(MODEL bqml_tutorial.nyc_citibike_arma_model_default,
              TABLE bqml_tutorial.nyc_citibike_time_series,
              STRUCT(7 AS horizon, TRUE AS perform_aggregation))
```

Once you get the hang of using SQL to forecast time series data sets using BigQuery, it becomes easy to chain together a complete pipeline ranging from the original problem formation to data ingestion, to modeling, to the conclusion.

Let's look at how to do those steps using the [BigQuery examples found here](#). First, let's view the total Wikipedia page views but filter out queries we don't query about, like the front page:

```
SELECT
  views,
  title
FROM
  `bigquery-public-data.wikipedia.pageviews_2023`
WHERE
  DATE(datehour) = "2023-04-18"
  AND NOT (title LIKE "Main_Page"
           OR title LIKE "Special:%"
           OR title LIKE "Wikipedia:%"
           OR title LIKE "Wikidata:%")
```

```

OR title LIKE "Cookie_(informatique)"
OR title LIKE "Wikipédia:Accueil_principal"
OR title LIKE "メインページ")
ORDER BY
  views DESC
LIMIT
  1000

```

You can see the Google BigQuery interface in [Figure 9-3](#); notice how you can also “explore the data;” this option even allows you to export it into a Colab notebook.

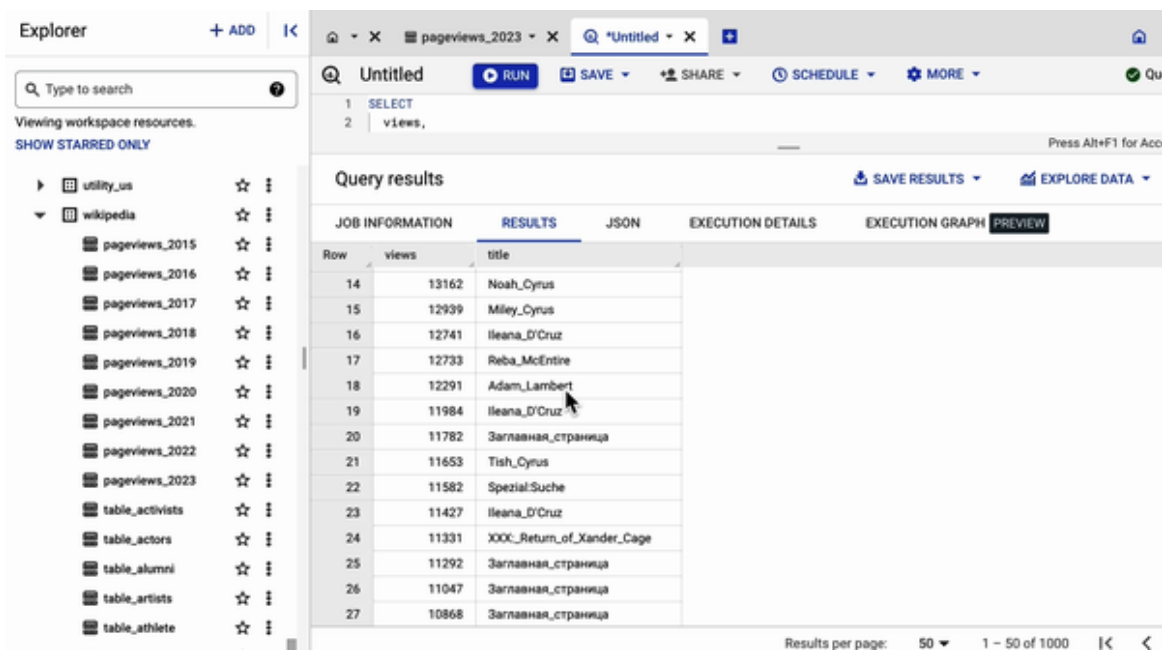


Figure 9-3. Top Wikipedia pageviews

The code for Colab with visualization follows; notice how easy it is to convert the results of the previous query into a DataFrame that we chart with seaborn.

```

from google.colab import auth
from google.cloud import bigquery
from google.colab import data_table

project = 'platinum-lead-379722' # Project ID
location = 'US' # Location
client = bigquery.Client(project=project, location=location)
data_table.enable_dataframe_formatter()

```

```

auth.authenticate_user()

job = client.get_job('bquxjob_3f8ffc04_18795d1ee67') # Job ID
print(job.query)

results = job.to_dataframe()

# Group by title and sum the views
results = results.groupby("title").sum().reset_index()

# Sort by views in descending order
results = results.sort_values(by="views", ascending=False)

# Print the result
print(results.head(10))

# Now visualize
import seaborn as sns
import matplotlib.pyplot as plt

# Visualize
# Select top 25 pages by views
top25 = results.head(25)

# Create a bar plot using seaborn
sns.set(style="whitegrid")
plt.figure(figsize=(12, 8))
ax = sns.barplot(x="views", y="title", data=top25)
ax.set_title("Top 25 Most Viewed Wikipedia Pages on 2023-04-18")
ax.set_xlabel("Number of Views")
ax.set_ylabel("Title")
plt.show()

```

Figure 9-4 shows the results.

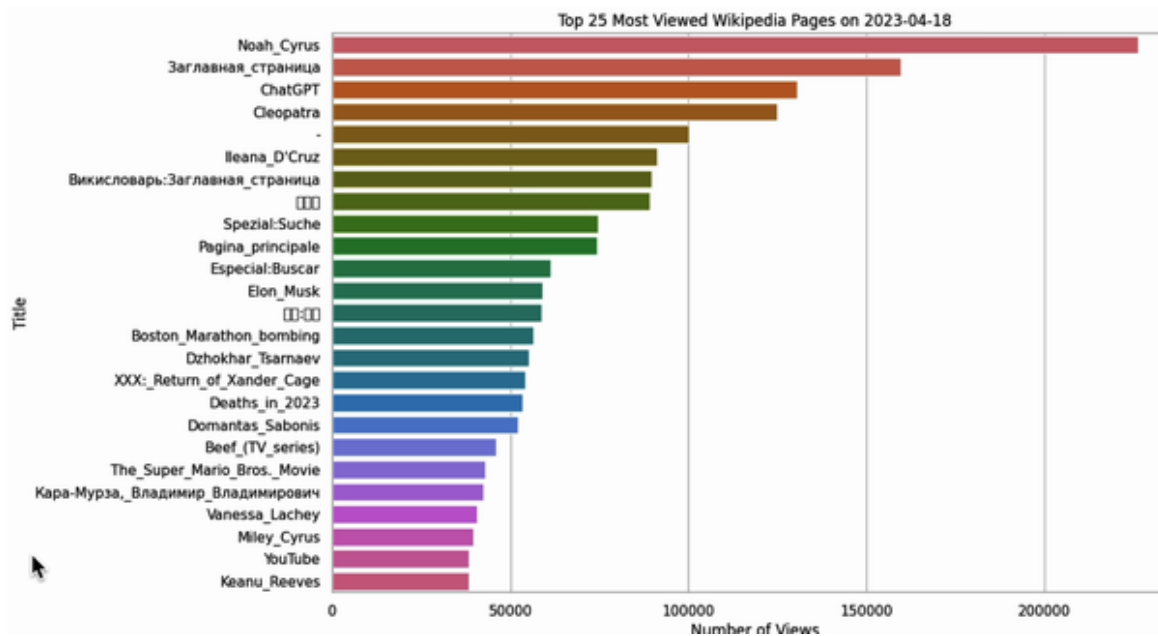


Figure 9-4. Top Wikipedia pageviews chart

Google BigQuery lets you run an entire pipeline for time series data that fits within an MLOps workflow via SQL queries.

Build Versus Buy for MLOps NLP Problems

NLP is ubiquitous in organizations looking to implement ML solutions. There are many ways to integrate solutions with NLP, including building custom models from scratch or purchasing pretrained models from vendors.

As language models become more sophisticated, deciding whether to *build* or *buy* has significant implications for businesses. This decision impacts cost, resources, time, customization, and control.

This section delves into the considerations surrounding this decision in the context of LLMs. These models, trained on vast amounts of data, have shown an impressive ability to analyze and generate human-like text, making them a potent tool for many applications.

The *build versus buy* decision in the context of LLMs is especially salient, given their resource requirements for training and the expertise needed for customization. We will explore these and other factors to provide a

framework for making this critical decision in your enterprise's MLOps strategy.

Build Versus Buy: The Hugging Face Approach

In the age of LLMs, Hugging Face has emerged as an influential player. Its hub acts as a repository for ML, offering models, datasets, and demos. It's a one-stop shop for ML resources, akin to BigQuery's expansive data warehouse.

But what if you're unsure whether to build your solution or leverage Hugging Face's offerings? Let's delve into this dilemma:

Models

Hugging Face provides a wide range of pretrained models for NLP, vision, and audio tasks. Think of it as a ready-to-use AWS library but focused on ML. Model cards provide information about limitations, biases, tasks, and languages. They're like BigQuery's schema descriptions but for ML models.

Datasets

Similar to BigQuery's vast datasets, Hugging Face hosts over 5,000 datasets in over 100 languages. Dataset Cards and Dataset Previews offer insights into the data, just as BigQuery provides table previews.

Spaces

Hugging Face Spaces are interactive apps to showcase your models. Imagine Spaces as an AWS Lambda function but for demoing your ML models. You can create your own space, upgrade it for GPU support, and even showcase it at conferences or meetings.

Organizations

Similar to AWS Organizations, Hugging Face provides a feature for grouping accounts and managing datasets, models, and Spaces. This feature enables better collaboration and resource management across teams and projects.

Security

Hugging Face has robust security features, similar to AWS's IAM roles and BigQuery's access control. User Access Tokens, access control for organizations, commit signing with GPG, and malware scanning ensure your work is secure.

Hugging Face has emerged as a powerful and adaptable platform for executing ML workflows. Building your solution from scratch or harnessing an existing one is a tough choice. Hugging Face achieves a desirable equilibrium between ready-to-use resources and customization opportunities that cater to your requirements by allowing you to use a model as is or fine-tune it. Numerous examples of Hugging Face exist throughout this book for your perusal. However, to avoid redundancy and conserve space, we will forego adding another Hugging Face example here.

Exploring Natural Language Processing with AWS

Before we dive into a hands-on example, let's first set the context for the role of NLP within MLOps. NLP is a branch of AI that helps machines analyze, interpret, and generate human language. In MLOps, NLP can play a crucial role in various tasks, such as sentiment analysis, text classification, language translation, and information extraction. Organizations can make more data-driven decisions, improve customer experiences, and optimize business processes by automating these tasks.

Similar to our exploration of time series analysis, we'll examine how AWS provides a comprehensive suite of ready-to-use solutions for NLP tasks, offering an alternative to platforms like Hugging Face. AWS's services enable rapid implementation of NLP tasks, making it a practical choice for MLOps workflows.

Now, let's get our hands dirty with some NLP. Start by downloading the [Sentiment Labelled Sentences dataset](#) from the [UCI Machine Learning Repository](#). After downloading, open the *yelp_labelled.txt* file. Next, navigate to the [AWS Comprehend interface](#). To see AWS Comprehend in action, paste a row from the *yelp_labelled.txt* file into the AWS Comprehend console, as shown in [Figure 9-5](#). This platform allows you to quickly prototype and visualize NLP tasks such as sentiment analysis. The shift from the traditional notebook or Python SDK methodologies to a more direct, user-friendly interface underscores AWS's commitment to providing practical and accessible tools for NLP tasks within MLOps workflows.

The figure provides a graphical representation of the AWS Comprehend console interface. This web-based user interface allows you to interact with AWS Comprehend without requiring programming or command-line interactions. You can input text data directly into the console, which the service analyzes to identify key sentiment metrics.

Input text

Crust is not good.

18 of 100000 characters used.

Clear text

Analyze

Insights Info

< Entities Key phrases Language PII Sentiment Targeted sentiment >

Analyzed text

Crust is not good.

▼ Results

Sentiment

Neutral	Positive	Negative	Mixed
0.00 confidence	0.00 confidence	0.99 confidence	0.00 confidence

Figure 9-5. AWS Comprehend sentiment analysis

A sentence from the Yelp-labelled dataset enters into the console in this example. The console’s output shows the sentiment analysis results from AWS Comprehend, demonstrating the primary sentiments detected (Positive, Negative, Neutral, or Mixed), and the confidence scores associated with each emotion. These scores reflect the probability assigned by the model for each sentiment category. This visualization is an excellent example of how AWS Comprehend lets you quickly prototype and test sentiment analysis tasks, making it a valuable tool for MLOps. It shows the ease of transitioning from data input to insightful results, all within a user-friendly, intuitive interface.

NOTE

Sentiment analysis, or *opinion mining*, is a facet of NLP concerned with predicting words' emotional tone. This inference evaluates attitudes, opinions, and emotions expressed in text data. This technique is common in areas like the voice of the customer (VoC) analysis, brand monitoring, and social media monitoring. It's a powerful tool for understanding public sentiment, aligning closely with the concepts we've explored around time series analysis and forecasting. When paired with ML services like AWS Comprehend or tools like Hugging Face, sentiment analysis can provide valuable insights into trends, helping organizations make informed decisions. Just as we used AWS CLI for time series forecasting, AWS Comprehend can also be leveraged through similar means for sentiment analysis, underscoring the versatility of these platforms.

We previously utilized AWS CLI to conduct time series forecasting. Another service, AWS Comprehend, can perform sentiment analysis using the CLI or SDK, further demonstrating the adaptability and utility of these pretrained model services in the MLOps context.

Now, let's move to the hands-on part. It's crucial to understand that experimenting with the AWS Console and the Python SDK provides a valuable feedback loop. This iterative process allows you to better understand the ins and outs of AWS services, making integrating them into your MLOps workflows easier. An excellent example of this is as follows:

```
import boto3

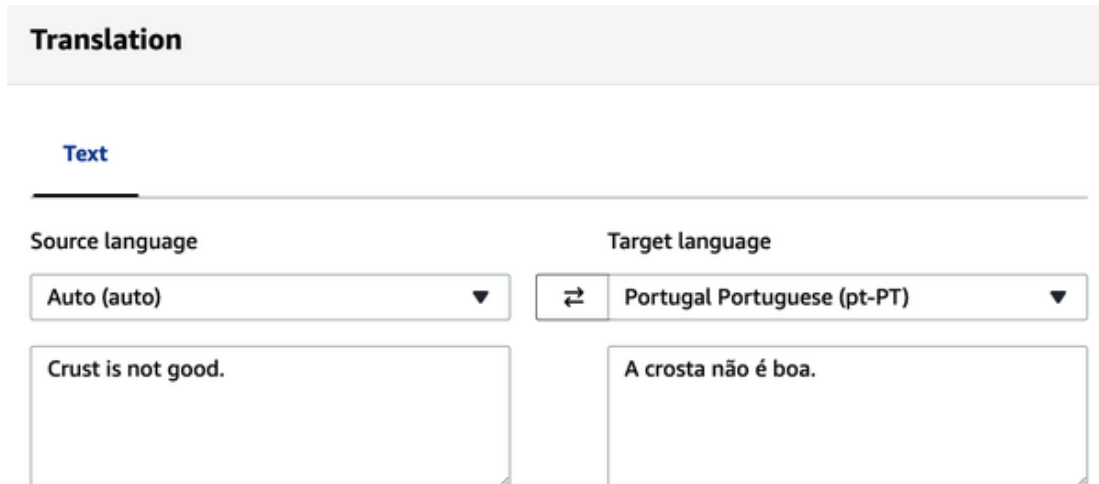
# initialize comprehend client
comprehend = boto3.client(service_name='comprehend',
    region_name='your_aws_region')

# input text for sentiment analysis
text = 'Crust is not good.'

# detect the sentiment of the input text
response = comprehend.detect_sentiment(Text=text, LanguageCode='en')

# print the sentiment score
print(response['SentimentScore'])
```

Let's use this approach again to start translating some text, first with the console as shown in [Figure 9-6](#). That exact string of text translates from English to Portugal Portuguese.



The screenshot shows the AWS Translate console interface. At the top, there is a header labeled "Translation". Below it, a tab labeled "Text" is selected. The interface is divided into two main sections: "Source language" and "Target language". Under "Source language", a dropdown menu shows "Auto (auto)". Under "Target language", a dropdown menu shows "Portugal Portuguese (pt-PT)". Below these dropdowns, there are two text input fields. The "Source language" field contains the text "Crust is not good." and the "Target language" field contains the translated text "A crosta não é boa.".

Figure 9-6. AWS Translate

Next, let's build several Python functions to help us translate text: the code lives in this [repository](#). First, let's make a function that lists the available languages to build a command-line tool that gives the end user many options:

```
def list_languages():  
    """List available languages"""  
  
    client = boto3.client("translate")  
    result = client.list_languages()  
    return result["Languages"]
```

The following function translates text:

```
def translate_text(text, source, target):  
    """Translate text from source to target language"""  
  
    client = boto3.client("translate")  
    result = client.translate_text(  
        Text=text, SourceLanguageCode=source, TargetLanguageCode=target  
    )  
    return (  
        result["TranslatedText"],
```

```

        result["SourceLanguageCode"],
        result["TargetLanguageCode"],
    )

```

Next, build a **click command-line tool** with two subcommands where each subcommand maps to the functions:

```

#!/usr/bin/env python

from awstools.translatelib import translate_text, list_languages
import click
from random import choices

# build out click group
@click.group()
def cli():
    """A simple command line interface for AWS Translate"""

# build out click command to list languages
@cli.command("languages")
def cli_list_languages():
    """List available languages"""

    colors = ["red", "green", "blue", "yellow", "magenta", "cyan", "white"]
    languages = list_languages()
    for language in languages:
        # randomly select a color
        color = choices(colors)
        # print the language name in the randomly selected color
        result = f"{language['LanguageName']}, {language['LanguageCode']}"
        click.secho(result, fg=color[0])

@cli.command("translate")
@click.argument("text")
@click.option("--source", default="en", help="Source language")
@click.option("--target", default="es", help="Target language")
def translate(text, source, target):
    """Translate text from source to target language

    Example:
    ./translator_cli.py translate "Hello World" --source en --target es

    """

```



```

text, source, target = translate_text(text, source, target)
# use colored text to highlight the source and target languages
click.secho("Source: {}".format(source), fg="blue")
click.secho("Target: {}".format(target), fg="yellow")
click.secho(text, fg="white")

# run the cli
if __name__ == "__main__":
    # pylint: disable=no-value-for-parameter
    cli()

```

Language translation, an integral part of NLP, is another task that MLOps professionals often encounter. It is beneficial in multilingual data environments, where understanding and translating data across languages are critical for global data-driven decision making. AWS, Google Cloud, and other cloud services offer solutions for this, but the choice of tool largely depends on your specific needs and the context of your project.

The colored output in [Figure 9-7](#) showcases the diversity of languages supported by our chosen tool. This visualization was created using a simple trick of randomizing the alternate lines to give a rainbow color effect (note that print readers will need to run this yourself to see the colors on a terminal).

```
bash - "ip-172-31-33-166.1" x Immediate (Javascript (brow x +)
Serbian, sr
Sinhala, si
Slovak, sk
Slovenian, sl
Somali, so
Spanish, es
Swahili, sw
Swedish, sv
Tagalog, tl
Tamil, ta
Telugu, te
Thai, th
Turkish, tr
Ukrainian, uk
Urdu, ur
Uzbek, uz
Vietnamese, vi
Welsh, cy
(.venv) ec2-user:~/environment/assimilate-aws (main) $
```

Figure 9-7. Output of the translation command

Next, we demonstrate a translation example from the command line using a custom-built Python tool. The following command translates the phrase “Crust is not good” from English (en) to Portuguese (pt-PT):

```
python translator_cli.py translate --source en --target pt-PT "Crust is not
good"
```

The translated text output is shown in **Figure 9-8**, again with the customized terminal work.

```
(.venv) ec2-user:~/environment/assimilate-aws (main) $ python translator_cli.py translate --source en --target pt-PT "Crust is not good"
Source: en
Target: pt-PT
A crosta não é boa
```

Figure 9-8. Translated text

This example illustrates how easy it is to automate language translation in an MLOps context, emphasizing the role of command-line tools in facilitating these tasks. While focusing on a custom tool, remember that cloud-based services offer similar functionalities, often with more robust

and scalable solutions. The choice between using a custom tool or a cloud service depends on your specific requirements and the scale of your data.

Exploring NLP with OpenAI

OpenAI, a leading NLP player, provides robust solutions that can significantly augment your MLOps workflow. Leveraging their readily available tools, we will explore constructing an “off-the-shelf” MLOps solution using OpenAI’s **Python SDK**. This **CLI tool** exemplifies how pre-existing solutions can effectively employ in an MLOps context.

The following code does a few things: it grabs a URL, parses the text, and summarizes it. Then it acts like a question/answer bot:

```
#!/usr/bin/env python

import openai
import os
import click
import urllib.request
from bs4 import BeautifulSoup

def extract_from_url(url):
    req = urllib.request.Request(
        url,
        data=None,
        headers={
            "User-Agent": ("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_3) "
                           "AppleWebKit/537.36 (KHTML, like Gecko) "
                           "Chrome/35.0.1916.47 Safari/537.36")
        },
    )
    html = urllib.request.urlopen(req)
    parser = BeautifulSoup(html, "html.parser")
    text = ''.join(paragraph.text for paragraph in parser.find_all("p"))
    return text[:1500] # return a max of 1500 characters

def submit_to_openai(text):
    openai.api_key = os.getenv("OPENAI_API_KEY")
    result = openai.Completion.create(
```

```

        prompt=text,
        temperature=0,
        max_tokens=300,
        top_p=1,
        frequency_penalty=0,
        presence_penalty=0,
        model="text-davinci-002",
    )
    return result["choices"][0]["text"].strip(" \n")

def summarize(text):
    return submit_to_openai(f"{text}\n\nTl;dr")

def submit_question(text):
    return submit_to_openai(text)

@click.group()
def cli():
    """An OpenAI tool to answer questions"""

    @cli.command("question")
    @click.argument("text")
    def question(text):
        print(submit_question(text))

    @cli.command("summarize")
    @click.argument("url")
    def summarize_url(url):
        print(summarize(extract_from_url(url)))

if __name__ == "__main__":
    cli()

```

First, let's parse a website, then use OpenAI to summarize it:

```

./openaiAnswerbotCli.py summarize \
    "https://en.wikipedia.org/wiki/2020_Summer_Olympics"

```

The utility of OpenAI in the realm of NLP is indeed profound, as the tool significantly simplifies many complex tasks. OpenAI is an invaluable resource for both developing and seasoned MLOps professionals. The following example showcases the tool's ability to summarize lengthy text concisely:

"The 2020 Summer Olympics were postponed to 2021 due to the global COVID-19 pandemic. The event was largely held behind closed doors with no public spectators permitted due to the declaration of a state of emergency in the Greater Tokyo Area in response to the pandemic. The Games were the most expensive ever, with total spending of over \$20 billion."

Other tools in the market also offer similar capabilities, such as AWS Comprehend, Google Cloud's Natural Language API, and Microsoft's Azure Text Analytics. Each tool has its strengths and weaknesses, and the choice between them often boils down to your project's specific requirements, budget, and comfort level with the respective platforms.

OpenAI stands out due to its powerful machine learning models like GPT-4 and user-friendly Python SDK, making it accessible for developers with varying levels of expertise. Its ability to generate human-like text makes it an exceptional tool for summarizing text, developing content, and answering context-based questions.

However, while OpenAI simplifies many tasks, it doesn't negate the need for a deeper understanding of NLP and its underlying principles. A strong foundation in NLP will enable you to leverage OpenAI's capabilities more effectively and tailor its use to your specific needs.

Next, we'll explore another intriguing aspect of OpenAI: its capabilities for image generation.

Video Analysis, Image Classification, and Generative AI

OpenAI's repertoire also includes a remarkable tool called DALL·E 2. This model leverages the power of AI to generate images from textual

descriptions, demonstrating a significant leap in AI image generation. It's essentially an artist at your command, creating visuals from mere sentences.

DALL·E 2's capacity to translate text into visual representations can be valuable in numerous MLOps scenarios. For instance, one use case is to generate data for training other models, visualizing concepts for better understanding, or creating illustrations for documentation or presentations. Its capabilities can enhance data understanding and exploratory data analysis and fuel creativity in the MLOps space.

Its accessibility via the OpenAI API means it can incorporate into existing MLOps workflows. This capability facilitates swift prototyping and iterative development, aligning with the principles of MLOps. Let's now dive into Python and **code a solution** that utilizes DALL·E 2 via the OpenAI API:

```
#!/usr/bin/env python
"""
A command line tool that uses click to generate images using OpenAI.
"""

import click
from oalib.image_gen import generate_image

# build a click command line tool that takes a prompt and returns an image
@click.group()
def cli():
    """A command line tool that uses click to generate images using OpenAI."""

    @cli.command("generate")
    @click.option("--prompt", help="The prompt to use for image generation.")
    @click.option("--size", default="1024x1024", help="The size of the image.")
    def generate(prompt, size):
        """Generate an image using OpenAI's API."""
        image_url = generate_image(prompt, size)
        print(image_url)

if __name__ == "__main__":
    cli()
```

Next, let's run this code example:

```
./imageGen.py generate --prompt "cats playing with dogs"
```

One of the exciting aspects of building your tools is their level of customization and flexibility. You can tailor the AI solutions to your specific needs, intertwining APIs with your trained models or even integrating multiple APIs. Combining AI solutions allows for creation of robust, multifaceted systems that can handle complex tasks.

For MLOps professionals, this level of customization and integration can be precious. For instance, DALL·E 2 can generate synthetic data for model training or create visual representations of complex concepts for better comprehension. Combining different AI tools, you can build a comprehensive system covering various aspects of the MLOps lifecycle, from data collection and preprocessing to model training and evaluation. The command prompt in **Figure 9-9** provides an example of what is achievable.



Figure 9-9. Cats playing with dogs

Image Classification Techniques with CreateML

Let's now look at a training-based prototyping tool, Apple's CreateML. We'll undertake a high-level image classification task to demonstrate its capabilities with a practical example. You can [download the cats and dogs dataset from Kaggle](#) or use any other image dataset you have.

As our final demonstration, [Figure 9-10](#) shows how rapidly a modest dataset of 200 images trains using these high-level tools. It's important to

note that speed and efficiency are paramount in MLOps. Quick model training allows for more iterations, ultimately leading to better-performing models. It also facilitates rapid prototyping and testing of different models and hypotheses. This process underscores the value of these high-level tools in an MLOps workflow, as they can significantly streamline operations and increase productivity.

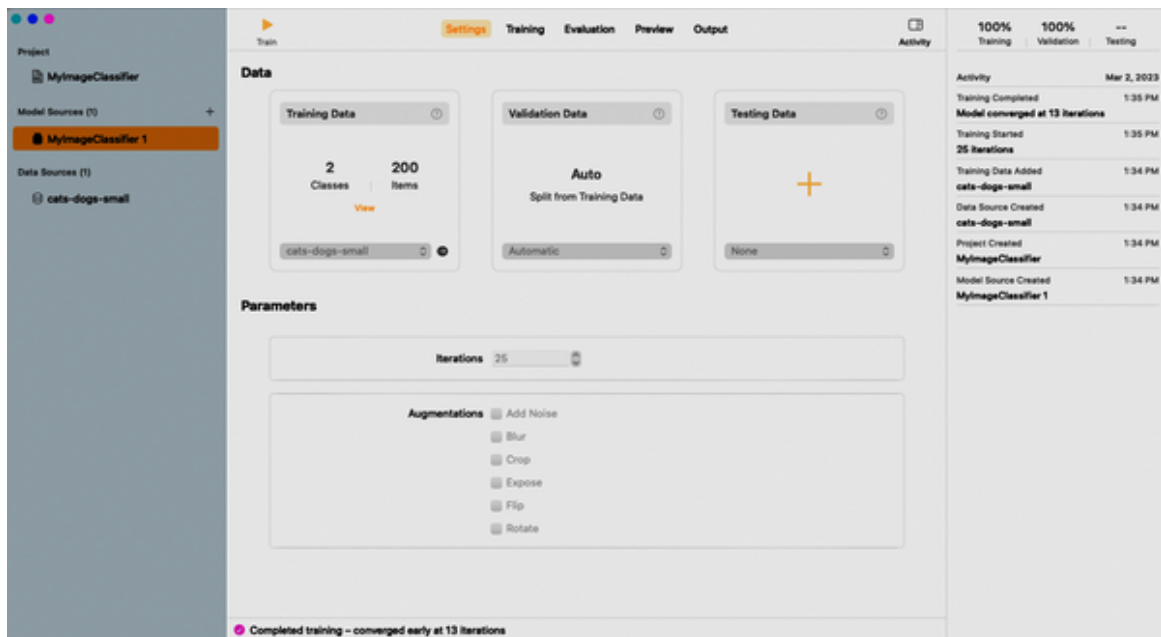


Figure 9-10. Image classification

Instead of starting from scratch, an experienced MLOps professional might use pretrained models, like those found on [Kaggle](#), as part of their workflow. These models have been previously trained on extensive datasets and can provide a solid foundation for many ML tasks.

Pretrained models can save considerable time and computational resources, as training is primarily complete. They can be instrumental when working with limited data, where training a complex model from scratch might be challenging.

Figure 9-11 illustrates the various pretrained models available on Kaggle. They cover a range of tasks from image and text classification to regression and more. These models can be easily integrated into your workflow, tuned

to your specific job, and deployed, making them a high-level tool that bolsters efficiency in the MLOps landscape.

Models

Search and discover hundreds of trained, ready-to-deploy machine learning models in one place.

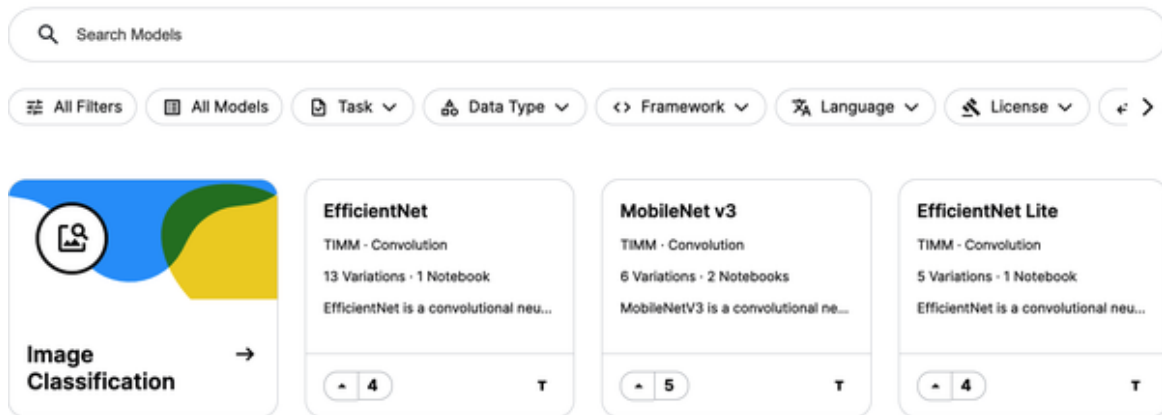


Figure 9-11. Availability of pretrained models on Kaggle

Composite AI

Composite AI is an emerging approach in artificial intelligence that combines various AI technologies and models to solve complex problems. It's more than just the sum of its parts; it's about creating systems that can leverage the strengths of various AI techniques, such as NLP, image recognition, and machine learning, to deliver more comprehensive and practical solutions.

In the context of MLOps, Composite AI can be a game-changer. It allows for developing more sophisticated models that can handle multifaceted tasks and significantly enhance your AI solutions' robustness and flexibility.

Composite AI combines different AI techniques and integrates AI with other technologies, such as cloud computing, big data analytics, and the

Internet of Things (IoT). This technique enables processing vast amounts of data, real-time decision making, and automating complex processes.

The potential of Composite AI is enormous. It can help organizations tackle challenges that would be difficult or even impossible to solve using a single AI technique, acting as a significant driver of innovation in the AI industry.

While numerous platforms and tools are available for implementing Composite AI solutions, AWS offers an exceptionally comprehensive and versatile ecosystem. AWS combines various services, from low-level tools like SageMaker to high-level AI and ML services like **AWS Comprehend** for NLP or **AWS Rekognition** for image analysis.

However, AWS is one of many players in the field. Platforms like Google Cloud, Microsoft Azure, and IBM Watson also provide tools and services for building Composite AI solutions. As an MLOps professional, understanding the capabilities of these different platforms and how to leverage them effectively is crucial to building robust, efficient, and innovative AI systems.

Next, let's explore how AWS builds a Composite AI solution.

Getting Started with Serverless for Composite AI

The foundation of serverless on AWS is Lambda, a service that allows you to run your code without provisioning or managing servers. It executes your code only when needed and scales automatically, making it an ideal component of a serverless architecture.

Our focus is on how Lambda is part of a hybrid AI solution. To illustrate this, let's consider a simple example: Lambda responds to events triggered by a timer, processes some data, and places the results in a queue.

This architectural structure is shown in **Figure 9-12**.

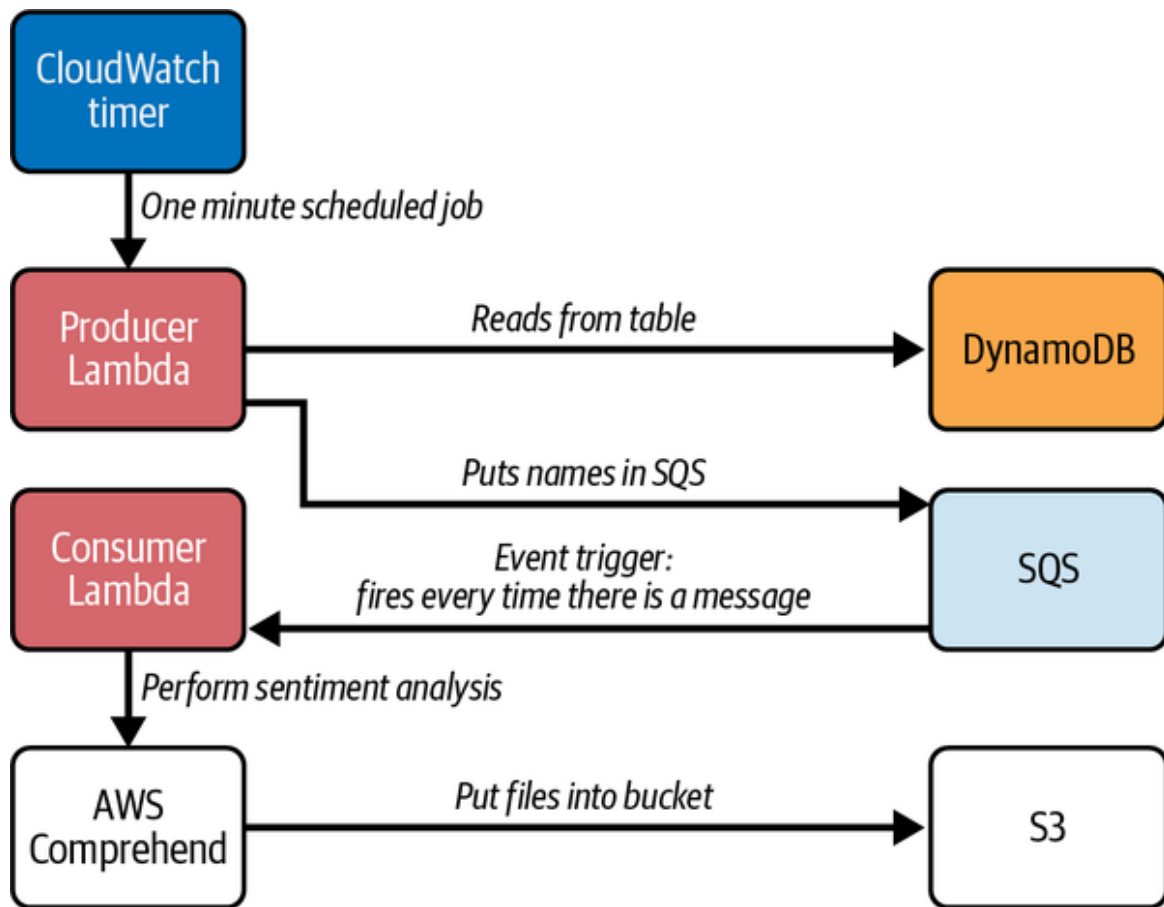


Figure 9-12. Building a simple serverless AI Lambda

This figure shows how the event triggers the Lambda function, which processes the data and places the results in an Amazon Simple Queue Service (SQS) queue.

Figure 9-13 provides a visual representation of how the SQS queue fits into the serverless architecture. Once the Lambda function has processed the data, it places the results in the SQS queue, ready to be consumed by other system components.

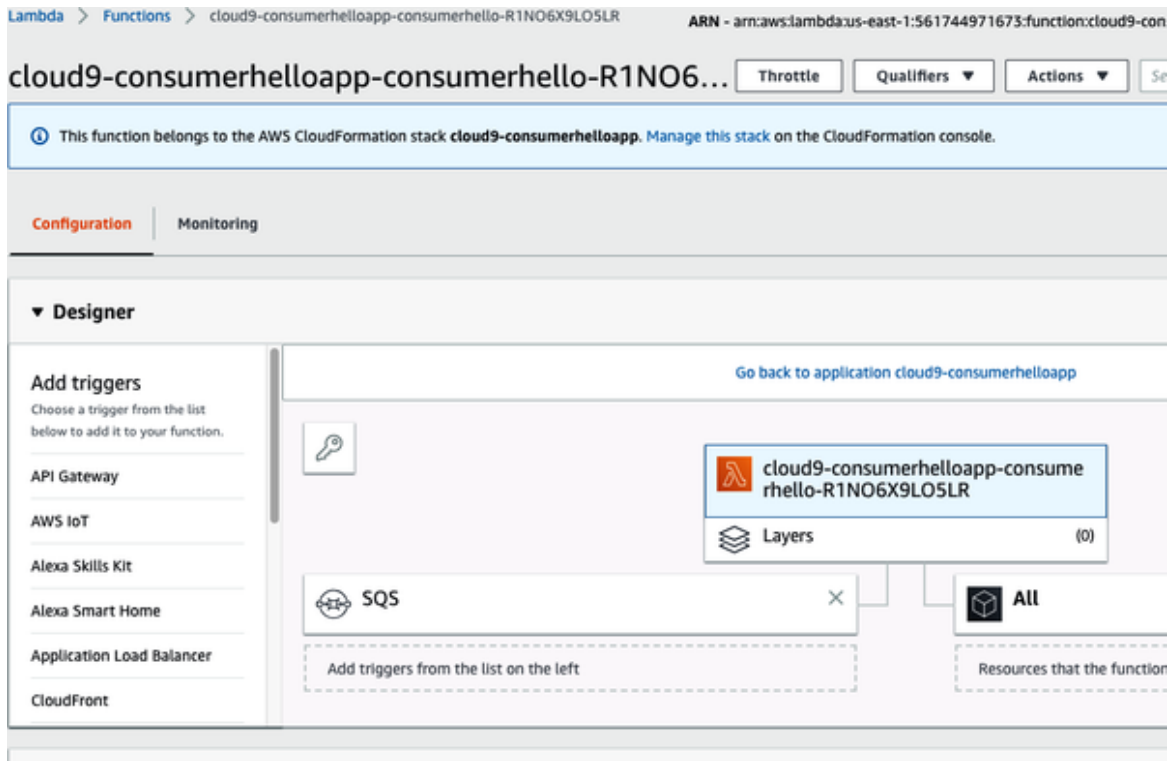


Figure 9-13. Triggering an SQS queue in a serverless architecture

The complete code repository is in [GitHub](#), but let's look at the highlights of this Python example:

```
def lambda_handler(event, context):
    """Entry Point for Lambda"""
    LOG.info(f"SURVEYJOB LAMBDA, event {event}, context {context}")

    receipt_handle = event['Records'][0]['receiptHandle'] # sqs message
    event_source_arn = event['Records'][0]['eventSourceARN']

    names = [] # Captured from Queue

    # Process Queue
    for record in event['Records']:
        body = json.loads(record['body'])
        company_name = body['name']

        # Capture for processing
        names.append(company_name)

    extra_logging = {"body": body, "company_name": company_name}
    LOG.info(
        f"SQS CONSUMER LAMBDA, splitting sqs arn with value:"
    )
```

```

        f" {event_source_arn}", extra=extra_logging
    )
    qname = event_source_arn.split(":")[-1]
    extra_logging["queue"] = qname
    LOG.info(
        f"Attempting Deleting SQS receiptHandle {receipt_handle} "
        f"with queue_name {qname}", extra=extra_logging
    )
    res = delete_sqs_msg(queue_name=qname, receipt_handle=receipt_handle)
    LOG.info(
        f"Deleted SQS receipt_handle {receipt_handle} with res {res}",
        extra=extra_logging
    )

# Make pandas dataframe with wikipedia snippets
    LOG.info(f"Creating dataframe with values: {names}")
    df = names_to_wikipedia(names)

# Perform Sentiment Analysis
    df = apply_sentiment(df)
    LOG.info(f"Sentiment from FANG companies: {df.to_dict()}")

# Write result to S3
    write_s3(df=df, name=names.pop(), bucket="fangsentiment")

```

This code is a Python script that serves as the entry point for an AWS Lambda function. The function triggers by receiving a message from an Amazon SQS queue. The purpose of the function is to process the message, perform sentiment analysis on a list of company names contained in the message, and write the result to an Amazon S3 bucket.

The script logs the event and context that triggered the Lambda function. It extracts the receipt handle and event source ARN from the SQS message. After processing, the receipt handles and deletes the message from the queue. The event source ARN determines the name of the SQS queue.

Next, the script loops through all the records in the event and extracts the company name from each record. The company names pass to a function named `names_to_wikipedia`, which returns a pandas DataFrame with the names and their corresponding Wikipedia snippets.

The script then calls a function named `apply_sentiment` to perform sentiment analysis on the DataFrame. The sentiment analysis result is logged and then written to an S3 bucket named *fangsentiment*.

Finally, the script deletes the processed SQS message using the `delete_sqs_msg` function, passing in the queue name and receipt handle. With a primary case out of the way, let's build on this and dive deeper into a hybrid AI solution.

Use Cases of Composite AI with Serverless

Composite AI with serverless technologies presents a powerful combination for tackling complex tasks cost-efficiently. AWS provides various services, making it an ideal platform for deploying these solutions. One such scenario is hosting a pretrained machine learning model on AWS's Elastic File System (EFS) and leveraging AWS Lambda, a serverless computing service, for inference.

The advantage of this setup is the ability to use a low inference, low-memory language, which optimizes cost. Serverless technology, particularly when combined with well-optimized, high-performance code, can be cost-effective due to AWS Lambda's billing model, which charges based on the number of requests and the duration of code execution. In addition to Lambda and EFS, other AWS services like Rekognition, which makes it easy to add image and video analysis to applications, can be incorporated for more comprehensive solutions. An example of such a composite AI solution is shown in [Figure 9-14](#).

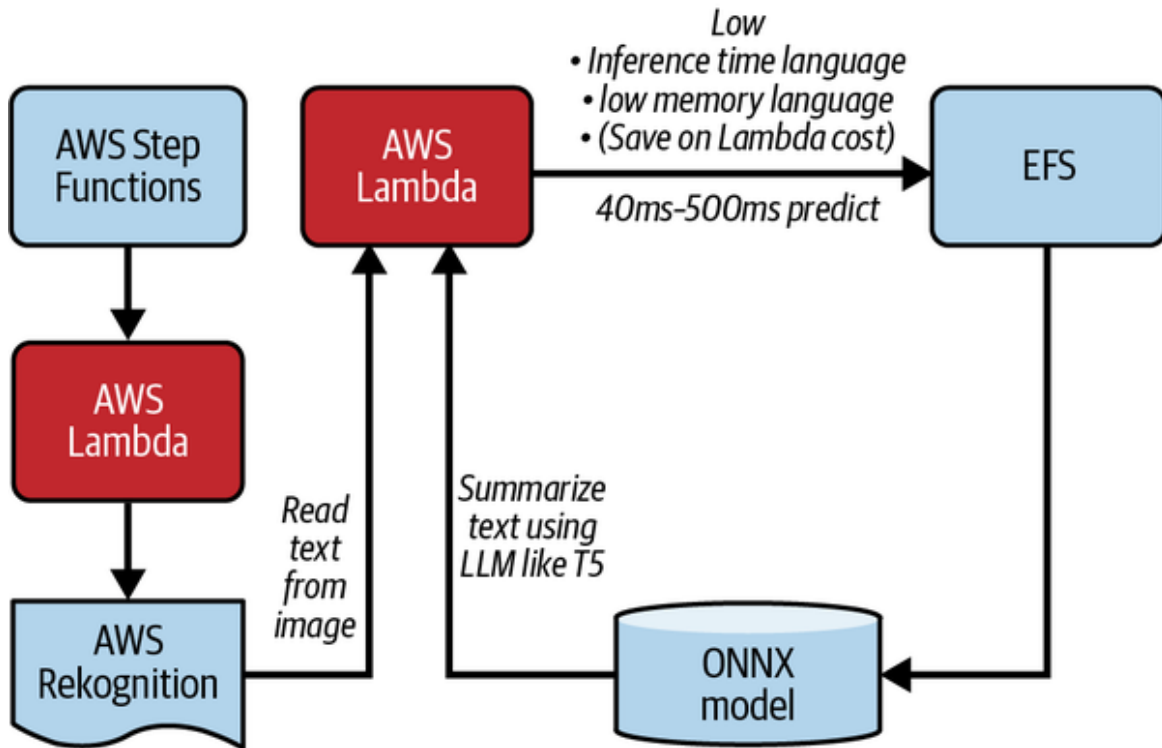


Figure 9-14. Using EFS with AWS Lambda for Inference

While this chapter focuses on AWS services, it's worth noting that similar concepts can be applied using other cloud platforms. For instance, Google Cloud offers Cloud Functions as a serverless execution environment, and Azure provides Azure Functions. Both used their respective storage and AI/ML services to create composite AI solutions, as shown in Figure 9-15.

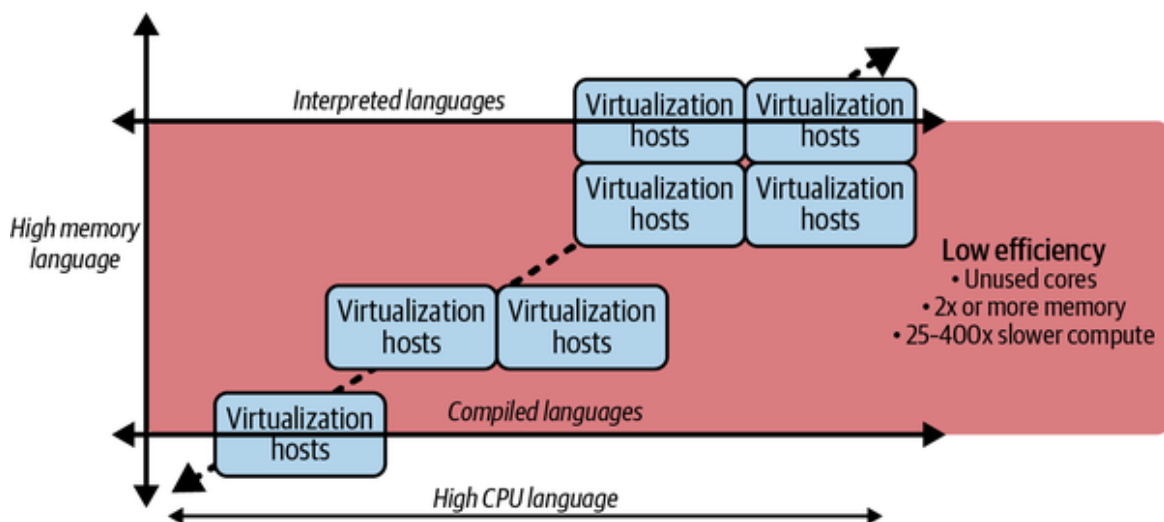


Figure 9-15. Lost efficiency from high memory runtime

Let's wrap up the chapter and reflect on the key takeaways.

Conclusion

This chapter embraced diving into advanced data types and doing so pragmatically, using APIs, GUI tools, and pretrained models. We also showed how AWS is an excellent base of operations for doing quick and dirty prototypes that later get put into command-line tools.

Next, go through the critical thinking questions and exercises to hammer home the ideas of this chapter.

Critical Thinking Discussion Questions

- What are the advantages and disadvantages of using time series analysis for data interpretation and prediction?
- How can LLM NLP tools be used to address specific challenges in sentiment analysis and text classification?
- What are the key considerations when using video analysis and image classification in real-world applications?
- What are the critical components of a composite AI solution, and how can they be integrated to solve complex data problems?
- How can AWS be used to support the entire lifecycle of a data science project, from data processing to model deployment and monitoring?

Exercises

- Train a time series forecasting model using Amazon SageMaker and compare its performance to a traditional time series analysis approach.
- Build an NLP-based sentiment analysis model using AWS Comprehend and evaluate its accuracy on a real-world dataset.

- Train an image classification model using AWS DeepLens and deploy it to a real-time video stream for object detection.
- Use AWS Rekognition to perform object recognition in video streams and compare its performance to other video analysis techniques.
- Create a composite AI solution using AWS services such as SageMaker, Comprehend, and Rekognition to analyze and interpret complex data sets.

Chapter 10. Implementing MLOps Using Rust

Operational efficiency must be at the core of any technology system. MLOps builds upon DevOps, which builds on the concept of *kaizen*, the Japanese word for continuous improvement. Without continuous improvement, you wouldn't have DevOps or, by extension, MLOps.

At the heart of continuously improving operations is a simple question: “Can we improve operational performance—from training and inference to packaging and delivery—by ten times or more?” If the answer is yes, as it will be with many organizations using Python for data science, the next question should be: “Why are we not doing it?”

For decades, organizations had few options other than pure C, C++, or C# and Python for machine learning solutions. C++ may provide more efficiency in terms of performance, but Python is generally easier to learn, implement, and maintain, which is why Python has taken off in data science. The hard choice between the performant but complex C++ and the easy-to-learn but comparatively slow Python ultimately results in many companies choosing Python.

But there's another way. Rust consistently ranks among the **most performant and energy-efficient languages**. It's also among the most loved languages in **Stack Overflow's annual developer survey**. Though some Python libraries widely used in data science are written in C and can provide some of the performance benefits of running a compiled language, Rust provides a more direct route to bare metal while using a single language.

Rust is also far easier to learn and use than C or C++, which makes it a realistic solution for those who want the performance of a compiled language. That's especially true when using **GitHub Copilot**. This AI-

powered pair programming assistant uses the OpenAI Codex to suggest code and entire functions in real time to developers while they code. Let's discuss this strategy next.

NOTE

The phrase *AI-powered pair programming assistant* refers to an improvement over the classic pair programming style where you sit next to another developer and write code together. With emerging developer tools like **GitHub Copilot X**, you can chat with a coding assistant to get ideas on a coding project as well as get coding suggestions as you type.

The Case for Rust for MLOps

GitHub Copilot is a revolutionary new change in the way developers work. GitHub Copilot and tools like it are game changers since they minimize the impact of syntax on productivity. With Rust, you spend more time compiling code, which is an investment in future returns, much like saving for the future in a retirement account. Rust has excellent performance and safety, but the syntax can be challenging. With GitHub Copilot, the syntax becomes less of an issue since the suggestions eliminate many of the difficulties in programming. Additionally, because of the robustness of the Rust toolchain for linting, formatting, and compiling, any errors or false starts from GitHub Copilot are caught by these tools, making the combination of Rust and GitHub Copilot an emergent front-runner in AI-assisted coding.

NOTE

There are several reasons to consider Rust other than performance. Rust is a modern language that first appeared in 2010. It lacks the baggage that older languages carry, but it is mature enough that it isn't going anywhere anytime soon. Further, other trends are supporting a hard look at Rust.

Rust was designed from the ground up to support modern computing capabilities, like multicore threads, that are often “bolted on” to older languages like Python. By designing the language to support these features from the start, Rust can avoid awkwardness in many other languages. A great example of how simple multicore threads are in Rust is the following snippet from the [Rust rayon library](#):

```
use rayon::prelude::*;
fn sum_of_squares(input: &[i32]) -> i32 {
    input.par_iter()
        .map(|i| i * i)
        .sum()
}
```

There are no code gimmicks or hacks; the threads “just work” across all the machine cores, and the code is just as readable as Python.

Likewise, Rust natively supports typing, so the entire toolchain, from the linter to the editor to the compiler, can leverage this capability. Rust also makes packaging a breeze. Cargo provides a Python-esque “one obvious way” to install packages.

Of course, there are still areas where Python excels. It's fantastic for API documentation and readability in general. If you need to try out an idea, it is hard to beat using Python in an interactive prompt, like [IPython](#), to explore a concept. But MLOps is more sensitive to performance requirements than other data science fields and heavily dependent on software engineering best practices conducive to implementation in Rust. A new superset of Python called [Mojo](#) might solve many performance and deployment issues soon, but it's still in development while Rust is available here and now.

One common objection to using Rust is that it doesn't have as large and established an ecosystem as Python for working with data. But remember that this ecosystem isn't necessarily optimal for the needs of MLOps. In particular, the stack I call #jcpennys (Jupyter, Conda, pandas, NumPy, sklearn) is straight from academia, heavyweight, and optimized for use with small data. In academics, much is to be said for a "God environment" with everything in one spot. But in real-world production MLOps, you don't want extra packages or brittle tools that are difficult to test, like notebooks. Meanwhile, the Rust ecosystem is growing. For example, **Polars** is a performant data frame library taking the data space by storm.

Leveling Up with Rust, GitHub Copilot, and Codespaces

You can see how to use the GitHub ecosystem to level up to a more robust language in Rust in **Figure 10-1**. Prompt engineering occurs when you ask GitHub Copilot in step B to generate code; you then test the idea for a CLI to ensure it works and clean up any suggestions with a series of Makefile commands.

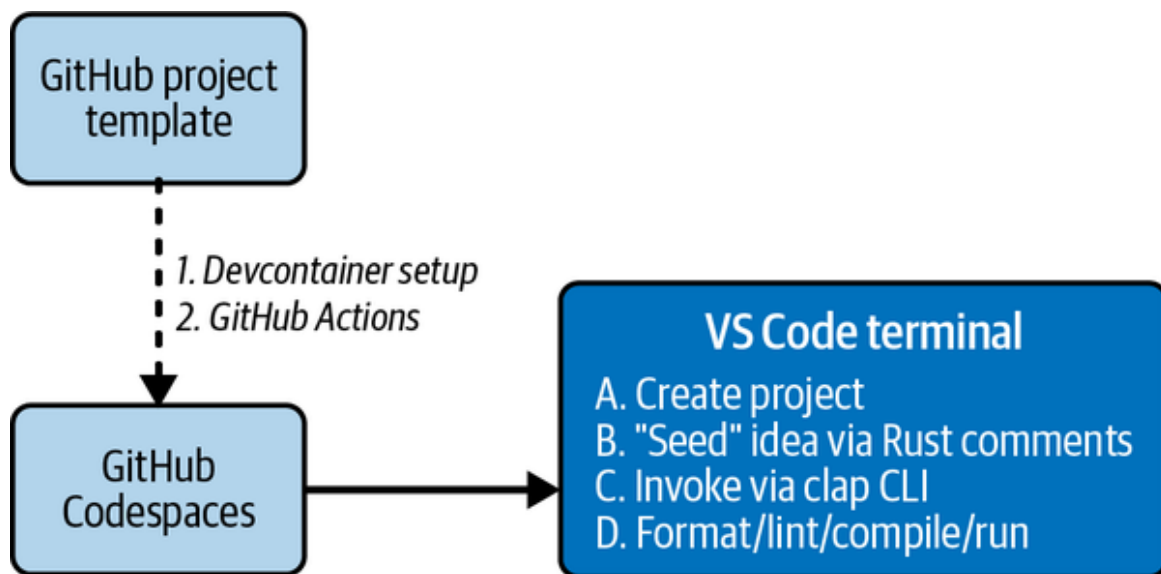


Figure 10-1. Prompt engineering with GitHub ecosystem

All Rust projects follow this pattern:

1. Create a new repo using **Rust New Project Template**.

2. Next, create a new **Codespace** and use it.
3. Use *main.rs* to call the handle CLI and *lib.rs* to handle logic and import `clap` in *Cargo.toml* as shown in this project.
4. Use `cargo init --name 'hello'` or whatever you want to call your project.
5. Put your “ideas” in as comments in Rust to seed GitHub Copilot (i.e., add a comment as shown: `//build an add function`).
6. Run `make format` (i.e., `cargo format`).
7. Run `make lint` (i.e., `cargo clippy --quiet`).
8. Run project: `cargo run -- --help`.
9. Push your changes to allow GitHub Actions to: format check, lint check, and other actions like binary deploy.

NOTE

The video “[Using VS Code, Copilot, and Codespaces to Level Up to Rust from Python](#)” shows this workflow using an [example in GitHub](#).

Here’s an **example repository**. A good starting point for a new Rust project is the following pattern.

To run: `cargo run -- marco --name "Marco"` Be careful to use the *name* of the project in the *Cargo.toml* to call *lib.rs* as in:

```
[package]
name = "hello"
```

For example, see the name `hello` invoked alongside `marco_polo`, which is in *lib.rs* code:

```

/* A Marco Polo game. */

/* Accepts a string with a name.
If the name is "Marco", returns "Polo".
If the name is "any other value", it returns "Marco".
*/
pub fn marco_polo(name: &str) -> String {
    if name == "Marco" {
        "Polo".to_string()
    } else {
        "Marco".to_string()
    }
}

```

main.rs code:

```

fn main() {
    let args = Cli::parse();
    match args.command {
        Some(Commands::Marco { name }) => {
            println!("{}", hello::marco_polo(&name));
        }
        None => println!("No command was used"),
    }
}

```

This style is an emerging pattern ideal for systems programming in Rust, as certain combinations lead to further advances. GitHub Copilot's suggestions, a next-generation compiled language like Rust, and its ecosystem of formatting, linting, and packaging tools, can lead to a more robust software development experience than Python. Let's consider some potential benefits that Rust brings to the world of MLOps:

Performance

Rust has a very efficient memory model with no garbage collector, which can significantly increase the speed of your MLOps pipelines. This capability is critical for MLOps tasks that quickly handle large volumes of data.

Concurrency

Rust's memory safety guarantees enable safe concurrency, enabling you to quickly leverage multiple cores to speed up your processing tasks.

Interoperability

Rust has excellent interoperability with C and can call C libraries directly. This capability could allow MLOps developers to leverage existing C libraries for numerical computation and ML tasks.

Security

Rust's emphasis on memory and type safety can lead to more secure applications. This process is crucial for MLOps tasks where security is critical, such as healthcare or finance.

Robustness

Rust's compile-time error checking can catch many errors before your code runs. This process can lead to more robust MLOps pipelines less prone to runtime errors.

Developer productivity

Rust has a steeper learning curve than Python, so tools like GitHub Copilot can help you write Rust code more quickly and easily.

Of course, Rust is not a silver bullet, and Python will continue to play an essential role in MLOps. However, for MLOps tasks that need the performance, security, and robustness that Rust can provide, it is an option worth considering.

Another bolt-on problem with Python is the packaging. Even though the Python standard library includes two tools that make it relatively straightforward to install packages in **pip** and **virtualenv**, as I **demonstrate in a Python MLOps repository**, there is an explosion of tools to handle “edge

cases.” Ironically, the **Zen of Python**, actually cautioned against this decades ago in the statement, “There should be one—and preferably only one—obvious way to do it.” Python means *except packaging*, which has an almost exponential way of doing things. Additionally, distributing a binary command-line tool efficiently in Python is nontrivial since this capability is not part of the language; this is different from a default workflow included in the language, like it is with Rust or Go. An excellent example of good binary tool distribution is the **Hugo framework** written in Go. One reason it is so popular is it is easy to install.

These critiques of Python don’t mean it is an imperfect solution to what it excels at; it just isn’t *the* solution to every single problem. Python is fantastic for API documentation and readability in general. If you need to try out an idea, it is hard to beat using Python in an interactive prompt, like **IPython**, to explore a concept. It struggles with packaging, performance, and language safety, all things Rust excels at. This strength is why Rust is an ideal language for enterprise MLOps.

What could be the alternative for MLOps if there is a better solution than academic “data science” tools?

- Is there a more performant DataFrame library? (Rust has **Polars**.)
- Why not have a **compiler to optimize code**?
- Why not have a simple packaging solution with “one way to install?” (**Rust has Cargo**.)
- Why not have a breakneck computational speed for ML? (**Some benchmarking shows 25X speed improvements**.)
- Why not be able to write both for the **Linux kernel** and general-purpose scripting?
- Why not see if there is a better solution than Python (which is essentially two languages: scientific Python and regular Python)?

Python is more or less the least green language in energy efficiency, and Rust is more or less the best. With the rise of machine learning, it is important to consider carbon footprint and sustainability goals.

NOTE

O'Reilly has several fantastic books on or involving Rust worth referring to if you want to dive deeper:

- *Command-Line Rust* by Ken Youens-Clark
- *Programming Rust* by Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall
- *WebAssembly: The Definitive Guide* by Brian Sletten (includes Rust Web Assembly examples)

Two GitHub repositories you might look at are this [brief Rust tutorial](#) and [Rust MLOps template](#). With the case for Rust made theoretically, let's dive into more detail next.

In the Beginning Was the Command Line

What could MLOps and data science look like without Jupyter Notebook and complex install tools as the center of the universe? It could be the command line. The command line was at the beginning of computing in the 1970s and 1980s, and it may be the best solution for the domain of MLOps.

In 1999, science fiction author Neal Stephenson wrote the essay “In the Beginning ...Was the Command Line.” The following excerpt highlights his thoughts at the time about the emergence of GUI-based systems:

What would the engineer say after you had explained your problem and enumerated all the dissatisfactions in your life? He would probably tell you that life is a very hard and complicated thing; that no interface can change that; that anyone who believes otherwise is a sucker; and that if you don't like having choices made for you, you should start making your own.

Similarly, with the constantly evolving domain of MLOps, it is easy to get caught in the trap of thinking of a notebook-based workflow alone as the solution to every problem. In a recent seminar, Noah discussed MLOps and some of the drawbacks of notebooks, with one data scientist “cutting and pasting” code from a notebook to a script file. In the early days of web development in the 1990s, the concept of cut-and-paste coding was debunked as an anti-pattern. But in 2023, there are advocates for it in MLOps, and with a straight face!

Ultimately the flexibility of command-line interfaces, coupled with systems programming approaches like those with C, C++, Go, and Rust, are too valuable to ignore, especially if the language is approachable for a developer more familiar with high-level languages. According to an April 2021 [newstack article](#) about Rust for the Linux kernel, developer Miguel Ojeda introduced a Request for Comments (RFC) on making Rust a part of Linux. Linus Torvalds, the creator of Linux, said, “Unless something odd happens, it [Rust] will make it into 6.1” (i.e., late 2022 or early 2023). And this happened in 6.1, and Rust has continued updates in each subsequent release.

Similarly, Amazon is active in open source Rust development via the [Firecracker project](#), which is “an open source virtualization technology.” One example of the power of this technology is a [firecracker demo repo](#), which shows 4,000 simultaneous VMs launching. Amazon also has an alpha release of a [Rust SDK on GitHub](#).

Finally, because Rust makes command-line tools so easy to distribute, doing MLOps for the CLI is an optimal use case for Rust. One of the more remarkable examples is the [diffusers-rs project](#) that invokes Stable Diffusion using Rust and [PyTorch](#):

```
cargo run --example stable-diffusion --features clap --\
  --prompt "A rusty robot using the command-line terminal and throwing\
  away notebooks"
```

The reason solutions like the Stable Diffusion example exist along with bindings for [Rust with PyTorch](#) is the exponential growth of Rust modules,

as shown in Figure 10-2.

Module Counts

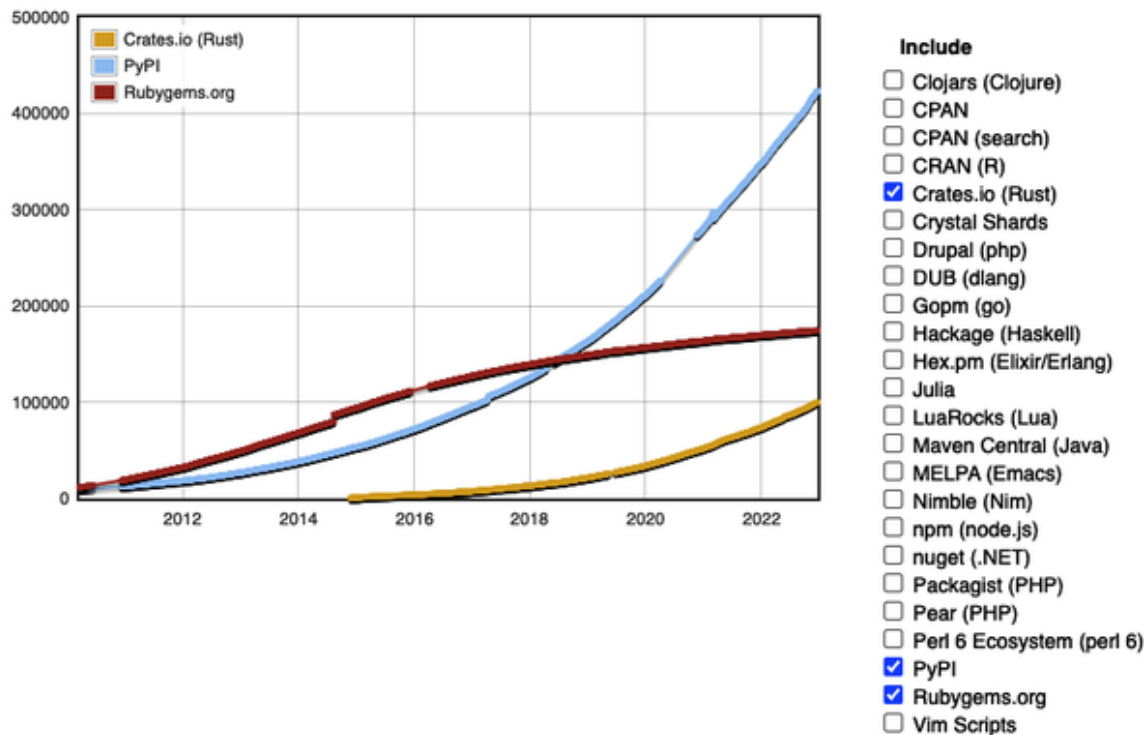


Figure 10-2. Exponential growth of Rust modules

Now that the case for Rust for MLOps exists, let's dive into getting started with Rust itself.

Getting Started with Rust for MLOps

Rust is one of the more accessible languages from an installation perspective. You run the `rustup` command, and that is all you need to install! Another option is to use GitHub Codespaces as a development environment. One of the advantages of Codespaces is that it has a generous free tier and allows for easy customization. You can create a new repository for developing with Rust using the Rust `new project template` we made available for this book.

Once you install, check to see if things work by running `rustc --version`.

Another option is to **create a Makefile** and put key commands in it such as `make rust-version`, which checks both the cargo and rust version. Several tools help you get things done in Rust:

```
rust-version:
  @echo "Rust command-line utility versions:"
  rustc --version          #rust compiler
  cargo --version          #rust package manager
  rustfmt --version        #rust code formatter
  rustup --version         #rust toolchain manager
  clippy-driver --version  #rust linter
```

To run everything locally, you can do `make all`, which will format/lint/test all projects **in this repository**.

Next, to build a hello-world example, you can use the built-in `cargo` command. Cargo is one of the most substantial and valuable parts of the Rust ecosystem.

Create a project directory: `cargo new hello`. The cargo command creates a structure you can see with `tree hello`:

```
hello/
├─ Cargo.toml
└─ src
   └─ main.rs
1 directory, 2 files
```

The *Cargo.toml* file is where the project configuration lives (i.e., if you need to add a dependency). The source code file has the following content in *main.rs*. It looks like Python or any other modern language, and this function prints a message:

```
fn main() {
    println!("Hello, world MLOps!");
}
```

To run the project, you `cd` into *hello* and run `cargo run`. The output looks like the following:

```
@noahgift > /workspaces/rust-mlops-template/hello (main) $ cargo run
Compiling hello v0.1.0 (/workspaces/rust-mlops-template/hello)
Finished dev [unoptimized + debuginfo] target(s) in 0.36s
Running `target/debug/hello`
Hello, world ML0ps!
```

Finally, run without all the noise: `cargo run --quiet`. If you want to run the binary created, the following command executes it:

`./target/debug/hello`. It is important to note how awesome this is coming from Python because the binary distribution of your code comes for free. In Python, the concept of binary executable isn't a workflow supported by native Python. Instead, the closest way to achieve the same portable executable would require packaging Python in a Docker workflow.

A big takeaway with this hello-world project is that the Rust ecosystem takes care of many complex programming parts, namely linting, testing, formatting, and binary deployment within Rust **Cargo package manager**. If the Rust code passes lint and compiles, it should work when you run it. You cannot say the same for Python since there is no compiler. In addition to being more reliable, it gives you C-level speed with a readable syntax similar to Python. Finally, many features of the language, like immutable variables and rational concurrency design, make the code safer because the compiler will not compile code that isn't safe.

One more way to make Rust programming more effective is including **GitHub Copilot** or a similar tool like **Amazon CodeWhisperer** in the initial phase of code creation. The synergy of Copilot with the robust Cargo ecosystem is a recipe for productivity, as shown in **Figure 10-3**.

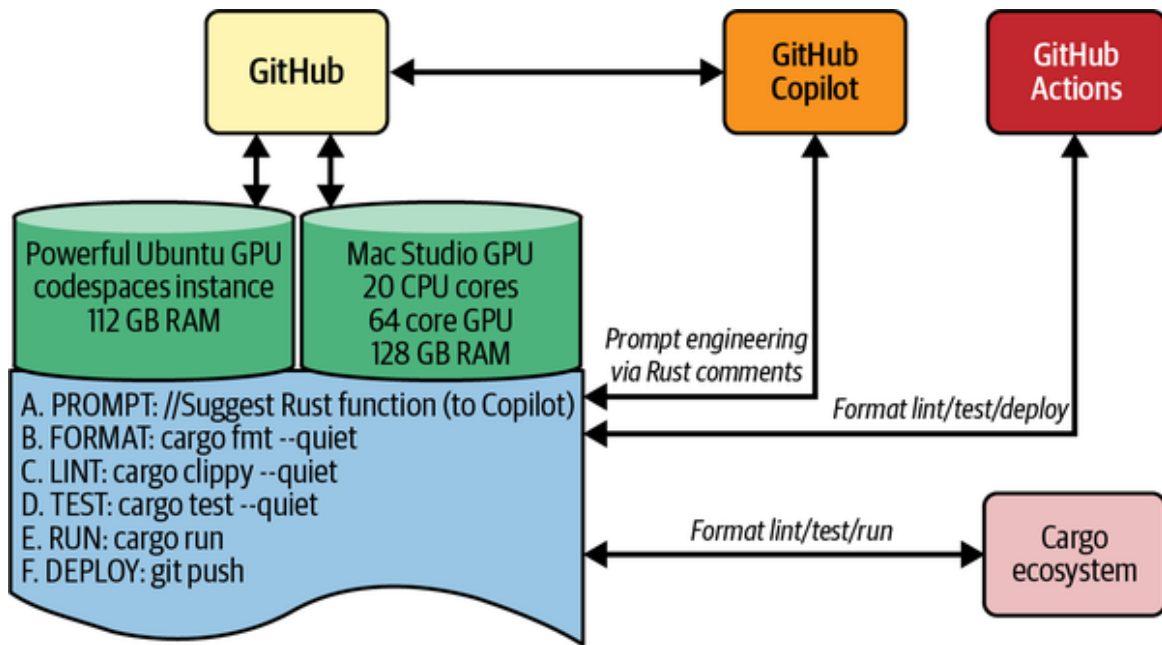


Figure 10-3. Modern Rust development with Copilot

Further, by including automation in the workflow via **GitHub Actions**, these necessary automation steps also occur as part of CI/CD. The foundation of MLOps is the automation of software engineering best practices happening at the build system. This core capability enables further automation down the line:

```
name: Rust CI/CD Pipeline
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
env:
  CARGO_TERM_COLOR: always
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v1
      - uses: actions-rs/toolchain@v1
        with:
          toolchain: stable
          profile: minimal
          components: clippy, rustfmt
          override: true
```



```
- name: update linux
  run: sudo apt update
- name: update Rust
  run: make install
- name: Check Rust versions
  run: make rust-version
- name: Format
  run: make format
- name: Lint
  run: make lint
- name: Test
  run: make test
```

One of the reasons a `Makefile` is an excellent component in a project is that the steps locally, installing, linting, formatting, and testing run in the same way on the build system. This methodology eliminates potential errors in building software automatically.

This style of Rust development is both new and an essential advancement in developer productivity. With tools like GitHub Copilot and AWS CodeWhisperer, switching to a higher-performance language like Rust, C#, or Go can be more manageable. This new form of software engineering enables *prompt engineering* as a valid first phase in software development. Next, the code formatted by the Rust Cargo tool allows a fresh look from a service like Copilot, potentially increasing the quality of the recommendation.

Once that phase completes, the Cargo lint tool Clippy further digs into the quality of the code and enhances it. Finally, running tests ensures the business logic works, and then a last step of compilation makes optimal and safe code.

Now let's go beyond a simple hello-world tool with PyTorch and Hugging Face.

Using PyTorch and Hugging Face with Rust

Hugging Face is an emerging platform for building solutions with LLMs, and it works with Rust and PyTorch. For this example, we put together a

reasonably realistic demo of the type of tool that would be useful using a pretrained summarization model; the example [project lives here](#). The general structure of this demo is as follows. The lyrics for “En El Muelle De San Blas” a song by Maná, are in the *lyrics.txt* file:

```
|— Cargo.toml
|— Makefile
|— README.md
|— lyrics.txt
└— src
    |— lib.rs
    └— main.rs
```

2 directories, 6 files

You can see the general architecture in [Figure 10-4](#). The idea is the performance of Rust alongside an SQLite database, and Hugging Face is a repeatable pattern to analyze song lyrics using a pretrained LLM.

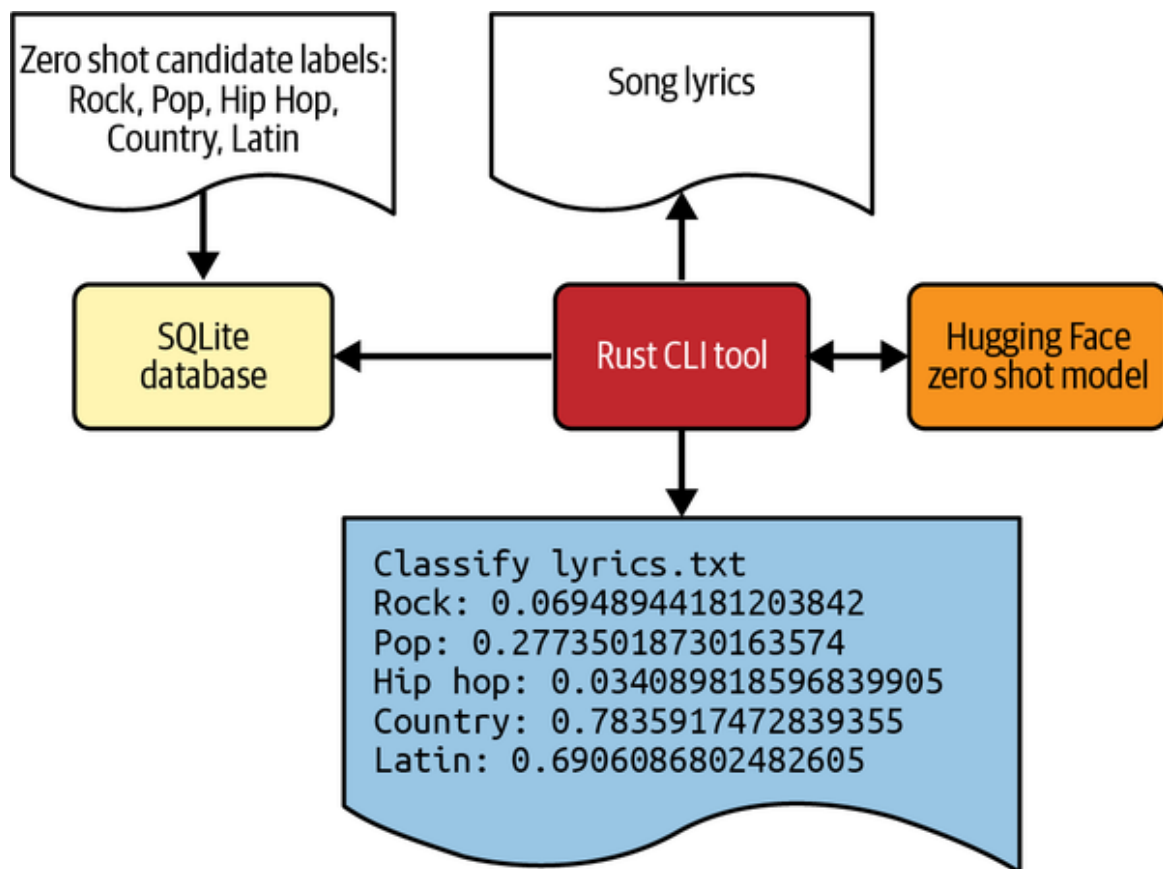


Figure 10-4. Hugging Face PyTorch Zero Shot

A common problem in NLP is building solutions that analyze text. The following example shows how to make a reproducible, efficient, and secure solution to analyze song lyrics:

```
use rust_bert::pipelines::sequence_classification::Label;
use
rust_bert::pipelines::zero_shot_classification::ZeroShotClassificationModel;
use std::fs::File;
use std::io::BufRead;
use std::io::BufReader;

fn create_db() -> sqlite::Connection {
    let db = sqlite::open(":memory:").unwrap();
    db.execute(
        "CREATE TABLE zeroshotcandidates (id INTEGER PRIMARY KEY, label
TEXT)",
    )
    .unwrap();
    ["rock", "pop", "hip hop", "country", "latin"].iter().for_each(|&x| {
        db.execute(&format!(
            "INSERT INTO zeroshotcandidates (label) VALUES ('{}')",
            x
        ))
        .unwrap();
    });
    db
}

pub fn get_all_zeroshotcandidates() -> Vec<String> {
    let db = create_db();
    let mut candidates: Vec<String> = Vec::new();
    db.iterate("SELECT label FROM zeroshotcandidates", |pairs| {
        for &(_column, value) in pairs.iter() {
            candidates.push(value.unwrap().to_string());
        }
        true
    })
    .unwrap();
    candidates
}

pub fn read_lyrics(file: &str) -> Vec<String> {
    let mut lyrics: Vec<String> = Vec::new();
    let file = File::open(file).expect("Unable to open file");
    let reader = BufReader::new(file);
    for line in reader.lines() {
```

```

        lyrics.push(line.unwrap());
    }
    lyrics
}

pub fn classify_lyrics(lyrics: Vec<String>) -> Vec<Vec<Label>> {
    let temp_candidates = get_all_zeroshotcandidates();
    let candidate_labels: Vec<&str> =
        temp_candidates.iter().map(|s| s.as_str()).collect();
    let lyrics_str: String = lyrics.join(" ");
    let lyrics_ref: &str = lyrics.as_ref();
    let zero_shot_model = ZeroShotClassificationModel::new(Default::default())
        .unwrap();
    zero_shot_model.predict_multilabel([lyrics], candidate_labels, None, 128)
}

```

To run everything, you see the different commands exposed via cargo:

```

cargo run -- candidates
    Finished dev [unoptimized + debuginfo] target(s) in 0.13s
    Running `target/debug/sqlitehf candidates`
1
rock
2
pop
3
hip hop
4
country
5
latin
cargo run -- lyrics
    Compiling sqlitehf v0.1.0 (/Users/noahgift/src/rust-mlops-template/sqlite-
hf)
    Finished dev [unoptimized + debuginfo] target(s) in 0.76s
    Running `target/debug/sqlitehf lyrics`
Lyrics lyrics.txt
Uh-uh-uh-uh, uh-uh
Ella despidió a su amor
@noahgift > /workspaces/rust-mlops-template/sqlite-hf (main)
$ cargo run -- classify
    Compiling sqlitehf v0.1.0 (/workspaces/rust-mlops-template/sqlite-hf)
    Finished dev [unoptimized + debuginfo] target(s) in 8.76s
    Running `target/debug/sqlitehf classify`
Classify lyrics.txt
rock: 0.06948944181203842

```

```
pop: 0.27735018730163574
hip hop: 0.034089818596839905
country: 0.7835917472839355
latin: 0.6906086802482605
```

Yet another example of Rust for MLOps is using PyTorch to load pretrained models and create image predictions as shown in the *main.rs* file. The PyTorch bindings are straightforward to turn into a pretrained model tool:

```
/*
Hello world Rust pytorch
Download pretrained model here:
https://github.com/LaurentMazare/tch-rs/releases/download/mw/resnet18.ot
*/

use anyhow::{bail, Result};
use tch::nn::ModuleT;
use tch::vision::{resnet, imagenet};

pub fn main() -> Result<()> {
    let args: Vec<_> = std::env::args().collect();
    let (weights, image) = match args.as_slice() {
        [_, w, i] => (std::path::Path::new(w), i.to_owned()),
        _ => bail!("usage: main resnet18.ot image.jpg"),
    };
    // Load the image file and resize it to the usual imagenet dimension
    // of 224x224.
    let image = imagenet::load_image_and_resize224(image)?;

    // Create the model and load the weights from the file.
    let mut vs = tch::nn::VarStore::new(tch::Device::Cpu);
    let net: Box<dyn ModuleT> = match
        weights.file_name().unwrap().to_str().unwrap() {
        "resnet18.ot" => Box::new(resnet::resnet18(
            &vs.root(), imagenet::CLASS_COUNT)),
        _ => bail!("unknown model, use a weight file named e.g. resnet18.ot"),
    };
    vs.load(weights)?;

    // Apply the forward pass of the model to get the logits.
    let output = net.forward_t(
        &image.unsqueeze(0), /* train= */ false
    ).softmax(-1, tch::Kind::Float); // Convert to probability.

    // Print the top 5 categories for this image.
    for (probability, class) in imagenet::top(&output, 5).iter() {
```

```

        println!("{:50} {:.5.2}%", class, 100.0 * probability)
    }
    Ok(())
}

```

To run this example, do the following. Pretrained model: cd into *pytorch-rust-example* and then run: `cargo run -- resnet18.ot Walking_tiger_female.jpg`. You can see the results in [Figure 10-5](#).

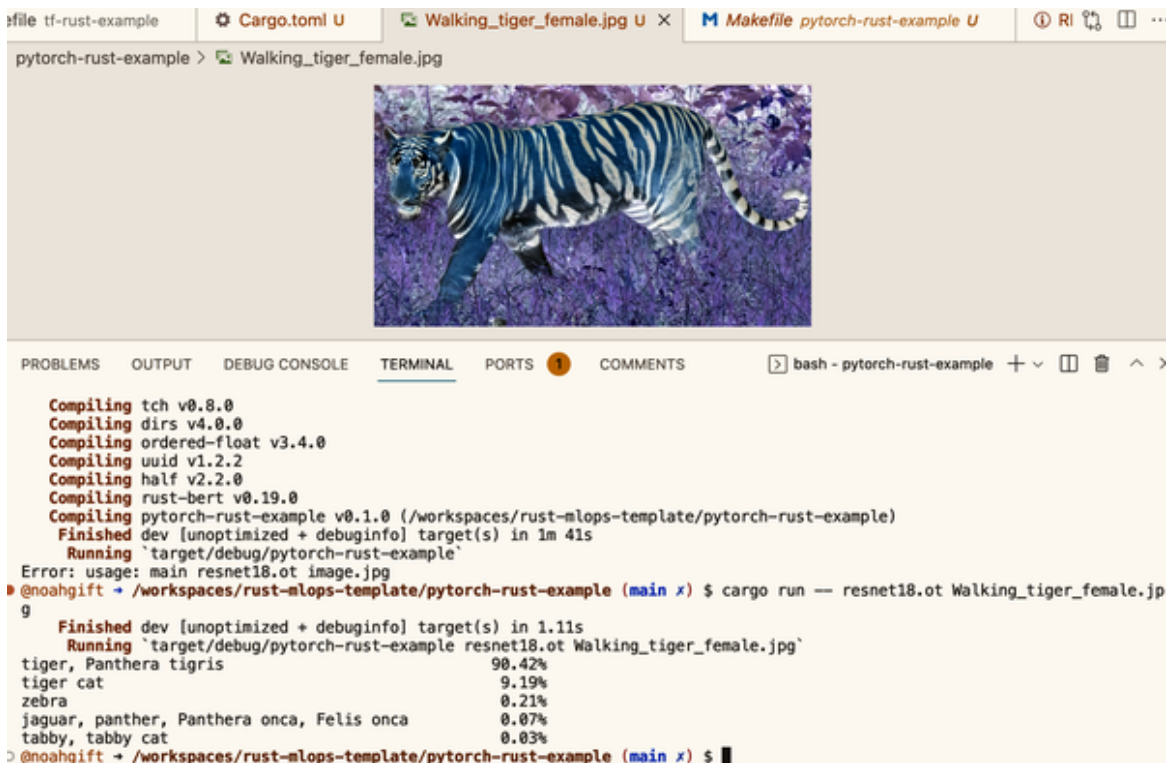


Figure 10-5. PyTorch pretrained model

Using Rust to Build Tools for MLOps

With some solid ideas on using Rust for deep learning, let's look at the ecosystem around Rust as it relates to MLOps.

Building Containerized Rust Command-Line Tools

Another Rust capability is containerizing command-line tools. Let's look at a regular containerized command-line tool and a Rust command-line tool

with PyTorch.

The repo for the project is [here](#). The *lib.rs* file holds a function that returns the string “Polo” if the string “Marco” passes in:

```
/* A Marco Polo game. */

/* Accepts a string with a name.
If the name is "Marco", returns "Polo".
If the name is "any other value", it returns "Marco".
*/
pub fn marco_polo(name: &str) -> String {
    if name == "Marco" {
        "Polo".to_string()
    } else {
        "Marco".to_string()
    }
}
```

To invoke the command-line tool, the same pattern as most examples in this chapter works where the library contains the logic, and the main maps the functions to subcommands, in this case, the subcommand `Play`:

```
//A command-line tool to play Marco Polo
use clap::Parser;

#[derive(Parser)]
#[clap(version = "1.0", author = "Noah Gift", about = "A Marco Polo game")]
struct Cli {
    #[clap(subcommand)]
    command: Option<Commands>,
}

#[derive(Parser)]
enum Commands {
    #[clap(version = "1.0", author = "Noah Gift")]
    Play {
        #[clap(short, long)]
        name: String,
    },
}

fn main() {
    let args = Cli::parse();
```

```

match args.command {
  Some(Commands::Play { name }) => {
    let result = containerized_marco_polo_cli::marco_polo(&name);
    println!("{}", result);
  }
  None => println!("No subcommand was used"),
}
}

```

Finally, the Dockerfile is **tiny** to turn this project into a containerized tool. The first section of the Dockerfile builds the project, and then a smaller container image, `debian:buster-slim` allows for a reduced footprint:

```

FROM Rust:latest as builder
ENV APP containerized_marco_polo_cli
WORKDIR /usr/src/$APP
COPY . .
RUN cargo install --path .

FROM debian:buster-slim
RUN apt-get update && rm -rf /var/lib/apt/lists/*
COPY --from=builder /usr/local/cargo/bin/$APP /usr/local/bin/$APP
ENTRYPOINT [ "/usr/local/bin/containerized_marco_polo_cli" ]

```

Building and running the container is straightforward now:

```

docker build -t marco-polo .
docker run --rm -it marco-polo --help
docker run --rm -it marco-polo play --name Marco
Polo

```

With this knowledge in our toolkit, let's package our previous PyTorch Rust pretrained model into a container. The complete example project lives **here**.

The main relevant addition is the **Dockerfile**, which leverages the existing Cargo ecosystem to install PyTorch:

```

FROM Rust:latest as builder
ENV APP pytorch-rust-docker
WORKDIR /usr/src/$APP
COPY . .
RUN apt-get update && rm -rf /var/lib/apt/lists/*

```



```
RUN cargo install --path .  
RUN cargo build -j 6
```

The steps to invoke this container involve the following commands to build and run it:

```
docker build -t pytorch-rust-docker .  
docker run -it pytorch-rust-docker  
#runs inside of container  
cargo run -- resnet18.ot Walking_tiger_female.jpg
```

GPU PyTorch Workflows

The synergy of performance gains from Rust really shines when a GPU comes into the mix. This [GitHub repository](#) has many repeatable GPU examples thanks to the excellent toolchain from GitHub Codespaces.

As long as the environmental variable export `TORCH_CUDA_VERSION=cu117` exists, cargo builds an NVIDIA CUDA project version. The following Modified National Institute of Standards and Technology (MNIST) database project shows a very brief [training snippet](#):

```
// CNN model. This should reach 99.1% accuracy.  
  
use anyhow::Result;  
use tch::{nn, nn::ModuleT, nn::OptimizerConfig, Device, Tensor};  
  
#[derive(Debug)]  
struct Net {  
    conv1: nn::Conv2D,  
    conv2: nn::Conv2D,  
    fc1: nn::Linear,  
    fc2: nn::Linear,  
}  
  
impl Net {  
    fn new(vs: &nn::Path) -> Net {  
        let conv1 = nn::conv2d(vs, 1, 32, 5, Default::default());  
        let conv2 = nn::conv2d(vs, 32, 64, 5, Default::default());  
        let fc1 = nn::linear(vs, 1024, 1024, Default::default());  
        let fc2 = nn::linear(vs, 1024, 10, Default::default());  
        Net { conv1, conv2, fc1, fc2 }  
    }  
}
```

```

    }
}

impl nn::ModuleT for Net {
    fn forward_t(&self, xs: &Tensor, train: bool) -> Tensor {
        xs.view([-1, 1, 28, 28])
            .apply(&self.conv1)
            .max_pool2d_default(2)
            .apply(&self.conv2)
            .max_pool2d_default(2)
            .view([-1, 1024])
            .apply(&self.fc1)
            .relu()
            .dropout(0.5, train)
            .apply(&self.fc2)
    }
}

pub fn run() -> Result<()> {
    let m = tch::vision::mnist::load_dir("data")?;
    let vs = nn::VarStore::new(Device::cuda_if_available());
    let net = Net::new(&vs.root());
    let mut opt = nn::Adam::default().build(&vs, 1e-4)?;
    for epoch in 1..100 {
        for (bimages, blabels) in m.train_iter(256)
            .shuffle()
            .to_device(vs.device()) {
            let loss = net.forward_t(&bimages, true)
                .cross_entropy_for_logits(&blabels);
            opt.backward_step(&loss);
        }
        let test_accuracy = net
            .batch_accuracy_for_logits(
                &m.test_images, &m.test_labels, vs.device(), 1024);
        println!("epoch: {:4} test acc: {:.5.2}%",
            epoch, 100. * test_accuracy);
    }
    Ok(())
}

```

To run, cd into the *pytorch-mnist* directory and run `cargo run -- conv`. The result shows a lightning fast training of a model as shown in [Figure 10-6](#). The `nvidia-smi -l 1` command enabled GPU monitoring, verifying that the GPU is doing the heavy lifting.

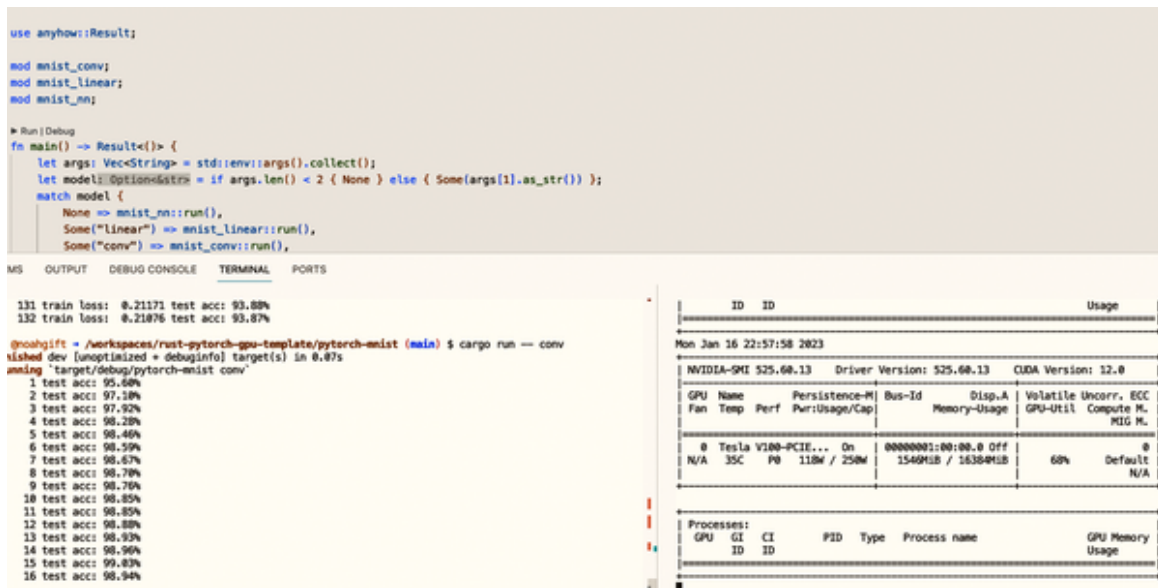


Figure 10-6. MNIST PyTorch GPU saturation

Yet another effective Rust GPU workflow is to run the latest version of Stable Diffusion. First, clone the [repo](#) and then follow the [setup instructions](#).

Next, download the model weights and then run:

```

cargo run --example stable-diffusion --features clap -- --prompt
"A very rusty robot holding a fire torch to notebooks"

```

Yet again, you see the power of Rust and GPU as the GPU gets completely saturated, as shown in [Figure 10-7](#).

The result in [Figure 10-8](#) is a robot picture that can easily integrate into a complex pipeline due to the high-performance nature of Rust.

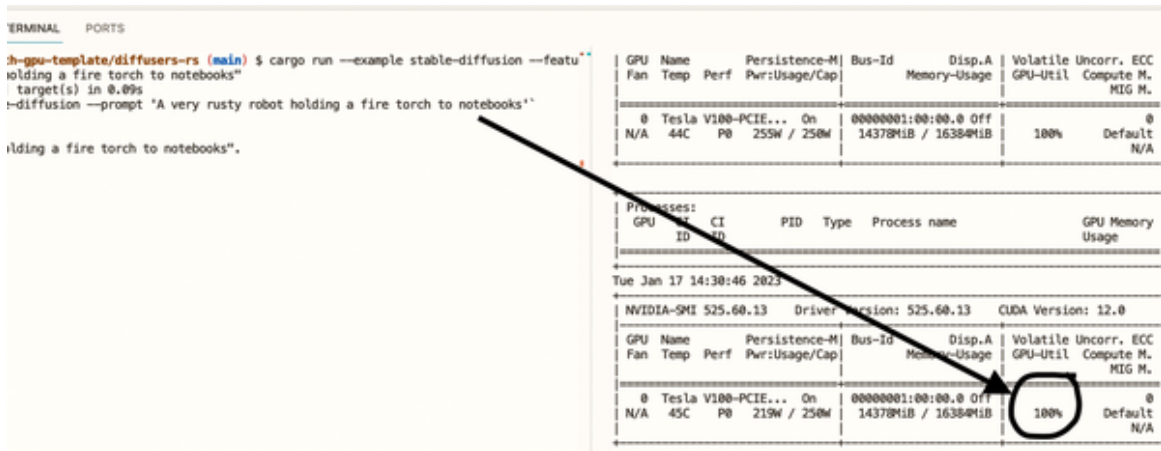


Figure 10-7. Stable Diffusion completely saturating the GPU



Figure 10-8. Rusty Robot with flame about to torch notebook

Using TensorFlow Rust

You don't have to just use PyTorch. To run TensorFlow with Rust, look at this example in [GitHub](#).

First, look at the *Cargo.toml* file. Notice that the tensorflow crate is all that's needed to get started:

```
[package]
name = "tf-rust-cli"
version = "0.1.0"
```

```
edition = "2021"  
[dependencies]  
tensorflow = "0.19.1"
```

Next, in the *main.rs* file, a simple example involves only a few lines of code. To see more ideas, visit the official crate [documentation page](#), including enabling GPU support (something easily accomplished with GitHub Codespaces with GPU):

```
/*Rust TensorFlow Hello World */  
  
extern crate tensorflow;  
use tensorflow::Tensor;  
  
fn main() {  
    let mut x = Tensor::new(&[1]);  
    x[0] = 2i32;  
    //print the value of x  
    println!("{:?}", x[0]);  
    //print the shape of x  
    println!("{:?}", x.shape());  
    //create a multidimensional tensor  
    let mut y = Tensor::new(&[2, 2]);  
    y[0] = 1i32;  
    y[1] = 2i32;  
    y[2] = 3i32;  
    y[3] = 4i32;  
    //print the value of y  
    println!("{:?}", y[0]);  
    //print the shape of y  
    println!("{:?}", y.shape());  
}
```

Doing k-means Clustering with Rust

Rust also contains high-performance scientific libraries for machine learning. An excellent example of a library is [linfa](#). There are many benchmarked examples.

Simple examples show a similar amount of code as machine learning in Python using [scikit-learn](#). A key and critical point from the library author is the statement, “No need to have a second language for performance

reasons,” as in you don’t need C and Python; everything is fast with just Rust. The following example from their website shows how simple and clean the code is:

```
let (train, valid) = linfa_datasets::diabetes()
    .split_with_ratio(0.9);

// train pure LASSO model with 0.1 penalty
let model = ElasticNet::params()
    .penalty(0.1)
    .l1_ratio(1.0)
    .fit(&train)?;

println!("z score: {:?}", model.z_score());

// validate
let y_est = model.predict(&valid);
println!("predicted variance: {}", y_est.r2(&valid)?);
```

A blog post of their k-means example [shows 25 times faster inference](#) than scikit-learn. Noah created a command-line tool example, available in [GitHub](#).

Final Notes on Rust

Let’s discuss a few additional notes on Rust.

Ruff Linter

A new Rust-based linter called [Ruff](#) can significantly decrease the time it takes to lint a codebase. You can see in [Figure 10-9](#) that Ruff gives speed improvements from 12 to 120 times regular Python.

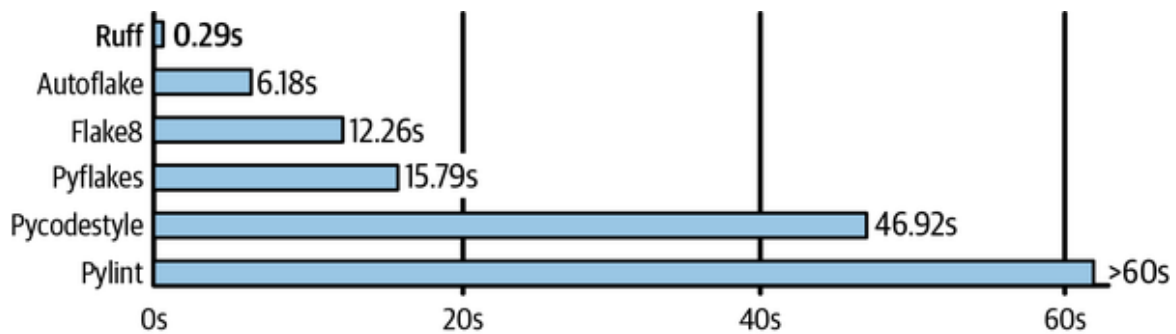


Figure 10-9. Rust-based linter in Python

According to Charlie Marsh, the added speed in linting makes “instant” possible for millions of lines of code. In a [blog post about the tool](#), he wisely says, “Ultimately, my goal with Ruff is to get the Python ecosystem to question the status quo. How long should it take to lint a million lines of code? In my opinion: it should be instant. And if your developer tools were instant, what would that unlock?”

rust-new-project-template

For beginners, a good idea is for Rust projects to follow this pattern:

1. Create a new repo using [Rust New Project Template](#).
2. Create a new Codespace and use it.
3. Use `main.rs` to call the handle CLI and `lib.rs` to handle logic and import `clap` in `Cargo.toml` as shown in this project.
4. Use `cargo init --name 'hello'` or whatever you want to call your project.
5. Put your “ideas” in as comments in Rust to seed GitHub Copilot: for example, `//build an add function`.
6. Run `make format` i.e. `cargo format`.
7. Run `make lint` i.e. `cargo clippy --quiet`.
8. Run `project: cargo run -- --help`.

9. Push your changes to allow GitHub Actions to: format check, lint check, and other actions like binary deploy.

This new emerging pattern is ideal for systems programming in Rust.

Using this [repository](#), you can do the following:

```
cargo run -- marco --name "Marco"
```

Be careful to use the NAME of the project in the Cargo.toml to call lib.rs as in:

```
[package]
name = "hello"
```

For example, see the name hello invoked alongside marco_polo, which is in lib.rs.

lib.rs code:

```
/* A Marco Polo game. */

/* Accepts a string with a name.
If the name is "Marco", returns "Polo".
If the name is "any other value", it returns "Marco".
*/
pub fn marco_polo(name: &str) -> String {
    if name == "Marco" {
        "Polo".to_string()
    } else {
        "Marco".to_string()
    }
}
```

main.rs code:

```
fn main() {
    let args = Cli::parse();
    match args.command {
        Some(Commands::Marco { name }) => {
            println!("{}", hello::marco_polo(&name));
        }
    }
}
```



```
    }  
    None => println!("No command was used"),  
  }  
}
```

Conclusion

The purpose of this chapter has been to critically examine the role of Python as the only option for MLOps. As Dr. Patterson illuminates, we need new languages, new hardware, and new ideas. The practice of MLOps is different from data science. It orients toward software engineering, especially that of distributed computing systems.

As for MLOps with LLMs, deployment of Rust is a big deal. For example, once you use [whisper.py](#) from [OpenAI](#) and see how helpful it is for building accurate transcriptions of audio and video files, the next logical goal is for a user to try to make it go 25X faster. Suddenly, Rust makes a ton of sense; this *is* 25X more performant. The “secret sauce” is GitHub Copilot; it makes Rust syntax a breeze (we cannot overstate this). After all, you don’t want to be the person telling a client or boss “sorry, we cannot go faster,” when you can!

Additionally, the idea that one language, Python, is a panacea for all software engineering problems (even though it is tremendously popular and valuable) is magical thinking. Bolting more and more nonnative components onto Python is a suboptimal strategy versus choosing a new language when appropriate. Additionally, the old paradigm of suggesting people mix C with Python needs to be reevaluated if a developer can replace both with Rust and use one language.

The pragmatic practitioner looks for tools that efficiently solve problems, which can mean having to give up a favorite, familiar tool. Languages like Go and Rust have emerged as solutions for high-performance computing, and Rust, in particular, shines at cybersecurity safety, a weakness of languages like C and Python. A fair question to ask is if learning a new language that is similarly or slightly more complex is worth improvements

in performance, energy efficiency, package deployment, and cybersecurity? Rust makes a compelling case that there is.

In distributed computing, performance matters, as does cybersecurity, energy usage, and binary software distribution. Rust has many compelling use cases for MLOps, and additional examples are in the [Rust MLOp repo](#) as well as a Rust tutorial in the appendix.

Rust combines a low-level language's performance with a high-level language's readability. Its emphasis on memory safety, type safety, and error checking can lead to more robust and secure MLOps pipelines. Furthermore, its interoperability with C and its growing ecosystem make it a strong contender for MLOps tasks that need to handle large volumes of data quickly and securely. As the MLOps field evolves, it will be interesting to see how Rust's role develops.

Remember, the best tool for the job depends on the job at hand. Python has its strengths and will continue to have wide adoption in MLOps. But if you're looking for a language that offers more performance, security, and robustness, Rust is worth a look.

Critical Thinking Discussion Questions

- What issues does Rust solve as a systems programming language that provide an advantage over Python?
- How could inference performance impact choosing Rust as a core component in an MLOps pipeline?
- What advantages are in play in using pretrained models, such as LLMs, versus exclusively training these models yourself? Additionally, how could Rust be an advantage in deploying solutions with LLMs versus Python?
- Two of the most popular deep learning frameworks are TensorFlow and PyTorch. What are the pros and cons of these frameworks from a

sustainability (i.e., energy efficiency) standpoint? What about Rust versus Python?

- How could deploying a statically linked binary solution be advantageous over a scripting language solution?

Exercises

- Build a command-line tool in Rust that uses a PyTorch pretrained model. You can use [this project](#) as a reference.
- Using the [linfa crate](#), build a k-means clustering command-line tool that analyzes a well-known public dataset. You can use [this example](#) as a starting point.
- Build a command-line summarization tool using the [rust-bert Hugging Face bindings](#). You can use [this project](#) as a starting point.
- Deploy a PyTorch pretrained model to AWS Lambda using Rust.
- Run Stable Diffusion in Rust, using [this project](#) as the starting point.

Appendix A. Job Interview Questions

The following section contains questions and answers that may arise during a job interview. *Python for DevOps* (O'Reilly), by Noah Gift, Kennedy Behrman, Alfredo Deza, and Grig Gheorghiu, covers these topics in more detail.

What is the primary purpose of DevOps?

The primary purpose of DevOps is to increase the speed and quality of software development while also reducing costs. DevOps combines software development (Dev) and information technology operations (Ops) to shorten the time it takes to deliver customer features and updates.

What is an excellent example of the fundamental processes necessary to implement MLOps?

An excellent example of the fundamental processes necessary to implement MLOps would be establishing a clear understanding of your organization's goals for using machine learning. Once you understand, you can develop the strategies and infrastructure to support your ML models. This process can include setting up a data pipeline, establishing a model training and testing process, and deploying models into a production environment.

What is a feature store?

A feature store is a database that stores the features machine learning models can use. A feature store stores features extracted from data sources and components generated by feature engineering processes.

What is a Model Registry?

A Model Registry is a database that stores machine learning models: a Model Registry stores models trained on data and models generated by model development processes.

What are the best practices for operationalizing a microservice?

Best practices for operationalizing a microservice include making sure that:

- Your microservice is well-designed and well-tested before deploying it.
- You plan how you will update and deploy your microservice.
- You have a way to roll back changes to your microservice if necessary.
- You have a way to scale your microservice if necessary.
- You have a way to monitor your microservice for errors and performance issues.

What is GitHub Actions, and what are the primary use cases?

GitHub Actions is a tool that allows you to automate your software development workflows. GitHub Actions integrates with GitHub, so you can use it to trigger actions in your workflow when certain events occur, such as when code pushes to a repository. Two primary use cases for GitHub Actions are:

- Continuous integration/continuous delivery. You can use GitHub Actions to build, test, and deploy your code when code pushes to a repository.
- Automated security testing.

What is a data pipeline?

It performs an extract for data, transforms, and then loads to a new destination. A data pipeline is a set of processes that extract data from one or more sources, convert the data into a format that can be used by downstream processes, and load the data into one or more destinations.

What are the primary use cases for Jupyter Notebook?

Jupyter Notebook allows you to create and share documents that contain live code, equations, visualizations, and explanatory text. It is helpful for data analysis, machine learning, and scientific computing.

What is the purpose of linting Python code?

Linting is the process of checking Python code for errors and potential problems. Linting can improve the quality of your code and help find bugs that might otherwise be difficult to find.

Why are cloud-based development environments like GitHub Codespaces and AWS Cloud9 useful?

Cloud-based development environments are helpful because they allow developers to work from anywhere in a uniform environment with deep integration into the deployment environment. Cloud-based development environments also make sharing code and collaborating with other developers easy. Finally, cloud-based development environments can quickly scale up or down to meet the needs of a project.

What is Big O Notation?

Big O Notation is a way to measure the efficiency of an algorithm. Big O Notation is used to describe the worst-case scenario for an algorithm. It is important to note that Big O Notation is not a measure of the actual time or space an algorithm takes but rather a way to compare the efficiency of different algorithms.

What are business use cases for the mathematical field of optimization?

There are many business use cases for the mathematical field of optimization. Some examples include:

- Finding the best route for a delivery driver
- Scheduling employees to minimize overtime

- Determining the most efficient production schedule for a factory
- Planning the layout of a store to maximize customer traffic
- Optimizing a website for search engine ranking

What is the traveling salesman problem?

Given a list of cities and the distances between each pair of cities, the goal is to find the shortest possible route that visits each city and returns to the origin city. The traveling salesman problem is a classic problem in computer science and mathematics.

Describe how the gradient descent algorithm works?

The gradient descent algorithm is an optimization algorithm used to find a function's local minimum. The algorithm works by starting at a random point on the function and then moving in the direction of the gradient (the function's derivative) until it reaches a point where the gradient is zero.

The greedy coin problem is what type of programming problem?

The greedy coin problem is a classic programming problem that can be solved using a greedy algorithm, an algorithm that makes the locally optimal choice at each step to try to optimize the overall goal. In the case of the greedy coin problem, the goal is to minimize the number of coins needed to make a given amount of money.

What are the advantages of containers?

Containers are more efficient than virtual machines because they don't require the overhead of a complete virtualization solution. Containers are portable and run on any platform that supports container technology. Containers provide isolation between applications, so one application cannot interfere with another.

What is an HTTP API?

An HTTP API is an interface for communication between two systems using the Hypertext Transfer Protocol (HTTP). Another way to describe it would be a system that allows two applications to communicate over the internet.

What are the advantages of containerized ML applications?

There are several advantages to containerized machine learning applications, including:

Increased portability and flexibility

Containers can be moved between different environments, making testing and deploying machine learning applications in various settings easy.

Improved resource utilization

Containers allow for more efficient use of resources, like multiple applications running on a single server or cluster of servers.

Isolation and security

Containers isolate applications from each other and the underlying operating system, providing an additional layer of security.

Reduced development and deployment time

Containers are quick to create and deploy, making it possible to iterate rapidly on machine learning applications.

What are the advantages of using ONNX for model interoperability?

There are several advantages to using ONNX for model interoperability:

- Its open standard is supported by many tools and frameworks.

- Models are easily exported from one framework to another, allowing the best tool use for each task.
- Models are portable and deployable on various devices and platforms.
- It provides a consistent interface for model development, to easily switch between frameworks as needs change.

What are the use cases for edge-based machine learning models?

There are many potential use cases for edge-based machine learning models; for example, these models could improve the accuracy of predictions made by IoT devices or provide real-time feedback to users based on their interactions with a system. Additionally, edge-based machine learning models could be used to monitor and optimize the performance of industrial equipment, for example, to automatically detect and diagnose problems with machinery.

What is a Spark Cluster?

A group of machines that work together to run Spark applications. A primary use case for Spark is data analytics on large datasets. Leading platforms include Amazon EMR and Databricks.

What problems does PySpark solve?

PySpark is a Python API for Spark that lets you harness the simplicity of Python and the power of Apache Spark to harness Big Data. PySpark solves the problem of learning multiple languages to work with Big Data. Further, it allows for a more streamlined and efficient workflow than when working with Spark alone, as the power of Python is combined with it.

What are the critical components of the Databricks platform?

The Databricks platform consists of three key components: Databricks Runtime, Databricks Workspace, and Databricks CLI. Databricks Runtime is a managed environment for running Apache Spark applications and includes a version of Spark and all the necessary libraries and dependencies

required to run Spark applications. Features involve creating and running Spark applications and tools for managing and sharing data.

Databricks Workspace is a web-based interface for interacting with Databricks. Databricks CLI is helpful to develop and run Spark applications, as well as to manage and share data.

What are the critical components of MLflow?

The critical components include tracking, model registry, and project format:

Tracking Server

Tracks the experiments and runs performed using MLflow.

Tracking API

Allows MLflow integration with other tools and systems.

Model Registry

Stores and manages models trained using MLflow.

Project Format

Defines the structure of an MLflow project.

What is the critical difference between a Spark DataFrame and a pandas DataFrame?

A Spark DataFrame is a distributed data collection organized into named columns. A pandas DataFrame is a local collection of data collected into named columns. Spark DataFrames can be created from various sources, including Hive tables, Parquet files, and JSON data, while pandas DataFrames can only be from local data.

What is kaizen?

Kaizen is a Japanese business philosophy that emphasizes continuous improvement in all aspects of an organization. Historically, kaizen has been associated with manufacturing and production settings, but the principles apply to any business or organization. The basic idea behind kaizen is that even minor improvements can significantly impact over time and that everyone in an organization should constantly look for ways to improve.

What is a data warehouse?

A data warehouse is a database that stores data for reporting and analysis. Data warehouses store data from multiple sources, such as sales, financial, and customer data. Examples of commercial platforms include Microsoft SQL Server, Oracle, and IBM DB2.

What is a scheduled data pipeline job?

Scheduled data pipeline jobs are jobs that run on a daily, weekly, or monthly schedule. Platforms that support this functionality include cron and Windows Task Scheduler. From a data perspective, these include jobs that extract data from a database or other data source, transform it, and load it into another database or data store.

What is data engineering?

Data engineering is designing, building, and maintaining systems that collect, store, and process data. Examples include data warehouses, data lakes, and data pipelines. Data engineers work with data scientists and other stakeholders to understand the needs of the business and design systems that meet those needs.

What is DataOps?

There is no one-size-fits-all answer to this question, as DataOps can mean different things to different organizations, depending on their specific needs and goals. However, DataOps generally refers to the processes and tools used to manage data throughout its lifecycle, from acquisition and ingestion to storage, processing, and analysis. This process includes operational tasks

such as data backup and recovery and data-driven tasks such as data mining and machine learning.

What is Kubernetes?

Kubernetes is a system for managing containerized applications running in a cluster across a group of servers. It provides a platform for automating deployment, scaling, and operations of application containers across clusters of hosts, providing container-centric infrastructure.

Why are microservices a good fit for Kubernetes?

Kubernetes works well for microservices applications because it supports declarative configuration and self-healing capabilities. Organizations can achieve greater flexibility and scalability in their application development and deployment process by wrapping each microservice in a container and deploying it on a Kubernetes cluster.

What is observability in software engineering?

Observability measures how well external outputs can infer the system's internal state or, put another way, how well a system's internal state presumes from its external outcomes. In software engineering, observability refers to instrumenting code to emit data for monitoring the system's health and performance.

What are the critical components of Kubernetes?

Kubernetes consists of control plane components that contain the cluster and a group of worker nodes that run applications. The control plane components include the API server, scheduler, and controller manager. The worker nodes run the applications and include management by the control plane components.

What is the Kubernetes API?

The Kubernetes API is an API that manages the deployment and scaling of the application. The Kubernetes API is a set of RESTful APIs that expose

the functionality of the Kubernetes system. The API allows clients to interact with the Kubernetes system to manage application deployment and scaling.

What are the core components of a cloud native architecture?

Scalability, elasticity, self-healing, and observability are core components of a cloud native architecture. Scalability is the system's ability to handle the increased load by adding resources. Elasticity is the system's ability to dynamically scale up or down in response to changes in demand. Self-healing is the ability to detect and recover from faults automatically. Observability is the ability to monitor the system's internal state to infer its health and performance.

What are three cloud native data components?

Three cloud native data components include persistence, streaming, and batch. Persistence is the system's ability to store data permanently. Streaming is the system's ability to process data in real time as it is generating. Batch is the system's ability to process data in batches.

What are the common fallacies of distributed computing?

Common fallacies of distributed computing include:

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology doesn't change.
- There is one administrator.
- Transport cost is zero.
- The network is homogeneous, as stated by **Sun Microsystems**.

How do you access network storage in Docker?

You access network storage in Docker using a volume driver, which allows you to connect to network-attached storage or cloud storage.

What is block storage?

Block storage is storage used with an operating system or application data, organized into blocks, the smallest unit of storage accessible. Block storage stores frequently accessed data, such as operating system files or application data.

[OceanofPDF.com](https://oceanofpdf.com)

Appendix B. Enterprise MLOps Interviews

The entire series of [Enterprise MLOps Interviews](#) is available online.

Shubham Saboo and Sandra Kublik

Interview with [Shubham Saboo and Sandra Kublik](#).

Piero Molino

Detailed [conversation about declarative AutoML with Piero Molino](#), author of Ludwig and cofounder of Predibase.

Asaf Somekh

Talk at Duke MIDS MLOps course with Asaf Somekh, CEO of Iguazio, [“Life of a Model: Or the Brutal Reality of Applying ML in Enterprises and How to Deal with It”](#).

Javier Luraschi and Pedro Luraschi

Discussion about [MLOps with Javascript](#), including no-code and low-code approaches and Tensorflow.js, with the cofounders of Hal9.ai.

Malcolm Smith Fraser

Interview with [Malcolm Smith Fraser](#), Duke MIDS alumnus, about [real-world MLOps](#). Also available on [YouTube](#) and on the [“52 Weeks of Cloud”](#) podcast.

Jon Reifschneider

[MLOps interview](#) with [Jon Reifschneider](#), head of the Duke Artificial Intelligence for Product Innovation master’s degree program. Also available on [YouTube](#) or on the [“52 Weeks of Cloud”](#) podcast.

Julien Simon

Discussion with Julien Simon of Hugging Face about the **use of MLOps via pretrained models and how to use MLOps to build and deploy models and create a career**

Shubham Saboo

Live Coding OpenAI with Shubham Saboo covers AWS and OpenAI in Codespaces.

Brian Ray

A discussion of **Enterprise MLOps** with Brian Ray, managing director of Maven Wave an Atos Company.

Simon Stebelena

A discussion of **Enterprise MLOps** with Simon Stebelena, lead MLOps engineer at Transaction Monitoring Netherlands.

Bindu Reddy

Bindu Reddy, the CEO of Abacus AI, talks about the role of MLOps in the AI industry and how to use MLOps to build and deploy models and opportunities in the space.

Dhanasekar Sundararaman

Dhanasekar Sundararaman, Duke PhD in computer engineering and Microsoft researcher, discusses how to process numbers in NLP models and discusses Hugging Face.

Ville Tuulos

Ville Tuulos, the CEO of Outerbounds and the Open Source Framework Metaflow, has a conversation about MLOps, including his time building systems for Netflix.

Lewis Tunstall and Leandro von Werra

Lewis Tunstall and Leandro von Werra of Hugging Face **discuss MLOps with Hugging Face** and how to use MLOps to build and deploy models and create a career.

Arvs Lat

Discussion with Arvs Lat, the author of *Machine Learning Engineering on AWS*.

Julien Simon, Yaron Haviv, and Noah Gift

“How to Easily Deploy Your Hugging Face Model to Production at Scale”, from the MLRun Hugging Face MLOps Seminar.

Nic Stone

Nic Stone, CTO of Crul, **discusses building automated data pipelines**.

Doris Xin

Discussion with Doris Xin, founder of Linea AI.

OceanofPDF.com

Index

A

A/B testing deployment strategy, [Deployment Strategies](#)

advanced data types

- build versus buy for MLOps NLP problems, [Build Versus Buy for MLOps NLP Problems-Exploring NLP with OpenAI](#)
- composite AI, [Composite AI-Use Cases of Composite AI with Serverless](#)
- ML problem framing with time series, [ML Problem Framing with Time Series-Time Series with the GCP BigQuery and SQL](#)
- solutions for, [Solutions for Advanced Data Types-Conclusion](#)
- video analysis, image classification, and generative AI, [Video Analysis, Image Classification, and Generative AI-Image Classification Techniques with CreateML](#)

AI Infrastructure Alliance (AIIA), [MLOps in the Cloud](#)

AI, business transformation by, [Identifying the Business Use Case and Goals-Defining Goals and Evaluating the ROI](#)

AI-powered pair programming assistant, [Implementing MLOps Using Rust algorithms, choosing, Choose Your Algorithm-Choose Your Algorithm](#)

Amazon

- Firecracker project, [In the Beginning Was the Command Line](#)

- revenue from AI-powered recommendations, [Identifying the Business Use Case and Goals](#)

Amazon DynamoDB, [Navigating Time Series Analysis with AWS](#)

Amazon EMR, [Navigating Time Series Analysis with AWS](#)

Amazon Forecast, [Navigating Time Series Analysis with AWS](#)

Amazon Forecast DeepAR+ (see DeepAR+)

Amazon Kinesis Data Analytics, [Navigating Time Series Analysis with AWS](#)

Amazon Kinesis Data Streams, [Navigating Time Series Analysis with AWS](#)

Amazon QuickSight, [Navigating Time Series Analysis with AWS](#)

Amazon SageMaker, [Navigating Time Series Analysis with AWS](#)

- code example for model registration, [SageMaker Example](#)
- model monitoring with, [Amazon SageMaker](#)
- model serving, [Amazon SageMaker-Amazon SageMaker](#)

Amazon SageMaker Ground Truth, [Data Labeling Solutions](#)

Anno-Mage, [Data Labeling Solutions](#)

Apache Airflow, [Batch Data Processing-Batch Data Processing](#)

Apache Beam, [From Model Endpoints to Application Pipelines](#)

- defining sliding window for stream processing, [Stream Processing Frameworks](#)
- implementing scalable application pipelines with, [Apache Beam](#)

Apache Hadoop, [Batch Data Processing](#)

application pipelines, [From Model Endpoints to Application Pipelines-From Model Endpoints to Application Pipelines, Scaling and Productizing Projects](#)

- advanced pipelines, [Advanced Serving and Application Pipelines-Model Optimization and ONNX](#)
- Apache Beam for implementing, [Apache Beam](#)
- AWS Step Functions for implementing, [AWS Step Functions-AWS Step Functions](#)
- batch application pipelines, [Batch application pipelines](#)
- building with Enrichment and Ensemble, [Building an Application Pipeline with Enrichment and Ensemble-Building an Application Pipeline with Enrichment and Ensemble](#)
- custom model serving class, [Defining a Custom Model Serving Class](#)
- defined, [Application Pipeline Development](#)
- deploying and testing, [Deploying and Testing the Real-Time Application Pipeline](#)
- LLM project, [Application and model serving pipeline-Application and model serving pipeline](#)
- ML project example, [Application Pipeline Development-Batch application pipelines](#)
- MLRun Serving Graph, [MLRun serving graphs-MLRun serving graphs](#)
- model optimization and ONNX, [Model Optimization and ONNX](#)
- model routing and ensembles, [Model Routing and Ensembles-Model Routing and Ensembles](#)

- real-time application pipelines, [Real-time application pipelines, Real-Time Application Pipeline-Deploying and Testing the Real-Time Application Pipeline](#)
- scalable pipeline implementation, [Implementing Scalable Application Pipelines-MLRun serving graphs](#)
- testing locally, [Testing the Application Pipeline Locally](#)

auto-logging, [Auto-Logging-Auto-Logging](#)

automator's law, [AutoML](#)

AutoML (Automated Machine Learning), [AutoML-AutoML](#)

- benefits, [AutoML](#)
- key components, [AutoML](#)

AutoMLOps, [Tracking and Comparing Experiment Results](#), MLOps

[Automation: AutoMLOps-Conclusion](#)

- building/running multi-stage workflows, [Building and Running Multi-Stage Workflows-Building and Running Multi-Stage Workflows](#)
- efficient management of computation resources, [Managing Computation Resources Efficiently-Managing Computation Resources Efficiently](#)
- handling training at scale, [Handling Training at Scale](#)
- running/tracking ML jobs using Azure Databricks, [Example: Running and Tracking ML Jobs Using Azure Databricks-Example: Running and Tracking ML Jobs Using Azure Databricks](#)

AWS

- market share, [The Key Players in Cloud Computing](#)

- navigating time series analysis with, [Navigating Time Series Analysis with AWS](#)
- NLP with, [Exploring Natural Language Processing with AWS](#)
- view of cloud computing as it relates to MLOps, [AWS view of cloud computing as it relates to MLOps](#)

AWS CLI, [Navigating Time Series Analysis with AWS](#)

AWS Cloud Shell, [Key Cloud Development Environments](#)

AWS DeepAR+ (see [DeepAR+](#))

AWS Elastic File System (EFS), [Use Cases of Composite AI with Serverless](#)

AWS Lambda

- composite AI with, [Getting Started with Serverless for Composite AI](#)
- network effect and, [Use Cases of Composite AI with Serverless](#)
- [MLOps in the Cloud](#)

AWS Step Functions, [From Model Endpoints to Application Pipelines](#),
[AWS Step Functions](#)

Azure

- market share, [The Key Players in Cloud Computing](#)
- view of cloud computing as it relates to MLOps, [Azure view of cloud computing as it relates to MLOps](#)

Azure Databricks, [Example: Running and Tracking ML Jobs Using Azure Databricks](#)

Azure Machine Learning Studio, [Managing Computation Resources Efficiently](#)-Managing Computation Resources Efficiently

B

bagging, [Model Routing and Ensembles](#)

batch data processing, [Batch Data Processing](#)-Batch Data Processing

Beam (see Apache Beam)

Bernanke, Ben S., [Understanding ROI in Enterprise Solutions](#)

BigQuery, [Time Series with the GCP BigQuery and SQL](#)-Time Series with the GCP BigQuery and SQL

Blue/Green deployment strategy, [Deployment Strategies](#)

boosting, [Model Routing and Ensembles](#)

building a production-grade MLOps project, [Building a Production Grade MLOps Project from A to Z](#)-Conclusion

- automated training/validation pipeline, [Building and Running an Automated Training and Validation Pipeline](#)-Building and Running an Automated Training and Validation Pipeline
- batch data ingestion for tests and training, [Batch data ingestion \(for tests and training\)](#)
- building an application pipeline with Enrichment and Ensemble, [Building an Application Pipeline with Enrichment and Ensemble](#)-Building an Application Pipeline with Enrichment and Ensemble
- CI/CD and continuous operations, [Model Monitoring](#)
- credit transaction dataset, [Preparing the Credit Transaction Dataset](#)-Preparing the Credit Transaction Dataset

- credit transactions data pipeline, [Building the Credit Transactions Data Pipeline \(Feature Set\)-Building the Credit Transactions Data Pipeline \(Feature Set\)](#)
- custom model serving class, [Defining a Custom Model Serving Class](#)
- data ingestion and preparation using a feature store, [Data Ingestion and Preparation Using a Feature Store-Real-time data ingestion \(for production\)](#)
- exploratory data analysis, [Exploratory Data Analysis-Exploratory Data Analysis](#)
- extracting labels and training a model, [Extracting Labels and Training a Model-Extracting Labels and Training a Model](#)
- feature vector creation/evaluation, [Creating and Evaluating a Feature Vector-Creating and Evaluating a Feature Vector](#)
- ingesting data into the feature store, [Ingesting Data into the Feature Store](#)
- interactive data preparation, [Interactive Data Preparation-Extracting Labels and Training a Model](#)
- model monitoring, [Model Monitoring](#)
- model training and validation pipeline, [Model Training and Validation Pipeline-Building and Running an Automated Training and Validation Pipeline](#)
- real-time application pipeline, [Real-Time Application Pipeline-Deploying and Testing the Real-Time Application Pipeline](#)
- real-time data ingestion for production, [Real-time data ingestion \(for production\)](#)
- target labels data pipeline, [Building the Target Labels Data Pipeline \(FeatureSet\)](#)

- user events data pipeline, [Building the User Events Data Pipeline \(FeatureSet\)](#)
- user events dataset, [Preparing the User Events \(Activities\) Dataset](#)

business impact, measurement of, [Measuring the Business Impact](#)

business incentives, [MLOps and Business Incentives](#)

C

C++, [Implementing MLOps Using Rust](#)

Caffe, [Building Scalable Deep Learning and Large Language Model Projects](#)

Canary deployment strategy, [Deployment Strategies](#)

champion/challenger (shadow) deployment strategy, [Deployment Strategies](#)

chatbots, [Identifying the Business Use Case and Goals, Finding the AI Use Case](#)

CI/CD (see continuous integration/continuous delivery)

classification algorithms, [Choose Your Algorithm](#)

cloud

- AWS view of cloud computing as it relates to MLOps, [AWS view of cloud computing as it relates to MLOps-AWS view of cloud computing as it relates to MLOps](#)
- Azure view of cloud computing as it relates to MLOps, [Azure view of cloud computing as it relates to MLOps](#)
- GCP view of cloud computing as it relates to MLOps, [GCP view of cloud computing as it relates to MLOps](#)

- key cloud development environments, [Key Cloud Development Environments-Key Cloud Development Environments](#)
- key players in cloud computing, [The Key Players in Cloud Computing-GCP view of cloud computing as it relates to MLOps](#)
- MLOps in, [MLOps in the Cloud-GCP view of cloud computing as it relates to MLOps](#)

clustering algorithms, [Choose Your Algorithm](#)

Collins, Jim, [Understanding Risk and Uncertainty in the Enterprise](#)

command line interface

- building containerized Rust command-line tools, [Building Containerized Rust Command-Line Tools-Building Containerized Rust Command-Line Tools](#)
- Rust and, [In the Beginning Was the Command Line-In the Beginning Was the Command Line](#)

composite AI, [Composite AI-Use Cases of Composite AI with Serverless](#)

computation resources, efficient management of, [Managing Computation Resources Efficiently-Managing Computation Resources Efficiently](#)

concept drift

- defined, [When to Retrain Your Models](#)
- monitoring, [Monitoring Data and Concept Drift-Monitoring Data and Concept Drift](#)

containerized Rust command-line tools, [Building Containerized Rust Command-Line Tools-Building Containerized Rust Command-Line Tools](#)

continuous integration/continuous delivery (CI/CD)

- building a production-grade MLOps project, [Model Monitoring](#)

- continuously monitoring data and models, [Continuously monitoring data and models](#)
- flow automation and, [Flow Automation \(CI/CD for ML\)-Flow Automation \(CI/CD for ML\)](#)
- integrating with a CI/CD service, [Integrating with a CI/CD service](#)
- MLOps and, [What Is MLOps?](#)

COVID-19, disruption of ML models by, [Model Retraining](#)

CreateML, [Image Classification Techniques with CreateML-Image Classification Techniques with CreateML](#)

customer sentiment analysis, [Finding the AI Use Case](#)

cybersecurity, AI for, [Finding the AI Use Case](#)

D

DALL·2, [Video Analysis, Image Classification, and Generative AI-Video Analysis, Image Classification, and Generative AI](#)

Dask, [Batch Data Processing](#)

data

- sources of, [Working with Data and Feature Stores](#)
- versioning and lineage, [Data Versioning and Lineage-Other Frameworks](#)

data annotation (see data labeling)

data collection/preparation stage of MLOps, [Data Collection and Preparation-Feature Stores](#)

- data exploration/preparation, [Data Exploration and Preparation-Data Exploration and Preparation](#)

- data labeling, [Data Labeling-Data Labeling](#)
- data storage/ingestion, [Data Storage and Ingestion-Data Storage and Ingestion](#)
- feature stores, [Feature Stores-Feature Stores](#)

data drift, [MLOps Versus DevOps](#)

- defined, [When to Retrain Your Models](#)
- monitoring, [Monitoring Data and Concept Drift-Monitoring Data and Concept Drift](#)
- pipelines as way of managing, [Design Your Pipelines](#)

data exploration/preparation, [Data Exploration and Preparation-Data Exploration and Preparation](#)

data gathering, for deep learning projects, [Data Gathering, Labeling, and Monitoring in DL](#)

data ingestion

- batch data ingestion for tests and training, [Batch data ingestion \(for tests and training\)](#)
- credit transactions data pipeline, [Building the Credit Transactions Data Pipeline \(Feature Set\)-Building the Credit Transactions Data Pipeline \(Feature Set\)](#)
- data ingestion and preparation using a feature store, [Data Ingestion and Preparation Using a Feature Store-Real-time data ingestion \(for production\)](#)
- real-time data ingestion for production, [Real-time data ingestion \(for production\)](#)
- target labels data pipeline, [Building the Target Labels Data Pipeline \(FeatureSet\)](#)

- user events data pipeline, [Building the User Events Data Pipeline \(FeatureSet\)](#)

data labeling, [Data Labeling-Data Labeling](#)

- best practices, [Data Labeling Best Practices-Data Labeling Best Practices](#)
- commercial and open-source solutions for, [Data Labeling Solutions-Data Labeling Solutions](#)
- for deep learning projects, [Data Gathering, Labeling, and Monitoring in DL](#)
- pitfalls to avoid, [Data Labeling Pitfalls to Avoid-Data Labeling Pitfalls to Avoid](#)

data lineage, [How It Works-How It Works](#)

data monitoring, [Continuously monitoring data and models, Data and Model Monitoring-Standalone Model Monitoring Solutions](#)

data parallelism, [Distributed Deep Learning](#)

data pipelines, [Scaling and Productizing Projects](#)

- credit transactions data pipeline, [Building the Credit Transactions Data Pipeline \(Feature Set\)-Building the Credit Transactions Data Pipeline \(Feature Set\)](#)
- ML project example, [Data and Model Pipeline Development-Data and Model Pipeline Development](#)
- target labels data pipeline, [Building the Target Labels Data Pipeline \(FeatureSet\)](#)
- user events data pipeline, [Building the User Events Data Pipeline \(FeatureSet\)](#)

data preparation and analysis

- at scale, [Data Preparation and Analysis at Scale-Stream Processing Frameworks](#)
- batch data processing, [Batch Data Processing-Batch Data Processing](#)
- credit transaction dataset, [Preparing the Credit Transaction Dataset-Preparing the Credit Transaction Dataset](#)
- data ingestion and preparation using a feature store, [Data Ingestion and Preparation Using a Feature Store-Real-time data ingestion \(for production\)](#)
- distributed data processing architectures, [Distributed Data Processing Architectures](#)
- extracting labels and training a model, [Extracting Labels and Training a Model-Extracting Labels and Training a Model](#)
- interactive data preparation, [Interactive Data Preparation-Extracting Labels and Training a Model](#)
- interactive data processing, [Interactive Data Processing-Interactive Data Processing](#)
- stream processing, [Stream Processing-Stream Processing](#)
- stream processing frameworks, [Stream Processing Frameworks-Stream Processing Frameworks](#)
- structured/unstructured data transformations, [Structured and Unstructured Data Transformations-Structured and Unstructured Data Transformations](#)
- user events dataset, [Preparing the User Events \(Activities\) Dataset](#)

data processing architectures, [Distributed Data Processing Architectures](#)

data retraining, [Strategies for Data Retraining-Strategies for Data Retraining](#)

data storage/ingestion, [Data Storage and Ingestion](#)-[Data Storage and Ingestion](#)

data tagging (see data labeling)

data transformations, [Structured and Unstructured Data Transformations](#)-[Structured and Unstructured Data Transformations](#)

Data Version Control (DVC), [Data Version Control](#)-[Data Version Control](#)

data versioning

- common ML tools for, [Common ML Data Versioning Tools](#)-[Other Frameworks](#)
- Data Version Control, [Data Version Control](#)-[Data Version Control](#)
- MLflow Tracking, [MLflow Tracking](#)-[MLflow Tracking](#)
- MLRun, [MLRun](#)-[Other Frameworks](#)
- Pachyderm, [Pachyderm](#)-[Pachyderm](#)

deep learning (DL) projects, [Building Scalable Deep Learning and Large Language Model Projects](#)-[Build Versus Buy Deep Learning Models](#)

- build versus buy solutions for training DL models, [Build Versus Buy Deep Learning Models](#)-[Build Versus Buy Deep Learning Models](#)
- data gathering/labeling/monitoring, [Data Gathering, Labeling, and Monitoring in DL](#)-[Monitoring DL Models with Unstructured Data](#)
- distributed deep learning, [Distributed Deep Learning](#)-[Ray](#)
- foundation models/generative AI/LLMs, [Foundation Models](#), [Generative AI](#), [LLMs](#)-[Adding a web interface](#)
- generative AI risks and challenges, [Risks and Challenges with Generative AI](#)-[Risks and Challenges with Generative AI](#)

- MLOps pipelines for efficiently using and customizing LLMs, [MLOps Pipelines for Efficiently Using and Customizing LLMs-MLOps Pipelines for Efficiently Using and Customizing LLMs](#)
- monitoring DL models with unstructured data, [Monitoring DL Models with Unstructured Data](#)

DeepAR+

- time series forecasting with, [Diving into Time Series with DeepAR+-Diving into Time Series with DeepAR+](#)

demand forecasting, AI for, [Finding the AI Use Case](#)

deployment of models and AI applications, [Deployment of Models and AI Applications-Conclusion](#)

- advanced serving/application pipelines, [Advanced Serving and Application Pipelines-Model Optimization and ONNX](#)
- business impact measurement, [Measuring the Business Impact](#)
- data/model monitoring, [Data and Model Monitoring-Standalone Model Monitoring Solutions](#)
- deployment strategies, [Deployment Strategies-Deployment Strategies](#)
- model registry/management, [Model Registry and Management-MLRun Example](#)
- model retraining, [Model Retraining-Model Retraining in the MLOps Pipeline](#)
- model serving, [Model Serving-MLRun Serving](#)

deployment stage of MLOps, [Deployment \(and Online ML Services\)-Online Data Preparation](#)

- from model endpoints to application pipelines, [From Model Endpoints to Application Pipelines-From Model Endpoints to Application](#)

Pipelines

- online data preparation, [Online Data Preparation-Online Data Preparation](#)

development environments, cloud-based, [Key Cloud Development Environments-Key Cloud Development Environments](#)

DevOps, MLOps versus, [MLOps Versus DevOps-MLOps Versus DevOps](#)

Deza, Alfredo, [Managing Computation Resources Efficiently-Managing Computation Resources Efficiently](#)

diffusers-rs project, [In the Beginning Was the Command Line](#)

dimensionality reduction, [Choose Your Algorithm](#)

distributed deep learning, [Distributed Deep Learning-Ray](#)

- Horovod, [Horovod](#)
- Ray, [Ray](#)

distributed training, [Distributed Training and Hyperparameter Optimization](#)

DL projects (see deep learning projects)

doccano, [Data Labeling Solutions](#)

drift (see data drift; concept drift)

DVC (Data Version Control), [Data Version Control-Data Version Control](#)

E

EDA (exploratory data analysis), [Exploratory Data Analysis-Exploratory Data Analysis, Exploratory Data Analysis-Exploratory Data Analysis](#)

EFS (AWS Elastic File System), [Use Cases of Composite AI with Serverless](#)

enterprises

- MLOps in, [MLOps in the Enterprise](#)
- risk/uncertainty in, [Understanding Risk and Uncertainty in the Enterprise-Understanding Risk and Uncertainty in the Enterprise](#)
- ROI in enterprise solutions, [Understanding ROI in Enterprise Solutions-Understanding ROI in Enterprise Solutions](#)
- startup companies versus, [MLOps in the Enterprise](#)

ETL (Extract, Transform, Load), [Batch Data Processing](#)

Excel, [ML Problem Framing with Time Series](#)

experiment tracking, [Experiment Tracking-Experiment Tracking](#)

exploratory data analysis (EDA), [Exploratory Data Analysis-Exploratory Data Analysis, Exploratory Data Analysis-Exploratory Data Analysis](#)

F

Feast, [Using Feast Feature Store-Using Feast Feature Store](#)

feature engineering, [Data Collection and Preparation, Data Exploration and Preparation, Model Development and Training, Deployment \(and Online ML Services\), AutoML, Building and Running Multi-Stage Workflows, Data Ingestion and Preparation Using a Feature Store, Building Scalable Deep Learning and Large Language Model Projects, ML Problem Framing with Time Series](#)

feature stores, [Feature Stores-Feature Stores, Feature Stores-Feature Retrieval \(for Training and Serving\)](#)

- architecture and usage, [Feature Store Architecture and Usage-Feature Store Architecture and Usage](#)
- basics, [Feature Stores](#)

- capabilities, [Feature Stores](#)
- data ingestion and preparation using a feature store, [Data Ingestion and Preparation Using a Feature Store-Real-time data ingestion \(for production\)](#)
- Feast feature store, [Using Feast Feature Store-Using Feast Feature Store](#)
- feature retrieval for training and serving, [Feature Retrieval \(for Training and Serving\)-Feature Retrieval \(for Training and Serving\)](#)
- feature storage, [Feature Storage](#)
- ingesting data into, [Ingesting Data into the Feature Store](#)
- ingestion and transformation service, [Ingestion and Transformation Service-Ingestion and Transformation Service](#)
- MLRun feature store, [Using MLRun Feature Store-Using MLRun Feature Store](#)
- solutions and usage example, [Feature Stores Solutions and Usage Example-Using MLRun Feature Store](#)

feature vectors

- creating and evaluating, [Creating and Evaluating a Feature Vector-Creating and Evaluating a Feature Vector](#)
- feature retrieval with, [Using MLRun Feature Store](#)
- feature stores and, [Feature Stores](#)

Figure Eight, [Data Labeling Solutions](#)

fine tuning, [Foundation Models, Generative AI, LLMs](#)

Firecracker project, [In the Beginning Was the Command Line](#)

foundation models (FMs), [Foundation Models](#), [Generative AI](#), [LLMs-Foundation Models](#), [Generative AI](#), [LLMs](#)

fraud prediction, AI for, [Finding the AI Use Case](#)

G

GCP (see [Google Cloud Platform](#))

GCP BigQuery (see [BigQuery](#))

generative AI, [Video Analysis](#), [Image Classification](#), and [Generative AI-Video Analysis](#), [Image Classification](#), and [Generative AI](#)

- defined, [Foundation Models](#), [Generative AI](#), [LLMs](#)
- risks and challenges, [Risks and Challenges with Generative AI-Risks and Challenges with Generative AI](#)

Gift, Noah, [ML Problem Framing with Time Series](#), [Julien Simon](#), [Yaron Haviv](#), and [Noah Gift](#)

GitHub Actions

- adding to [Azure Machine Learning Studio](#), [Managing Computation Resources Efficiently-Managing Computation Resources Efficiently](#)
- automating CI project tests, [CI/CD and Continuous Operations](#)

GitHub Codespaces, [Key Cloud Development Environments](#), [Getting Started with Rust for MLOps](#)

GitHub Copilot, [The Case for Rust for MLOps](#), [Leveling Up with Rust](#), [GitHub Copilot](#), and [Codespaces-Leveling Up with Rust](#), [GitHub Copilot](#), and [Codespaces](#), [Getting Started with Rust for MLOps](#)

goals, defining/identifying, [Identifying the Business Use Case and Goals-Defining Goals and Evaluating the ROI](#)

[Google Cloud Platform \(GCP\)](#)

- market share, [The Key Players in Cloud Computing](#)
- view of cloud computing as it relates to MLOps, [GCP view of cloud computing as it relates to MLOps](#)

Google Cloud Shell Editor, [Key Cloud Development Environments-Key Cloud Development Environments](#)

Google Vertex AI, [Google Vertex AI](#)

governments, AI use by, [Identifying the Business Use Case and Goals](#)

GPU workflows, [GPU PyTorch Workflows-GPU PyTorch Workflows](#)

Gradio, [Adding a web interface-Adding a web interface](#)

ground truth

- bias in, [Monitoring Model Performance and Accuracy](#)
- defined, [Monitoring Model Performance and Accuracy](#)

H

Hadoop, [Batch Data Processing](#)

hallucination (generative AI risk), [Risks and Challenges with Generative AI](#)

Hasty, [Data Labeling Solutions](#)

Haviv, Yaron, [Julien Simon, Yaron Haviv, and Noah Gift](#)

healthcare, AI and, [Finding the AI Use Case](#)

Horovod, [Horovod](#)

Hugging Face

- building an end-to-end application with, [Building an End-to-End Hugging Face Application-Building an End-to-End Hugging Face Application](#)

- NLP and, [Build Versus Buy: The Hugging Face Approach-Build Versus Buy: The Hugging Face Approach](#)
- Rust and, [Using PyTorch and Hugging Face with Rust-Using PyTorch and Hugging Face with Rust](#)

hybrid cloud deployment, [MLOps in Hybrid Environments](#)

hyperparameter optimization, [Distributed Training and Hyperparameter Optimization-Distributed Training and Hyperparameter Optimization, ML pipelines and hyperparameter optimization](#)

hyperparameter tuning, [Hyperparameter Tuning-Hyperparameter Tuning](#)

I

image classification, [Image Classification Techniques with CreateML-Image Classification Techniques with CreateML](#)

incentives, [MLOps and Business Incentives](#)

inefficient systems, [MLOps: What Is It and Why Do We Need It?](#)

ingestion (see data ingestion)

Intel OpenVINO, [Model Optimization and ONNX](#)

interactive data processing, [Interactive Data Processing-Interactive Data Processing](#)

interviews with enterprise MLOps experts, [Job Interview Questions-What is block storage?](#)

J

job interview questions, [Job Interview Questions-What is block storage?](#)

Jupyter Notebook, [Writing and Maintaining Production ML Code](#)

K

k-means clustering, [Doing k-means Clustering with Rust](#)

kaizen, [What Is MLOps?](#), [Implementing MLOps Using Rust](#)

Keras, [Building Scalable Deep Learning and Large Language Model Projects](#)

Knight, Frank, [Understanding Risk and Uncertainty in the Enterprise](#)

Knightian uncertainty, [Understanding Risk and Uncertainty in the Enterprise](#)

Kubernetes, [Enterprise MLOps Strategy](#), [Model Development and Training](#), [Batch Data Processing](#), [Exercises](#), [Seldon Core-Seldon Core](#), [Data and Model Monitoring](#), [Deployment Strategies](#), [Building and Running an Automated Training and Validation Pipeline](#), [Distributed Deep Learning](#), [Defining an MLRun project and running the pipeline](#), [What is Kubernetes?](#)

Kublik, Sandra, [Shubham Saboo and Sandra Kublik](#)

L

Label Studio, [Data Labeling Solutions](#)

Labelbox, [Data Labeling Solutions](#)

labeling (see data labeling)

Lambda (see AWS Lambda)

language translation, [Exploring Natural Language Processing with AWS-Exploring Natural Language Processing with AWS](#)

large language model (LLM), [Foundation Models](#), [Generative AI](#), [LLMs](#)

- (see also natural language processing [NLP])

- application and model serving pipeline, [Application and model serving pipeline](#)
- build versus buy for MLOps NLP problems, [Build Versus Buy for MLOps NLP Problems-Exploring NLP with OpenAI](#)
- data preparation, [Data preparation](#)
- data preparation and tuning pipeline, [Data preparation and tuning-Defining an MLRun project and running the pipeline](#)
- defining an MLRun project and running the pipeline, [Defining an MLRun project and running the pipeline](#)
- fine-tuning (application example), [Application Example: Fine-Tuning an LLM Model-Adding a web interface](#)
- Hugging Face, [Build Versus Buy: The Hugging Face Approach-Build Versus Buy: The Hugging Face Approach](#)
- MLOps pipelines for efficiently using and customizing LLMs, [MLOps Pipelines for Efficiently Using and Customizing LLMs-MLOps Pipelines for Efficiently Using and Customizing LLMs](#)
- model evaluation, [Model tuning and evaluation](#)
- model tuning, [Model tuning and evaluation](#)
- web interface for, [Adding a web interface-Adding a web interface](#)

Lat, Arvs, [Arvs Lat](#)

LATAM Airlines, [Identifying the Business Use Case and Goals](#)

Luraschi, Javier, [Javier Luraschi and Pedro Luraschi](#)

Luraschi, Pedro, [Javier Luraschi and Pedro Luraschi](#)

MAB (Multi-Armed Bandit) algorithms, [Model Routing and Ensembles](#)
machine learning (ML) (see ML entries)

MapReduce, [Batch Data Processing](#)

marketing, generative AI for, [Finding the AI Use Case](#)

metadata, [Saving Essential Metadata with the Model Artifacts-Saving Essential Metadata with the Model Artifacts](#)

Microsoft Azure (see Azure)

ML engineering, defining, [What Is ML Engineering?](#)

ML jobs

- comparing with MLflow, [Comparing ML Jobs: An Example with MLflow-Comparing ML Jobs: An Example with MLflow](#)
- running/tracking using Azure Databricks, [Example: Running and Tracking ML Jobs Using Azure Databricks-Example: Running and Tracking ML Jobs Using Azure Databricks](#)
- running/tracking/comparing, [Running, Tracking, and Comparing ML Jobs-Auto-Logging](#)

ML project, [Getting Started with Your First MLOps Project-Conclusion](#)

- application pipeline development, [Application Pipeline Development-Batch application pipelines](#)
- approving/prototyping, [Approving and Prototyping the Project-Approving and Prototyping the Project](#)
- CI/CD and continuous operations, [CI/CD and Continuous Operations-Integrating with a CI/CD service](#)
- data and model pipeline development, [Data and Model Pipeline Development-Data and Model Pipeline Development](#)

- defining goals and evaluating ROI, [Defining Goals and Evaluating the ROI](#)
- exploratory data analysis, [Exploratory Data Analysis-Exploratory Data Analysis](#)
- finding the AI use case, [Finding the AI Use Case-Finding the AI Use Case](#)
- how to build a successful ML project, [How to Build a Successful ML Project-Project Structure and Lifecycle](#)
- identifying business use case and goals, [Identifying the Business Use Case and Goals-Defining Goals and Evaluating the ROI](#)
- ML pipelines and hyperparameter optimization, [ML pipelines and hyperparameter optimization](#)
- project structure/lifecycle, [Project Structure and Lifecycle-Project Structure and Lifecycle](#)
- scaling/productizing projects, [Scaling and Productizing Projects-Scaling and Productizing Projects, Scaling and Productizing the Project-ML pipelines and hyperparameter optimization](#)
- steps in development of, [ML Project Example from A to Z-Integrating with a CI/CD service](#)

MLflow

- code example for model registration, [MLflow Example](#)
- comparing ML jobs with MLflow, [Comparing ML Jobs: An Example with MLflow-Comparing ML Jobs: An Example with MLflow](#)

MLflow Tracking, [MLflow Tracking-MLflow Tracking](#)

MLOps (generally)

- anti-patterns, [What Isn't MLOps?](#)

- automation with AutoMLOps, [MLOps Automation: AutoMLOps-Conclusion](#)
- basics, [What Is MLOps?-What Isn't MLOps?](#)
- building a production-grade project (see building a production-grade project)
- business incentives and, [MLOps and Business Incentives](#)
- defined, [What Is MLOps?](#)
- DevOps versus, [MLOps Versus DevOps-MLOps Versus DevOps](#)
- enterprise strategy, [Enterprise MLOps Strategy-Enterprise MLOps Strategy](#)
- hybrid cloud deployment, [MLOps in Hybrid Environments](#)
- implementing using Rust (see Rust)
- in the cloud, [MLOps in the Cloud-GCP view of cloud computing as it relates to MLOps](#)
- in the enterprise, [MLOps in the Enterprise](#)
- mainstream definitions, [Mainstream Definitions of MLOps-MLOps and Business Incentives](#)
- ML engineering, [What Is ML Engineering?](#)
- on-premise solutions, [MLOps On-Premises](#)
- project example (see ML project)
- risk and uncertainty in the enterprise, [Understanding Risk and Uncertainty in the Enterprise-Understanding Risk and Uncertainty in the Enterprise](#)
- ROI in enterprise solutions, [Understanding ROI in Enterprise Solutions-Understanding ROI in Enterprise Solutions](#)

- stages of (see stages of MLOps)

MLOps pipelines

- designing, [Design Your Pipelines](#)
- for efficiently using and customizing LLMs, [MLOps Pipelines for Efficiently Using and Customizing LLMs-MLOps Pipelines for Efficiently Using and Customizing LLMs](#)
- model retraining in, [Model Retraining in the MLOps Pipeline](#)

MLRun, [MLRun-Other Frameworks](#)

- automated training/validation pipeline, [Building and Running an Automated Training and Validation Pipeline-Building and Running an Automated Training and Validation Pipeline](#)
- CI and, [CI/CD and Continuous Operations](#)
- code example for model registration, [MLRun Example-MLRun Example](#)
- data ingestion and preparation using a feature store, [Data Ingestion and Preparation Using a Feature Store-Real-time data ingestion \(for production\)](#)
- defining a project/running the pipeline, [Defining an MLRun project and running the pipeline-Defining an MLRun project and running the pipeline](#)
- feature store, [Using MLRun Feature Store-Using MLRun Feature Store](#)
- model monitoring, [MLRun, Model Monitoring](#)
- model serving, [MLRun Serving-MLRun Serving](#)
- model training and validation pipeline construction, [Model Training and Validation Pipeline-Building and Running an Automated Training](#)

and Validation Pipeline

- real-time application pipeline construction, [Real-Time Application Pipeline-Deploying and Testing the Real-Time Application Pipeline](#)

MLRun Serving Graph, [From Model Endpoints to Application Pipelines](#),
[MLRun serving graphs-MLRun serving graphs](#)

model accuracy monitoring, [Monitoring Model Performance and Accuracy-Monitoring Model Performance and Accuracy](#)

model behavior, [Data Collection and Preparation](#), [Build Versus Buy Deep Learning Models](#), [Risks and Challenges with Generative AI](#), [Time Series with the GCP BigQuery and SQL](#)

model deployment, [GCP view of cloud computing as it relates to MLOps](#), [AutoML](#), [Building and Running Multi-Stage Workflows](#), [Model Registry and Management](#), [Model Serving](#), [Seldon Core](#), [Deployment Strategies](#)

model development pipeline, [Scaling and Productizing Projects](#)

model development/training, [Model Development and Training-Building and Testing Models for Production](#)

- (see also production, developing ML models for)
- as stage of MLOps, [Model Development and Training-Building and Testing Models for Production](#)
- building/testing models for production, [Building and Testing Models for Production-Building and Testing Models for Production](#)
- distributed training and hyperparameter optimization, [Distributed Training and Hyperparameter Optimization-Distributed Training and Hyperparameter Optimization](#)
- tracking/comparing experiment results, [Tracking and Comparing Experiment Results-Tracking and Comparing Experiment Results](#)

- writing/maintaining production ML code, [Writing and Maintaining Production ML Code](#)-Writing and Maintaining Production ML Code

model ensemble, [Model Routing and Ensembles](#)

model monitoring, [Continuously monitoring data and models](#), [Data and Model Monitoring](#)-Standalone Model Monitoring Solutions, [Model Monitoring](#)

- Amazon SageMaker, [Amazon SageMaker](#)
- Google Vertex AI, [Google Vertex AI](#)
- integrated solutions, [Integrated Model Monitoring Solutions](#)-MLRun
- MLRun, [MLRun](#)
- standalone solutions, [Standalone Model Monitoring Solutions](#)-[Standalone Model Monitoring Solutions](#)

model optimization, [Model Optimization and ONNX](#)

model parallelism, [Distributed Deep Learning](#)

model performance monitoring, [Monitoring Model Performance and Accuracy](#)-Monitoring Model Performance and Accuracy

model registry, [Model Registry and Management](#)-MLRun Example

- defined, [Model Registry and Management](#)
- MLflow example, [MLflow Example](#)
- MLRun example, [MLRun Example](#)-MLRun Example
- SageMaker example, [SageMaker Example](#)
- solution examples, [Solution Examples](#)-MLRun Example

model retraining, [Model Retraining](#)-Model Retraining in the MLOps Pipeline

- in the MLOps pipeline, [Model Retraining in the MLOps Pipeline](#)
- strategies for data retraining, [Strategies for Data Retraining-Strategies for Data Retraining](#)
- when to retrain, [When to Retrain Your Models-When to Retrain Your Models](#)

model serving

- advanced serving/application pipelines, [Advanced Serving and Application Pipelines-Model Optimization and ONNX](#)
- MLRun Serving Graph, [MLRun serving graphs-MLRun serving graphs](#)
- model retraining, [Model Retraining-Model Retraining in the MLOps Pipeline](#)
- model routing and ensembles, [Model Routing and Ensembles-Model Routing and Ensembles](#)
- optimization and ONNX, [Model Optimization and ONNX](#)

model serving endpoints, [From Model Endpoints to Application Pipelines-From Model Endpoints to Application Pipelines](#)

model serving frameworks, [Model Serving-MLRun Serving](#)

- Amazon SageMaker, [Amazon SageMaker-Amazon SageMaker](#)
- MLRun, [MLRun Serving-MLRun Serving](#)
- Seldon Core, [Seldon Core-Seldon Core](#)

model serving pipeline, [Application and model serving pipeline-Application and model serving pipeline](#)

model training

- automated pipeline for, [Building and Running an Automated Training and Validation Pipeline](#)-Building and Running an Automated Training and Validation Pipeline
- feature vector creation/evaluation, [Creating and Evaluating a Feature Vector](#)-Creating and Evaluating a Feature Vector
- handling training at scale using AutoMLOps, [Handling Training at Scale](#)
- hyperparameter tuning, [Hyperparameter Tuning](#)-Hyperparameter Tuning
- model training and validation pipeline, [Model Training and Validation Pipeline](#)-Building and Running an Automated Training and Validation Pipeline
- retraining, [Model Retraining](#)-Model Retraining in the MLOps Pipeline

model validation pipeline, [Model Training and Validation Pipeline](#)-Building and Running an Automated Training and Validation Pipeline

models, pipelines versus, [Design Your Pipelines](#)

Molino, Piero, [Piero Molino](#)

monitoring

- continuous model/data monitoring stage of MLOps, [Continuous Model and Data Monitoring](#)-Monitoring Model Performance and Accuracy
- data/concept drift, [Monitoring Data and Concept Drift](#)-Monitoring Data and Concept Drift
- data/model monitoring, [Data and Model Monitoring](#)-Standalone Model Monitoring Solutions
- DL models with unstructured data, [Monitoring DL Models with Unstructured Data](#)

- integrated model monitoring solutions, [Integrated Model Monitoring Solutions-MLRun](#)
- model performance/accuracy, [Monitoring Model Performance and Accuracy-Monitoring Model Performance and Accuracy](#)
- standalone model monitoring solutions, [Standalone Model Monitoring Solutions-Standalone Model Monitoring Solutions](#)

Multi-Armed Bandit (MAB) algorithms, [Model Routing and Ensembles](#)

N

natural language processing (NLP), [Data Gathering, Labeling, and Monitoring in DL](#)

- (see also large language model [LLM])
- AWS, [Exploring Natural Language Processing with AWS-Exploring Natural Language Processing with AWS](#)
- build versus buy for MLOps NLP problems, [Build Versus Buy for MLOps NLP Problems-Exploring NLP with OpenAI](#)
- data labeling in, [Data Gathering, Labeling, and Monitoring in DL](#)
- Hugging Face, [Build Versus Buy: The Hugging Face Approach-Build Versus Buy: The Hugging Face Approach](#)
- OpenAI, [Exploring NLP with OpenAI-Exploring NLP with OpenAI](#)

“naughty child” data drift problem, [MLOps Versus DevOps](#)

negative externalities, [MLOps and Business Incentives](#)

Netflix, [Identifying the Business Use Case and Goals](#)

nonprofits, AI use by, [Identifying the Business Use Case and Goals](#)

Nuclio, [MLRun Serving, MLRun serving graphs](#)

NVIDIA TensorRT, [Model Optimization and ONNX](#)

O

Ojeda, Miguel, [In the Beginning Was the Command Line](#)

on-premise MLOps solutions, [MLOps On-Premises](#)

online data preparation, [Online Data Preparation-Online Data Preparation](#)

online ML services, [Deployment \(and Online ML Services\)-Online Data Preparation](#)

ONNX, [Model Optimization and ONNX](#)

ONNX Runtime, [Model Optimization and ONNX](#)

open source solution (see Anno-Mage) (see Apache Airflow) (see Apache Beam) (see Apache Hadoop) (see Caffe) (see Dask) (see doccano) (see DVC (Data Version Control)) (see Feast) (see Firecracker project) (see GitHub Actions) (see Horovod) (see Hugging Face) (see Keras) (see Label Studio) (see MLflow) (see OpenLabeling) (see Pachyderm) (see PyTorch) (see Seldon Core) (see Spark) (see TensorFlow) (see Trino) (see VGG Image Annotator)

OpenAI

- exploring NLP with, [Exploring NLP with OpenAI-Exploring NLP with OpenAI](#)
- video analysis, image classification, and generative AI, [Video Analysis, Image Classification, and Generative AI-Image Classification Techniques with CreateML](#)

OpenLabeling, [Data Labeling Solutions](#)

overfitting, [Choose Your Algorithm, AutoML, Data Labeling Pitfalls to Avoid](#)

P

Pachyderm, [Pachyderm-Pachyderm](#)

pipelines (see specific types, e.g. application pipelines)

Porter, Michael, [The Strategy of Pretrained Models](#)

predictive maintenance, AI for, [Finding the AI Use Case](#)

PrestoDB, [Interactive Data Processing](#)

pretrained models

- building an end-to-end Hugging Face application, [Building an End-to-End Hugging Face Application-Building an End-to-End Hugging Face Application](#)
- strategy of, [The Strategy of Pretrained Models-Building an End-to-End Hugging Face Application](#)

price optimization, AI for, [Finding the AI Use Case](#)

production, developing ML models for, [Developing Models for Production-Conclusion](#)

- auto-logging, [Auto-Logging-Auto-Logging](#)
- AutoML, [AutoML-AutoML](#)
- AutoMLOps, [MLOps Automation: AutoMLOps-Managing Computation Resources Efficiently](#)
- comparing ML jobs with MLflow, [Comparing ML Jobs: An Example with MLflow-Comparing ML Jobs: An Example with MLflow](#)
- experiment tracking, [Experiment Tracking-Experiment Tracking](#)
- GitHub Actions with Azure Machine Learning Studio, [Managing Computation Resources Efficiently-Managing Computation Resources Efficiently](#)

- hyperparameter tuning, [Hyperparameter Tuning-Hyperparameter Tuning](#)
- running/tracking/comparing ML jobs, [Running, Tracking, and Comparing ML Jobs-Auto-Logging](#)
- saving essential metadata with the model artifacts, [Saving Essential Metadata with the Model Artifacts-Saving Essential Metadata with the Model Artifacts](#)

production-grade project (see building a production-grade MLOps project)

project example, MLOps, [Getting Started with Your First MLOps Project-Conclusion](#)

prompt engineering, [Foundation Models, Generative AI, LLMs](#)

Python, Rust versus, [Implementing MLOps Using Rust, The Case for Rust for MLOps, Leveling Up with Rust, GitHub Copilot, and Codespaces, Getting Started with Rust for MLOps](#)

PyTorch, [Building Scalable Deep Learning and Large Language Model Projects](#)

- Rust and, [Using PyTorch and Hugging Face with Rust-Using PyTorch and Hugging Face with Rust, Using PyTorch and Hugging Face with Rust](#)
- Rust GPU PyTorch workflows, [GPU PyTorch Workflows-GPU PyTorch Workflows](#)

R

Ray, [Ray](#)

Ray, Brian, [Brian Ray](#)

real-time application pipelines, [Real-time application pipelines](#)

real-time data pipelines, [Data Collection and Preparation](#)

recommendation systems, [Choose Your Algorithm](#), [Finding the AI Use Case](#)

Reddy, Bindu, [Bindu Reddy](#)

regression algorithms, [Choose Your Algorithm](#)

Reifschneider, Jon, [Jon Reifschneider](#)

resource optimization, [Finding the AI Use Case](#)

retraining (see data retraining; model retraining)

risk

- in startups, [MLOps in the Enterprise](#)
- in the enterprise, [Understanding Risk and Uncertainty in the Enterprise-Understanding Risk and Uncertainty in the Enterprise](#)

ROI (return on investment)

- in enterprise solutions, [Understanding ROI in Enterprise Solutions-Understanding ROI in Enterprise Solutions](#)
- evaluating for business use case, [Defining Goals and Evaluating the ROI-Defining Goals and Evaluating the ROI](#)
- hiring of data scientists and, [MLOps and Business Incentives](#)

Ruff (Rust-based linter), [Ruff Linter](#)

Rust, [Implementing MLOps Using Rust-Conclusion](#)

- building containerized Rust command-line tools, [Building Containerized Rust Command-Line Tools-Building Containerized Rust Command-Line Tools](#)

- building tools for MLOps with, [Using Rust to Build Tools for MLOps-Doing k-means Clustering with Rust](#)
- command line and, [In the Beginning Was the Command Line-In the Beginning Was the Command Line](#)
- getting started with, [Getting Started with Rust for MLOps-Getting Started with Rust for MLOps](#)
- GPU PyTorch workflows, [GPU PyTorch Workflows-GPU PyTorch Workflows](#)
- k-means clustering with, [Doing k-means Clustering with Rust](#)
- leveling up with Rust, GitHub Copilot, and Codespaces, [Leveling Up with Rust, GitHub Copilot, and Codespaces-Leveling Up with Rust, GitHub Copilot, and Codespaces](#)
- potential benefits for MLOps, [Leveling Up with Rust, GitHub Copilot, and Codespaces](#)
- PyTorch and Hugging Face with, [Using PyTorch and Hugging Face with Rust-Using PyTorch and Hugging Face with Rust](#)
- reasons to use for MLOps, [The Case for Rust for MLOps-In the Beginning Was the Command Line](#)
- Ruff linter, [Ruff Linter](#)
- rust-new-project-template, [rust-new-project-template](#)
- TensorFlow with, [Using TensorFlow Rust](#)

S

Saboo, Shubham, [Shubham Saboo and Sandra Kublik, Shubham Saboo SageMaker](#) (see Amazon SageMaker)

sales forecasting, AI for, [Finding the AI Use Case](#)

Scale AI, [Data Labeling Solutions](#)

scaling ML projects, [Scaling and Productizing Projects-Scaling and Productizing Projects](#), [Scaling and Productizing the Project-ML pipelines and hyperparameter optimization](#)

- adding unit tests, [Adding tests](#)
- ML pipelines and hyperparameter optimization, [ML pipelines and hyperparameter optimization](#)

Seldon Core, [From Model Endpoints to Application Pipelines](#), Seldon Core-
[Seldon Core](#)

sentiment analysis, [Finding the AI Use Case](#)

serverless architecture, [Getting Started with Serverless for Composite AI-Use Cases of Composite AI with Serverless](#)

serving class, defining, [Defining a Custom Model Serving Class](#)

shadow (champion/challenger) deployment strategy, [Deployment Strategies](#)

Simon, Julien, [Julien Simon](#), Julien Simon, Yaron Haviv, and Noah Gift

Smith Fraser, Malcolm, [Malcolm Smith Fraser](#)

social media influence patterns, [ML Problem Framing with Time Series](#)

Somekh, Asaf, [Asaf Somekh](#)

Spark, [Batch Data Processing](#)

Spitznagel, Mark, [Understanding Risk and Uncertainty in the Enterprise](#)

SQL, time series analysis with, [Time Series with the GCP BigQuery and SQL-Time Series with the GCP BigQuery and SQL](#)

Stable Diffusion, [In the Beginning Was the Command Line](#)

stages of MLOps, [The Stages of MLOps-Flow Automation \(CI/CD for ML\)](#)

- algorithm choice, [Choose Your Algorithm-Choose Your Algorithm](#)
- continuous model/data monitoring, [Continuous Model and Data Monitoring-Monitoring Model Performance and Accuracy](#)
- data collection/preparation, [Data Collection and Preparation-Feature Stores](#)
- deployment and online ML services, [Deployment \(and Online ML Services\)-Online Data Preparation](#)
- first step, [Getting Started](#)
- flow automation (CI/CD for ML), [Flow Automation \(CI/CD for ML\)-Flow Automation \(CI/CD for ML\)](#)
- getting started, [Getting Started-Design Your Pipelines](#)
- model development/training, [Model Development and Training-Building and Testing Models for Production](#)
- pipeline design, [Design Your Pipelines](#)
- strategy of pretrained models, [The Strategy of Pretrained Models-Building an End-to-End Hugging Face Application](#)

startup companies, enterprise companies versus, [MLOps in the Enterprise](#)
 Stebelena, Simon, [Simon Stebelena](#)

Stephens-Davidowitz, Seth, [MLOps: What Is It and Why Do We Need It?](#)

Stephenson, Neal, [In the Beginning Was the Command Line](#)

Stone, Nic, [Nic Stone](#)

stream processing, [Data Storage and Ingestion, Online Data Preparation, Stream Processing-Stream Processing](#)

- basics, [Stream Processing-Stream Processing](#)

- defined, [Data Collection and Preparation](#)
- frameworks, [Stream Processing Frameworks-Stream Processing Frameworks](#)

structured data

- storage, [Data Storage and Ingestion](#)
- transformations, [Structured and Unstructured Data Transformations](#)

Sundararaman, Dhanasekar, [Dhanasekar Sundararaman](#)

SuperAnnotate, [Data Labeling Solutions](#)

supervised learning, algorithm choice for, [Choose Your Algorithm](#)

sustainability, AI and, [Identifying the Business Use Case and Goals](#)

systems, [MLOps: What Is It and Why Do We Need It?](#)

T

Taleb, Nassim, [Understanding ROI in Enterprise Solutions, MLOps Versus DevOps](#)

TensorFlow, [Building Scalable Deep Learning and Large Language Model Projects, Using TensorFlow Rust](#)

time series analysis, [Time Series with the GCP BigQuery and SQL-Time Series with the GCP BigQuery and SQL](#)

time series data

- forecasting with DeepAR+, [Diving into Time Series with DeepAR+-Diving into Time Series with DeepAR+](#)
- ML problem framing with, [ML Problem Framing with Time Series-Time Series with the GCP BigQuery and SQL](#)

- navigating time series analysis with AWS, [Navigating Time Series Analysis with AWS](#)-[Navigating Time Series Analysis with AWS](#)

Torvalds, Linus, [In the Beginning Was the Command Line](#)

training (see model training)

training vectors (see feature vectors)

transfer learning, [Model Retraining in the MLOps Pipeline](#), [Build Versus Buy Deep Learning Models](#), [Foundation Models](#), [Generative AI](#), [LLMs](#), [MLOps Pipelines for Efficiently Using and Customizing LLMs](#)

translation, [Exploring Natural Language Processing with AWS](#)-[Exploring Natural Language Processing with AWS](#)

Trino, [Interactive Data Processing](#)

Tunstall, Lewis, [Lewis Tunstall and Leandro von Werra](#)

Tuulos, Ville, [Ville Tuulos](#)

U

uncertainty in the enterprise, [Understanding Risk and Uncertainty in the Enterprise](#)-[Understanding Risk and Uncertainty in the Enterprise](#)

unit tests, [Adding tests](#)

unstructured data

- labeling, [Data Labeling](#)
- monitoring DL models with, [Monitoring DL Models with Unstructured Data](#)
- storage, [Data Storage and Ingestion](#)
- transformations, [Structured and Unstructured Data Transformations](#)

unsupervised learning, algorithms for, [Choose Your Algorithm](#)

V

versioning (see data versioning)

VGG Image Annotator (VIA), [Data Labeling Solutions](#)

virtual drift (see data drift)

von Werra, Leandro, [Lewis Tunstall and Leandro von Werra](#)

W

workflows, multi-stage, [Building and Running Multi-Stage Workflows-Building and Running Multi-Stage Workflows](#)

X

Xin, Doris, [Doris Xin](#)

[OceanofPDF.com](#)

About the Authors

Yaron Haviv is a serial entrepreneur who has been applying his deep technological experience in data, cloud, AI, and networking to leading startups and enterprise companies since the late 1990s. As the cofounder and CTO of Iguazio, Yaron drives the strategy for the company's data science platform and leads the shift toward real-time AI. He also initiated and built Nuclio, a leading open source serverless platform with over 4,000 Github stars, and MLRun, Iguazio's open source MLOps orchestration framework.

You can follow Yaron's work through his LinkedIn profile:

[linkedin.com/in/yaronh](https://www.linkedin.com/in/yaronh) and on Twitter: @yaronhaviv.

Noah Gift is the founder of Pragmatic A.I. Labs. He lectures at MSDS, Northwestern, Duke MIDS Graduate Data Science Program, the Graduate Data Science program at UC Berkeley, the UC Davis Graduate School of Management MSBA program, UNC Charlotte Data Science Initiative, and University of Tennessee (as part of the Tennessee Digital Jobs Factory). He teaches and designs graduate machine learning, MLOps, AI, and data science courses, and consults on machine learning and cloud architecture for students and faculty. These responsibilities include leading a multicloud certification initiative for students.

You can follow Noah's work through his LinkedIn profile:

[linkedin.com/in/noahgift](https://www.linkedin.com/in/noahgift) and website: noahgift.com.

OceanofPDF.com

Colophon

The animal on the cover of *Implementing MLOps in the Enterprise* is an Indian hog deer (*Axis porcinus*). This deer gets its name from the way it runs through the forest with its head held low like a hog, so that it can duck under obstacles instead of leaping over them as most deer do.

Indian hog deer live in tall grasslands near rivers. They graze on grasses, flowers, and fruits. They are mostly solitary animals, but have been observed grazing in groups of up to 80 deer. Females have one to two fawns per year. The fawns are weaned at six months and reach maturity within their first year. The longest observed lifespan of a hog deer is 20 years old.

Hog deer are currently found across Pakistan, India, Nepal, Bhutan, Bangladesh, and Cambodia. Over the last few decades, the hog deer population has been rapidly declining. They are considered extinct in several countries in which they once lived. They are hunted for food and trophies and face habitat loss due to human settlements, agricultural development, and flooding. As a result, many deer populations have become isolated in severely fragmented portions of their former range.

Like many animals on O'Reilly covers, the conservation status of the Indian hog deer is endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on an antique line engraving from *English Cyclopedia*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

OceanofPDF.com